PHP Gallery

Complete Product Documentation

Installation, administration, architecture, security, maintenance, and development reference

Application version:	1.1.2
Manual edition:	13 November 2026
Document format:	A4, indexed, linked PDF
Project author:	Rudolf Klusal
License:	MIT License

Guide to creating, organizing, presenting, protecting, sharing, and preserving a photographic collection with PHP Gallery version 1.1.2.

Contents

1 Purpose and reading guide **1**
1.1 How to use this manual . . . 1
1.2 Further project references . . . 1
1.3 Terminology . . . 1

2 Product overview **3**
2.1 Core capabilities . . . 3
2.2 Filesystem and database partnership . . . 3

3 Requirements and environment **4**
3.1 Minimum runtime . . . 4
3.2 Permissions . . . 4

4 Installation and initial setup **5**
4.1 Browser installation . . . 5
4.1.1 Installation sequence . . . 5
4.2 Command-line setup . . . 6
4.2.1 Local commands . . . 6
4.3 First administrative review . . . 6
4.4 Clean URLs and rewrite compatibility . . . 6

5 Your first hour with PHP Gallery **7**
5.1 Open the Admin area . . . 7
5.2 Temporary administrator support access . . . 7
5.3 Create a first gallery . . . 7
5.4 Preview as a visitor . . . 8
5.5 Complete the first-hour checklist . . . 8

6 Planning a gallery collection **9**
6.1 Choose a structure people recognize . . . 9
6.2 Decide how deep the hierarchy should be . . . 9
6.3 Prepare titles and descriptions . . . 9
6.4 Plan privacy before uploading . . . 10

7 Creating and editing galleries **11**
7.1 Create a gallery from Admin . . . 11
7.1.1 Creation sequence . . . 11
7.1.2 Remembered defaults and SimBrief drafts . . . 12
7.1.3 Working in the compact editor . . . 12
7.1.4 Attach and convert a PDF document . . . 13
7.1.5 Global Music Manager . . . 16
7.1.6 Music storage . . . 17
7.2 Edit identity and context . . . 19
7.2.1 Review date suggestions across galleries . . . 19
7.3 Gallery editor control reference . . . 19
7.4 Choose a cover and visual identity . . . 23
7.5 Move and reorder galleries . . . 23
7.6 Delete a gallery thoughtfully . . . 24

8 Smart Galleries **25**
8.1 Relationship safety and repair . . . 26
8.2 Presentation settings . . . 26
8.3 Lightbox across result pages . . . 27
8.4 Downloads and safety limits . . . 27

9 Cooperative galleries **28**

10 Windows companion uploader **29**
10.1 Durable capture queue and recovery . . . 29
10.2 Uploader layout . . . 30
10.3 Windows startup and one running instance . . . 39
10.4 Update notice and indicators . . . 40
10.5 Windows simulator location . . . 41
10.6 Flight recording and reports . . . 42

11 Adding, describing, and organizing photographs **46**
11.1 Choose an upload method . . . 46
11.2 Import photos from iCloud . . . 46
11.3 Write useful photo titles . . . 47
11.4 Use tags as a second organization layer . . . 47
11.5 Arrange the viewing order . . . 48
11.6 Review EXIF and location information . . . 48

12 Publishing, sharing, and audience scenarios **49**
12.1 Public portfolio . . . 49
12.2 Private family archive . . . 49
12.3 Client delivery . . . 49
12.4 Event and community gallery . . . 49
12.5 Travel and location archive . . . 49
12.6 Historical and institutional archive . . . 50

13 Understanding your gallery data **51**
13.1 What lives in gallery folders . . . 51
13.2 What lives in the database . . . 51
13.3 How users benefit from the database . . . 51
13.4 Backup as one complete set . . . 52

14 Everyday ownership and care **53**
14.1 After every important upload . . . 53
14.2 Monthly review . . . 53
14.3 Before a major public launch . . . 53
14.4 When another person helps administer . . . 53

15 Public visitor experience **54**
15.1 What a visitor sees . . . 54
15.1.1 Local history and new content . . . 54
15.1.2 Typical visit . . . 55
15.2 Gallery hierarchy and pagination . . . 55
15.2.1 Direct content and stable ordering . . . 55
15.3 Configurable public content widgets . . . 55
15.4 Search, tags, and navigation . . . 56

16 Media and gallery administration **58**
16.1 Gallery inventory and quick controls . . . 58
16.1.1 Upload preferences and mobile connections . . . 58
16.1.2 Appearance, language, and custom stylesheets . . . 58
16.1.3 Card orientation and operational status . . . 59
16.2 Admin workspace . . . 61
16.3 Reliable in-place side-panel changes . . . 62
16.4 Admin navigation . . . 63
16.5 Central Settings hub . . . 64
16.5.1 Guided Setup Wizard . . . 64
16.5.2 Correcting the website address . . . 65
16.6 Tag metadata administration . . . 65
16.7 Feature switches . . . 66
16.8 Gallery management . . . 67
16.9 Metadata organizer . . . 67

16.10 Media renamer . . . 68
16.11 Image management . . . 69
16.12 Uploads and discovery . . . 71
16.12.1 The gallery.json metadata sidecar . . . 71
16.13 Generated metadata and automation . . . 74

17 Thumbnail generation and public rendering **76**
17.1 What a thumbnail is . . . 76
17.2 Why several thumbnail sizes exist . . . 76
17.3 Why JPEG and WebP are available . . . 76
17.4 When thumbnails are created . . . 77
17.5 DNG and RAW display masters . . . 77
17.6 What visitors receive . . . 78
17.7 Responsive browser selection . . . 78
17.8 Progressive thumbnail sharpening . . . 78
17.8.1 What happens on the screen . . . 78
17.8.2 The transfer tradeoff . . . 79
17.9 Which mode should an owner choose? . . . 79
17.10 Loading priorities and stable layout . . . 79
17.11 Thumbnail quality and storage . . . 80
17.12 Thumbnail compatibility and maintenance modes . . . 80
17.12.1 Guarded public thumbnail warmup . . . 81
17.13 If a thumbnail is missing . . . 82

18 Lightbox, maps, EXIF, and voting **83**
18.1 Compact browsing and keyboard focus . . . 83
18.2 Asynchronous lightbox metadata . . . 83
18.3 Adjusting slideshow speed . . . 84
18.4 Zooming and panning a photograph . . . 84
18.5 EXIF and map policy . . . 85
18.6 Flight maps and navigation data . . . 86
18.7 Voting . . . 89
18.8 Picture Game . . . 89

19 Duplicate Photo Detector **90**
19.1 Evidence categories . . . 90
19.1.1 Exact and possible findings . . . 90
19.2 Persistent review ledger . . . 90

20 Access control and security **91**
20.1 Layered access evaluation . . . 91
20.2 Passwords and share links . . . 91
20.3 Viewer accounts and invite-only registration . . . 91
20.4 Administrator account and mail settings . . . 94
20.5 Database readiness contract . . . 95
20.6 Security-sensitive exceptions . . . 96
20.7 Writes, maintenance, and credentials . . . 96
20.8 Optional features . . . 97
20.9 Operational security . . . 97
20.10 Media storage boundaries and recoverable migration . . . 97

21 Theme, presentation, and localization **99**
21.1 Theme control reference . . . 99
21.2 Advanced appearance and manual CSS overrides . . . 103
21.3 Lightbox browsing modes . . . 106
21.4 Choosing the interface language . . . 107
21.4.1 Customizing the viewer selector . . . 107
21.4.2 Multilingual gallery and photo text . . . 107
21.4.3 Available languages and their current state . . . 108

21.4.4 How translation packs work . . . 108
21.5 Gallery hero tags . . . 109

22 Downloads, sharing, and transfer 110
22.1 Optional original PDF and music files in ZIP downloads . . . 110
22.2 Gallery-to-gallery migration . . . 110
22.3 Mobile WebDAV transfer . . . 111
22.4 Public discovery and selected-photo tools . . . 111

23 Maintenance, updates, and recovery 113
23.1 Migrations . . . 113
23.2 How the gallery checks database readiness . . . 113
23.3 Gallery trash and recovery . . . 114
23.4 Maintenance Center . . . 114
23.5 Thumbnail maintenance . . . 115
23.6 SEO request guard and public query hygiene . . . 116
23.7 Database maintenance . . . 116
23.8 Storage statistics and Complete report . . . 116
23.9 Telemetry and privacy controls . . . 117
23.10 Runtime diagnostics, integrity, and gallery benchmark . . . 118
23.11 Scheduled site maintenance . . . 119
23.12 Cron Overview and visitor scheduling . . . 122
23.13 Admin log history and archives . . . 123
23.14 Complete current feature reference . . . 124
23.15 Application updates . . . 130
23.16 Integrity manifest . . . 134

24 Optional background: how PHP Gallery works 135
24.1 Bootstrap, routing, and dispatch . . . 135
24.2 Controllers . . . 136
24.3 Services . . . 136
24.4 Views and browser modules . . . 137
24.5 Public selected-gallery render pipeline . . . 137

25 Optional background: what the database remembers 138
25.1 Principal domains . . . 138
25.2 Relations and cascades . . . 138
25.3 Application settings . . . 138

26 Gallery-scoped API-key reference 139
26.1 Scope and support boundary . . . 139
26.2 Operation index . . . 139
26.3 Connection, keys and request conventions . . . 140
26.4 Upload photographs . . . 141
26.5 Inventory: compare supplied file candidates . . . 143
26.6 Partial results, timeouts and recovery . . . 144
26.7 Revoke the current key . . . 145
26.8 Specialized AI worker contract . . . 145
26.9 Claim one AI job: `ai_next_job` . . . 145
26.10 Renew an AI claim: `ai_heartbeat` . . . 146
26.11 Complete or fail an AI job: `ai_complete` . . . 147
26.12 Fetch the claimed AI input: `ai_asset` . . . 148
26.13 Look up a PDF report receipt: `flight_report_status` . . . 148
26.14 Attach a PDF report: `flight_report` . . . 149
26.15 Migration: authority and data contract . . . 149
26.16 Export the manifest . . . 150
26.17 Download one source asset . . . 152
26.18 Download a source ZIP package . . . 152
26.19 Prepare the destination job . . . 153

26.20 Upload one asset to the destination . . . 153
26.21 Upload a planned ZIP package . . . 154
26.22 Reconcile destination progress . . . 155
26.23 Complete the destination job . . . 155
26.24 HTTP errors, limits and compatibility . . . 156
26.25 Complete Python upload, inventory and migration example . . . 157
26.26 Implementation map and maintenance . . . 162

27 Optional reference for developers **163**
27.1 Repository organization . . . 163
27.2 Coding conventions . . . 163
27.3 Authoring code before source checks . . . 163
27.4 Adding a feature . . . 165
27.5 Preparing source candidates for CI . . . 165

28 Checking your gallery before publication **167**
28.1 A visitor-centered publication check . . . 167
28.2 Automated checks for software maintainers . . . 167
28.3 Manual browser verification . . . 168
28.3.1 Network and interaction checks . . . 168
28.4 Release hygiene . . . 169

29 Making the gallery feel fast **170**
29.1 Practical ways to improve speed . . . 170
29.2 Slow-connection test . . . 170
29.3 Technical measurements for maintainers . . . 170

30 Troubleshooting **172**
30.1 Application does not start . . . 172
30.2 Gallery or images are missing . . . 172
30.3 Thumbnails are missing or soft . . . 172
30.4 Admin actions leave the side panel . . . 172
30.5 Migration fails . . . 172
30.6 PDF manual compilation fails . . . 172

31 Backup and recovery checklist **173**
31.1 Validate a recovery set . . . 173

32 Maintainer verification and release evidence **174**
32.1 Source qualification and selected feature workflows . . . 174
32.2 Release assembly from qualified develop . . . 174
32.3 Review, version and complete attachments . . . 176

33 Glossary **177**

34 References **178**

Index **180**

1 Purpose and reading guide

PHP Gallery is a self-hosted photo gallery. The photographs remain on storage you control, while the application adds the catalog, navigation, access rules, thumbnails, maps, search, downloads, and administrative tools around them. A single installation can be used for a public portfolio, a family archive, client delivery, event photographs, travel collections, or a large personal library.

The manual is organized around work that an owner actually performs. The early sections cover installation and the first gallery; the middle of the manual deals with publishing, media, privacy, presentation, and maintenance. Architecture and developer material is kept near the end so it does not interrupt ordinary administration.

For a new installation, begin with Section 5. Collection planning is covered in Section 6; routine gallery and photo editing is in Sections 7 and 11. Section 12 is useful before publishing because it compares several common audience types. The contents and index are intended for jumping directly to a setting or workflow when the site is already in use.

1.1 How to use this manual

Admin names such as **Theme** and **System Health** are shown in bold. Monospaced text denotes filenames, setting names, values, or commands. Procedures are written as ordered lists where sequence matters; checklists and reference material use bullets or tables. Technical implementation notes are included only where they explain a visible behavior, a recovery decision, or a maintenance requirement.

1.2 Further project references

Developer and hosting references remain separate from this owner manual. The References section lists the repository documents used for architecture, database, source ownership, testing, and contribution rules [1, 2, 3, 4, 6, 7].[1]

The project website offers the published download and documentation in English, Czech, German, and Swedish. Its light/dark switch changes the presentation page; the enlargeable application screenshots retain their captured language, appearance, and version. Screenshot source links lead to the public gallery or the corresponding documentation. These controls do not change your installed gallery's theme.

1.3 Terminology

Gallery A named collection of photographs. A gallery can contain photographs and smaller galleries.

Photograph or image One item in a gallery, together with its title, description, tags, visibility, and available camera information.

Thumbnail A smaller copy created for quick display in gallery grids, search results, covers, and previews.

Visitor A person viewing the public side of the site.

Administrator A trusted person who signs in to create and manage content and settings.

[1] Those files are the better source when changing code, inspecting schema details, or preparing a release.

Selected gallery The gallery currently open on the screen.

Gallery branch A gallery together with every smaller gallery contained beneath it.

2 Product overview

PHP Gallery turns a folder collection into a browsable photographic website. Visitors see covers, titles, descriptions, photo grids, full-screen viewing, maps, tags, search, and downloads according to the choices made by the owner. Administrators receive private tools for adding photographs, arranging collections, controlling privacy, improving descriptions, and keeping the installation healthy.

Ordinary PHP/MySQL web hosting is the primary deployment target. The owner keeps the source photographs in recognizable folders, while the database supplies the catalog and application state needed for browsing.[2]

2.1 Core capabilities

- Create galleries inside other galleries, for example *2026 / Italy / Rome* or *Clients / Northwind / Final selection*.
- Give every gallery a title, description, date range, cover, public address, privacy choice, and individual appearance.
- Upload photographs through the browser or discover files copied into gallery folders.
- Add titles, stories, tags, visibility, ordering, dates, and location information to photographs.
- Let visitors search, browse tags, view photographs full screen, follow maps, vote, and download permitted collections.
- Protect family, client, or sensitive collections with access controls and share methods.
- Find likely duplicate photographs and review them with direct links to both locations.
- Keep the installation current through guided updates, health checks, backups, and maintenance tools.
- Automate scoped uploads, exchange galleries with compatible installations, and prepare selected-photo or gallery downloads.
- Generate optional AI metadata and translation drafts, while keeping every generated value reviewable before publication.
- Publish crawler-friendly robots and image-sitemap output for the public content that visitors can actually access.

2.2 Filesystem and database partnership

PHP Gallery stores persistent data in both the filesystem and the database. Gallery directories hold the source photographs in a recognizable tree. The database stores the information that does not belong in the file itself: titles, descriptions, tags, ordering, access settings, votes, dates, generated-media records, and other application state.

Neither side is a complete backup on its own. Restoring only the database leaves catalog entries without their source files; restoring only the gallery tree loses the administrative and presentation state associated with those files. Backup and recovery should therefore treat the database, gallery directories, and local configuration as one set. Section 13 returns to this in more detail.

[2] The project overview provides the short installation and feature summary [1].

3 Requirements and environment

Use this section when choosing a hosting plan or checking an existing account. The requirements table is also suitable for sending directly to a hosting provider.

3.1 Minimum runtime

Component	Requirement
PHP	Version 8.1 or newer.
Database	MySQL 8.4 LTS, MariaDB 10.11 LTS, or MariaDB 11.4 LTS.
PDO	PDO MySQL extension for application queries and migrations.
Image processing	GD for common thumbnail generation. Additional local image tooling can support specialized formats.
Archives	ZipArchive for package, download, and transfer workflows that produce or consume ZIP data.
Filesystem	Writable application data, gallery, cache, and generated-media locations.
Web server	Apache-style rewriting is recommended for clean URLs; query-string routing supports compatible environments without rewrites.

The maintained support matrix names MySQL 8.4 LTS, MariaDB 10.11 LTS, and MariaDB 11.4 LTS as representatives directly exercised by required CI. The canonical support document lists MySQL 8.0 and MariaDB 10.4–10.10 as legacy best-effort series without a required project database workflow or project regression and security assurance; they are outside the maintained qualification set. MySQL 5.7 or older and MariaDB 10.3 or older are unsupported. Other database-server series remain unqualified until required CI tests them directly; a higher version alone does not establish compatibility. See `docs/DATABASE_SUPPORT.md` for the full policy.

Outbound HTTPS access supports application updates and remote release metadata. Local operation and manually deployed releases can function with a more restricted outbound policy.[3]

3.2 Permissions

The web process needs read access to runtime code and write access to designated mutable locations. Keep runtime code read-only where hosting permits it, except during a controlled updater operation. Gallery source files deserve backup coverage equal to the database because the complete application state spans both domains.

Important. Create backups of the database, gallery files, and installation configuration together. A database-only backup preserves metadata while omitting the authoritative media tree.

[3] Network policy should follow the installation owner's update and privacy requirements. Update code validates packages and preserves recovery data according to the local updater implementation.

4 Installation and initial setup

Internal application, tooling, documentation, cache, data, and log directories carry their own Apache authorization guards, independent of URL rewriting. The host must honor those directives or provide equivalent server rules; PHP's development server does not read them. Direct HTTP requests to CLI commands receive an empty 404 before application startup. Continue to use the public front controller for browser routes and the shell for maintenance commands. See `docs/HTTP_ENTRYPOINTS.md`.

Build requested production folders or ZIPs with `scripts/package_production.php`. The PHP 8.1+ builder uses `app/production-files.json`, requires current integrity data, and verifies exact membership and complete SHA-256 content before publication. Existing outputs, unsafe paths and filesystem links are refused. Tests, private media, installed configuration and runtime state cannot enter the package. WinApp sources retain their separate companion inventory. Historical interactive and FTP deployment scripts have been removed. See `docs/PRODUCTION_FILES.md`.

Companion font/license resources use the optional `companion_resource_files` inventory field. Installed readers can ignore this addition while the current package policy validates its exact allowed paths. The original companion field stays readable by older companion-aware updaters; neither field grants CMS replacement ownership. Direct updates use a complete newer package and all pending migrations, without requiring each intermediate release. This does not qualify arbitrary historical hosting or database states. Cancel a failed pre-activation job before selecting a corrected newer package; retrying the same incompatible build cannot repair its inventory.

4.1 Browser installation

4.1.1 Installation sequence

The browser installer gathers database settings, initializes the schema through migrations, creates the first administrator, prepares writable locations, and writes local configuration. A fresh request redirects to installation when required configuration is absent.[4]

Migrations finish before the first administrator, `config.php`, and the installation lock are created. Repairs use the verified installer connection and the shared gallery-writer lock; before configuration exists, gallery storage is the installation's `galleries/` directory. If a migration fails, correct its cause and submit the administrator step again. Completed migrations remain recorded and the failed callback is retried. Preserve the migration ledger and leave completed installations' configuration and locks in place.

Typical steps are:

1. Create an empty MySQL or MariaDB database using the hosting control panel.
2. Upload the application package to the intended web directory.
3. Open the site and follow the installer.
4. Enter database connection values and the initial administrator credentials.
5. Confirm writable directories and extension checks.
6. Complete installation and enter the Admin area.

[4] Installation state and generated configuration should receive the same protection as credentials. Source archives should continue excluding the local `config.php`.

4.2 Command-line setup

4.2.1 Local commands

The repository provides plain PHP scripts for migration and administrator creation:

```
php scripts/migrate.php
php scripts/create_admin.php <username> <password>
```

Command-line setup follows the same persistent schema model as browser setup. Review `config.example.php`, create the local configuration, provision the database, run migrations, and create the first administrator.

4.3 First administrative review

After installation, review:

- Site name, language, theme, page width, and public rendering preferences.
- Gallery root permissions and discovery results.
- Thumbnail formats, dimensions, quality, and maintenance state.
- Access, password-reset, email, telemetry, update-channel, and backup preferences.
- Rewrite compatibility and canonical public URLs.
- System Health diagnostics and pending migrations.

4.4 Clean URLs and rewrite compatibility

PHP Gallery supports clean public paths when the hosting environment applies the project's rewrite rules. Clean URLs are a presentation and routing preference, not a requirement for the gallery data model. Query-string routes remain the compatibility path and continue to work when rewriting is disabled or cannot be verified reliably.

The URL rewriting setting controls which form PHP Gallery emits in generated links. When rewriting is off, helpers use query-style addresses for galleries, photographs, thumbnails, and other application routes instead of assuming that the web server will translate a clean path. Slugs, gallery folders, database rows, and source photographs do not need to be renamed when this setting changes.

If clean paths fail on a new host, first disable URL rewriting in Admin and confirm that the query-string form, such as `?page=gallery`, works. On Apache or LiteSpeed, then verify that the project's `.htaccess` is honored and that rewrite support is available. On other web servers, keep the compatibility mode unless equivalent routing rules have been configured.

5 Your first hour with PHP Gallery

For the first session, keep the scope small: create one gallery, add a few photographs, and open the result from a browser that is not signed in as Admin. Theme work and deeper organization can wait until that basic path is known to work.

5.1 Open the Admin area

Choose **Admin login** from the public header and enter the account created during installation. The dashboard is the starting point for everyday ownership. It summarizes galleries, photos, system state, and available maintenance actions.[5]

After authentication, a return link can open a local page of this installation. External URLs and return paths containing backslashes are refused; the application instead uses the safe local fallback of the login workflow.

Spend a few minutes identifying these areas:

- **Galleries**, where collections are created, discovered, edited, and arranged;
- **Theme**, where the public site's appearance and presentation defaults are selected;
- **Tags**, where reusable subjects and categories are maintained;
- **System Health**, where migrations, thumbnails, storage, and consistency are reviewed;
- **Account**, where credentials, recovery email, and mail delivery are managed.

5.2 Temporary administrator support access

A permanent administrator can open **Account**, choose **Temporary admin access**, and generate a single-use support link. The link is shown only at creation and expires exactly 24 hours after issuance, regardless of when the recipient first accepts it. Share it privately with a trusted support person. The recipient should use a separate browser profile and explicitly accept the invitation; no new permanent administrator account is created.

The Admin header displays a temporary-access notice and UTC expiry. The recipient can inspect diagnostics and work on permitted gallery/media tasks, but cannot change administrator credentials, global security settings, persistent tokens or updater configuration. The owner can revoke the grant at any time; the next authorization check refuses every associated session. Expiration and revocation require no scheduled cleanup.

Security note. The unused invitation link is a bearer credential. Never publish it in logs, tickets or screenshots. Existing permanent Admin sessions remain unaffected by revocation. Apply pending database migrations before issuing support invitations.

5.3 Create a first gallery

Open gallery administration and create a gallery with a short title such as *Summer Walk*. Review the proposed public path and folder name, then add one sentence describing what the collection

[5] The exact dashboard contents depend on enabled features, completed migrations, and development settings. A healthy installation can still display informational notices that invite an administrator to complete optional work.

contains.

For this first check, make the gallery public and leave advanced branding or inherited overrides at their defaults. Add three to ten photographs. That is enough to verify ordering, thumbnail generation, titles, and the lightbox without turning the exercise into a large import.

5.4 Preview as a visitor

Use the public-gallery link or visitor-preview control. For a realistic check, open the public address in a private browser window where the Admin session is absent. Confirm that the title, description, photographs, and navigation appear as intended. Open a photograph, move forward and backward in the viewer, and return to the gallery.

Administrator views can contain edit controls and additional assets. Anonymous preview gives the clearest picture of the experience received by family, clients, or the public.[6]

5.5 Complete the first-hour checklist

1. Set the public site name and language.
2. Create one gallery with a title and description.
3. Upload a small group of photographs.
4. Add a meaningful title to at least one photograph.
5. Reorder two photographs and confirm the new order publicly.
6. Open the gallery as an anonymous visitor.
7. Check the gallery on a phone-sized screen.
8. Confirm that System Health reports no urgent migration or permission problem.
9. Create a coordinated backup after the initial setup is complete.

[6] Protected galleries and age-restricted content deserve a separate anonymous test because an active administrator session already has broader access.

6 Planning a gallery collection

A thoughtful structure helps visitors understand a collection and helps the owner maintain it over time. Start with the way people will look for photographs. Folder names and database details can follow that human organization.

6.1 Choose a structure people recognize

Common structures include:

By year and event A top-level year contains weddings, trips, celebrations, and projects. This suits family and historical archives.

By place Countries contain regions and cities. This suits travel, aviation, landscape, and documentary photography.

By client or project Each client contains assignments and delivery galleries. This suits professional work with separate access rules.

By subject Wildlife, architecture, portraits, aircraft, and similar subjects form enduring categories. Tags can provide a second way to browse across them.

By workflow Incoming, selected, delivered, and archive galleries represent stages. Visibility settings keep working collections private while finished selections become public.

Use galleries for strong navigational boundaries and tags for relationships that cross those boundaries. A photograph from Prague can live in *Travel / Czech Republic / Prague* and carry tags such as *architecture*, *night*, and *tram*.[7]

6.2 Decide how deep the hierarchy should be

Nested galleries support deep collections, yet most visitors browse more comfortably when each level has a clear purpose. Two to four levels serve many sites well. A deeper archive remains practical when breadcrumbs, meaningful titles, and consistent dates guide the reader.

Create a child gallery when it deserves its own title, description, cover, access policy, date range, or public address. Keep photographs together when a new level would add only one extra click.

6.3 Prepare titles and descriptions

A gallery title should make sense outside the Admin interface. A description can answer three questions:

- What does this collection show?
- Where and when was it created?
- What context makes the photographs meaningful?

Descriptions can be brief for casual albums and extensive for documentary work. Use the first sentence as the essential summary because visitors often scan before reading the full text.

[7] A photograph has one gallery location in the normal content model, while reusable tags provide several semantic connections. Copy and move tools support deliberate changes to physical organization.

6.4 Plan privacy before uploading

Choose whether a collection is public, unpublished for administrative preparation, or protected for a known audience. Parent-gallery policy can influence descendants, so place collections with similar audiences together where practical.

Examples include:

- a public portfolio with carefully selected photographs;
- a password-protected family branch;
- an unpublished client proofing gallery during preparation;
- a public event gallery with selected individual photographs marked private;
- an age-gated branch for content that needs visitor confirmation.

7 Creating and editing galleries

7.1 Create a gallery from Admin

Use the + control on a public gallery or **Create gallery here** in its editor to start a child gallery. The right-side Admin panel asks only for the new title; it already knows the parent gallery that supplied the control. The new gallery starts **Unpublished**. After creation, the same panel opens its complete editor for description, date, language, tags, access, display, media, and photographs. The public page does not navigate away during this enhanced workflow.

To start from the standalone Admin create page, enter a title and optionally provide SimBrief details, description, source language, and tags before submitting. Its closed **More options** section contains the folder name, initial visibility, date range, parent, voting, filename display, and contained-picture badge. A short summary below it shows the current visibility and parent. The dedicated create-and-upload form remains available when photographs should be uploaded during creation. On a validation error, submitted non-secret values and parent context stay available for correction.

As you type at least two characters in the title field, Admin may show the remaining text of a matching existing gallery as a faint inline suggestion. Recent titles under the selected parent are preferred; matches elsewhere are considered only after the bounded sibling search is exhausted. Older titles can be omitted, so no suggestion does not prove that a title is unique. The form requests at most eight candidates instead of loading the complete title catalog.

With the caret at the end and no text selected, press unmodified `Tab` or **Right Arrow**, or select the suggested text, to accept it. Modified shortcuts and input-method composition keep their ordinary behavior. The first `Escape` dismisses completion; a later press can reach the surrounding panel. Translated help and a polite announcement make suggestions discoverable without changing the field name.

Unicode matching accepts only safe whole-character boundaries. Some non-ASCII suggestions are omitted when server extensions or browser segmentation support are unavailable. A slow, failed, or outdated lookup never prevents ordinary typing. Acceptance changes only the title field: it does not submit the form, rename an existing gallery, or alter its access rules. The same behavior applies after a side-panel form is replaced. See `docs/TITLE_COMPLETION.md` for matching limits and technical details.

After the name-only creation, the gallery is already a real collection in the gallery tree, even before additional editor fields are saved. Its physical folder can receive photographs through the browser, filesystem transfer, discovery, or another supported upload workflow. Fill and save the editor fields before publishing. The database stores descriptive and presentation settings; the normal access policy determines what visitors can see.

Each generated create or classic-upload form also carries a private operation identity. If a connection fails after the server may have finished the work, retry the same form instead of opening a new one: the same administrator, operation, and content receive the original completed result. A freshly opened form represents a new intentional operation and can still create a deliberate duplicate. This identity is not a password or a replacement for sign-in and CSRF protection.

7.1.1 Creation sequence

1. Open the intended parent and choose its + control, or start a root gallery from Admin.
2. Enter a visitor-friendly title; optionally accept a title suggestion.

3. Create the Unpublished gallery. The full editor opens in the same panel when JavaScript is available.
4. Add the description, date or date range, source language, tags, and optional SimBrief data in **Identity**.
5. Review **Access**, **Display**, **Media management**, and the gallery folder/parent values in Advanced where relevant.
6. Save the gallery, add photographs, and choose a cover after suitable photographs exist.
7. Publish when ready, then preview the result anonymously.

7.1.2 Remembered defaults and SimBrief drafts

The editor can remember the current SimBrief identifier and source language for the signed-in administrator. Select the nearby **Remember for future galleries** choice only for a value you want to reuse. Later galleries show that value already selected with a small note saying it came from your saved defaults. The value remains editable; leaving Remember unchecked preserves the previous saved choice. Defaults belong to one administrator and do not publish credentials or copy access rules. The normal migration action must apply `202609250001_gallery_creation_preferences.php` before Remember can save a new default; gallery creation still works without that optional table.

The visible SimBrief field accepts a Pilot ID or pilot name. Digits alone are treated as a Pilot ID, while other text is treated as a name. On a new or existing gallery, **Generate description draft** can fetch the latest OFP and fill an editable Markdown description in the selected content language. When gallery localization is enabled and its translation storage is ready, the draft also fills the maintained-language description fields. Review and edit the description fields before saving. The import keeps a private reference for 30 minutes, limited to the same administrator and session; changing the identifier invalidates that reference. For an existing gallery, the OFP and available route coordinates are attached only after you save the gallery. If an optional attachment fails, the gallery remains saved and the panel reports a warning. Re-import when the draft expires or the session changes.

7.1.3 Working in the compact editor

The ordinary work areas are **Identity**, **API**, **Access**, **Display**, and **Media management**; image and specialized tools follow as available. The sticky top navigation keeps the disk **Save** button visible while the settings form or photo list scrolls. Changing tabs does not save automatically. Use the button after editing, then reopen the gallery if you want to verify the stored values. Longer explanations sit behind keyboard-accessible question-mark controls, and less frequent choices sit under **Advanced** disclosures. A direct Admin editor follows the same fields, while a browser without JavaScript uses the normal form submission.

Identity keeps title, description, date, source language, SimBrief, and tags readily available. Its Advanced section includes slug, folder, parent, sort order, and AI gallery text when enabled. **API** displays a copyable upload endpoint, the key label/action, and a newly generated key only once; existing keys appear only when there are any. AI metadata regeneration, migration, and the global API manager live under **Advanced API tools**. **Access** groups visibility, password, share-link controls, and the NSFW checkbox, with longer policy explanations under question marks. **Display** uses compact translucent groups for the grid, photo cards, gallery cards, navigation and maps, visitor interactions, and thumbnail quality. In **Photo cards**, choose Default, 16:9, 21:9, 4:3,

1:1, 4:5, or 9:16. Default preserves the existing 4:3 desktop and square mobile previews; explicit formats apply on every screen, only to photos inside this gallery. Originals, lightbox images, and gallery cover cards remain unchanged. Save gallery commits the format and other staged controls. File-name, voting, and Picture Game switches save independently with JavaScript; recursive thumbnail copying and the coupled grid controls remain staged. A failed or unverified switch save keeps drafts and blocks shared-form replay until the current editor is reviewed. Help opens beside its question mark; breadcrumb previews remain in a disclosure. Flight-route text remains beside SimBrief in **Identity**.

In **Admin > Features**, **Gallery PDF documents** and **Gallery music** are independent switches. Both default ON for new installations and upgrades without a saved choice; an explicit OFF remains saved. OFF hides the matching tables/players and prevents uploads, delivery and PDF conversion, including stale forms. Files, playlists and preferences remain stored for re-enabling. With both OFF, core Presentation settings remain when lightbox is enabled. With lightbox also OFF, the empty tab is omitted; old tab URLs select Identity. Existing converted page photographs keep their normal access rules. SimBrief can still refresh descriptions/routes with PDF OFF and keeps an old original.

In **Media management > Music**, the catalog is on the left and this gallery's playlist on the right; narrow editors stack them. Play appears before each song title and Add at the row end. Select several items with checkboxes and use the Add icon beside search. Existing songs remain playable but their Add controls are disabled, including songs supplied by live albums. Albums use a files icon and a distinct chevron; only expanding them loads their indented songs. Preview does not assign music. Batch additions run in order and report partial progress if an item fails. Removing a gallery source restores available Add actions without deleting shared originals. The shared header holds the player and gallery-wide slideshow music switch. In **Display**, **Newest uploads first** is off by default. When enabled, an upload or changed scan orders this gallery by registration time, newest first. Equal timestamps use the image ID. The same preference is available in **Media management > Images**; both switches control one saved setting. Without JavaScript, use the adjacent save button. Images also offers a one-time sort; manual order lasts until the next changed scan. Registration time is local to the image row; re-import can assign a new time. In **Media management > Images**, a large existing thumbnail does not replace the required 300px preview. Use the thumbnail tools to create gallery thumbnails, then retry Images when that size is missing. Disable browser-side rebuilding to generate on the server. Loading editor previews never downloads originals as a fallback.

A failed music action shows a reference matching **Admin > Logs**, category **Media**. It records the gallery, selection, action, stage and whether the write completed, without raw exceptions, SQL, credentials or absolute storage paths. If Admin logging fails, the safe record goes to PHP's error log. Missing originals require restoring or uploading audio; a dirty source requires the music catalog task in Maintenance Center. Move SQL, `data/music/` and gallery sidecars together between machines. After a saved-but-unrefreshed result, reopen the editor first.

7.1.4 Attach and convert a PDF document

Expanded conversion controls keep the document column bounded. If the previous target is missing or changed, confirm a new private result; earlier files and galleries are preserved.

Translucent cards keep PDF file count, total size and total pages visible, along with the music track count and combined duration. Unknown metadata remains a dash; totals never silently omit an unreadable original. Cards use the gallery's available width: wide windows place them beside each other, narrower windows stack them and move them below the description.

Smart Galleries use the same hero cards, **Presentation** button and share panel. PDFs and OFPs

come from accessible physical sources contributing matching photos to the complete collection; hidden documents stay absent even for an administrator on the public Smart page. PDF links retain source access checks. The shared link preserves language and a selected source filter, omits page numbers, and starts at matching photo one. Explicit launch defaults to autoplay and music ON with medium speeds without changing ordinary Smart slideshow music. The same black introduction, activation confirmation and exclusive mode apply.

Below the cards, **Presentation** starts at the first photo with configured speeds and soundtrack for that launch. The saved slideshow music preference stays unchanged. The share icon opens an overlay that creates a compact gallery link at the application root, e.g. `/Galerie/presentation=1a2b3c4d`, without an album path, pagination or preview state. The eight lowercase hexadecimal characters identify a stored snapshot of the playback options, language and any Smart source filter. The same gallery and settings reuse the same code; another gallery receives its own code. Links have no automatic expiry and never grant gallery access. Without clean URLs, the short code uses the `presentation` query parameter. Run the normal database migration after upgrading; unavailable storage shows an error and disables copying. Older long links remain supported. The database stores versioned JSON and a full SHA-256 fingerprint separately from the surrogate primary key, preserving supported snapshots when options grow. Generated gallery URL slugs reserve eight-character hexadecimal names and presentation prefixes, adding `gallery-` when necessary; displayed gallery titles remain unrestricted. Playback retains the short root address. The share overlay uses compact translucent Theme colors, small switches, thin speed sliders and an icon-only Copy action with visible keyboard focus. Opening it starts the same presentation after ordinary access checks. Browser autoplay restrictions may require **Play**; denied clipboard access reveals a selectable link for manual copying. Anonymous visitors can use the same link after ordinary access checks. If native fullscreen or sound needs activation, the black opening waits for **Start presentation**. After confirmation, unsupported native fullscreen retains the browser viewport; **Play** can retry later audio failures. The start confirmation matches the share panel with compact transparent Start/Close actions, visible keyboard focus and larger touch targets on narrow screens. Use **Edit gallery > Media management > Presentation** and its transparent **Presentation settings** card to choose public button visibility, music, photo speed and transition speed. The Save icon uses the ordinary guarded gallery form. Preview and Share use current selections, including unsaved ones. New links store music and both speeds; later gallery edits do not change those snapshots. Visitors can change playback controls temporarily. `autoplay=1` also starts a presentation. If sound needs activation, the player hides with the idle HUD; movement shows it again. Click **Play** or the photo to retry sound and requested native fullscreen. Stop or Close cancels pending audio. Hiding public buttons does not revoke links. Defaults use versioned `galleries.presentation_json` after the column migration. Explicit writes require verified schema; older galleries retain enabled, music and medium defaults until configured. Autoplay defaults ON for every gallery until explicitly disabled in the card. The share overlay has speed sliders and switches for autoplay, music, fullscreen and **Without controls**; its Copy icon copies the current URL. Share drafts do not save gallery settings. `autoplay=0` opens photo one paused and silent. Fullscreen and exclusive choices use `presentation_fullscreen=1|0` and `presentation_exclusive=1|0`. Without controls defaults OFF; enabling it hides controls, metadata and diagnostics while Esc exits. Browser sound restrictions still apply. Each launch holds black for at least one second and until photo one decodes, then checks requested fullscreen and music. If the browser needs activation, black remains with a compact **Start presentation** confirmation. One click starts music and requests fullscreen before the selected reveal. This confirmation also appears for links without controls; it then disappears. Esc or the close icon cancels. Autoplay OFF stays paused and silent. This opening never repeats at the gallery loop. Closing cancels pending work. Native fullscreen may need a click; the fallback fills the browser viewport.

PDF and Music launchers show file counts beside the description, or below it on narrow screens.

Click a launcher to open a small overlay. Its list has a maximum height of 12rem and scrolls independently. PDF rows retain size, page count and small action icons; a title or eye opens the protected viewer. Administrators can expand conversion, hide/show and confirm deletion. Hidden PDFs remain denied through direct links; generated photos remain after deletion. Escape, the close icon, launcher or an outside click dismisses the applet. Native details keeps a bounded inline list without JavaScript or Popover support. Reduced motion disables disclosure animation. In-place replacement retains the open list and scroll.

The existing-gallery Upload photos chooser and drop queue also accept PDFs. By default, an upload attaches only the original document; it does not create a subgallery or PNG pages. To convert during upload, open **Advanced** and explicitly check **Also convert PDF pages to PNG**, then select a private child gallery or append pages and choose the DPI. Opening Advanced alone does not enable conversion. Mixed selections upload photos first, then attach PDFs and perform only the selected conversions. The panel stays open with progress and cancellation controls. Retries retain the original conversion choice; for an enabled conversion, unchanged inputs automatically recreate a missing empty child only when no page was acknowledged.

The gallery editor combines photos and media in **Media management**, with **Images**, **Music**, **PDF**, **Presentation**, **Artwork** and ZIP preferences. Its single **Upload files** picker accepts enabled photo, PDF and audio formats together and routes each original automatically. PDF replacement, deletion and conversion remain in **PDF**; the optional OFP panel remains there too. Artwork and presentation defaults retain their gallery settings form. The sticky top navigation includes a disk **Save** button, available above long photo lists. Uploads and photo ordering still save independently. The compact upload drawer hides support and instructions under a plain question mark; Escape or an outside click closes help. **Advanced** contains iCloud and processing options; PDF page conversion remains an unchecked opt-in. Former Images links remain supported.

An administrator can attach a general PDF to a physical gallery even when it has no SimBrief data. The gallery keeps the original file and a small index with its stable document ID, filename, title, provenance, exact size and SHA-256 digest. The protected same-origin viewer and download link follows the gallery's visibility, password, share-link and NSFW access rules. SimBrief OFPs keep their existing manifest and links.

Choose **Convert PDF pages** to render the document in the browser. PDF.js creates PNG pages sequentially and the existing upload process reports progress, acknowledges batches, resumes interrupted work and can be cancelled. New conversions use 150 DPI by default; choose 96, 150, 200, 300, 600, 1200 or 2400 DPI. There is no page-count limit. A page above the conservative 16,384-pixel edge or 32-million-pixel advisory threshold shows a warning but does not block rendering or silently downscale the page. Its memory estimate covers only the canvas RGBA buffer at four bytes per pixel, not total browser memory. Actual browser canvas allocation and upload limits can still make rendering or upload fail.

Choose either a reserved private child gallery or append after the source gallery's last photograph. A child stays private until you change its visibility explicitly; publication is blocked while its PDF conversion is incomplete or cancelled, so finish or safely reconcile the conversion first. Appending keeps the source gallery's visibility and existing photographs, then places converted pages at the end. Conversion never replaces the original PDF. Progress markers live in the source gallery's hidden conversion sidecar for both destinations and record the target gallery ID. Photo ZIP exports and gallery migration do not transfer these general PDF originals. A full filesystem backup must include hidden sidecar folders. Existing JPG page galleries remain separate. The former server-side OFP conversion request now returns HTTP 409 and directs you to reload; its rasterizer helper is for CLI and test compatibility only and is not called by browser actions. See `docs/GALLERY_PDF.md` for the storage and conversion contract.

7.1.5 Global Music Manager

Each song has an independent Play/Pause action. The compact player provides Previous, Play/Pause, Stop, Next, Repeat and Mute. Its timeline shows elapsed time and the original duration; click, drag or use the keyboard to seek. Seeking while paused keeps playback paused; Play resumes the same cursor. Stop releases the source and resets its timeline. Listening never assigns files or changes gallery defaults. Repeat cycles All, One, Off and back to All. All starts enabled and follows the whole saved album, including songs beyond the displayed page. One restarts the current song; Off stops after the last song. Active modes have an accent tint and line; the One icon adds 1 and Off crosses the loop arrows out. Tooltips name the current mode. Changing mode preserves Pause and the cursor; Next/Previous still select another song. Each list retains its choice when switching players; reloading resets All. If the catalog changes during a continuation, reload before continuing. Mute carries between player scopes on the same page, while JavaScript is required for these controls. Native editing and upload forms remain available without JavaScript.

In **Albums & playlists**, select or create an album, then use **Upload files** below the existing catalog-song selector. Both ways of adding songs remain available. Choose up to twenty audio files, at most 64 MiB each, and press **Upload**; no gallery is required. Hosting limits also apply. Identical original bytes reuse the existing song even after renaming. A song can belong to several albums independently, and removing its last membership retains its original in the catalog. Reuploading an existing member preserves its order. Progress and individual results remain visible; a failure stops further files. Reload before retrying an unverified response. Without JavaScript, the native form submits the whole selection in one request. Uploading never starts playback.

Open **Admin > Galleries > Music Manager** for the compact song catalog. Reconcile existing playlists in bounded batches; originals and playlist order remain intact. Search by title, filename or music tag as you type; only the latest search response updates the results. The table separates Title, **Files**, **Locations**, **Tags**, and Actions. Click a song title to open its existing rename editor, add-to-album control, and shortcuts to tag, file, and location details; its independent checkbox selects it for merge. The **Songs** tab is a full-width catalog. **Albums & playlists** places the tree and creation form left of the selected album and its songs, stacking on narrow screens. With no selection, a prompt invites you to choose or create an album. Legacy album links open this tab. Creating an album selects it automatically; deleting the selected album clears the selection, and a stale album link returns to the prompt. Adding from the title popup appends a song to an album; an existing membership stays in its current order. Click a song title in the album to edit its title or open its tag editor, using the same song metadata as the catalog. Drag a song by its grip to change album order; an insertion line marks the destination. Up/down buttons remain available for keyboard use and single-step moves. The Files disclosure lists physical variants. **Locations** expands to direct gallery and music-album placements: two variants in one gallery count as one location, and derived Smart inclusion adds nothing. Gallery links in both disclosures open the public gallery using its canonical URL, current language and the site's configured rewrite style; a missing gallery has no link. Each music tag appears as its own compact chip. Header links sort Title, Files, Locations, or Tags in either direction across the complete result set before pagination, with stable ID tie-breaking. Tags sort alphabetically by the song's first music tag. The narrow action column appears only when a visible song has variants to split. The manager shares the site's typography and accent colors, with compact translucent surfaces and softly rounded borders. Each song title sits beside its file count, shown with a stacked icon and number. Variant and **Locations** disclosures follow the global Admin side-panel duration; 0 makes them immediate. Floating row disclosures close on an outside click, a repeat click on the same trigger, Escape, or when another row disclosure opens. Interaction inside an open disclosure leaves it open, and the compact row layout is preserved.

The catalog-heading gear contains the global filename-extension preference. The adjacent new-album gear hides parent and description fields until needed. Native disclosures keep collapsed song and album rows compact; the pending reconciliation count appears only when nonzero. Import keeps its text label for an empty catalog and becomes icon-only after entries exist. Icon-only actions retain accessible names and visible keyboard focus.

Click a song title to open its inline editor near the pointer. Clicking the same title or outside closes it; clicking another title switches editors. Text is not selected automatically. The editor stays within the viewport and wraps long titles. Enter saves the title as one line; textarea wrapping is visual only and never renames the original file. The global **Show file extensions in song titles** switch defaults OFF, including after upgrade, and affects filename-derived names only. Explicit titles keep their text. Select songs to preview an explicit same-song relationship and choose surviving metadata. The survivor chooser offers only selected songs, and the chosen survivor remains selected only while it is still in the selection. Split a variant to undo a wrong link. This never grants access to another source or changes its download permission.

Create albums and nested subalbums, then navigate the compact tree with native disclosure controls or use breadcrumbs. Selecting an album opens its ancestor path automatically. Its context gear opens native details for the album title, description, move and delete controls; song membership and ordering stay visible. Add, remove or reorder songs. The tree supports eight levels and 2,000 albums. A pending gallery count identifies snapshots awaiting reconciliation; an unreadable source prevents a complete import. Removing relationships do not delete centrally stored originals. Albums are Admin collections without a new public library. Music tags use the shared taxonomy with a separate media type; image/gallery tags and public image searches remain isolated. PDF tagging is reserved. Existing tag slugs remain globally unique. Back up SQL, `data/music/` and hidden legacy indexes together; run normal forward migrations and bounded reconciliation after recovery. Resolve stale-edit refusals by reopening current state. Disabling **Gallery music** preserves this metadata and all audio. Local Trash restore reconnects current tracks to existing song identities after gallery IDs change. Old snapshots or removed identities create fresh catalog entries; transferred archives never supply trusted logical IDs.

7.1.6 Music storage

Open **Edit gallery > Media management > Music** to upload audio and manage this gallery's playlist. New uploads store immutable originals in the protected central `data/music/` repository; audio never enters image conversion. Direct song assignments remain independent items. Album assignments are live playlists: later album membership and order changes appear in every assigned gallery. The gallery stores ordered references and per-item download permission, not a copy of the shared original. MP3, M4A, AAC, Ogg/Opus, WAV, FLAC and WebM are accepted; browser codecs determine playability. Playback and download use the destination gallery's current access rules and that item's download permission; source-gallery privacy does not authorize unrelated requests.

The table shows each track's duration and one Play/Stop icon before its title. The same icon changes with playback; there is no Pause action or separate playback column. Long titles use an ellipsis and full tooltip; only the playing title scrolls horizontally. Reduced motion retains static text. Closing the list keeps music running and the existing miniplayer visible. Compact Previous/Play/Stop/Next icons are available on the page and in the photo viewer whenever audio exists. One shared player prevents overlapping music between rows, editor and slideshow. The whole playlist repeats and navigation wraps. Automatic slideshow music is initially off; an administrator can enable it for this gallery. The switch saves its preference automatically and keeps the editor open while reporting save status. An enabled slideshow replaces other music and starts at the first track. Stopping or closing the viewer releases its music; independently

requested table playback can continue. Photo and speed changes leave music running. Browser refusal permits an explicit Play retry; a completely unplayable playlist stops after one pass.

Smart Galleries merge visible music from all accessible physical sources contributing matching photos. Pagination and temporary source filters do not narrow this union; no Smart audio folder is created. SHA-256 deduplicates identical original bytes even after renaming. Equal names or durations alone never remove different audio. Different encodings remain distinct unless an administrator explicitly links them in Music Manager. Hidden tracks are excluded, while visible download-locked tracks remain playable. Existing and new tracks default visible. Protected source URLs recheck access and visibility. In the Smart editor's presentation overrides, enable its independent slideshow music preference; it starts off and does not inherit source autoplay settings.

Each uploaded track initially forbids downloads. Small administrator icons beside each track allow or lock its download; the header icon changes all current tracks. Later uploads still start locked. Public download icons appear only for allowed tracks, and the endpoint rechecks permission and gallery access. Playback delivers audio bytes to the browser, so this setting cannot prevent a technically capable visitor from capturing those bytes.

Durations use a silent browser metadata reader, one visible row at a time, with a ten-second deadline. Unknown durations show a dash. Browsers without visibility observation use the same serialized reader for connected rows. No server decoding occurs. Upload retries preserve acknowledged photos, PDFs and audio; an unknown audio response requires checking the gallery before submitting again.

The existing gallery playlist index remains in its hidden `.gallery-music/` folder; immutable audio bytes live centrally in `data/music/`. The optional **Maintenance > Maintenance Center** task migrates legacy originals in bounded, resumable steps. It verifies each central copy before switching the reference and preserves track IDs, order, visibility, download flags and slideshow settings; uncertain local files are not deleted. Back up SQL, `data/music/` and hidden legacy indexes together. Each central asset counts once in storage totals regardless of gallery links; unmigrated local originals count separately. Deleting a gallery removes its references but retains shared originals. Gallery ZIP exports and transfers omit music. See `docs/GALLERY_MUSIC.md`.

Removing a catalogued local track first preserves and verifies its original centrally; it can then be added again without uploading. Failed preservation keeps the old playlist and file. After relocation, the selected maintenance task removes at most 20 catalog records per step without gallery use or repository assets. Dirty sources and registered assets remain protected, including assets whose files need restoration. Album/tag-only records do not retain an unbacked song. Cleanup deletes metadata only and logs its count. Analyze again before approving this revised task. Audio already deleted by an older removal needs an original or backup; maintenance cannot recreate it.

The same Maintenance Center includes a default-selected optional reconciliation for legacy public PDF/music card counts. It resumes in steps of at most 25 galleries and hashes at most one legacy OFP candidate up to 25 MiB per step. Missing or dirty results stay omitted. Analysis does not read originals; the task does not change them. Manual filesystem edits require reconciliation. If a legacy OFP lacks a digest, the task hashes it under the shared persistent source lock and atomically saves the digest in its manifest; the PDF and audio bytes remain unchanged. Music reconciliation may create the private

ilepath.gallery-music/ lock directory. A new gallery with no source metadata can start with ready-zero counts.

If the durable initial-pass receipt is missing, the first successful public HTML request for the home, a gallery, or a Smart Gallery queues the media-count pass after the response. It processes at most 25

galleries per background step and continues while work remains; a later public request can recover interrupted or busy work after its bounded retry delay. This is a one-time pass, not periodic scanning. The separate Maintenance Center task remains available for explicit reconciliation after later manual filesystem edits.

7.2 Edit identity and context

The editor lets you improve a gallery over time. Titles and descriptions can change as a collection develops. Date fields can represent a single day, a trip, a season, or a longer historical period. EXIF date suggestions can examine photographs and descendants, then offer an editable range for approval.

The complete editor records the exact gallery revision that supplied its fields. If another tab, administrator, upload, move, scan, or maintenance workflow changes the gallery first, an older save is refused instead of silently overwriting the newer state. The conflict view preserves entered non-secret text and shows a safe current comparison. Review and copy the intended changes into the latest editor; re-enter passwords and reselect files because private values are never copied into conflict details.

Use date suggestions as a helpful starting point. Scanned photographs, screenshots, edited exports, and cameras with an incorrect clock can produce dates that deserve human review.[8]

7.2.1 Review date suggestions across galleries

The dedicated **Gallery dates** maintenance page applies the same EXIF evidence across many galleries. It can review the whole installation or one selected gallery branch. Each row shows the current manual date range, the suggested range, the number of EXIF photographs and gallery nodes that contributed evidence, and editable **From**/**To** fields. Rows without an existing manual date are selected for application by default; rows that already have a manual date are left unchecked so a bulk review does not silently overwrite owner-entered chronology.

Only checked rows are saved. Unchecked suggestions are ignored, and the administrator can edit either date before applying the selection. If older imported photographs do not have scanned EXIF capture dates yet, run the normal gallery image scan/import first. The same focused suggestion action is available inside an individual gallery editor and can refresh that editor in place.

Admin date fields keep the native browser date input as the submitted value but add a compact calendar opener plus **Today** and **Delete** helpers. **Today** writes the browser's current local calendar date; **Delete** clears the field. The enhancement is re-applied to forms loaded into the Admin side panel, while the underlying native date field remains usable without the extra JavaScript controls.

7.3 Gallery editor control reference

Schedule publication The enhanced picker uses your browser's named timezone, so its current-time suggestions refer to your local clock. A UTC preview below it shows the exact stored instant. No timezone preference is saved. Without JavaScript, the field remains explicitly UTC and requires UTC input. Nonexistent spring clock-change times are rejected. For a new edit in a repeated autumn hour, check the UTC preview: the browser chooses the earlier occurrence. An unchanged saved later occurrence is preserved.

[8] Applying an EXIF-derived suggestion updates the proposed gallery fields through the existing editor workflow. The administrator remains responsible for the final dates shown to visitors.

In **Access**, enter a future date and time in the native **Schedule publication** field, following its displayed timezone, and save. The pending indicator shows the exact UTC deadline. Edit it to reschedule; leave it empty to cancel. Choosing **Published** also clears the schedule. For the unenhanced UTC field, convert your intended local time to UTC before entering it.

An unpublished gallery retains its normal direct URL before the deadline; a private gallery retains its private access rules. At or after the deadline, the next ordinary PHP page GET/HEAD, including an anonymous crawler, changes canonical visibility to public. Passwords, NSFW, share tokens, expiry, and parent restrictions stay unchanged. Scheduled publication also restores public listing, just like choosing **Published** manually. Incomplete PDF children remain protected and children are not automatically published. No traffic means no execution; static/CDN hits cannot trigger it. A large backlog catches up in batches of 16 across visits. No cron, daemon or browser timer is needed. Run normal database migrations before using this control. Schedules stay in the database; sidecars retain resulting visibility without transferring pending deadlines to another installation. The lightweight check reuses the existing gallery model and access service; publication writers load only when an indexed due row exists.

The gallery editor is the authoritative place for settings that belong to one physical gallery. The ordinary tabs are Identity, API, Access, Display, and Media, followed by Images and optional Organizer/Renamer tools where available. Advanced disclosures retain infrequent settings without removing them from the shared save form. The following table summarizes the current gallery-owned controls in Version 1.1.2.

Area	Control	Current behavior
Identity	Title and description	Public editorial text. Maintained translations can be stored separately without changing the source text.
Identity	Manual date and date range	Optional event or shooting date plus supported start/end range. EXIF-derived suggestions remain editable before save.
Identity	Slug	Controls the public gallery URL identity. Keep established slugs stable when possible because bookmarks and external links can depend on them.
Identity	Folder name	Read-only physical name derived from the sanitized gallery title. Saving a title change renames the folder and its descendant paths through the normal mutation service.
Identity	Parent and sort order	Changes hierarchy and sibling ordering. Moving a gallery physically relocates its folder tree where the configured gallery root permits it.
Identity	Tags	Reusable tags separated by commas. Suggestions are weighted by nearby galleries, photographs, and folder context rather than being a flat random list.

Area	Control	Current behavior
Identity	SimBrief identifier and defaults	A single visible field accepts numeric Pilot ID or pilot name. The administrator may remember that identifier and the source language for future galleries; pre-filled values are marked.
API	Upload endpoint and keys	Copy the gallery-scoped endpoint, generate a key with a label, and copy the new raw value immediately. Active keys appear only when present; the raw value is not shown again. AI regeneration and migration are under Advanced.
Access	Visibility	**Public** is listed normally. **Unpublished** is hidden from normal listings but can still open from its normal URL when access rules allow it. **Private** is administrator-only except through supported direct-token access.
Access	Password lock	Independent from visibility. A public, unpublished, or private gallery can have its own password policy where the access schema is available. Leaving the new-password field empty preserves the existing password.
Access	Share link	Can be generated, regenerated, revoked, and optionally given an expiry. Stored token material is hash-protected; an older or non-decryptable token may need regeneration before a copyable link can be shown again.
Access	NSFW / 18+	Marks the gallery branch for anonymous age confirmation. The protection also applies to subgalleries and served media according to the effective access policy.
Display	Picture Game	Visitor-interaction switch for the comparison game in this gallery branch. Enabling the game also requires image voting to be available.
Display	Image voting	Enables public image likes for this gallery. Disabling it leaves stored scores intact but removes new vote controls and blocks new submissions.
Display	Show file names	Off by default. Raw uploaded filenames remain hidden unless enabled; custom photo titles and descriptions are still shown.

Area	Control	Current behavior
Display	Photo card format	Default preserves 4:3 desktop and square mobile previews. Explicit 16:9, 21:9, 4:3, 1:1, 4:5, and 9:16 crop photo previews only inside this gallery on every screen. Originals and lightbox remain unchanged.
Display	EXIF/GPS display	**Use global default**, **Force on**, or **Force off**. The effective state controls public GPS pins, gallery EXIF maps, and GPS coordinate display.
Display	Breadcrumb style	**Default / inherit from Theme** follows the Theme choice; a gallery can select any of the nine registered styles. An invalid gallery override inherits the Theme choice, then the built-in `chevron` fallback.
Display	Gallery-card description format	Inherits Theme by default and can override the card layout for this gallery where supported.
Display	Contained-picture badge	Inherits Theme or overrides the stacked-picture icon and contained-image count shown on gallery cards.
Display	Lightbox browsing mode	Inherit Theme, classic single image, picture strip, or 3D carousel. Fullscreen and slideshow behavior remain available across the modes.
Display	Custom grid	First visible Display control: optional columns and rows for this gallery. Without an explicit grid, the nearest parent grid that permits subgallery inheritance applies, then Theme.
Display	Responsive thumbnail bounds	Optional minimum/maximum thumbnail-size guardrails. A save can copy gallery bounds recursively to descendants; recursive save is off by default.
Display	Flight route	Advanced manual route text supports local identifier lookup and explicit `NAME@latitude,longitude` points. SimBrief can save an OFP, description draft, and resolved route coordinates.
Media management	Title picture	Automatic or a selected photograph, including eligible photographs from descendants when the picker offers them.
Media management	Uploaded gallery thumbnail	Separate gallery-card media asset stored independently from ordinary gallery photographs.

Area	Control	Current behavior
Media management	Background source	Inherit Theme, upload a new background, choose an existing gallery image, or generate a collage from public galleries.
Media management	Branding assets	Optional gallery-specific banner/logo/separator assets override Theme fallbacks only for that gallery where configured.
Media management	Scan/import	Re-scans the physical gallery folder and synchronizes supported media into the catalog.
Media management	Create all thumbnails	Rebuilds thumbnails for this gallery. Select **Include subgalleries** to cover its descendants; other gallery branches remain outside the job. Browser generation is selected by default when available, with a server-generation option.
Tools	Organizer, Renamer, API	Date organizer, physical Media Renamer, upload API key management, AI regeneration, and gallery migration appear only when their schemas/features are available.

Important. Gallery-specific controls are intentionally not flattened into the global Settings hub. Inheritance is part of the data model, so a global search result may link to the gallery editor without turning that gallery-owned value into a site-wide setting.

7.4 Choose a cover and visual identity

A cover acts as the gallery's visual invitation. Choose an image that remains recognizable as a thumbnail and represents the collection fairly. Faces, strong silhouettes, clear subjects, and uncluttered compositions often work well at small sizes.

Gallery branding can provide a logo, banner, background, separator, or presentation override according to available controls. Begin with inherited theme styling, then add gallery-specific assets where a collection has a genuine identity, such as a wedding, exhibition, client, or long-running project.

7.5 Move and reorder galleries

Moving a gallery changes its place in the hierarchy and can affect inherited access or presentation. Review the destination before confirming the move. Reordering changes the sequence shown among siblings and supports a curated reading order.

Moving photographs between galleries uses a durable operation journal. The Organizer physically moves each original and verifies its recorded identity before changing catalog ownership. If a worker or connection stops partway through, System Health and Runtime Diagnostics show a bounded action state without exposing private paths. A failed move gives a safe reason on screen; trusted Admin logs retain more detailed private diagnostics. Stop competing storage work, preserve

a backup, and follow `docs/IMAGE_MOVE_RECOVERY.md`; recovery retains changed or unverifiable files instead of overwriting them.

Ordering may be chronological, geographical, narrative, alphabetical, or manually featured. Within one parent gallery, keep the choice consistent enough that visitors can anticipate the next item.

A signed-in administrator can also reorder the direct subgallery cards on the public gallery page. The compact public reorder toolbar is limited to the cards visible on the current pagination page. The save request includes that page's offset and visible count, and the server refuses the change if the submitted IDs no longer match the same current slice. It changes sibling `sort_order` values only; it does not change gallery parents or nest galleries.

When an administrator is signed in while viewing a public gallery, direct child galleries that have a filled **From** date can also be preview-sorted from oldest to newest or newest to oldest. Child galleries without a usable date keep their existing positions. The preview is temporary until **Save** is chosen; saving writes the result as the real sibling order seen by all visitors. At least two dated direct child galleries are required before the date order can be saved.

The public header provides **View as anonymous** for a signed-in administrator and **Exit preview** while visitor rules are active. It preserves the current public destination and locale. There is no separate preview or idle reorder instruction bar. Card handles retain drag ordering; progress, saved results and failures remain visible when relevant. Date sorting keeps its compact translucent bar, anchored question-mark help and explicit **Save this order**.

7.6 Delete a gallery thoughtfully

Gallery deletion can affect source files, descendants, metadata, thumbnails, links, and visitor bookmarks. Review the exact selected gallery and its children, create a backup, and use the normal Admin deletion workflow. A temporary visibility change can provide a safer preparation period when a gallery may still be needed.

The gallery trash bin is enabled by default. Deleting a gallery through a normal administrator gallery-delete action moves the selected folder and its complete descendant tree out of the live gallery root, records the metadata required for recovery, and lists the entry under **Maintenance > Trash**. Public pages stop discovering that subtree immediately, but an administrator can restore it to its original path while that path remains free. Restore never overwrites a new live gallery occupying the same location.

From the Trash panel, choose **Restore**, **Delete permanently**, or **Empty Trash**. Permanent deletion and Empty Trash cannot be undone. Empty Trash works in bounded batches so large trash collections do not require one unbounded request. Individual-photo deletion is not part of this gallery-level recovery feature and remains permanent. A backup remains necessary protection against storage failure, administrative permanent purge, or damage outside PHP Gallery.

8 Smart Galleries

A Smart Gallery is a saved query, not a folder. Open **Admin > Galleries > Smart Galleries**, add conditions, and combine them with nested **AND**, **OR**, and **NOT** groups. Fields cover source galleries and descendants, tags, capture dates, EXIF/GPS, text, AI metadata, duplicate state, file characteristics, and private editorial ratings.

Use the existing **Tag** condition when a rule should include photo tags and tags assigned to the physical gallery or an applicable ancestor. Add a **Photo tag** condition to match only tags assigned directly to each photo. Its operators for one tag, its absence, any or all selected tags, and untagged photos ignore gallery, music, and PDF assignments. The all-tags condition requires every selected tag on the same photo. Existing saved **Tag** rules keep their previous results and the rule format does not need a version change. Where the typed tag schema is available, photo-only conditions accept image-category tags; confirmed legacy schemas without that column still use direct image assignments, while an unknown schema blocks the condition safely.

When inline administration is enabled for a signed-in administrator, a single-row quick-add field appears on eligible photo cards and in the normal lightbox/fullscreen viewer. It adds one direct photo tag. The last successfully added tag is prefilled in the same field for the next photo, but opening a photo never applies it automatically. Eligible photo cards also show a passive preview of the first three direct tags. If more tags exist, a fourth non-clickable ... chip indicates that others are omitted; a small horizontal gap separates the chips. Suggestions appear only while typing; use **Enter** or the small add control to apply the selected or new tag. The adjacent close control or clicking the same tag button again dismisses the row without applying the current input. The row does not show a tag list or removal controls; use the existing photo-edit pencil to inspect or remove tags. Press **T** while viewing a photo to focus the add field. The row closes after the server confirms the add and the view synchronizes; a rejected, unconfirmed, or saved-but-unrefreshed result keeps it open with the entered value. In the ordinary lightbox, **Escape** closes the row. In native fullscreen, browsers that support page handling of **Escape** let the first press close the row while the viewer remains fullscreen; the next press exits fullscreen. If that support is unavailable or rejected, native fullscreen may exit first and the row closes with it. Quick adding is not available in slideshow/presentation mode or document/PDF viewers. In a Smart Gallery, a tag change applies to the authorized physical source photo and the saved rule results update dynamically.

Use **Preview** to see the current match count and real matching photo cards, then save and optionally publish. Images and files remain in their physical galleries, so metadata or rating changes alter membership immediately. The editorial rating is private and independent from visitor voting. Administrators may also use **Save search as Smart Gallery** from public search.

Choose **Unlisted** for URL-only access, **Root gallery** to show the Smart Gallery on the homepage, or **Subgallery placement** to let physical galleries attach it. In each physical gallery editor, select whether an attachment belongs **Above gallery content** or **Below gallery content** and give it an order value. Above attachments render first, then ordinary subgalleries and photos, then Below attachments. Equal order values are resolved consistently by Smart Gallery ID. Existing attachments default to Below with order 0.

The physical editor's **Identity** tab shows **Smart Gallery attachments** before **Advanced gallery settings**. Only attached rows remain visible. Use **+ Smart Galleries**, type a name, and activate a matching result; at most eight matches appear, with a displayed/total count for broader searches. Each row has its own placement, order and **Detach** action. These edits remain a draft until **Save gallery**. **Create Smart Gallery** opens the definition editor in a new tab. Keyboard arrows and Enter reach the results; Escape closes the anchored panel. Native selection fields remain available without JavaScript. Slideshow waits for the next decoded image

and uses additive fades in supporting browsers to avoid a dark pulse between equally bright images; older browsers retain the ordinary fade.

The same virtual gallery may be attached beneath several physical galleries, with independent placement and order in each parent. The Smart Gallery editor's **Used in physical galleries** section shows every parent and lets you save its placement/order or **Detach** it without changing other parents. When the editor is opened in the Admin side panel, these actions stay in that panel; ordinary form submission remains available when JavaScript is disabled. A public Smart Gallery placed beneath a private or otherwise restricted parent still inherits that parent's public access boundary for the card.

Public results and counts always apply the same source-access policy as ordinary public gallery content. A matching photograph contributes only when its physical gallery is publicly listed and accessible to the current visitor and the photograph itself is public. Existing password, share-link, inherited visibility, unpublished-gallery, and age-restriction rules therefore continue to apply. Query-string routes work without rewriting and clean `/smart/<slug>` URLs are optional.

8.1 Relationship safety and repair

A Smart Gallery must not resolve back into a physical gallery that attaches it. The server checks the complete relationship graph whenever rules, attachments, or gallery hierarchy are changed; browser filtering is not treated as security. This includes indirect paths through physical descendants and other attached Smart Galleries. Filesystem-derived hierarchy synchronization and public-path parent repair validate their complete proposed parent map before any new parent link is written. The current rule editor can reference physical galleries but does not offer a direct Smart-Gallery-reference condition.

A complete Admin drag-and-drop tree is validated once in its final form. The subsequent folder moves use that prevalidated map and defer hierarchy synchronization until the final tree exists, so a temporary intermediate state cannot reject an otherwise valid batch. Every committed physical hierarchy mutation clears the request-local Smart Gallery graph cache before later graph-dependent work continues.

If older or manually modified database data already contains a cycle, public rendering stops that invalid relationship instead of recursing. The Admin attachment list marks the relationship for repair, and **Detach** remains available. Validation and runtime traversal both use conservative depth/node limits and request-local deduplication, so malformed data cannot repeatedly expand the same Smart Gallery. The normal result query remains paginated and does not copy all matching photographs into PHP merely to perform cycle detection.

Migration `202608170002_smart_gallery_attachment_ordering.php` adds the per-parent placement and order fields. Until that migration is available, existing attachments keep their historical Below behavior, while Admin placement mutations are refused rather than partially saved.

8.2 Presentation settings

By default a Smart Gallery follows the current Theme and site presentation. Enable **Override Theme defaults for this Smart Gallery** when this virtual collection needs its own columns, rows, pagination, thumbnail size limits, thumbnail renderer, gallery-card layout, metadata overlays, lightbox behavior, slideshow, download, voting, or breadcrumb style. The breadcrumb selector appears with its presentation controls; **Default / inherit from Theme** follows the Theme choice. Sorting and direction remain part of the Smart Gallery definition.

A Smart Gallery does not inherit presentation from the physical gallery in which its card appears. One Smart Gallery can appear beneath several physical galleries, so there is no single parent presentation with clear precedence. When no override exists, or when stored presentation data is missing or invalid, the current Theme/site defaults remain the fallback. Physical gallery and photo thumbnail limits still win if a Smart Gallery limit conflicts with them.

8.3 Lightbox across result pages

The large viewer is not limited to the photographs rendered on the current HTML page. Smart Gallery cards remember each photograph's position in the complete authorized result order. As the visitor moves forward or backward, the browser requests nearby metadata in small authorized groups from the server. The server repeats the same Smart Gallery rules, source access checks, sorting, and stable tie-break used by the page.

This keeps large virtual collections practical: opening a Smart Gallery does not preload all originals or all result metadata. Existing keyboard, touch, fullscreen, zoom, close/reopen, and slideshow behavior is shared with ordinary galleries. If slideshow is disabled for that Smart Gallery, its slideshow controls are omitted while normal galleries keep their usual defaults.

8.4 Downloads and safety limits

When site downloads and the Smart Gallery download option are both enabled, the server can prepare a ZIP from the current authorized matching originals. It re-evaluates the saved rules and access policy instead of exposing filesystem paths to the browser. The archive path refuses collections above 5,000 matching images and also enforces an aggregate original-file byte ceiling. Existing installations use a 2 GiB ceiling unless `smart_gallery_zip_max_source_bytes` is set in `config.php`. Concurrent requests for the same archive signature are serialized with a file lock; after the lock is obtained the cache is checked again, a unique partial ZIP is written, and the completed archive is published with an atomic rename. Failure logs keep only bounded reason/class diagnostics rather than raw exception messages or private server paths.

Rules are readable versioned JSON, never user SQL: fields/operators are allowlisted, values are prepared parameters, depth is limited to five, and no more than 50 conditions are accepted. Deleted references match nothing and malformed or unknown rule versions are rejected. The normal migration workflow adds Smart Gallery presentation storage; existing Smart Galleries inherit Theme/site defaults until an override is saved.

9 Cooperative galleries

Cooperative galleries connect independently operated HTTPS installations while each owner retains their albums and original files. Apply pending migrations through the normal administration workflow, then enable **Cooperative galleries** in **Features** on each server. The feature is off by default; turning it off preserves identities, friendships, proposals and preferences.

Open **Settings > Advanced > Friendly galleries** and pair every participating server directly. Friendship alone shares no photographs. In **Album collaborations**, select a public, listed, password-free, non-NSFW album and prepare its reference code. One participant combines the references into an immutable proposal; every participant reviews the exact members and permissions and explicitly approves locally. **Deliver and verify** advances a bounded sequence in the existing panel. Email invitations use the configured account-mail transport; opening an invitation never grants consent.

After activation, open the shared photographs from the collaboration panel or the local album's shared-trip link. Each source retains attribution and pagination. Existing JPEG/WebP thumbnails and previews are authorized independently; originals, protected albums and remote writes are unsupported. Ordinary links work without JavaScript. Missing derivatives need the normal thumbnail workflow. New photographs receive complete path-based identities; old names remain intact without bulk rebuilding, but ambiguous ownership is refused.

Withdrawal, changed source protection and local friendship revocation block subsequent authorized reads. An unreachable peer remains bounded by the maximum 120-second verification lease; downloaded pixels cannot be recalled. Adding a participant creates a separate proposal requiring fresh approval from everyone. Ordinary use can advance technical renewal on demand; unattended renewal is optional. Missing or unknown required schema refuses sharing and mutations. See `docs/COOPERATIVE_GALLERIES.md` for operational limits and deployment qualification.

10 Windows companion uploader

10.1 Durable capture queue and recovery

Connect once, start capture and press **Finish** once at the end. A local producer records screenshots throughout upload, renewal, outages and rate-limit waits. Windows notifications have a one-second reconciliation backup. Each source generation has a durable ID and immutable FIFO sequence before its hash is known. Only the oldest unresolved item may query inventory, connect or upload. Partial, missing, waiting, unauthorized and unknown heads block newer files. Import uses the same consumer; ZIP extraction stays local and each HTTP request carries one original plus optional thumbnails.

New groups use recognized Steam capture time, otherwise birth time or first observation, with deterministic ties. Mtime never reorders accepted work. Late old files cannot change past transfers. A new session still excludes historical images. Restored confirmed bytes are deduplicated per destination; changed historical bytes without an original fingerprint and old unordered multi-path queues require review rather than an invented chronology.

The compact **Refresh queue** icon reconciles the same session and safely wakes processing. Repeated requests coalesce. An active request retains its owner until it actually ends. FIFO, target/scope, retry deadlines, uncertainty and Finish cutoff remain binding. Finish persists its first cutoff, waits for eligible unfinished sources and requires a new final scan after the last acknowledgement. Photo queues can wait beyond five minutes. Verified same-target screenshot Finish intent resumes after restart; legacy key revocation retains its separate renewed-confirmation rule. Linked flight reports remain local.

An unknown head shows **Needs attention**. **Resume** in Watcher, **Force Continue** in progress and **Review pending uploads** after restart open the same review for the exact retained source and target. Local preview and authorized remote observation are shown separately; a matching image, gallery number or hash is not an upload receipt. **Confirm already uploaded** requires the exact durable acknowledgement. **Skip photo** records an explicit skip, keeps the original and unknown history, and allows the next FIFO item. Skips are counted separately from successful uploads. Cancel leaves the item pending. Each later unknown needs its own decision. After submitting a decision, **Hide (saving decision)** only hides progress; the already authorized durable publication finishes normally. In a short window, evidence scrolls while the decision and close controls remain visible below it. After a worker restart, **Resume pending uploads** also releases safe pending work in the same session, preserving its retry deadline. Explicit recovery after a failed Stop save also restores reconciliation and the required final scan under the original cutoff.

An earlier unknown can also block a new session's first photo before it is sent. Relinking and deleting local files do not clear that history. Review must distinguish the older unknown from the current unsent source; reviewing the older operation cannot acknowledge the new one. **Inspect folder (read-only)** shows separate filesystem findings without changing the queue, startup exclusions, cutoff or live status. Progress always offers **Hide**. **Stop and keep pending** stops admission and retains accepted work and its cutoff for explicit recovery. It does not revoke access or delete files. A request still running keeps its owner; the UI reports that wait instead of claiming successful completion. Ownership remains held until the retained cutoff has been saved and read back; a failed save keeps processing paused under the same owner.

Inventory/upload no longer share 60 requests per IP per 300 s. Upload grant budgets count requests; account/IP/global storage budgets charge every original atomically, including existing multi-original multipart requests. New 60-second windows and locks allow 120/60 per verified grant/account/gallery, 240/120 per account and IP, and 600/120 globally, respectively. Combined

pre-authentication admission allows 240 per IP and 600 globally per minute. Two 45-image clients fit including controls and safe head checks; this does not measure hosting capacity. Retry-After reaches the same head; missing hints use automatic 5–300 s backoff without exhausting completion intent. Speculative Photos preload/prefetch yields during upload blocks. Local preparation failures can retry; unknown dispatched uploads require review without resend or the next image. Hash inventory and request-local IDs are not operation receipts. Idle updates omit capture continuation; active updates retain strict nonce/version checks. Diagnostics retain head, counts, producer/consumer progress, next retry and historical failure. Feature BASIC checks source only; installed Windows, layout and deployed server behavior require separate acceptance. The permanent contract is `docs/UPLOADER_QUEUE.md`.

10.2 Uploader layout

While Watcher capture is active, a red dot pulses on the tray icon in step with the in-app indicator. All Watcher markers share a 267 ms phase interval, or about 534 ms for a complete bright/dim cycle; the update badge remains independent.

The normal launch opens on **Photos**; the navigation order is **Photos**, **Watcher**, **Import**, **Activity**, **AI**, and **Settings**. With an active gallery link, Photos loads the gallery tree on the left and opens an initial gallery on the right. Expand or collapse a branch or the whole tree. Selecting a gallery shows its photos in the same window. The default-on **Subgalleries** switch shows direct child-folder cards above the photos. Each cover uses the first authorized direct thumbnail. When none is available, the card shows a neutral folder. Photos keeps gallery listings in memory and uses the current link grant. When disconnected, **Connect** opens the existing address and browser approval flow. The active link can then browse public photos directly. Choose the independent upload destination with **Use for uploads**. A logon `-startup` launch remains in the system tray until restored. On macOS, source launches use the normal Tk window. Temporary links end with the process.

Large trees initially load at most 500 galleries, then continue automatically from the saved position, one page every five seconds. Loaded galleries remain usable while the loading status shows remaining work. Errors pause continuation and offer an explicit retry; normal browsing needs no **Load more** click. Loading status and a small smooth spinning ring sit above the tree. They remain visible during initial reads and paced continuation, then stop on completion, error or disconnection without estimating a percentage. Newer servers can advertise an optional complete JSON tree with a revision. They recheck current account permissions before returning the tree or an unchanged reply. The existing paged API remains available for older servers, unavailable data and trees beyond the complete-response limit. The client first shows the last-known saved structure and restores the account's last explicitly selected gallery. It requests the optional tree only when advertised; invalid or unsupported replies retain the saved tree and use the paged API, including with web 1.0.3. Every successful page and confirmed creation saves known metadata, preserving earlier rows even if loading is interrupted. Accepted local writes finish on normal exit. Only a complete successful refresh removes missing rows. The circular arrow beside expand, collapse and add refreshes the tree, including renamed galleries and changed parents, while preserving expansion, selection and the photo view. If the selected gallery moves, its new ancestor path opens to keep it visible. Repeated refreshes ignore obsolete responses. Short UI batches update changed rows. Incremental loading preserves numeric sibling order, including later rows. A row whose parent arrives later moves under that parent. Tooltips appear only while the pointer remains over their trigger; keyboard focus alone does not open them. Windows protects saved titles with user-scoped DPAPI; unsafe or corrupt local storage is a cache miss. Media still needs live authorization; new upload/create actions wait for fresh gallery confirmation. This metadata does not grant offline photo access. Maintainer diagnostics can select `native_only` in the hosted

WinApp workflow. The central `winapp-native` suite records its narrower native viewer scope and cannot qualify complete WinApp coverage. Default complete discovery and its existing time limit remain unchanged. Contextual panels keep their existing keyboard and outside click dismissal. Windows clips their rounded boundary without dark outer corner blocks; if native shaping is unavailable, a flat opaque themed panel remains usable.

The Watcher screen shows the gallery target. A verified API-mode title survives refreshes of the same saved connection; credential or upload-mode changes invalidate it. Selecting a Photos album does not change the upload title. Its top API-key control opens a compact key dialog with a masked key and **Save & Test**; this control remains available in the header on every screen. The separate ordinary header **Link gallery** flow remains the upload/search path. Unsaved edits do not replace the saved key. The app header also has Settings, updates, and help controls. The Watcher screen keeps the selected folder path, watcher status, and source-delete warning visible. If the configuration cannot be read or verified, startup stops before work begins. Preserve the original file and restore a valid backup; a missing file still starts with first-run defaults. Windows reparse points and power-loss durability have not yet been qualified. Click **Link gallery** in the main window. Paste the gallery address from your browser into the dialog and confirm with **Link gallery**. You can also enter a host and installation path without a scheme. On confirmation, WinApp adds `https://` and shows the normalized address before checking the gallery. An HTTP address on `localhost` can link to a gallery on the same computer. Remote installations use verified HTTPS. The dialog shows progress while waiting for approval. If the connection fails, it displays the error and lets you try again. Sign in as an administrator in the browser and click **Approve**. Return to WinApp when the browser shows **Approval received**. WinApp shows the linked site's address and loads the first gallery page automatically. **Cancel** declines the request.

If linking fails, copy the WinApp diagnostics. The separate browser-linking entry retains the last failure, request phase, HTTP status and response type, including recognized hosting verification pages. It distinguishes a blocked request from a gallery reporting linking unavailable. Credentials and raw response bodies are excluded. Use the public website URL, including its public subdirectory when needed. Internal hosting directories do not belong in the link address. Servers with internal path rewriting need PHP Gallery 1.0.1 for public-mount verification.

Use the ellipsis beside **Unlink gallery** to open the links chooser. Select an entry under **Linked now** to switch using its current approval. The profile keeps up to 20 **Recent addresses** across restarts. Select one to start a new approval. The power icon unlinks the selected site and the cross removes a recent address. The upload destination has its own selection.

Open **Choose gallery...** beside the address and type in **Search galleries**. Suggestions favor names, then descriptions and gallery tags. Names tolerate a small typo. Quoted phrases and `AND`/`OR` work as on the website. Indentation and the selected row's parent titles show the hierarchy. The arrow loads more matches. With basic list access you can filter the current page by name. Approve a new link on a supporting server for catalog search.

With approved upload access, choose a result and click **Use for uploads**. On a supported local profile, including a source run on macOS, WinApp uses the approved upload access automatically. After verification, **Upload gallery** shows the destination in Watcher and Import. The ellipsis reveals more gallery actions. **Unlink gallery** ends the current link. After restart, reconnect to the same account and select the gallery again. Use **Manage link...** in Settings for advanced access. API-key uploads use their own **Gallery URL** and gallery-scoped key.

Separately select **Read gallery details** and approve the read permission to use **Read selected details** for a selected gallery. It shows its ID, title, revision and stored visibility, access and listing settings, not inherited effective access. A read already checked by the server can finish after concurrent revocation. Older servers may not support this permission or endpoint; the client reports unsupported or unavailable without silently reducing scope. The saved `pgu_` upload path

remains available. Real Windows, deployed HTTPS and database behavior remain unqualified. Delegated upload success requires a new image row and the observed original file at its final gallery path. If confirmation fails, the file may already exist and the client keeps the attempt unresolved without automatic retry. Revocation during an in-flight upload cannot roll back an already stored file. The new linked client requires the exact `fresh_insert_v1` upload evidence marker; an older linked server leaves the attempt unresolved. The `pgu_` upload path remains unchanged. A live reply may replace the write-ahead uncertainty only for the same positive attempt, unchanged source identity and hash, and a validated fresh image insertion. It must report one uploaded image and a nonnegative scanned count; an unknown or lost reply, hash check or inventory never clears the marker. No retry, source deletion or exactly-once guarantee follows from this correction. Server file identity is not byte-exact proof against outside writers. Client source identity and hash checks are not atomic against local writers; Windows reparse and power-loss behavior remain unqualified. Original ZIPs and extracted staging directories remain retained regardless of age or upload confirmation for manual review; automatic age-based staging cleanup is disabled because visible job IDs do not prove complete history. For exactly one uncertain linked upload, **Check uncertain upload** asks for the file size in bytes, up to 64 MiB, and checks its stored hash once without opening the local file. A match reports observed bytes, not completion or database registration; an unknown result does not prove absence. Neither result clears the record, deletes files or retries the upload. Older servers may not support this check. To review one current unknown upload, first inspect the gallery and original source. The optional hash check observes bytes only; it does not confirm registration or prove absence. The explicit review prompt defaults to No. Review preserves the uncertainty marker, original and ZIP. The same source path or bytes stay blocked after restart, while other eligible sources can continue. Review does not resend or delete the unknown source. Linked uploads require an unredirected profile on a fixed local Windows drive or an owner-controlled local macOS volume. Close older uploaders using the same profile first. The separate `gallery:visibility` permission is offered only when the server advertises it, and requires explicit browser approval. Before a change, the client confirms the current gallery state. A stale revision or schedule conflict causes a read-only refresh; the change is not replayed. Older servers omit this permission. The operator command `inspect_gallery_visibility.php` is read-only by default. Only an explicit current `-reconcile=FINGERPRINT` can repair eligible version-two sidecar evidence and retire its marker after synchronized readback; it does not write the database row or revision. Version-one markers are inspection-only. Candidate `8de8e50a` passed BASIC run `42107055888` and WinApp run `42107055797`. Installed GUI, MariaDB, and deployed behavior remain unqualified. An active gallery listing link permits browsing listed public, non-sensitive photos. Supported servers offer explicit `presentation:read` consent in the normal link flow for presentation settings and visible music. For public-only links, every ancestor must be public, listed, normal-access, and non-NSFW. Separate administrator media consent lets Photos display hidden, restricted and NSFW media. The server rechecks the live administrator grant. Internal pages carry up to 64 photo descriptors, or eight on older servers. Binary responses have a 16 MiB cap and playlists carry up to 32 visible MP3/WAV tracks. If the smallest thumbnail is missing, the server may use another existing rendition or the bounded public display file. The photo pane and lightbox share the main WinApp window. An incomplete HTTP 304 cache reply triggers at most one fresh authorized read; the rejected reply never grants access to stale image bytes. The responsive grid scrolls continuously and requests only visible thumbnails; it does not request detail photos while browsing. Explicit **Preload gallery** downloads both thumbnails and lightbox renditions. Click a photo to select it and show its metadata in the adjustable right pane. Double-click or press Enter to open the lightbox. Arrows navigate the grid; Space toggles a photo. Control-click (Command-click on macOS) toggles individual selections; Shift-click or Shift-arrow selects an anchored range within the 64-photo limit. Control-arrow (Command-arrow on macOS) moves focus without clearing selection. Only selected photos have a small rounded outline fitted to the actual preview and its shared rim, without an empty caption area below the image. Selected formats keep

a compact 6-pixel gap between card rims in both directions; remaining viewport width stays after the last column. Default retains its stable natural-preview row reservation. Filename hints use a compact contrasting tooltip. The pane and context information share authorized metadata and formatting, with a short bounded session cache. Left/Right loops through the whole album. Close the lightbox or press Escape to restore the tree and grid, including after an image error. By default, thumbnails trim detected black bands and fit the whole remaining scene. Seven visible shape buttons in the album toolbar apply Default, 16:9, 21:9, 4:3, 1:1, 4:5 or 9:16 immediately, without a Save step. Each rectangle illustrates its ratio; a quiet accent marks the active choice. Explicit ratios center-crop grid previews; originals and the lightbox retain their existing presentation. Choices stay local to WinApp, separately for each gallery and verified site/account. Website and upload settings stay independent. A small **!** indicates a local save failure; the chosen view remains applied and the tooltip explains how to retry. Switching uses retained previews. Photos and thumbnails have a separate five-minute limit of 600 requests per IP and 3,600 globally. On HTTP 429, Photos displays the limit and pauses speculative requests until **Retry**. Source launches use Pillow from the same Python interpreter. The optional server capability `media:thumbnail-batch-v1` is controlled in **Admin > Features**. It defaults to enabled for fresh installations and upgrades without a saved choice; an administrator can turn it off, and saved choices survive updates. While no choice is saved, the legacy `PHP_GALLERY_THUMBNAIL_BATCH_V1` value is used: unset or 1 means on; 0 or another value means off. A saved choice always takes precedence. It uses the existing bearer media authority without a new grant and accepts a fixed read-only request of up to 32 photos, 8 MiB total and 4 MiB per image. Responses are `private, no-store`. SHA-256 verifies bytes but grants no cache authority; Admin preload remains separate. The server buffers at most one 4 MiB image at a time. It charges N requested photos against the existing 600/IP and 3,600/global limits and the 600/account limit per five minutes. When negotiated, WinApp starts with groups of eight and uses up to sixteen after a healthy group; slow, large or exceptional groups return to eight. Unverified originals retain individual confirmation. Target FPM, proxy and HTTPS acceptance remains unverified; the default does not establish host acceptance. Existing single reads remain the fallback for mixed versions and exceptional images. Retry reloads failed content. Use the wheel, a two-finger scroll or + and − to zoom. Press 0 to reset the view and drag to pan. F or F11 toggles fullscreen. Pointer motion briefly reveals the controls. When permitted coordinates exist, an anchored location pin offers **Open in OpenStreetMap** in the system browser. Browsing prepares the previous and next details in bounded memory only while the lightbox is open. Closing it clears queued prefetch; an in-flight bounded read may finish. **Fit by content** crops detected black bands in both views. **Show original frame** restores the full frame. The image keeps its aspect ratio in the available area. During a resize, the lightbox updates a preview and refines it when the window settles. On macOS 15 or later, compatible HDR source data can use AppKit on an EDR display when PyObjC and the Tk window bridge are available. Other configurations use the 8-bit Pillow/Tk SDR renderer, including Windows. Windows fullscreen shortcuts also work with NumLock, CapsLock and ScrollLock or without a numeric keypad. Control/Alt combinations and text entry retain their behavior. Large JPEG/MPO details use a reduced decoder raster before RGB allocation, within the same 36-million-pixel budget and display bound. This also applies below the pixel ceiling when the source exceeds the display target; supported JPEG scaling reduces the raster before conversion. If a detail still exceeds the budget, one authorized thumbnail can provide a limited preview, including on older webs. Authorization, cancellation, throttling and corrupt data do not use this fallback. Copied diagnostics retain safe decode reasons, dimensions and limits, plus a separate bounded F/F11 trace. The initial authorized photo page appears before optional presentation settings are loaded through the same worker. A green preload indicator means verified encoded files; detail decoding and required private server checks still occur. These paths are shared by Windows and macOS; installed latency needs a live test. Recent album metadata appears immediately from the same live session while a fresh server read follows, without client visibility filtering. Unchanged refreshes keep tiles and update their verification ages. Large JPEG thumbnail variants

also reduce the decoder raster before RGB allocation. The shared-header **Thumbnails** indicator and **Photo storage** control use the compact Watcher-pill style: icons, short counts and simple bars, without long labels. The storage popup opens on demand below its header control and aligns with the right edge of the visible main-window client area. Window resizing and changed values reposition the whole panel inside that area. It shows all stored typed thumbnails and their bytes; its detail separates ordinary quota and **Keep preloaded**, listing retained thumbnails, other media and galleries separately. RAM means this process's Windows working set or macOS resident set; unknown VRAM remains unknown. Selecting a quota saves it immediately, without a Save button or confirmation. A save failure keeps the previous value and shows the error inline. Accounting scopes do not change. The photo grid combines repeated scroll/layout signals into one scheduled viewport update. Visible thumbnails load first, with a bounded neighboring-row buffer for quick reversals. Decoding stays in a worker and Tk image updates stay on the GUI thread. Copied diagnostics include event/update counts, image creations/releases, cache hits and the longest viewport callback. These figures are local observations; installed scrolling still needs manual qualification. One thumbnail worker publishes ready images in bounded serial batches without a separate GUI polling wait between photos. Ready results and decoded-cache hits attach in small GUI slices; foreground work and viewport changes interrupt remaining reads. A brief local pause after a transient transport failure resumes pending thumbnails only after the delay has elapsed and the prior request has finished; an HTTP 429 still waits for the existing explicit **Retry** action. The session request gate still owns authenticated transport. Diagnostics separate disk access, gate wait, credential renewal, media transport, decoding, queue wait and Tk application. Compare cold and warm browsing separately; these measurements do not establish FPS. Current authorized disk hits use one independent disk/decode worker with a four-result queue; Admin reuse needs a fresh, account-bound catalogue proof. The decoded LRU accounts four bytes per pixel within 16 MiB and 256 entries. Visible entries have eviction priority; full-cache overscan cannot displace recent history. Tk images remain limited to the admitted row band. Sequential media GETs may reuse one HTTP/1.1 connection and one verified SSLContext, with both serialization gates retained. Only fully consumed, validated responses can be reused. Server closure, incomplete bodies, cancellation and renewal discard the socket. One stale borrowed GET may reconnect before a response, within its deadline. The initial idle bound is 15 seconds; real hosting support and latency remain to be measured. Diagnostics count requests, connections and TLS handshakes separately, including discarded work, and split context, DNS, TCP, TLS, headers and body. The cache menu offers **Preload thumbnails only** for the selected direct gallery. It resumes current authorized cache data, yields to browsing, respects Retry-After and never requests full photos or subgalleries. Its completion does not mean lightbox renditions are preloaded; the original **Preload gallery** keeps that two-rendition meaning. First viewing uses the negotiated verified Admin binary protocol directly; saving full private files still requires explicit local preload intent. A newer authenticated page with the same photo and strong validator can confirm an older mounted descriptor, using the new page's real proof time. Full preload revalidates stale stored full responses instead of skipping them because their files exist. A valid 304 refreshes age without a body download; final coverage still means verified files, not simultaneous fresh deadlines. The current photo or page leads background refresh. Superseded image reads cancel, and adjacent lightbox preparation leads remaining grid thumbnails. Leaving a view withdraws its cancelled waiting tickets from the serial gate. Both preload modes yield to the current view before catalogue pages and images. Viewed thumbnails save automatically in the account-bound ordinary cache under the shared 4 GiB default limit; unrelated gallery pins cannot bypass that limit. Cache events update its measured count without waiting for slower retained/RAM accounting. Full preload counts each confirmed stored photo during transfer, then reconciles final coverage. Selecting an image already being prefetched adopts its prepared result instead of restarting the same request. Diagnostics separate detail/prefetch disk, network, 304, bytes and RAM/adoption. An isolated Windows pixel comparison checks the settled view after scroll; it does not prove removal of transient strips during motion or higher FPS. One per-user disk cache stores opted-in photos and tracks with

verified content digests. Ordinary thumbnails have a separate configurable 4 GiB/262,144-entry area. Full/detail photos and audio retain the 1 GiB/4,096-entry area. The compact control beside Watcher opens a shared per-user limit of 1/2/4/8/16/32/64 GiB. The main count and bytes include ordinary and typed retained thumbnails. Used/limit in the detail covers ordinary files only; pinned galleries, detail/audio and decoded/Tk memory remain separate. Loading/unavailable does not mean zero or offline permission. SQLite accounting updates totals by delta under the existing lock; dirty mutations and changed directory generations require bounded cancelable metadata recovery off Tk. The index never grants access or replaces SHA-256. Preload shows activity before listing and during checks, foreground waits and Retry-After. Processed, current-hit and confirmed-write counts differ; final checks reuse only exact unchanged verified file identities. Failed writes and no-store replies cannot claim complete thumbnails. Serial FIFO admission preserves viewport/Watcher turns; speculative warm queues cannot postpone explicit preload indefinitely. Hosting speed and actual rendition size remain live-test questions. The right inspector saves its own preferred width in the existing layout file, including hide/restart/resize. Optional RAM means current-process Windows working set or macOS RSS, without children; per-process VRAM remains unavailable. Decoded memory remains bounded at 16 MiB. Each evicts only its own least recently accessed entries; large photos cannot displace ordinary thumbnails. Gallery retention and private-preload settings remain shared. Unpinning checks capacity for all destination copies before retained cleanup. Old thumbnails migrate by their exact rendition identity without resetting freshness. A matching validator from the current album catalogue reuses a local thumbnail immediately. Photos without this proof use the usual 60-second freshness and authorized ETag revalidation. The current authorized playlist can reuse a track by SHA-256, size and format. Media available only to administrators uses transient storage by default. Cache icons update automatically from loaded metadata: cloud means online, half-circle partial, filled circle local and dotted circle unchecked. A partial catalogue cannot prove full coverage. The heading's cache icon opens the gallery menu. **Preload gallery** prepares direct photos, including thumbnails and lightbox images, in the background with progress and Cancel. Subgalleries are separate targets. **Keep preloaded** retains successfully prepared files outside ordinary cleanup, entry limits and byte limits, including after restart. Disabling it returns the gallery to ordinary cache policy, which may evict excess files. Cancel keeps prepared files and the retention preference without claiming full coverage. Existing verified bytes are reused when retrying after a server limit. No-store media remains transient; retained bytes grant no offline access or expired-link permission. One explicit preload accepts up to 100,000 direct photos as a bounded work operation. An explicit Admin Photos preload can also retain private or draft photos on supporting servers. The photo, gallery and complete ancestry must be verified as non-NSFW. The saved preference and private files belong to the verified account and installation. A current authorized catalogue proof can reuse matching thumbnails for at most 60 seconds without another binary request. Ordinary full-image HTTP reads and explicit preload still require their own verified body within the 60-second freshness window and normal 200/304 revalidation. The full-image viewer can separately use an explicitly downloaded snapshot when saved full-preload intent, the exact issuer, verified account, gallery and photo, a hash-valid file and a live catalogue proof no older than 60 seconds all match. The downloaded body's original HTTP age does not limit this viewer-only path and is never rewritten. An expired page proof refreshes the small photo metadata page for the group, not each full-image body. Missing or corrupt files use the normal server read. Prepared details keep their 15-second bound; authentication, account changes, cancellation, known rejection and opt-out still block reuse. Unknown proof, renewal and known rejection otherwise require the normal server path. Changes since the last page are detected at the next authenticated read, not instantly. Older servers and ordinary restricted browsing remain transient. A server-directed retry delay shows a cancelable countdown; exhausted retries retain completed files. Zero stored photos report a safe reason instead of partial success. The presentation button opens fullscreen and loops through the album at the server's enabled interval or a five-second default. The authorized soundtrack starts with the presentation. The music controls play, pause and change tracks. The player

reads the local file and prepares the next track. Right-click a gallery or photo for information and authorized visibility changes. Click outside the panel or press Escape to close it. Admin photo information includes format, dimensions, stored EXIF and GPS when allowed by the gallery. Thumbnail quality shows the server's current generation settings. The gallery panel shows direct inventory and local cache coverage. A tree icon appears when coverage is known. Use the add icon in the tree toolbar for a root gallery and the add icon beside the album title for a subgallery. You can also add a subgallery from a gallery menu or blank photo space. The wheel and trackpad scroll the grid and zoom the lightbox. **Edit gallery** opens the selected gallery's web Admin editor in the system browser's right-side panel. It uses the browser's own Admin login and sends no WinApp credentials. Older servers open their normal full-page editor. The tree restores expanded and collapsed branches for each linked site and account. Use the circular arrow beside the album title to refresh its photos. WinApp checks the visible Admin album's index about every 30 seconds and reuses unchanged thumbnails. Older servers support manual refresh. Scrolling uses bounded RAM and the verified persistent disk cache. Thumbnail reuse is bound to the issuer, session, gallery, photo, public eligibility, strong validator and private account. Current proof timestamps and row epochs authorize reuse; proof renewal may retain unchanged content, while expired or denied access cannot reuse it. Before per-photo reads, WinApp coalesces expired visible-thumbnail proof renewal through the existing bounded catalogue refresh. This supports public and eligible non-NSFW Admin media; older servers and no-store responses keep their existing authorized reads. One retry per existing revision interval bounds repeated refresh. Select photos with Control-click on Windows, Command-click on macOS or the photo menu. The context menu can delete one photo or the selection. WinApp uses the current Admin Photos link. Confirm each batch of up to 64 photos in WinApp. It uses the same permanent deletion operation as web Admin. Refresh and review the album after an interrupted response before confirming another deletion. Use **Use this gallery for Watcher** in a gallery context or the Watcher rail menu. WinApp starts capture with a temporary gallery key and shows a red REC indicator. Photos remains selected. A small header display uses icons for Watcher state, the observed accepted-photo count and the last scan time. A dash means the value is not yet available. Hover for the full gallery, state and queue description. This display sits just left of **Thumbnails**. Its Stop icon uses the existing safe queue drain and report finalization, so ordinary capture needs no visit to the Watcher tab. Open that tab whenever its full controls are needed. The Watcher rail icon pulses red on every tab. A filled red light marks the capture album and its heading. A hollow light marks a collapsed branch that hides it. SimBrief is available for a detected flight simulator. Finish drains screenshots, saves available flight reports for a simulator session and revokes the capture key. The website stays linked. The Watcher panel reports preparation or a reason it could not start. A new gallery or a recovered creation confirmed by the current connection can use this action immediately, without reloading Photos. Cached rows still need live confirmation. If the gallery or upload access changes while the menu is open, an explanation stays in the Photos panel; reconnect or refresh Photos before reopening the action. During startup, a busy indicator names the preparation phase. Wait for REC before creating a new screenshot. Stop and Finish show preparation immediately before the upload drain and report phases. Recent Activity rows are saved in bounded background batches. Normal exit attempts a flush within its deadline. Storage failure, timeout or a crash can lose buffered rows; failures are logged and the rotating log is written immediately. Destination selection checks accepted imports in the local upload history. It uses the same 256 MiB history limit as state updates. Unverifiable history or unfinished work for another destination still blocks the change. The updated native viewer needs a current installed Windows click-through. Earlier protocol evidence predates this UI revision. The filesystem writer fence is an inactive prototype and does not protect gallery operations until its coordination has been qualified. The experimental separate `galleries:create` permission requires explicit consent and verified scope storage. Its source refuses occupied targets, canonical catalog drift and catalogs over 10,000 rows including the new gallery; slug search checks at most 128 candidates. Neither is a time limit. A pending intent is not automatically repeated after an uncertain result. A pending

linked-create reservation temporarily blocks Admin folder imports, moving discovered photos, deleting discovered folders, copying selections, moving galleries to or restoring them from Trash, metadata-organizer moves, consistency repairs and ordinary gallery creation, even for another path. Do not delete the reservation: safe automatic cleanup is not yet available. Existing API-key upload remains available. This same-database guard is not a filesystem fence. Before creating or checking a retained intent, the client requires a fresh server identity matching the selected or saved Admin account. The maintenance CLI can inspect private original evidence from standard input and report ledger correlation only; it does not prove that a worker stopped or authorize another creation. Before sending, the client verifies a complete staged intent and publishes it without replacing another key. An interruption may leave a stale lease that blocks work; Windows filesystem and power-loss behavior remain unqualified. **Resume original creation** shows the original title and parent for explicit confirmation, then sends the same operation key and inputs to the same HTTPS issuer and verified Admin account. Existing pending or reconciliation claims remain blocked and can require administrator reconciliation. This protection requires the original server ledger; restoring an older database backup can remove it. The unconfirmed status offers explicit resume or **Close unresolved creation** after reviewing the gallery and storage. Closure shows the original title and parent for explicit confirmation. It removes the local intent only after the server confirms a durable closure of the same key. An unknown result leaves the intent in place. Closure is refused while the server claim is pending. Closure does not prove that no gallery or files exist or that earlier work has stopped. Verify the original work has stopped and inspect the gallery and storage before starting a separate creation. None starts automatically; a verified completed result instead remains as a receipt for acknowledgement. An invalid or unreadable local intent remains blocked and requires administrator assistance because its original key cannot be verified. Before sending a new gallery, Photos checks earlier work in the same local profile. A pending result is checked only with the original site and Admin account; another site offers reconnection. This check never repeats creation. For a completed earlier gallery, confirm **Continue** before creating the next one. Only the exact local completion is retired; the earlier gallery and upload target stay unchanged. The current connection and selected parent are checked again. Submitting the same title and parent on the same site and account instead checks the completed result without creating a duplicate. An unknown result or failed local confirmation keeps the saved work for another check. A confirmed receipt remains until explicit acknowledgement. The client writes and checks a complete staged receipt before exclusive publication; uncertainty retains the intent and any published receipt. A local binding to the original intent prevents a stale selection or acknowledgement from applying to another operation with the same gallery details; the operation key is not displayed. These checks do not exclude unrelated filesystem changes or prove power-loss durability. On a supported local profile, **Use created gallery** rechecks the receipt and gallery before selecting it for Watcher and Import. This requires separate upload approval. The server compares the original ID, title and parent with current data. A rename yields unknown; a move, deletion or unverifiable state can be unavailable. A missing or unsafely redirected directory or sidecar also remains unknown; validation checks current safe files, not their original inode. Neither automatically starts a replacement. Where supported, new access is normal and listed with no scheduled publication. Database rollback cannot undo filesystem writes. Local flushes do not prove recovery after power loss; actual database, HTTPS and Windows creation remain unqualified. With separate explicit consent, a short-lived grant can list all galleries, including private and unpublished ones. An identity-only grant cannot list them. Selecting a gallery alone grants no upload permission. A separate upload scope explicitly covers all ordinary Admin galleries, including private and unpublished ones. The transient linked Watcher and Import are implemented but not qualified; the gallery-scoped key remains available independently.

The installer and updater provision the linking key in the database. The saved website address supplies the exact origin and mount. Existing explicit issuer and key settings keep precedence. Public discovery reads readiness. The browser saves a short-lived request handle in the session

before ordinary Admin sign-in. CSRF-protected binding leads to explicit consent. **Approve** delivers a one-use code to the verified callback for PKCE exchange. The main action requests account and gallery access, with image uploads where supported. On compatible servers, approve rolling access explicitly. It lasts seven days after the last successful authenticated use or refresh. WinApp renews the 15-minute access token using macOS Keychain or Windows Credential Manager. Verified HTTPS and direct `http://localhost` are supported. Older servers use a temporary link lasting up to 15 minutes. The owner can revoke grants in Connected applications. Unexchanged codes expire within 120 seconds. See `docs/APP_LINKING_PHASE_0.md`.

The scope-capacity migration widens four binary scope columns without changing existing grants. A partially completed migration retains verified short-scope support; new wide capacity is ready only when all four columns are verified. It does not grant new permissions. DDL is replayable, not atomic, and older runtime code may refuse the widened schema; no automatic narrowing is provided. Each metadata-lock wait is limited to at most five seconds; a stricter existing setting remains unchanged. A reduced setting is restored, but total DDL time is not limited. The callback lock ends before the runner writes its migration checkpoint.

The separate Windows HTTPS remembrance option uses the legacy offline scope and expires after 30 days or seven days of inactivity, whichever comes first. WinApp can resume an approved saved link at startup. **Reconnect saved link** also starts this check. The client rotates once and checks the server identity. If the same verified server explicitly rejects an expired or revoked rolling credential, WinApp discards that saved credential. Choose **Link gallery** to approve a new link in the browser. Startup leaves approval to you. An uncertain response or storage failure blocks reuse of the old credential. Review server links and use **Forget saved link** before linking again if needed. Local forget does not revoke the server grant, and the current session may remain active. The saved `pgu_` key remains independent. Real Windows and power-loss recovery have not been qualified. Rolling links keep one saved binding per exact issuer. The legacy Windows HTTPS option keeps one shared binding. Resolve pending work and forget the local link before replacing a saved account for the same issuer. The server grant can be revoked separately. The linked upload path uses separate history and an exclusive lock by issuer, account and gallery. Before sending, it records a pending result. After an uncertain transfer, inspect the gallery and source manually; do not automatically retry or delete the source. An experimental bounded server probe can report `present` only when matching bytes were observed during its read; it does not prove image registration. `unknown` is not proof of absence or permission to retry, and the client does not yet use the probe. Its time budget is checked between filesystem calls and cannot stop a blocked call. External filesystem changes are not atomically excluded. When the current WinApp process exits, the OS releases its native lock for the next Retry. For older locks, close all uploaders, review the gallery and sources, preserve the recovery JSON and archive only the verified inactive lock. Local filesystem and power-loss guarantees remain unverified. Linked flight data stays local and does not implicitly send AI work or PDFs with a saved legacy key. Configure proxy, web-server and PHP request limits because multipart parsing can consume resources before the application checks the request.

An idle Watcher starts after an explicit **Start Watcher** click; while active the same contextual control is **Stop Watcher**. While accepted work finishes, it opens **Show progress**. After an interruption, **Review pending uploads** opens the original queue. Reconnect and select the same upload gallery before choosing **Retry**. For a linked session, **Keep files locally** closes the queue after workers stop and preserves the photos, reports and upload history. Once the session is closed, **Start Watcher** is available again. **Check folder only** performs a folder dry run without uploads or starting continuous watching. The Watcher gear opens a settings sheet for folder choice, recursion, scan interval, file stability, simulator metadata, and the saved deletion setting. The sheet requires stopping the watcher before edits. **Choose folder...** saves the folder from the main Watcher view. **Start Watcher** asks for a folder when one is missing. **Save watcher**

settings stores the Watcher preferences for the next start. **Cancel**, Escape, or closing the sheet restores the opening folder, ignore count, last scan, and preferences. **Sync UTC time** remains an opt-in beside the main actions. Long pages and dialogs scroll with the mouse wheel or scrollbar; text and table controls keep their own scrolling.

Manual **Import** still requires preflight review and an explicit start. **Activity** contains the filtered event history, retry controls, redacted diagnostics, and log. A compact latest-event footer opens **Activity** when selected. Icon controls expose tooltips and keyboard focus; the question-mark help opens usage guidance. These presentation changes preserve the existing persistence, tray, update, and API-key safeguards. Pillow adds an ambient cool gradient and opaque glass-inspired card, footer, and pill-button artwork. This composited treatment simulates frosted glass without true alpha transparency or operating-system backdrop blur; text stays sharp, and input and table surfaces stay solid and high-contrast. It uses no whole-window opacity and adds no runtime dependencies. Without Pillow, readable flat ttk surfaces remain, and native controls and source-mode operation, including `-once`, stay supported. Windows prefers the installed Segoe UI Variable Text font, with Segoe UI and the native Tk font as fallbacks. Shared text styles preserve Windows font smoothing; fixed-width diagnostics stay monospaced. Common control, cache, pin and recording icons use supersampled artwork; without Pillow, readable native glyphs remain. The app downloads no fonts and changes no system preferences. Before creating windows, an unaware Windows process requests system DPI awareness and preserves an existing manifest context. This does not add per-monitor layout reflow. Restart on the destination display when moving between displays with different scaling. The main window remembers its normal size, position, maximized state and Photos tree width automatically. Minimizing, fullscreen and tray hiding preserve the last normal rectangle. Restoration keeps the window and divider within current display bounds. These view preferences do not change upload configuration or credentials. Switching albums cancels the old view. A new request waits within its original deadline for the existing single transport slot while earlier network cleanup finishes; canceling the new view also cancels that wait. Before its scan, `-once` verifies the saved upload mode with Windows Credential Manager. A verified legacy or unmigrated legacy profile remains eligible; linked, first-setup, unknown or unavailable evidence returns a nonzero status without scanning. Each inventory and upload attempt and redirect rechecks the mode; an already running request may finish. This in-process check does not fence other processes or protect graphical workers. API-key requests refuse redirects to another host, an unapproved port, or from HTTPS to HTTP. Use the final gallery URL if a server requires such a redirect. Actual Windows behavior remains unqualified. On Windows, watched-source deletion is enabled by default for new settings; an explicit saved off choice is retained. The Watcher settings switch controls both linked and API-key captures. After a successful upload response and local acknowledgement, a native handle verifies the same file identity and bytes before deleting that opened source. Changed, redirected, hardlinked or busy files remain with a warning. Inventory matches and uncertain uploads do not authorize deletion. Other platforms, manual imports and source ZIPs retain their sources. The Watcher deletion preference applies only to watched captures; it does not delete files selected for manual Import. Flight API-key revocation is temporarily unavailable. GUI mode protection and crash recovery are still being implemented and are not qualified.

10.3 Windows startup and one running instance

The installed uploader starts after Windows sign-in by default, minimized directly into the system tray. In **Settings > App & integrations**, **Start with Windows (minimized to tray)** saves immediately; switching it off removes this user's startup entry, and updates retain that choice. Setup applies the preference even if the final application-launch checkbox is cleared. Source/Python runs leave Windows startup unchanged. An idle Watcher and manual imports need explicit Start. An active Watcher can continue after normal Quit and restart: Exit settles

accepted writers, verifies the original session/queue and confirms closure of the old temporary Watcher key before saving a separate credential-free restart checkpoint. Within its existing 24-hour lifetime, startup restores the exact saved gallery/account and rechecks access before admitting the same queue and recorder with fresh contextual authority. Stop, Finish and unlink cancel this intent; stopped capture stays stopped. Startup waits for mode readiness. Temporary checkpoint reads or gallery verification failures keep the same intent for bounded retry; invalid state, changed identity/scope and revoked access still block capture. Stop or unlink also cancels pending retries and late results. Diagnostics identify the failed phase and a fixed safe reason without keys, checkpoint contents or local paths. A securely saved gallery grant or unchanged saved API key is required. Expired/revoked or unsaved transient links need normal reconnection; uncertain uploads and forced termination retain explicit recovery.

Opening the app again restores the existing window and keeps current work. One Windows process owns the shared app profile, including executable copies; another Windows session must open its existing process in that session. A logon launch keeps import-recovery prompts deferred until the window opens. If the tray cannot become available within five seconds, the window appears so the app remains accessible. Startup registration belongs to the user running the application; an installer explicitly started elevated may use that account.

10.4 Update notice and indicators

The app checks for uploader releases at startup and at most once an hour; **Check for updates** requests a manual check. A validated offer appears as a version banner and marked update control in the main header on every screen. **Later** hides the notice but leaves both indicators available to reopen it. When tray support is available, its icon shows a red update badge, its tooltip identifies the offered version, and the first menu action names that version and opens the same notice. Without tray support, the in-app indicators remain available. The app does not set a Windows taskbar overlay or show operating-system toast notifications.

A failed network recheck preserves the validated cached offer across restart; the offer is validated again before it is restored. A successful check with no current offer clears the banner, header mark, and tray badge.

Update operations automatically record their phase, stage, result and duration in `watcher.log`. Network failures retain the HTTP status and typed transport cause; transfers include known byte counts. Checkpoint, helper handoff and cleanup have separate diagnostic stages. Cleanup failure preserves the original update error. Credentials, signed download queries, response bodies, raw exception messages and local paths are excluded. The notice keeps its concise status; technical detail is available in the ordinary log without a debug switch.

Copy diagnostics includes the retained updater failure after later successful checks. It also includes current-session Photos failures, distinguishing catalogue reads, thumbnail transfer and image decoding. Album changes retain those facts; changing the approved live session resets the Photos snapshot. Preload failures also reach the ordinary log, visible Log and copied diagnostics, distinguishing listing, cache checks, transfer and verification. Terminal records include processed, stored and skipped counts. A running preload keeps its failure facts across album changes within the same approved session. Rejected media responses show bounded cache-header decisions without copying raw headers or relaxing permissions and local-storage rules. Equivalent repeated or split cache directives are accepted with the same authorization markers and private revalidation policy. Longer additional ages never extend the local 60-second cap; incompatible directives do not authorize storage, and no-store browsing remains transient. Worker events keep their FIFO order. The GUI yields between complete handlers after at most 64 messages or an eight-millisecond work budget, then continues after 16 ms; an empty queue keeps its 200 ms poll. One slow handler

can exceed this cooperative budget. Activity history and persistence remain immediate while table redraws are combined. Copied diagnostics expose numeric batch counts and durations for manual measurement, without claiming screen latency.

10.5 Windows simulator location

Sync UTC time, beside the Watch folder actions, is off by default and saves its own preference. While watching, a separate worker checks MSFS 2020 or 2024 every 10 seconds. It compares full simulator UTC date/time with Windows UTC; only an absolute difference above 60 seconds requests year, ordinal day, hour and minute correction, followed by fresh readback. The worker waits for a loaded, unpaused flight at 1x rate and five seconds of settled state. Menus, loading, pause and other rates suspend writes. Window/tray stop and complete exit cancel UTC work; hiding to tray preserves it. The selection survives stopping. Three failed corrections with 30-second backoff suspend that connection until reconnection. Uploads and photo-location acquisition remain independent.

Both generations use the same time events. The one-based ordinal day setter and any effect on Live Time/Live Weather still require live verification in two separately loaded flights per available simulator/build; native mocks do not establish those results. See `winapp/README.md`. Hosted installers explicitly use x64 CPython 3.14.x. Before packaging, the builder verifies its interpreter and the actual frozen EXE's patch version, implementation and pointer width through a pre-GUI report; a wrong or missing report fails preparation. The independent application source targets 0.10.0.

The bundled native SimConnect DLL comes unmodified from the official MSFS 2024 SDK Core 1.7.3. Its 84,480 bytes and SHA256 are documented in `winapp/README.md`. Static x64 and export checks preserve the previous 117 named exports; live camera, flight telemetry and UTC acceptance in MSFS 2020/2024 remain separate. SDK version identifies provenance because the DLL has no usable embedded file version. Microsoft SDK terms apply.

WinApp 0.3.2 automatically uses the MSFS 2024 camera world position and falls back to the aircraft position on bounded camera failure. MSFS 2020 uses the aircraft position directly through the same SimConnect runtime. Coordinates are optional: simulator unavailability does not prevent upload. Position-source diagnostics distinguish camera and aircraft results. The installer requests application shutdown before replacing files; complete pending work before updating. Live simulator and upgrade acceptance must be checked on the target PC.

The frozen updater loads Windows APIs explicitly from System32, avoiding a collision between the bundled VERSION text file and the Windows version DLL. When upgrading from 0.3.1, manually launch the verified 0.3.2 installer. The running older app copies its own helper before installation, so that old helper may still report its DLL error after replacing the app. Future updates started from the corrected 0.3.2 app use the repaired helper; verify the installed handoff on Windows.

Aircraft replies are matched by request and definition IDs; the request USER selector does not require a zero reply object ID. Bounded diagnostics record receive metadata, dispatch sequence and specific rejected-packet reasons. Failure reasons survive path redaction and activity messages display one SimConnect marker. The object-ID filter was a confirmed code bug, but the reported FS2020 timeout did not record the returned object ID. Retest on the affected PC; simulated tests do not prove a live fix.

The installed uploader checks its own stable GitHub installer version at most once per hour; **Check for updates** also starts a manual check. The last automatic attempt is saved independently for the current user and survives application restarts. CMS tags do not determine its version.

The highest numeric installer version requires SHA256 from the GitHub asset or a matching same-release `winapp-update.json`. Missing or conflicting integrity data refuses the update; identical installers in several CMS releases are accepted. Gallery credentials are never sent to GitHub.

Choose **Update** to download with progress and optional cancellation and verify the installer. An active Watcher finishes the current acknowledged photo operation, saves its queue and flushes the original recording before the helper starts. Unresolved work postpones installation after 120 seconds; the game-presence observer does not block it. Finish a manual import, AI task or report finalization first. Helper failure keeps the application open. After update, the same session, folder, target and original file boundary continue. Screenshots created during installation remain eligible. Telemetry keeps its original identity and records a gap. Explicit Stop or Quit cancels continuation. Linked capture requires the same account, gallery and scope. Older versions without saved running intent can offer **Resume Watcher** for verified unfinished work; they do not resume it without that explicit choice. Abandon and a replacement key are not required for continuation. Persisted settings and credentials remain available; unsaved Settings fields are not saved by an update. The helper rechecks the installer, waits for the installed executable to exit, requests Windows UAC approval, installs and verifies the new version, then restarts the uploader as the original user. Installation progress remains visible. There is no automatic rollback. Python/source runs offer the verified installer for manual launch. A required Windows reboot remains explicit; cancelling UAC reopens the existing installation. Older clients blocked waiting for workers need a manual installer launch after fully closing the old application.

10.6 Flight recording and reports

The uploader uses dark glass panels, an icon navigation rail, and sliding switches. Hover over icons for their descriptions. The header groups PHP Gallery with its autopilot subtitle. Settings gears sit directly beside their gallery, watch-folder, or flight control. The flight gear opens a local sheet for automatic reports, report languages, interface language, progress, and the report folder. Starting the watcher automatically arms recording; there is no separate flight-start action. The Record flight switch pauses and resumes acquisition during watching, preserving the session and marking the gap in the track. Starting with recording off opens no SimConnect connection. Photo uploads continue independently. Closing settings starts no operation. Glass artwork uses opaque RGB pixels for reliable Windows Tk; source mode without Pillow retains usable native controls for repair.

Watcher shows the folder path and **Choose folder...**. Saved paths, gallery URL, and feature choices are restored after restart. The interface language uses four directly visible choices. Scrollbars appear only when content exceeds the viewport; helper windows and tooltips use the same dark palette.

The Windows uploader records MSFS 2020 and 2024 aircraft telemetry through a shared read-only provider. Recording and report creation are enabled by default; recording can be opted out independently, and the saved language choices initially include English, Czech, German, and Swedish. The bundled ReportLab renderer and DejaVu fonts create a separate chart PDF for each selected language from one analysis. Reports remain local for retry. Unsupported signals stay unavailable rather than invented; live simulator, aircraft, and build acceptance remains separate for each generation.

The recorder requests `ABSOLUTE TIME` in seconds and load factor in `GForce`. The `E:` prefix belongs to RPN expressions. Photo-camera success does not establish flight recording. Copied diagnostics show the app version, independent recorder state, saved sample count, and last warning. Rejected SDK operations identify the variable, units, send ID, and parameter index. Known SDK/HRESULT codes include a name, explanation, and impact; unknown codes stay explicit.

Changed failure reasons stay in session warnings. Connection, first valid frame, and first saved sample have separate log messages. Missing historical samples cannot be recreated.

The Record flight row also shows a single compact IAS/ALT/V/S line in knots, feet, and feet per minute. It reads the existing recorder cache without another SimConnect connection. Paused, stopped, unavailable, or more-than-three-second-old values become dashes. The adjacent **PDF** shortcut opens saved flights.

Saved flights and PDFs lists start time, aircraft, checkpoint sample count, local PDF count, state, and retained errors. Refresh reads checkpoints only; Open app log uses the existing log. **Finish flight and create PDFs** creates all four editions and finishes accepted work and required delivery. It keeps the upload connection and works with automatic creation off.

Selected languages render simultaneously in separate processes, one per language (currently up to four). Flight analysis and map backgrounds are prepared once; the main application uploads only after the complete set succeeds. Failure keeps completed PDFs for recovery. Cancellation stops queued work, waits for running renders and retains their finished files.

With one version, expanding a flight shows its files directly. Multiple versions appear as expandable groups, always numerically descending (v5, v4, v3), regardless of column sorting. Refresh retains expanded flights/versions and selected files. This display does not move or rename files on disk.

For a stopped flight, **Create local PDF** or **Re-create local PDF** creates a numbered version under `reports/`. Earlier PDFs, raw samples, and upload receipts remain intact. No key is required, no upload/revoke occurs, and another running flight continues. Partial reports remain incomplete; active, unclosed, and empty recordings are refused. Local progress/errors are retained in `manual-report.json`; local creation does not resolve pending remote delivery. Cancellation retains finished editions. Shutdown/update drains this worker. Windows layout and live acceptance remain a separate manual check.

The Begin/Finish actions in Watcher and Import use **Begin flight** and **Finish flight** when SimConnect is available. If it fails, known game processes owned by the same Windows account provide an optional fallback, without admin rights. A Microsoft Flight Simulator process also selects flight wording. Otherwise the labels are **Begin game**/**Finish game**, with localized easter eggs for Cyberpunk 2077, the Mass Effect trilogy, Skyrim and Dragon Age. Unknown or ambiguous games keep general wording. The background observer checks every three seconds, even before watching starts or with recording/UTC sync disabled. It opens/closes SimConnect without telemetry or simulator writes; process names stay local. SimConnect probes, photo GPS, recorder connections and UTC sync require this account's running MSFS 2020 or 2024 process; otherwise they wait or skip silently. Prepar3D and X-Plane only select flight wording through process names. Outside a flight context, current recording controls and progress use neutral session wording. The Watcher UTC controls are hidden while their saved preference is retained; flight detection restores them. Historical flight reports keep their actual identity, and custom game action phrases remain. In idle API-key mode, Begin stays visible with or without a saved key or running game and opens the connection editor directly. A Photos browsing link does not hide it. Explicit linked upload mode offers gallery reconnection. Active capture, pending work and finalization retain Finish, Review or Progress before setup; an active manual import retains its own controls. SimBrief's Load/Unload control, settings icon and status appear only for a detected flight; hiding retains the loaded plan. Outside a flight context, Watcher, Import and progress use **Abandon game** with matching confirmation and status text. The existing local cleanup leaves the game running. The bundled offline process catalog is broad but incomplete. Add exact process names and optional four-language phrases in `\%APPDATA\%/PHPGalleryUploader/game-contexts.json`, then restart; no rebuild is needed. The schema, source and refresh instructions are in `docs/WINAPP_GAME_CONTEXTS.md`.

Watcher and Import show **Gallery:** followed by the current display title from the authenticated inventory API, wrapping long names within the available row. The physical folder name is always derived from the sanitized source title. Saving a title change renames the folder and updates descendant paths; a destination collision refuses the rename without overwriting data. Older APIs without title metadata display the final folder segment. Hover over the name to see the full physical folder hierarchy on separate indented lines, from root to gallery. Connection changes refresh both presentations; flight records and import history retain the complete identity.

The Finish button sits below the upload gallery during capture. Finish and Stop keep the connection available for another session. Manage server keys separately in PHP Gallery Admin. An active linked session uses the browser approval for uploads. After restart the verified saved address prefills reconnection.

In legacy API-key mode, **Abandon flight** in Watcher, Import and progress offers explicit local recovery after a gallery is deleted or its key revoked. After confirmation, workers stop and in-flight requests settle; pending delivery is abandoned and the saved key is removed from this PC without contacting the server. Recordings, photograph sources, PDFs and receipts remain local. It also works with no pending sessions. The main action becomes the Begin action. A still-valid server key needs separate revocation in Admin. Submitted revocation must settle first; shutdown or configuration-save failure keeps the key and recovery available for retry.

To upload retained PDFs, save and test a valid gallery key, including a new key for the same gallery. In the report browser select a stopped flight and PDF files with Ctrl/Shift, then choose **Upload saved PDFs**. Confirm the gallery and files; server inventory verifies the exact target before transfer. Existing PDFs are sent without rendering, stopping another flight, or revoking the key. Separate `upload-receipts.json` retains acknowledged checksums; retries reconcile server status. Old plans, bindings, and raw data remain intact. Cancellation retains finished editions. Protocol errors include redacted HTTP/action/type context and a bounded response excerpt. The client checks status before upload and requires matching checksum and size. If confirmation is uncertain or the local PDF changes, it retains the file and does not mark that edition complete. An earlier saved plan receipt records a past acknowledgement; it does not prove the PDF still exists on the server. The selection dialog identifies a file, not a hash captured at confirmation; the later file checks do not atomically exclude other writers.

Without a saved key the main Begin action opens API settings. Expand a flight and, when present, its version to see PDFs; double-click opens the default viewer. **Open folder with PDFs** opens only the PDF folder. Filenames contain recorded UTC, aircraft, language, and version. A restartable private catalog normalizes older layouts without changing bytes or upload identity; JSONs are in `reports/.metadata/`. Keep that folder with backups.

New reports use A4 landscape (297 by 210 mm), with a navy header, teal accents, pale background, and white panels. Embedded regular/bold DejaVu uses larger type: 28 pt titles, 12 pt body, 9.5 pt notes, and 9 pt plot axes. The overview places value cards beside the route; full-flight charts appear in pairs. Each landing has a summary and a separate detail page with four plots. Earlier PDF editions remain available; this layout changes no measurements, recording, transfer, or flight completion.

Reports use value cards and vector phase/configuration icons. Technical versions remain metadata; simulator build/date use small header text. Rounded Y axes and subtle grids show feet AMSL, feet AGL on approach, IAS knots, and fpm with 200/1,000 minor/major spacing. Long-flight time grids use ten minutes/hours. Route axes label coordinates, north, and first/last captured points. Scalar profiles bridge gaps up to 5 s in curve color and 5–300 s in amber dashes; pauses, reloads, and longer losses remain breaks. This display interpolation never enters landing measurements. No runway, score, or missing rate is invented.

Expanded PDF rows show page count and local creation date/time, with version and language first. A star marks legacy file modification time when creation metadata is unknown. Phase milestones normally fit one row. Landing profiles retain original samples before overview thinning; fitted touchdown rate and the simulator V/S are separate, with actual cadence and partial rollout disclosed. The compass compares true aircraft heading with wind-from HDG/KT, such as 020/05, and heading-relative components; it does not infer runway alignment. Descending below 100 ft AGL latches every acquired frame through rollout until 10 s at 15 kt or less on the ground. Earlier dense capture below 300 m remains; a climb above 500 m exits for a go-around. Target acquisition remains 20 Hz. PDF requests use the established uploader identification; gateway HTML 401/403 is reported separately from Gallery authentication. Completion requires server receipt IDs/checksums, not activity alone. Fuel defaults to kg, uses lb for Boeing/PMDG and gallons for common light GA. Measured simulator mass takes priority; old volume-only mass uses a visible 0.80 kg/L estimate. Raw data stays unchanged. Landing rollout cannot produce a false rejected-takeoff event.

Both actions use one durable coordinator. Standalone Stop stops the watcher, its owned UTC synchronizer, and flight recorder, then drains accepted photographs and completes required report delivery; unrelated AI, manual-import, and preflight workers remain running. Stop leaves the key configured. Explicit **Finish flight/game** in saved API-key mode also authorizes final revocation after required work. The captured revoke ticket remains active through the server response and is exchanged atomically for the configuration save reservation. Only the same operation, endpoint/key and config generation can clear that key; stale events and failed saves cannot claim success or clear a new credential. A running ordinary Stop must finish before a new Finish action. Confirmed Revoke runs the same Stop, also stops other authenticated producers, finishes accepted work, and requests scoped server revocation only after required receipts. Progress shows phase and elapsed time; ETA appears only when measurable. Failures retain tasks and local files for recovery, with explicit local-only resolution available. Restart restores work but never restores Revoke consent; a new confirmation is required. No empty report is created.

The gallery-scoped token advertises the optional `flight_reports_v1` capability. The JSON `flight_report_status` action accepts `report_id`, `language`, and `sha256`; the multipart `flight_report` action accepts those fields plus `session_id` and the PDF field `report`. The server confines writes to the token gallery and reuses general PDF validation, storage, and protected viewing without adding schema. Stable identity, checksum, and status reconciliation make retries idempotent. Reports are separate from OFP, inherit gallery visibility, and contain no raw telemetry.

11 Adding, describing, and organizing photographs

11.1 Choose an upload method

Browser upload suits everyday additions. Select a gallery, choose files, review the batch, and follow progress while the server validates media and prepares records and thumbnails. Filesystem discovery suits large existing folder trees or transfers made through FTP and hosting tools. Automation and WebDAV suit repeated or mobile workflows after their scoped credentials are configured.

In the right-hand Admin **Upload photos** panel, paste successive screenshots with Ctrl+V on Windows/Linux or Cmd+V on macOS, choose additional files, or drop images into the upload area. Each selected image appears in a local thumbnail queue before submission, together with its filename and size. Move images with the arrow controls, remove an unwanted screenshot, or clear the selection. The existing **Upload** button sends the originals through the normal upload process once; preview tiles are not replacement upload files. This queue is not a server-side draft: leaving the panel asks before discarding unsent files, a failed upload retains the selection for retry, and successful server acknowledgement clears it. Unsupported preview formats show a format tile without bypassing the server's upload checks. The standalone Admin upload page and no-JavaScript form continue to use their previous workflow.

For a large event, upload in meaningful groups. Smaller groups make errors easier to identify and allow titles, ordering, and visibility to be reviewed incrementally.

11.2 Import photos from iCloud

Open **Import from iCloud**, paste a public Photos Link at `https://share.icloud.com/photos/` or a public Shared Album at `https://photos.icloud.com/shared/album/`, select the destination and click **Download and import from iCloud** beside the link. This action recognizes the link and starts the existing upload form in one step; the footer shows the same action for an automatic source. Keep browser preparation enabled. Gallery checks public availability, requests Apple's compatible export, downloads it to temporary browser disk storage and imports supported photos automatically. You do not need to download or select a ZIP yourself. A new gallery must be Private or Unpublished; review an existing gallery's visibility because it also applies to partial imports.

The source types use separate acquisition paths. Photos Links can expire or be revoked; Shared Album availability is controlled by the owner, including optional temporary expiry. Gallery requests compatible JPEG/PNG/GIF representations; received JPEG/PNG/WebP/GIF images are checked by their actual signature. HEIC/HEIF/DNG/RAW, MP4/MOV, Live Photo motion and sidecars are counted as unsupported rather than renamed or silently converted. Compatible JPEG is not a preserved RAW original; capture/GPS metadata and Apple edits are not guaranteed. Apple's download protocol can change and a sign-in-only source cannot be imported anonymously.

Automatic transfer requires a supported browser on HTTPS with OPFS temporary disk storage. Known byte totals produce download progress; unknown totals show received bytes only. The response is written incrementally, never held as one whole-archive memory buffer. The existing ZIP worker reads bounded metadata and one image at a time, preparing and uploading small IndexedDB-backed batches. Stored and Deflate are supported; ZIP64 and multi-disk archives fail explicitly. Bounds include 10,000 entries, 4 MiB directory metadata, 2 GiB expanded data and 64 MiB/the server limit per image. Manually selected ZIPs use the same incremental reader: the 64 MiB bound applies to each image, together with the server limit. The archive itself is not uploaded or compared with the PHP request ceiling. Mixed selections preserve drawer

order or standalone natural image order. Retry retains the source cursor and exact pending batch; manual new-gallery visibility follows the explicit form and server. Decoders and thumbnails also use memory; these bounds are not a total heap measurement.

Cancel import stops acquisition and the active work; accepted photos remain. Keep the same form open and choose **Retry the same import** after an uncertain result: the disk source, cursor and unacknowledged batch/session identity are retained, so retry does not insert duplicates. Source and target controls remain locked until reconciliation. Safe failure before server work and successful completion remove temporary sources. Closing the tab loses in-page retry; the next acquisition removes abandoned unlocked source files older than one day. No background import or cross-session restart is promised.

If the automatic path is unavailable, open **Manual export and details** and choose **Open iCloud for manual export**. On Apple, choose **Most Compatible** when offered, download the ZIP or supported images and select them in the existing picker. Clear the source for ordinary local uploads. The link is cleared from its input after recognition and retained only in this form's memory; bearer links, Apple cookies and credentials are never sent to Gallery or stored in drafts. Sign in to Apple only on its official page.

11.3 Write useful photo titles

A title identifies the photograph quickly. A description or caption can explain people, place, activity, technical context, or a story outside the frame. Useful examples include:

- *Charles Bridge before sunrise*;
- *Arrival of the first train at the restored station*;
- *Family group in the garden, June 1998*;
- *Final approach during the Saturday display.*

Meaningful text improves search, accessibility, later identification, and the experience of readers who want more than a visual grid. File names can remain operational details when public display settings favor curated titles.

11.4 Use tags as a second organization layer

Create a small vocabulary before adding many near-duplicate tags. Prefer one stable term, such as *aircraft*, over several accidental variants. Tags can represent subjects, people, places, techniques, equipment, seasons, or editorial status.

For a travel archive, galleries can represent trips and tags can represent recurring themes. For a family archive, galleries can represent years and events while tags identify people. For a professional portfolio, galleries can represent clients and tags can identify deliverable type, location, and specialty.

Tag entry fields can suggest existing tags while you type. Reusing a suggestion avoids accidental spelling variants and keeps tag landing pages coherent. The suggestion is an editing aid only; the ordinary save action still decides which tags are stored for the gallery or photograph.

11.5 Arrange the viewing order

Ordering turns a collection into a sequence. Start with an inviting image, establish place or context, vary wide and close views, keep closely related details together, and finish with a natural conclusion. Chronological order works well for events, while visual rhythm can suit portfolios and exhibitions.

The public reorder controls help administrators adjust visible cards. Photo reordering on a public page is also scoped to the current visible pagination slice and saves through the authenticated image-order endpoint. When Picture Manager is active, dragging selected photo cards over a visible child-gallery card changes the gesture from reordering to a validated move into that child gallery. Pagination can divide a large collection, so review transitions around page boundaries as well as the first page.

11.6 Review EXIF and location information

EXIF can enrich a photograph with capture date, camera, lens, exposure, and GPS coordinates. This information supports maps, date suggestions, duplicate evidence, and technical context. Review GPS visibility before publishing photographs made at homes, schools, private workplaces, or sensitive natural locations.

12 Publishing, sharing, and audience scenarios

12.1 Public portfolio

A public portfolio benefits from a small number of strong top-level galleries, consistent covers, concise descriptions, and careful ordering. Use tags for specialties that cross projects. Keep source archives in private or unpublished branches and copy selected work into public presentation galleries where the workflow calls for separate curation.

Test the portfolio on a phone, a high-density display, and a slower connection. Progressive sharpening is the default and safe fallback for photo cards in the open gallery; responsive selection remains a supported alternative.

12.2 Private family archive

Organize by year, family branch, or event. Use password-protected parent galleries to establish an understandable private area, then place related descendants within it. Add names and dates while memories are available. Scanned photographs benefit especially from human descriptions because embedded camera metadata may be absent.

Share the protected address and password through an appropriate private channel. Explain that recipients can navigate child galleries without receiving an administrator account. Periodically review whether old links and passwords still serve the intended audience.

12.3 Client delivery

Create one protected gallery per client or assignment. Upload proofs, organize them into useful groups, and use descriptions to explain selection or download expectations. Voting can support lightweight preference gathering when enabled for that gallery. A final delivery gallery can contain approved files in the intended order.

Share links offer another controlled entry method where configured. Test the exact client journey in a private browser window before sending the address.

12.4 Event and community gallery

An event gallery can begin unpublished while photographs are arriving. Use child galleries for days, stages, teams, locations, or activities. Publish the parent when the first useful selection is ready, then add later groups without changing the familiar public address.

Public voting can highlight audience favorites. Tags connect recurring participants or subjects. Downloads provide a convenient collection transfer when the event policy allows it.

12.5 Travel and location archive

Country, region, city, and trip galleries provide natural navigation. Date ranges establish chronology, while GPS maps create a geographical reading path. Review coordinates for accommodation and private locations. A descriptive introduction can explain the route, purpose, and historical context of a trip.

12.6 Historical and institutional archive

Historical collections benefit from careful source notes, uncertain-date wording, names, locations, and provenance. Use descriptions to distinguish confirmed facts from family or institutional recollection. Tags can connect people, buildings, organizations, and periods across many galleries.

Create backups before large metadata projects and export information through available tools when planning external preservation. The gallery can provide an accessible presentation layer while original preservation practices continue alongside it.

13 Understanding your gallery data

Gallery owners interact with two cooperating stores: ordinary files and a database. Understanding their roles makes backup, migration, and troubleshooting easier.

13.1 What lives in gallery folders

The gallery folders contain the photographs and folder hierarchy, so the core collection remains recognizable through FTP, a hosting file manager, a backup tool, or a local copy. Moving or renaming files outside the application can require discovery or synchronization so the database learns the new state.

13.2 What lives in the database

The database remembers titles, descriptions, dates, visibility, passwords and access configuration, public paths, ordering, tags, votes, image dimensions, EXIF indexes, checksums, thumbnail records, administrator accounts, settings, audit information, and workflow state.[9]

The database makes search and administration fast. It also allows a file to have a friendly title, rich description, tags, a chosen order, and access rules without changing the original image bytes.

The `public_content_widgets` table persists authored titles, Markdown, page scopes, seven in-flow zones, floating anchors and normalized coordinates, publication state, ordering and optimistic revision numbers. An upgrade creates an empty table and displays no widgets until an administrator explicitly publishes one. Published records are rendered on eligible public home/gallery pages; drafts and disabled records are never included in public HTML. Back up these records together with the rest of the database.

13.3 How users benefit from the database

Everyday benefits include:

- searching words across gallery and photo descriptions;
- opening a gallery through a stable public path;
- displaying photographs in a curated order;
- remembering tags and visitor votes;
- protecting a gallery with inherited access rules;
- finding exact duplicate content through stored checksums;
- preparing suitable thumbnails without re-reading every source file on every page;
- recording migrations and maintenance decisions with an audit trail.

[9] Password values use hashes or protected configuration appropriate to their purpose. A database export still contains sensitive operational information and deserves secure storage.

13.4 Backup as one complete set

A complete backup includes the gallery tree, database export, local configuration, and installation-owned custom assets. Give these parts the same backup date so they describe one coherent state. Store a copy away from the web server and test restoration in a separate location.

14 Everyday ownership and care

14.1 After every important upload

Review the destination gallery, photo count, ordering, titles, visibility, and thumbnail appearance. Open several photographs in the lightbox and check the gallery anonymously. For a protected collection, repeat the password or share-link journey from a fresh private session.

14.2 Monthly review

A monthly maintenance check should include:

1. Review System Health and pending migrations.
2. Check recent Admin audit events.
3. Inspect available application updates and their release information.
4. Confirm scheduled or manual backups completed.
5. Open representative public and protected galleries.
6. Review thumbnail maintenance when source files or rendering settings changed substantially.
7. Remove or revoke credentials and share methods that have completed their purpose.

14.3 Before a major public launch

Check titles, descriptions, cover images, spelling, mobile layout, privacy, GPS exposure, downloads, maps, voting, search results, and contact expectations. Test through a slow network profile if the audience may use mobile connections. Create a backup immediately before the launch state.

14.4 When another person helps administer

Give each administrator an individual account where the account model supports the intended workflow. Explain gallery scope, privacy expectations, naming conventions, tag vocabulary, backup responsibilities, and deletion procedures. Individual accounts improve accountability in audit records and keep personal duplicate-review ledgers separate.

15 Public visitor experience

15.1 What a visitor sees

A visitor opens the site at the home page or follows a direct link to a gallery or photograph. The page presents the site's name, navigation, gallery title, description, breadcrumbs, tags, child galleries, and photographs that the visitor is allowed to see. Protected content asks for the appropriate access step before revealing private material.

Each gallery has a public address that can be bookmarked or shared. A direct photo address opens the gallery context and leads the visitor to that photograph. Meaningful titles and stable public paths help links remain understandable in messages, bookmarks, and search results.

15.1.1 Local history and new content

Small icons follow each gallery title, including when the title wraps. A bright dotted circle means unseen, a part-filled circle means partially visited, and a quiet checked circle means seen. A physical branch becomes seen only after its own page and all navigable descendants have been opened. Attached Smart Galleries count as their own items; opening one does not mark its physical sources visited. Covers, search results and background requests do not count as visits. Click the progress icon beside a card title to mark that gallery and its available descendants unvisited again. This includes attached Smart Galleries, whose state is shared across their placements, and preserves New-content markers. Opening a gallery again records a new genuine visit.

The separate sparkle means newly available content since this browser's first observation or last acknowledgement. It can accompany any visit state. Opening a parent does not acknowledge new content inside unopened descendants. Cosmetic changes and deletion alone do not create additions.

The header bell opens a small translucent panel immediately beside it, without dimming the page. Follow a gallery link to open its content. *Mark all new content as read* appears beside unread notifications and acknowledges only their content, preserving gallery visit states. The separate footer action defaults to *Mark all galleries as seen*. Only when every available gallery is seen does it offer *Mark all galleries as unseen*. Both directions preserve content markers and work without a confirmation dialog; neither dismisses notifications. Escape or a click outside closes the panel. Narrow-screen actions retain comfortable touch targets. The panel matches the PDF/music applets with a Theme glass surface, small accent icons and a compact outlined read-all action; the independent visit toggle remains below the list. When there are no unread notifications, the entire news section disappears, leaving only the compact visit action, Close and the local-history note. The panel grows with its content up to the actual viewport height, including browser zoom. Only then does its compact list scroll; heading and actions stay visible. There is no fixed item-count threshold.

History stays in this browser's local storage, independently for each installation. It does not synchronize between devices or send visitor history to the server. If storage is blocked, the current tab remains usable. Unavailable content evidence omits notifications and their action without a technical warning or a false empty-state claim. The visit toggle remains usable. Authenticated administrators never see this feature, including in anonymous preview; after logout they receive the ordinary visitor interface. Any future non-Admin Viewer account uses the same separate visitor identity boundary.

15.1.2 Typical visit

One common visitor path is:

1. Open the home page and choose an interesting gallery cover.
2. Read the gallery introduction and follow child galleries or tags.
3. Scroll through smaller preview images.
4. Open one photograph in the large viewer.
5. Move through nearby photographs with buttons, keys, a strip, or the selected viewing mode.
6. Open a map, vote, search, or download when the gallery offers those actions.
7. Use breadcrumbs to return to a broader collection.

15.2 Gallery hierarchy and pagination

15.2.1 Direct content and stable ordering

A gallery can show smaller galleries first and photographs afterward. For example, *Italy 2026* can contain *Rome*, *Florence*, and *Venice*, while also showing a few overview photographs directly on the Italy page.

Pagination divides a long collection into manageable pages. This helps the first page appear promptly and keeps scrolling comfortable. A small exhibition may look best on one page. A wedding with 1,500 photographs will usually benefit from pages or child galleries. The large photo viewer can continue through the wider gallery sequence while loading additional information as needed.[10]

Long public listings include a floating **Top** control after the visitor has scrolled into the gallery listing. It uses smooth browser scrolling, disappears outside the active listing, and stays hidden while the lightbox or fullscreen viewer is open so it does not compete with photo controls.

15.3 Configurable public content widgets

Open **Appearance** and **Widgets** in the Admin Zone to create independent content panels such as *Partner galleries*, announcements or useful links. Each widget has a title, a Markdown text body, a publication state, page scope, appearance and a saved position. Create or edit a draft, choose **Preview without saving**, and then explicitly **Save**. The editor also supports duplication as an unpublished draft, numeric ordering, disabling and confirmed deletion. A concurrent edit from another Admin tab is rejected rather than silently overwriting a newer revision.

Use headings, paragraphs, emphasis, lists and links with your own visible captions in one Markdown body. Both safe internal gallery URLs and external HTTP(S) links are supported. Executable URL schemes, arbitrary HTML, scripts and remote embeds are not permitted. Images or logo uploads are not part of the current widget editor. Widget authors must not place privileged information in public content.

Select the *Homepage*, *Gallery pages*, or *Homepage and galleries* scope. In page mode, the available zones are top/bottom of public content, left/right content rails, immediately before/after the homepage gallery grid, and the global footer. A homepage-only grid position on a gallery

[10] Pagination changes how many cards appear at once. It leaves the underlying photographs and their order intact.

page falls back to the bottom of the main content for an all-pages widget. Side rails remain part of the document layout and stack after the main content on narrow screens; the navigation header and hero are not widget targets.

In *Floating* mode choose one of eight viewport anchors or set custom X/Y coordinates from 0 to 1000. The published widget may become a fixed, dismissible panel on a sufficiently large desktop viewport. Up to two panels are enhanced at once; unsafe overlaps, small screens, zoom or insufficient height retain the same accessible article in normal document flow. The visitor's **Close** hides the panel only on that page view and does not update the administrator's publication settings. With JavaScript disabled, authored content and links remain in the initial HTML exactly once.

On public pages, floating placement is recalculated when protected controls or widget content change size, including an expanded Gallery toolbar and loaded fonts. Unsafe placement returns the same article to normal flow.

The protected Home/Gallery preview uses the real Theme layout and saved widget order while keeping public scripts disabled. If Home has no gallery cards, an after-grid draft remains ahead of content-bottom and floating zones; a new side rail follows the same responsive stacking behavior as the public layout. In the separate Admin placement map, drag with mouse, touch or pen, click, or use arrow keys to set custom coordinates. Shift with an arrow makes a larger step; **Reset floating position** restores a valid bottom-right preset. Desktop/tablet/mobile selectors and the content preview do not save the draft. The position map remains advisory. Use the actual public page for final visual verification after saving.

The EN/CS/DE/SV Admin interface strings are translated, but authored titles and Markdown remain in their selected source language without automatic content translation. Theme/Custom CSS can target `.public-content-widget` or a stable `data-public-widget-id` selector. Widget editing itself never stores arbitrary CSS or JavaScript.

15.4 Search, tags, and navigation

Search and pagination use compact translucent controls in the selected Theme, with a 32 px desktop search bar. The compact suggestion list preselects the highest-ranked result as results arrive. Enter opens it; Up/Down or pointer movement selects another result. An explicit choice follows that result when later phases reorder the list. Editing the query resets selection to its new top match and immediately disables old results. Long subtitles are truncated; mobile rows retain 44 px touch targets. Escape clears the query and closes suggestions. The current page has a subtle accent tint and line; keyboard focus remains visible. On narrow screens, the search scope moves below the field and touch targets grow. A small switch selects the search scope. Search scope and page navigation work as before.

Search can look across the whole public site or stay within the current gallery and its children. A visitor looking at *Family archive* can search only that branch for a person's name, while a visitor on the home page can search across every public collection. The current search service begins at two visible query characters, returns a bounded mixture of gallery and photo results, and ranks stronger title/name/tag matches ahead of weaker text matches. The default request returns up to 12 merged results and the service clamps a request to at most 30 results.

Searchable evidence includes public gallery/photo titles and descriptions, image filenames, gallery and photo tags and their public descriptions, active maintained-language content where its schema is available, and internal AI searchable text when AI metadata is ready. Access filtering happens as part of the query: private, inaccessible, or non-public image content is not returned merely because its text matches. A branch search also constrains the listing context to that gallery

subtree rather than searching the whole catalog and hiding unrelated matches afterward.

Tags provide another route through the archive. A *night photography* tag can connect Prague, London, and Tokyo even when those photographs live in separate travel galleries. Breadcrumbs show the current place in the hierarchy, and favorite shortcuts place important galleries directly in the site header.

16 Media and gallery administration

16.1 Gallery inventory and quick controls

The Admin Galleries workspace shows direct photographs, descendant photographs, and subgallery counts. Compact icons show each gallery's own feature settings; hover or focus to identify the state. Click Maps, File names, Voting, or Picture Game to save the selected gallery immediately. Maps cycle through inherited, on, and off; inherited descendants follow their parent. An inherited OFF is a muted icon; an explicit OFF is crossed out and turns orange when it overrides an inherited ON. The background icon shows state only. Picture Game requires voting: enabling the game enables voting, and disabling voting disables the game. Stored votes and game results remain intact. Use bulk actions for an explicit selection of galleries. If a save cannot be confirmed, check the reloaded state before trying again.

Quick password controls affect only the current gallery's own password. Enter a new password to enable or replace it, or remove the own password. Share-token protection, ancestor protection, visibility, and unrelated preferences stay in force. An outdated revision is refused. Ordinary forms remain available without JavaScript; panel actions complete in place and keep the panel open.

On the public home, signed-in administrators can create a root gallery. The first-gallery prompt requires a confirmed empty catalog. Creation groups content and location/visibility, with secondary settings disclosed; source language, tags, SimBrief drafts, and remembered defaults remain available.

16.1.1 Upload preferences and mobile connections

Open **Settings > Uploads** for format/rename preferences, browser workers and limits, ZIP batch tuning, thumbnail source chunks, and mobile WebDAV connections. Related limits are normalized together; size controls display MB. Enable the legacy upload-menu preference only if those original links are useful. Hidden links do not disable the existing routes.

Mobile actions refresh their own workspace. Copy a newly issued one-time password immediately. If the following connection-list refresh fails, the connection remains saved and the password stays visible in that response; retry the list separately. Disabled mobile uploads do not read the connection inventory.

16.1.2 Appearance, language, and custom stylesheets

Appearance combines grouped controls with a live preview and a bounded pointer/keyboard resizer on wide screens. Pane size is a local browser preference. Site images groups branding/background tools; Layout separates shortcuts, grids, card details, and viewer defaults. Language has Settings, Design, Editor, and Diagnostics. The design preview uses that form's offered languages and public default; changing a sample does not save a language preference.

All five Appearance subsections use compact translucent groups and question-mark help beside their controls. The shared live preview and its browser-local divider remain available. Reload and Back/Forward preserve the active subsection, including existing deep links. GPS pin/background, display-all-tags, tag scrollbar and header transparency switches save immediately and individually with factory-default tooltips. Other colors, text, selections and numbers remain drafts for **Save theme**. An unverified or refused switch write locks that control and blocks shared-form replay until passive reload, without automatic retry. Ordinary form saving remains available without JavaScript.

The Language subsections use compact groups, switches and contextual question-mark help. Admin language, public default and the viewer language list save immediately and independently, without submitting other Theme drafts. After changing the Admin language, use **Reload interface** to refresh labels. An unverified save locks its group and blocks shared-form replay until reload; it is not retried automatically. All language selectors show bundled flags. The active subsection survives reload and browser Back/Forward navigation through the page URL. Switcher design remains a draft applied with **Save theme**; JSON/import and diagnostics retain explicit actions. Native form submission remains available without JavaScript.

Keep the installed custom stylesheet by leaving replacement empty. Explicitly choose a preset or upload a valid CSS file to replace it; an upload takes precedence. Replacement/reset verifies storage first and restores prior bytes if saving the marker fails. If recovery is incomplete, retain the available copy and follow the bounded warning before retrying. Deployment preserves the installation-owned `public/assets/custom.css`.

16.1.3 Card orientation and operational status

Vertical cards place the photograph above the description; horizontal cards place it beside the description. The normal orientation migration preserves existing appearance across saved settings, physical and Smart Galleries, sidecars, imported metadata, and Trash. Back up the database, galleries, and Trash together. If conversion is interrupted, resolve the reported problem and retry; document markers prevent double conversion. Fresh installations keep the vertical default. Tags remain complete without JavaScript; browser expansion and scrolling also work on gallery cards.

Public search defaults on only without a saved choice and when its feature is enabled. Updates refresh versions, notes, and API status together; automatic metadata checks use the saved interval (five minutes by default), and existing background jobs continue independently of new discovery. OurAirports refresh uses weekly freshness and hourly failure backoff. Trash shows remaining retention and safe purge availability. Presentation-schema problems also require action in System Health.

Optional SimBrief loading sits beside Begin/Finish flight, with an adjacent identity gear. Enter one pilot ID or username; only a successful OFP fetch saves the identity. Loading is explicit and never required for recording. The staged plan is consumed by the next session; plans attached during recording belong only to that session. No previous flight plan is automatically reused. Saved flights also offer loading the latest OFP directly into the selected flight, or attaching an archived SimBrief JSON/XML export. A PDF-only OFP cannot supply machine-readable dispatch data. Create a new local PDF version afterwards; existing PDFs and delivery receipts remain intact. Reports include recorded OpenStreetMap route maps independently of SimBrief. An attached OFP adds a separate planned route, planned passengers and ramp fuel. These values never replace measured telemetry. Coordinates come from the OFP navlog and are retained in a bounded, AIRAC-scoped local SQLite evidence cache; ambiguous names are not guessed, and no external navigation subscription is required. Displayed OSM tiles are cached for at least seven days, with attribution and a bounded cache. Missing network tiles leave the route visible on a pale background. Mercator maps clamp polar latitudes; original coordinates remain in the saved raw recording. Online tile rendering sends viewport tile coordinates and the app User-Agent to OSM, without Gallery keys or SimBrief identity.

Both maps fit the complete recorded and planned route with a small margin. The overview omits the redundant note after the phase timeline, avoiding a separate page containing only that note. Detached jump-and-return position blocks are omitted from both maps with a visible notice; raw samples remain unchanged. Distant starting or ending segments and relocations without a local return stay visible.

The main screen offers Load SimBrief or Unload SimBrief. Unload clears the staged/current OFP and its route, including the current session snapshot; older sessions and existing PDFs remain intact. Loading/finalization blocks unloading. Stopped flights no longer show their route on the main control. The SimBrief identity sheet uses the dark app theme, validates input inline and accepts Enter/Escape. A checked loaded state and airport pair identify the staged/current flight's accepted plan; failed reloads preserve it. Flight settings, history, progress and confirmation dialogs open near their owner. Click saved-flight headings to toggle ascending/descending order. Starts include seconds; times sort by UTC instant, counts numerically and names without case. Created/page columns use the newest PDF time/total known pages for flight groups. Expanded flights and multi-selection survive sorting and refresh. Missing pypdf leaves page counts unknown while saved PDFs remain openable; missing ReportLab shows dependency-repair instructions for new rendering. Install requirements into the same Python used to launch the source app. Update progress closes itself only after matching installed version and successful restart; reboot instructions and errors remain visible until dismissed.

Report map backgrounds are composed from native tiles at least 300 dpi at their final PDF size. Recorded tracks and purple dashed OFP routes stay vector; the overview uses a light background and the last map full contrast. New PDF names may include the OFP flight number and departure/arrival ICAO. Existing editions, names and receipts remain preserved. Two planning pages show passengers, cargo/payload, registration, runways/alternate, cruise altitude, cost index, distance/time, scheduled UTC departure/arrival, fuel components and planned weights. Missing fields remain unknown. Timed navlog altitude and fuel can appear beside recorded curves: detected takeoff and captured simulator clocks align them even during simulator-rate changes. Known kg/lb units are converted explicitly; recording gaps break the comparison. Missing time anchors allow a labelled comparison at recorded route positions instead: at least two ordered OFP fixes must match captured legs within 10 NM. Gaps and position jumps still break lines; no takeoff time is invented. Missing matching data suppresses comparison with a visible explanation. OFP values remain planned evidence; no actual passengers, delay score or unrecorded fuel is inferred. Earlier OFP snapshots remain readable; explicitly reload/import them to obtain new fields they did not retain. Map requests stay limited to the displayed viewport with a 45-second network budget and stop on the first provider failure.

An attached OFP adds plan-versus-actual statistics in all four editions while retaining maps, profiles and landing detail. The final repeated-header flightplan keeps every OFP point with planned/actual elapsed time, simulator UTC, remaining fuel, pressure height, TAS, GS, wind and passage distance. Intermediate matches require ordered passages within 2 NM; airport rows use detected takeoff/landing within 10 NM. Missing matches remain unknown. Summary and phase tables distinguish airborne/block time, distance, fuel, weights and estimated delays. More initial fuel is separate from savings.

Vector deviation profiles show time, fuel onboard, load-adjusted net fuel use, pressure height and route separation at projected OFP positions. These planning estimates differ from the ordered final flightplan matches. Time-weighted histograms and bias, MAE, RMSE and absolute-error P95 use original saved samples, independently of graph thinning. P95 uses fixed 1-second, 0.1-NM and 25-ft bins; the overflow bin gives a lower bound. Amber map segments exceed the descriptive 5 NM band; height uses ±300 ft. These are comparison bands, not operating limits or pilot scores. Phase averages include IAS/TAS/GS, V/S and track-relative wind.

New optional pressure-height, parking-brake and Zulu-calendar observations remain separate from host UTC. Height differences require both airport transition references. Estimated OUT/IN requires sustained parking/motion evidence and does not identify a stand. Full flight totals require matched airports, one captured flight and complete relevant data. Gaps, fuel increases, incompatible dates and legacy missing fields suppress affected claims. Old recordings remain readable; explicit OFP reload/import or a new recording supplies new evidence. Earlier PDFs

and receipts remain preserved.

Compact analysis panels pair plan, actual and difference in one cell and group short boundary phases with climb, descent or arrival. Original statistics stay preserved. Green indicates lower time/net consumption or more remaining fuel; red indicates the opposite. Speed whiskers show observed min/max with weighted means; phase range bars compare planned and actual totals. These are descriptive ranges, not confidence intervals. Verified duplicate airport rows may share the same observed endpoint without suppressing valid summaries. Missing values use a dash. Legacy AMSL height is marked with an asterisk and never supplies pressure-height errors; absent simulator UTC leaves an explicit planned-only UTC column. The missing-channel context is explained once.

The main SimBrief control marks its loaded plan; saved-flight details show their own OFP attachment. Finish uses available loaded OFP for the current or sole pending flight. Manual generation uses it for the selected saved flight and creates four new editions without confirmation. With no loaded OFP, existing evidence stays unchanged; neither action fetches a plan or asks for an identity. Partial recordings retain incomplete facts. Success updates status without an OK message box and closes progress after durable completion/key cleanup. Errors stay visible; earlier PDFs remain intact.

The update notice resizes to fit its wrapped status text and complete button row when background checks change the message. Text and action-row backgrounds match the window; the primary **Update** action remains distinct.

The independent uploader source targets 0.8.0. A separately requested installer build will place the installer and matching generated `winapp-update.json` under `winapp/dist/0.8.0/`. The installer and metadata belong to the same hosted release. This build check does not replace acceptance testing of the installed Windows app or the simulator. Existing version folders, including 0.4.0, remain preserved.

Develop integration compares the complete installer inputs with current `main`. Changed WinApp runtime, assets, dependencies or build inputs require a version strictly greater than main's; tests, README and instructions are excluded. Several integrations may keep the same upcoming version while main retains its older version. After publication advances main, another runtime change needs another bump. Admission runs before FULL and again before protected publication; release continues to trust qualified develop.

16.2 Admin workspace

After signing in, an administrator sees private controls for the dashboard, galleries, photographs, theme, tags, maintenance, updates, logs, and account settings. Many edits open in the right-side panel, leaving the gallery visible while one item is changed and saved.[11]

On public pages, anonymous preview/Exit preview, Admin, Test run, Updates, and Logout use compact transparent outlined icon buttons: 32 pixels square with 16-pixel glyphs. Localized names appear as native hover tooltips and remain available to screen readers, including disabled Test run and pending Updates. Anonymous Admin login uses the same treatment, including in anonymous previews. Show gallery map uses a pin icon with its localized tooltip and accessible name. Header and gallery-map targets grow to 44 pixels on narrow or touch screens. Language choices remain governed by Theme settings.

The public gallery itself also exposes contextual Admin controls while a normal administrator session is active. Gallery and photo pencil controls open the same full editors in the side panel.

[11] A full Admin page remains available for workflows that need more space or when browser enhancement is unavailable.

Compact gallery removal uses the recoverable trash workflow when Trash is enabled; disabling Trash returns future gallery removals to the legacy permanent subtree-deletion service. Compact photo removal is deliberately labeled **Remove photo ... from CMS**: that quick public-page action removes the catalog image row and does not run the full source-file deletion service. Use the normal photo/bulk deletion workflow when the original file and generated derivatives should also be deleted. Anonymous-preview mode hides these contextual Admin controls.

The Admin gallery table can filter rows by visibility: all, unpublished, public, or private. Filtering is a browser-side view aid, not an access-policy change. A row hidden by the filter is also unchecked, and the master selection is cleared, so a hidden stale checkbox cannot accidentally enter the next bulk action. The summary reports the displayed/eligible gallery count.

The gallery list searches names locally while typing, ignoring case and accents. Longer words allow one insertion, deletion, replacement or adjacent swap; approximate results are labelled. Search includes collapsed descendants and shows their ancestors as non-selectable context. The status filter still applies to matches. Clearing search restores the saved collapse state. Only matching visible galleries are selected by **Select displayed**; hidden selections are cleared. Reordering is unavailable while search or a status filter is active. The bulk action chooser groups available operations with icons; choosing an action does not execute it until **Apply**. Removal of a selected gallery also affects its subgalleries, as the confirmation explains.

The gallery tree supports per-branch expand/collapse plus **Collapse all** and **Expand all**. It remembers which gallery branches the administrator collapsed. The browser sends the collapsed gallery IDs through an authenticated CSRF-protected action and the application stores that UI state, so returning to the Admin tree keeps the same branches closed rather than expanding the entire hierarchy again. Descendants hidden by a collapsed branch are also unchecked before bulk work. Reordering refreshes the tree controls when a gallery gains or loses children.

16.3 Reliable in-place side-panel changes

When JavaScript is available, a persistent action started in the Admin right-side panel completes in that panel. The drawer stays mounted and open, the address in the browser remains unchanged, and the page is not reloaded merely to show the result. This applies to gallery create/edit/move/delete, image upload/edit/delete/move/reorder/visibility/NSFW/cover actions, scan/import, EXIF date suggestions, thumbnail creation, Media Renamer, Metadata Organizer, Duplicate Photo Detector review/deletion actions, Upload Automation credentials, Gallery Migration target pulls, tag edits, and Smart Gallery create/edit/placement/detach/duplicate/delete actions. A standalone Admin page and ordinary POST/redirect behavior remain available when JavaScript is disabled or the route is opened directly.

Only the newest panel-open request may replace its contents; closing or opening another item cancels ownership by an older response. Keyboard focus remains inside the open drawer and returns to the control that opened it. Unsaved ordinary text can be held in bounded browser memory for the current page and restored when the same form returns. Passwords, tokens, API keys, file inputs, operation identities, and other private values are excluded, and nothing is persisted in browser storage.

The saved database/filesystem state remains authoritative. After persistence, the server describes the changed entity, the panel fragment that may need re-rendering, and every affected public context. Stable gallery, image, tag, and Smart Gallery identifiers are used instead of assuming that the old URL still names the saved object. This matters after a gallery folder/slug rename, gallery reparenting, or tag slug change: the application can fetch the new canonical render source while leaving the visible browser address untouched. Source and destination galleries are both described for moves; old/new parents and the moved gallery are described for reparenting; Smart

Gallery placement changes describe every affected root or physical parent context.

A successful fragment request is not by itself proof that the change is visible. The refreshed server-rendered HTML must satisfy an operation-specific observable condition where one exists. Examples include gallery/card presence or absence and full counts, exact effective visibility, saved gallery revision, image absence/visibility/NSFW state, gallery-wide image revision for off-page changes, cover image identity, persisted image order, tag identity, and Smart Gallery placement/order. Pagination-aware checks accept a directly visible matching card before using aggregate counts, while an off-page result still waits for the authoritative total to converge. Thumbnail cache repair is the intentional exception: it refreshes affected contexts but has no stable database-backed public-HTML condition.

Independent render requests can briefly return stale data on shared hosting. PHP Gallery therefore performs one centralized bounded retry sequence only after the mutation itself succeeded and the declared condition is not yet observable. Each completion receives a new operation generation; a newer mutation aborts or supersedes older fetches, retry wakeups, and panel responses for the same context. Structurally valid but stale HTML is verified before replacement, so an older response cannot overwrite a newer visible state. Refreshes bypass browser caches and preserve applicable pagination, sorting, language, filter, active-tab, drawer-scroll, and page-scroll state.

If persistence succeeds but the refreshed view cannot be verified within the bounded retry budget, the panel reports a synchronization problem and keeps the saved server change. It does not misreport the mutation as failed, silently reload the page, navigate away, or rewrite browser history. Authentication, authorization, CSRF, schema readiness, path safety, and validation failures return an expected structured error to the enhanced workflow and are not retried. Dynamically refreshed controls remain active because the panel uses lifecycle-safe delegated handling rather than bindings that disappear after fragment replacement.

Browser-assisted create/upload has additional safeguards. Selecting browser processing is an explicit execution choice: a missing browser capability or failed worker preparation stops before persistence instead of silently switching to server thumbnail generation. PHP validates that every required prepared thumbnail size/format is present before it stores originals. Once browser persistence starts, direct-page fallback metadata is never interpreted as permission to replay the classic create/upload request, preventing duplicate galleries and duplicate photographs. The configured ZIP batch size is a soft packing target: one atomic photograph package that exceeds that target can travel alone in one batch, but the effective PHP upload ceiling remains a hard limit. Windows upload automation may send the complete JPEG+WebP compatibility matrix; a modern WebP-only server accepts required WebP variants and safely skips valid extra JPEG variants, while malformed, mismatched, unknown-size, or unsupported thumbnail payloads remain hard failures.

16.4 Admin navigation

The desktop Admin menu starts expanded, with its existing function groups and icons. Use the double-chevron button at the top to collapse it to an icon rail or expand it again. The choice is remembered in this browser for this installation; it is not stored in the database or shared with other browsers. Clearing browser storage restores the expanded default. If storage is blocked, the choice lasts only for the current page. Both desktop states scroll with the page, without a separate menu scrollbar. Scroll the page to reach the last account actions.

On narrow screens, a labelled hamburger opens the same groups below its button. The panel scrolls independently and overlays the page, so you can reach content without first scrolling past the whole menu. Close it with the button, Escape, a click outside, or by choosing a destination. Keyboard focus returns to the button after Escape. Moving focus outside also closes it. Opening another page starts with the mobile menu closed; returning to desktop restores its separate

expanded or collapsed choice. Without JavaScript, all grouped links remain visible.

16.5 Central Settings hub

The Admin navigation includes a central **Settings** page for site-wide configuration. General now groups the website address, site name, Admin/public languages and navigation preferences. The other sections cover Public appearance, Content, Media and browsing, Uploads and automation, Privacy and diagnostics, and Advanced. Existing Website address links still reach General. Stable section links and ordinary forms remain available without JavaScript.

With JavaScript, sections load when opened and save in place. A sticky save bar shows changed settings and offers saving or reverting the current draft. Advanced summaries remain collapsed until needed. Address or Admin-language changes offer an explicit link to the new address or language. The dashboard first displays its shell, then loads Overview totals or the Galleries list as needed; failed reads offer a retry. Overview counts indexed originals rather than scanning files or resolving gallery covers.

A Spotlight-style field searches settings as you type. It matches names, descriptions, categories, and technical names, including controls that are owned by specialist Admin pages. Selecting a result opens the relevant section or specialist page and focuses the matching control. Arrow keys, Enter, Escape, and the clear button can be used without leaving the keyboard.

Frequently used global settings can be edited directly from the hub, including the site name, public language, URL rewriting, public search where available, thumbnail rendering, the global EXIF/GPS display default, and development diagnostics. Other settings remain on their specialist pages when they need more context, credentials, file uploads, destructive actions, or larger editors. Examples include Theme layout, gallery-scoped upload API keys, telemetry, account security, Google/OpenAI configuration, custom CSS, branding, language packs, and database maintenance.

The hub does not flatten existing inheritance. Tag-page layout can continue inheriting the global grid and card design, while gallery-specific card, lightbox, and EXIF/GPS choices stay with the gallery that owns them. Context-specific gallery and photo values are not turned into global settings.

Secrets are never shown in search results or summaries. Passwords, tokens, API credentials, and similar values appear only as safe states such as Configured, Not configured, or a link to their specialist page.

16.5.1 Guided Setup Wizard

Choose **Setup Wizard** at the top of **Settings** to start or resume a guided configuration draft. The wizard follows the same eight sections as the hub. Short subsections keep related choices together; **Advanced** and **Expert and specialist settings** reveal less common options. With JavaScript disabled, all subsection content remains readable and normal form navigation still works.

Each supported setting has an explanation and an example. Keep its inclusion checkbox selected to stage a value, or clear it to preserve the original setting. **Skip** preserves the whole section. **Next**, **Back**, and subsection navigation do not save application settings. Supported Theme choices share the existing live preview; a preview is temporary and does not change the public site.

The wizard supports safe central Settings values, basic Theme colors and layout, grid/card and hero-tag options, bounded upload and telemetry preferences, thumbnail warm-up, SEO guard, Gallery Trash, automatic updates, and Scheduled Maintenance preferences. Credentials, API-key

actions, uploaded branding, raw CSS, gallery-storage relocation, and destructive maintenance remain informational. Their specialist pages are not opened from the draft; leave the wizard before using those workflows. Per-gallery overrides remain unchanged.

The final review shows proposed changes first. Expand the unchanged details or use **Show all settings** to inspect preserved and informational items. Select the approval checkbox and apply only after reviewing the summary. **Cancel** or **Restart** discards the draft. A draft belongs to the signed-in administrator and the current browser session; it is not a shared configuration document.

Saving validates the current settings again. If another session changed an affected value, the wizard refuses to overwrite it. Required database storage must be verified; missing or unknown storage blocks the affected save. Application and telemetry rows use a transaction, while website-address changes have a reversible configuration write for failure recovery. Trash and Scheduled Maintenance preferences are saved through their existing owners as coherent groups. Secrets are never placed in the draft, summary, or wizard log.

16.5.2 Correcting the website address

Open **Settings > General** to correct the public installation URL after setup. Enter an absolute HTTP or HTTPS address such as `https://example.com`. Include a path only when the gallery is actually installed in that subdirectory. If hosting detection added an unwanted directory, remove it from this field and save.

The save changes only the top-level `base_url` string in `config.php`; it does not create or change a database setting. Other configuration values, comments, and formatting are preserved. Both the file and its containing directory must be writable. The application rejects addresses with credentials, query parameters, or fragments, and refuses to rewrite duplicate or computed configuration values. Edit a custom configuration manually when it cannot be updated safely.

After a successful save, the browser opens the same Settings section at the new address. Subsequent requests use the updated configuration. This setting corrects generated addresses; it does not move gallery files or configure DNS and web-server routing.

16.6 Tag metadata administration

The editor uses compact translucent groups for the tag catalog, selected tag and usage references. Groups fit beside one another on wide screens and stack on narrow screens. The arrow beside the catalog heading jumps to the editor below sticky navigation. Filter and sorting choices switch immediately through the native GET form. Apply remains available without JavaScript. A question mark beside each field explains its meaning in place; Escape or an outside click closes the help. Review content edits and use **Save tag**. Music tags retain their internal identifiers and song references without public tag-page actions.

Images and **Galleries** filter direct assignments over the same visual tag vocabulary. A shared tag can appear in both; its edit still affects every assignment. Counts and usage sorting follow the selected scope. **All tags** includes unassigned tags. **Documents** is a disabled placeholder for future PDF tagging; direct requests explain that document editing is not yet available.

Admin > Edit tags maintains the shared tag vocabulary. Selecting a tag exposes its canonical lowercase name, stable identifier and description. For visual tags the identifier forms the public tag URL and the description can appear on that page; music tags use internal metadata. Usage references link to the related public gallery, photo editor or song editor. Counts in the catalog

follow the selected usage filter rather than always combining scopes.

Renaming a tag or changing its slug updates the shared tag record, so every gallery/photo assignment continues to reference that one reusable tag. Names and slugs are normalized to the safe tag convention on save and a duplicate slug is rejected. Deleting a tag is different: the application transactionally detaches it from both galleries and photographs and then removes the tag record. Use **View public tag** to review the landing page and **Configure tag display** to open the Theme controls for tag-page/card and hero-tag presentation.

16.7 Feature switches

The specialized destinations for public live search, lightbox browsing, Smart Galleries, gallery PDF documents and gallery music use compact, translucent controls. Central Settings shares this treatment across categories; Smart Gallery editors retain it inside the side panel. Preference switches keep their native keyboard operation, with larger tap targets on narrow screens. Settings question marks open help beside the control; Escape or an outside click closes it. Each specialist retains its existing Save workflow; catalog row selections remain ordinary selection boxes.

Dashboard **Maintenance > Content and display** uses a denser layout: one toolbar and compact groups flow into one to three columns according to available width, retaining each group's own height. Groups match Features: quiet outlines, translucent surfaces, tinted headings with a left accent and thin row separators. Question marks retain the explanations; counts and warnings stay visible. Switches save immediately without confirmation or a Save button; their tooltips state the factory default. Saving, success and failure remain visible. Unverified completion requires a state reload instead of replaying the write. Without JavaScript, native help disclosures and the existing Save/POST forms remain available.

This is the default pattern for new or revised Admin preference interfaces, specified in `docs/ADMIN_DESIGN.md`. Use switches with immediate saving, compact headed groups and one to three independently sized columns. The reference uses a preferred column width of 300 CSS pixels, 38-pixel heading strips and 26 by 14-pixel switch tracks; narrow/touch interaction targets are at least 44 pixels high. A setting with its own configuration page has a clickable name and a trailing settings arrow leading to the same destination. Name, switch and question mark remain independent navigation, preference and help controls. Apply the pattern as each owning interface is edited; older forms may still await migration.

Appearance > Features presents optional application features in compact grouped tables. Whole groups flow into one, two or three columns according to the available workspace width, reading down each column before the next. Groups keep their own heights and are never split between columns; narrow pages use one column. Switches sit directly before feature names and save immediately, without a second confirmation or Save action. The tinted group header switches every editable feature in that group; a mixed switch indicates that only some are configured on. Linked feature names open their specialized settings where available. The plain **?** opens contextual description, configured/effective state, dependencies and technical details; Escape or an outside click closes it.

Each row shows its factory value and whether the configured preference differs. Factory means the explicitly declared new-installation default, otherwise the registry default; it can differ from an upgrade compatibility default. **Reset to factory defaults** above the table immediately saves all editable switches to those factory values. Feature data and specialized preferences are preserved. A configured switch can remain on while a disabled dependency makes the feature unavailable; enabling that dependency restores availability.

Saving, saved and failure feedback stays visible. A group write can partially succeed if a domain

owner refuses a later setting; the table then displays the authoritative persisted state. If a response is lost, **Reload state** reads the server before permitting another change and never repeats the uncertain write. Without JavaScript, the complete form retains an explicit Save action and factory reset uses a normal authenticated POST.

A disabled feature is hidden from normal navigation and its guarded routes refuse the feature workflow. Disabling a switch does not delete the feature's code, photographs, database records, credentials, or configuration. Re-enabling it therefore restores the normal entry points subject to the same schema, access, and configuration checks as before. Examples of switchable features include public search, lightbox browsing modes, Picture Manager, Inline Administration on public pages, Smart Galleries, voting, Picture Game, downloads, gallery and flight maps, navigation data, SimBrief, OpenAI assistance, local AI metadata, API uploads, mobile WebDAV, gallery migration, Media Renamer, and anonymous telemetry.

16.8 Gallery management

Gallery administration covers the full life of a collection: creation, discovery, naming, description, dates, tags, privacy, passwords, branding, covers, layout, ordering, moving, and eventual deletion. A child gallery can inherit useful choices from its parent. For example, every gallery inside a private family branch can follow the parent's protection, and an individual child can use its own presentation choice where the editor offers an override. The bulk gallery screen provides corresponding multi-gallery operations where the same change is appropriate for several collections at once.

Gallery descriptions can present safe external links written as ordinary Markdown links, `[link=URL]label[/link]`, `[url=URL]label[/url]`, or the short forms whose visible text is also the target. Only HTTP and HTTPS targets are linked; other schemes remain inert text. Recognized services use bundled local brand symbols. For other public hosts, saving the gallery can retrieve a small validated favicon into the local gallery cache. Retrieval is hostname-deduplicated, bounded by size, time, redirect, and retry limits, rejects private or reserved network destinations, and never occurs while a visitor renders the public description. If the cache or its database table is unavailable, the link remains usable without an icon.

Link targets retain underscores and query parameters. Emphasis in labels and literal inline code remain intact, and links stay in the surrounding paragraph.

When clean public-path storage is available, **Regenerate clean public paths** rebuilds the stored gallery and image paths from the current hierarchy and reports separate gallery/image counts. Reordering and selected editor saves also refresh paths when their changes require it. This maintenance action is useful after hierarchy repair, migration, or older imports; it does not turn URL rewriting on by itself, so query-string routing remains the fallback when rewriting is disabled or unsupported by the host.

16.9 Metadata organizer

The gallery editor includes an **Organizer** workflow for turning a flat set of direct photographs into date-based child galleries. The current implementation groups by EXIF capture date already stored in the image database. Location-based secondary grouping is reserved for future use and is not an active organizer strategy in Version 1.1.2.

Choose minimum and maximum EXIF dates, then build a preview. Previewing reads database metadata in small batches and does not scan, rename, or move source files. The draft shows the number of direct photographs, matching candidates, proposed child galleries, and photographs

ignored because they have no usable EXIF date or fall outside the chosen range. A minimum date is particularly useful for excluding photographs from a camera whose clock was unset.

For each accepted calendar date, the proposed child folder name is the sanitized source title. Its display title uses the gallery date-display formatter and falls back to a human-readable day/month/year value if that formatter cannot supply a label. Before proposing a new child, the organizer looks for an existing direct child with the same organizer title and reuses it. This makes a repeated preview/apply cycle append to the same daily destination instead of creating another equivalent day gallery.

Applying the draft requires a separate confirmation. PHP Gallery creates or reuses the proposed date child galleries and moves matching photographs through the same physical move service used by the ordinary bulk image mover. Originals and known generated derivatives therefore move together with the database ownership update. Large applies are split into bounded browser-driven requests instead of one long PHP request. Review the draft and maintain a backup before reorganizing a large gallery.

16.10 Media renamer

Choose the scope, set the filename pattern, then build the preview before confirming any physical rename. The compact gallery list supports local search without changing checked selections; it is shown for the checked-gallery scope. For a single gallery, use the dropdown. Question-mark controls reveal detailed help. The preview shows summary counts and expandable gallery groups; open each group to review its old-to-new mappings and statuses before applying. Search terms are combined with AND regardless of their order; enclose a phrase in double quotes to match its word order exactly. Visibility filters refresh the gallery selector without generating a rename plan. Build the plan explicitly with Preview rename plan. Empty galleries occupy compact summary rows; galleries with file mappings expand within a bounded result list. A visible progress bar accompanies requests: list refresh and preview show indeterminate activity, while batched checks and renames report measured progress.

Galleries > Media renamer, and the renamer panel available for an individual gallery, can normalize physical image filenames from gallery context and the current photo order. This is a filesystem operation, not a cosmetic title change, so the tool always begins with a dry-run plan. The preview lists each old and proposed filename and marks files that already match, are missing, would collide, or must be skipped by a safety rule.

Only direct image files inside the selected gallery folder are eligible. Missing sources, paths outside the gallery, and unsafe targets are not renamed. If the preferred target name is already occupied but a safe deterministic suffix can resolve the collision, the preview shows that adjustment. Applying a plan requires checking the review confirmation beside the preview. This is the single confirmation for gallery and site-wide applies, enforced by the browser and the server; no additional blocking dialog follows.

The default pattern is `\{gallery_path\}_\{seq4\}`. A custom pattern may use `\{gallery_path\}`, `\{gallery_title\}`, `\{photo_title\}`, `\{old_name\}`, `\{old_stem\}`, `\{image_id\}`, and sequence placeholders `\{seq\}` through `\{seq5\}`. The numbered sequence variants pad to two, three, four, or five digits. Textual components are normalized to lowercase filesystem-safe ASCII-style underscore stems, the source extension is preserved in sanitized form, and an empty custom pattern falls back to the default.

After a successful physical rename, PHP Gallery keeps the database row and public path references aligned, updates derived titles where the renamer owns them, moves or invalidates generated derivatives as appropriate, and removes stale cached ZIP archive records that refer to the old

source identity. Derivative cleanup warnings do not justify reverting an otherwise completed source rename because reproducible derivatives can be generated again. For site-wide selections, the browser advances the work in bounded portions and shows the result for each gallery.

Gallery and site-wide applies use requests of at most 15 images. The bar and completed/total counter advance only after server confirmation; an incomplete batch stops without reporting completion. Settings drafts remain in place. Progress uses the Admin language. The renamer's own controls remain accessible on narrow screens without the settings Save footer covering them. Exclusively owned JPG/WebP thumbnails move through temporary staging and retain their exact bytes. They become available under new names only after the new database identity owns them. Shared, unverified or locked cache files retain explicit invalidation/warning handling; originals never acquire another image's cached pixels. Results count preserved thumbnails and removed stale derivatives separately in one contextual status message. The compact retry action reloads existing thumbnails.

Temporary staging uses a short internal filename independent of the original name, so valid long source names remain usable during a rename. Final filenames keep the readable naming pattern. Sequence placeholders count the complete gallery order, including photographs outside the current batch. Changing the gallery title preserves those positions; later uploads continue their numbering after the existing photographs.

16.11 Image management

Photographs can be uploaded in the browser, copied into folders and discovered, received through configured automation, transferred from another compatible installation, or added through a scoped mobile workflow. After a photograph arrives, the gallery records its size and available camera information and prepares the smaller display copies used throughout the site.

The individual photo editor currently covers title, description, maintained-language variants, visibility, sort order, tags, NSFW / 18+ state, the private editorial rating used by Smart Gallery rules, optional per-photo responsive-thumbnail bounds, and read-only EXIF/GPS details. Where local AI metadata exists, the editor shows the generated model/source, searchable internal text, and raw metadata JSON separately from the public caption. OpenAI controls insert reviewable drafts into editable text fields and do not replace saved editorial text automatically.

Bulk image operations are intentionally narrower than the individual editor and perform server-side ownership checks again. Current bulk actions include:

- move selected photographs to an existing gallery;
- create a new destination gallery and move the selection into it, with cleanup of an empty newly created destination if the move fails before files are committed;
- delete selected photograph records together with their owned originals and known generated derivatives through the normal deletion service;
- set a selected photograph as the gallery title/cover image;
- change photo visibility among the supported draft/public/private states;
- turn per-photo NSFW protection on or off when its schema is known;
- create thumbnails for the selected photographs.

Photo ordering can be changed by dragging the row handle, and the gallery can be sorted by filename or EXIF capture date. Photos without a capture date stay last, and changes are saved

through the normal ordering route.

When opening **Edit gallery** in the right Admin drawer, ordinary settings remain usable while photos load in the background. A thin progress strip counts processed thumbnails; **Images** stays unavailable until the complete list and all thumbnail requests have settled. Failed thumbnails receive placeholders and an explicit **Retry images** action; settings can still be saved. Only the small table thumbnails are preloaded. Large previews remain on demand, and the File renamer prepares its plan when its tab is opened. Closing or replacing the drawer cancels obsolete work. Ordinary full-page navigation remains the complete fallback for browsers without JavaScript.

The gallery editor's **Images** tab keeps common photo work in one compact list. Accessible thumbnail previews can be opened by pointer, keyboard focus, or click; row selection supports a Shift-click range. Each row provides direct controls for visibility, choosing the gallery title picture, editing, and deleting that photo. These actions continue to use the existing authenticated server workflows. The **Show file names** option changes only the editor display; administrators can save an optional browser-wide default, while each gallery remembers its choice for the current editor session.

The title-picture star has three states when the database supports multi-photo covers: an outline star selects that photo alone (gold); clicking the gold star again keeps it selected and arms accumulation (blue); clicking another outline star adds it to the selection, up to four photos. Clicking any selected blue star keeps only that photo (gold). Each click saves immediately. The right drawer updates the stars in place, preserving its open tab, scroll position, other form drafts, and loaded image rows; the public gallery preview refreshes separately. After a lost or failed response, the editor first reads the authoritative cover state without changing the panel. If the state changed, it reconciles the stars before refreshing the public gallery; otherwise read the current state before retrying, because persistence may have succeeded before response construction failed.

Under **Media management** > **Title picture**, **Automatic (2 photos)** chooses up to two suitable direct photos and fills missing positions from accessible child galleries. The original/default mode preserves legacy behavior for galleries that have not explicitly selected Automatic or a manual cover. Public cards recheck current visibility and gallery access; unpublished or inaccessible photos are not revealed.

When the optional **Picture Manager** feature is enabled, a normal signed-in administrator can select photographs and direct physical subgalleries in the public gallery grid. Checkmarks select individual cards; Shift-click selects a range; Ctrl-click or Cmd-click toggles a card; **Select all**, **Clear**, and Ctrl/Cmd+A support larger visible-page selections. Smart Gallery cards are not part of this file-based selection. Dragging an unselected photo starts with that photo as the selection, while dragging an existing photo selection moves the selected photographs together. A visible child-gallery card becomes a drop target, and dropping there performs the same validated photo move as the destination picker. Physical gallery cards themselves are not drag sources.

From the toolbar the administrator can delete a mixed selection or create a real physical gallery from it. Creation accepts a searchable parent-gallery choice, copies selected photographs into the new root, and copies selected physical subgallery trees below it while leaving the sources unchanged. The server writes current gallery sidecars, rejects recursive destinations and existing target folders, reserves each target root before copying, rebuilds copied gallery rows from the filesystem, and removes the new partial result if the operation fails. Mixed deletion removes selected photographs through normal image cleanup and applies the configured recoverable Trash policy to selected gallery trees when available.

Move and copy-to-existing-gallery remain photo-only actions. If a physical gallery is selected, clear it before using those controls. Photo selections can also be shared or downloaded. **Share** fetches the authorized browser-displayable image versions as browser `File` objects and opens

the native Web Share sheet when the browser/device supports multi-file sharing. The browser chooses the available receiving apps, so PHP Gallery cannot force Instagram Story, Reel, or another specific target. If native sharing is unavailable, cannot accept the files, or fails after preparation, the manager starts the authenticated selected-photo ZIP download as a fallback. The same ZIP endpoint can therefore serve both an explicit selected-photo download and share fallback. Ordinary visitors never receive these controls, and the server revalidates the logged-in account, CSRF token, source ownership, direct-child gallery ownership, and destination gallery for every mutation or ZIP request.

16.12 Uploads and discovery

Filesystem discovery is useful when photographs have already been copied to the server through FTP, a hosting file manager, or a restored backup. The discovery screen shows what the application found and lets the administrator bring those folders and files into the catalog. Discovery runs as a session-backed browser job in small AJAX batches rather than scanning the entire tree during the initial Admin page request. The current default examines up to 80 directories per batch, accepts a bounded batch size up to 300, and expires abandoned discovery job state after two hours. This keeps a large folder tree responsive on slower hosting while still showing progress and a final actionable result list.

The scanner walks below the configured gallery root and does not follow directory symlinks. Hidden directories and known generated/cache directory names such as `cache`, `thumbs`, `thumbnail`, `thumbnails`, `preview`, and `previews` are ignored. A new candidate branch must contain at least one supported image somewhere below it. Empty structural parent folders can still become galleries when they are needed to preserve the hierarchy above image-bearing descendants. A folder containing only a `gallery.json` sidecar and no supported image anywhere in its branch is reported as metadata-only rather than imported as a gallery by the current discovery workflow. Already indexed gallery folders are suppressed from the candidate list, and an import candidate whose display title would duplicate an existing sibling title is not silently created.

For each unmanaged result, the administrator can import the folder tree in place, move supported photographs from selected unmanaged folders into an existing gallery, or delete selected unmanaged folders from disk. Import expands the selection in parent-to-child order and can include missing image-bearing ancestors and descendants. The move action avoids overwriting an existing destination filename by choosing a deterministic unique name, scans the destination gallery after successful moves, and removes source directories only when they become empty. The discovery-specific disk-delete action is deliberately restricted to safe non-root unmanaged paths below the gallery root and refuses folders that are already represented by database galleries. Use the normal gallery deletion workflow for an imported CMS gallery.

16.12.1 The gallery.json metadata sidecar

PHP Gallery keeps its active catalog and relationships in the database, while an optional `gallery.json` file beside a gallery folder preserves useful editable metadata with the filesystem. Normal gallery saves write the sidecar back to the gallery folder. Current sidecar output includes the title, description, normalized gallery tags, visibility, order, voting and filename-display state, maintained content language/translations when available, manual gallery dates, selected card/count/lightbox/grid overrides, responsive-thumbnail bounds, access/listing policy, cover references, and configured gallery branding assets where those schemas are available. Sidecar writing follows the same schema checks as the corresponding gallery features.

When Admin tag sidecar repair runs, it refuses targets outside the configured gallery root or

targets it cannot verify; links resolving inside remain usable. A refusal or failed sidecar write does not record its completion checkpoint. The writer refuses multiple hard links or unusable inode identity and checks the opened target and prior content before truncation; a later write failure or interruption can leave partial content. The whole JSON document is decoded and re-encoded, so formatting and number representation may change. These checks do not exclude unrelated filesystem changes between observations.

During discovery and missing-parent repair, PHP Gallery can read supported values from the sidecar instead of deriving everything from the folder name. Imported rows conservatively default to unpublished when no visibility is specified. The current import path restores the sidecar fields explicitly supported by the importer, including title/description/localization, visibility, manual dates, voting/filename presentation, selected card/count/lightbox/grid/thumbnail overrides, access/listing policy, order, and branding paths. The sidecar should therefore be treated as portable gallery-folder metadata, not as a complete database backup: relational state, credentials, logs, votes, game state, Smart Gallery definitions, and other database-owned records still require the normal backup procedure.

> **Security note.** The **Delete from disk** action on the discovery screen is a real recursive filesystem deletion for unmanaged candidate folders. Review the selected relative paths before confirming it. The guard against known imported gallery paths is an additional safety boundary, not a replacement for a backup.

Browser upload is the friendliest choice for routine work. It can upload into an existing gallery or create a child gallery and upload optional photographs in the same side-panel workflow. Select the destination, choose files, and follow the progress display. The application checks the destination and file information before accepting each result.

With JavaScript enabled, focus the upload area and paste an image or screenshot using Ctrl+V on Windows/Linux or Cmd+V on macOS. The focused or most recently used visible uploader receives the images; an open side panel owns its upload target. Paste appends all supported image items exposed by the browser to the same file selection used by the chooser. JPEG, PNG, GIF and WebP follow the picker policy; HEIC, HEIF and DNG additionally require an explicit server-capability hint. Browser-provided filenames are retained; unnamed image data receives a distinct timestamp/sequence name with the matching extension. Submit the ordinary form to upload through the selected browser/server pipeline, with its usual validation, size limits, metadata, thumbnails, progress and errors. Text fields keep ordinary text paste; unsupported content and paste during an active upload are ignored. If the browser cannot expose or add clipboard images, use the chooser or file drop. Side-panel uploads retain the open panel and current URL.

In the right-hand upload drawer, repeated paste, file-picker choices and drop append to one local preview queue. Inspect its count and total size; use Remove, Clear selection or the arrow buttons to adjust the FileList and upload order. Then press the ordinary Upload button once. Both existing-gallery and create-with-upload drawers use this queue; the dedicated full-page picker keeps its established behavior. The queue does not create a server draft and has no Commit publication action.

Preview work is lazy and serialized: one decoder, at most twelve miniature resources, and PNG miniatures no larger than 256 pixels per edge. Before decoding, only the first 256 KiB of headers are inspected; a source must fit within 24 MiB, 8192 pixels per edge and 16 * 1024 * 1024 pixels overall. Unsupported, animated, unreadable or larger candidates show an icon or warning while the original remains selected. These preview limits do not replace server upload limits. Selection never strips EXIF or replaces the original with a miniature.

Repeated filenames receive a warning but remain separate selected occurrences; removing a duplicate updates that warning. Empty files, unavailable previews and format/size warnings do not erase healthy files. A disabled file input also blocks clipboard and drop additions. The displayed total describes the local selection: classic uploads send one image per request and browser preparation follows its configured batch limits.

Closing or switching the drawer offers an explicit choice for unsent files; Escape cancels that warning and restores focus. Native Reset clears only an unsubmitted selection. After an uncertain or partially failed upload, removal, replacement and Reset are blocked to preserve the exact retry inputs and operation keys. Retry from the same form; only canonical acknowledgment clears the queue. Navigation releases preview resources. A browser history-cache return may restore page state, but an ordinary reload does not retain unsent files.

Compressed file size alone does not prove that an image is safe to decode. Before GD or optional Imagick allocates a full raster, PHP Gallery checks the format, dimensions, pixel arithmetic, configured/runtime memory budget, and pipeline limits. A file that cannot be admitted safely is refused before its original or generated derivatives are committed to the target gallery. This policy applies to classic and browser-prepared ingestion; it does not claim that every external RAW decoder has the same in-process memory model.

The upload settings distinguish the ordinary server path from browser-assisted preparation. **Allow all server-supported formats** leaves the picker aligned with the server's detected capabilities. **Prefer phone-rendered JPG/PNG/WebP, no RAW/DNG** asks mobile browsers for a rendered image representation when possible, which is useful when an iPhone ProRAW/DNG workflow produces unsuitable color. The browser hint is not a guaranteed transcoder, so the server still validates the actual uploaded media.

Browser-assisted preparation is enabled independently and is checked by default in the upload workflow. The browser can prepare optimized thumbnails and bounded ZIP batches before sending them. If browser preparation fails before any server-side write starts, ordinary image selections can fall back automatically to the classic server upload path. Administrators can bound the default and maximum browser worker counts, images per batch, ZIP-to-PHP-upload-limit ratio, maximum ZIP batch bytes, and source chunk size used by browser-side thumbnail rebuild. The normalizer also enforces a hard worker cap so an accidental setting cannot create unbounded parallel requests on shared hosting.

Server-side acceptance of browser-prepared ZIP batches also verifies the CRC-32 of every stored file and requires a complete central directory and end record matching entry names, offsets, sizes, and counts. A damaged or inconsistent prepared batch is rejected before gallery files are written; rebuild and retry the batch in the browser.

Current browser-pipeline defaults are 8 workers, an 80% target of the detected PHP upload limit for each store-only ZIP batch, at most 8 images per ZIP batch, and a 24 MiB absolute ZIP-batch cap. The validated ranges are 1–32 workers, 10–95% of the PHP upload limit, 1–64 images per batch, and 1–128 MiB for the absolute ZIP cap. Browser thumbnail rebuilding uses a 512 MiB source-ZIP chunk by default, clamped to 16 MiB–3 GiB, and the source builder normally caps a chunk at 96 items with an internal hard limit of 512. These are safety ceilings and defaults, not a recommendation to raise a slow shared host to the maximum.

When browser-assisted upload is enabled, the chooser also accepts ZIP archives such as an iCloud Photos export. The browser inspects the archive locally, adds supported JPEG, PNG, WebP, and GIF images to the ordinary preparation queue, and skips folders, hidden metadata, and unsupported entries. The selected ZIP itself is never uploaded to PHP. Stored and Deflate archives are supported; encrypted, multi-disk, ZIP64, damaged, suspiciously compressed, or excessively large archives are rejected before any server-side upload begins. ZIP import has no classic-PHP

fallback, so keep browser-assisted preparation enabled when selecting an archive.

The global automatic-upload rename option can run the same post-scan naming policy used by the Media Renamer after browser, API, or WebDAV ingestion. Leave it off when external filenames are part of your archival convention; enable it when all arrival paths should converge on the gallery's normalized naming scheme.

16.13 Generated metadata and automation

The optional local or worker-based image-analysis queue can record visual metadata separately from the human title, description, tags, and EXIF fields. Administrators can review that metadata in the photo editor and use it as searchable evidence without silently replacing editorial text. Configured OpenAI assistance can generate a photo or gallery description, clean up existing text, summarize context, describe visible content from safe generated thumbnails, or suggest a translation. These actions insert reviewable drafts; they do not publish or save text automatically unless the administrator completes the ordinary save operation.

For repeated transfers, create a scoped upload-automation token for the intended gallery and send supported files to the upload endpoint. Tokens can be revoked independently and must be treated like passwords. The raw API key is shown only when it is created; only its hash is stored afterward, so copy the new key to the intended client at creation time. The global **API manager** lists automation credentials across galleries and provides a central revocation workflow without exposing the original secrets. Gallery-level controls remain available for the same scoped credentials.

The public discovery query at the installation-relative address `index.php?page=app_linking_discovery` reports version 1 and checks the canonical HTTPS issuer, independent secret, core schema and optional capability readiness. If core linking is not ready it reports unavailable; unknown optional storage hides only that operation. It does not require an Admin session or reveal credentials or gallery lists. The existing gallery-scoped API key remains independent; discovery does not authorize uploads.

The Windows companion uses the same gallery-scoped upload contract. It has two normal upload modes. **Watch folder** ignores files that were already present when watching begins, waits for new files to become stable, remembers confirmed uploads locally, and retries transient failures with backoff. It can optionally attach the current Microsoft Flight Simulator camera or aircraft latitude/longitude/altitude through SimConnect immediately before each watched upload, and can delete a watched source only after the gallery confirms a successful upload. **Manual upload** accepts a file selection independently of the watch folder and can either generate responsive thumbnails on the PC with worker processes or ask the gallery server to generate them. Thumbnail work and network uploads are pipelined rather than staging an entire large selection first. Tray support lets the watcher or manual job continue while the main window is hidden.

The same Windows application can optionally run a low-priority AI metadata worker. It claims one authenticated analysis job, downloads only the claimed asset, produces internal searchable metadata with the selected local backend/model generation, and returns the result to the existing upload-automation endpoint. This worker is separate from human captions. Stopping waits until any in-flight heartbeat request actually ends; blocked HTTP or DNS can extend this wait beyond the socket timeout. A gallery-level **Force AI metadata regeneration** action clears the stored generated rows/old queue state for that gallery branch and queues fresh work; the heavy analysis still runs on the Windows worker.

Mobile WebDAV credentials are gallery-scoped and use HTTP Basic authentication plus an unguessable path token. Creating a connection shows its generated password once; the server keeps only a password hash afterward. The Admin page lists the connection label, target gallery, server URL,

and last-use time and can revoke the connection. The WebDAV endpoint supports the minimal `OPTIONS`, `PROPFIND`, `MKCOL`, and `PUT` behavior needed by photo-transfer clients such as PhotoSync. Current WebDAV ingestion accepts JPG, PNG, GIF, and WebP. Configure the mobile client to convert HEIC to JPEG before transfer where available. Uploaded media then enters the same scan, optional auto-rename, database, and thumbnail pipeline as other uploads.

17 Thumbnail generation and public rendering

17.1 What a thumbnail is

A thumbnail is a smaller display copy of a photograph. The original photograph may be several thousand pixels wide and occupy many megabytes. A gallery card on a phone may need only a few hundred pixels. Sending the full original for every small card would make visitors wait for detail they cannot see at that size.[12]

Imagine a gallery containing 60 photographs, each with a 12-megabyte original file. Loading every original for the first grid could require hundreds of megabytes. Small thumbnails can reduce that first viewing task dramatically. The original remains available for authorized full viewing or download, while the grid receives efficient previews.[13]

Thumbnails serve several everyday purposes:

- gallery grids appear sooner;
- scrolling uses less memory and network capacity;
- mobile visitors receive an image closer to the size of their screen;
- gallery covers and search results remain compact;
- the large viewer can begin with a suitable preview while a larger source is prepared;
- repeated visits can reuse files already stored in the browser cache.[14]

17.2 Why several thumbnail sizes exist

One small size cannot suit every screen. A narrow phone card may look clear with a 300-pixel thumbnail. A wide desktop card may need 600 or 800 pixels. A high-density screen uses more image pixels to draw the same physical area and may benefit from a larger candidate.

PHP Gallery can prepare common sizes such as 300, 600, 800, 960, 1280, and 1600 pixels, depending on configuration and source-image limits. The number describes the maximum long side of the generated copy. Portrait and landscape photographs keep their original proportions.[15]

The smaller candidates serve grids and covers. Larger candidates serve wide cards, high-density displays, search presentations, map previews, and the large viewer. The browser can select from the available group instead of receiving one fixed size for every situation.

17.3 Why JPEG and WebP are available

JPEG is widely compatible and familiar for photographs. WebP often provides similar visual quality with fewer transferred bytes. PHP Gallery can generate both so a compatible browser

[12] Thumbnail generation keeps the source photograph as the archival and full-viewing asset. The generated copy is a replaceable browsing aid.

[13] Actual savings depend on image content, quality, format, dimensions, and the browser's selected candidate. The example illustrates scale rather than promising one fixed compression ratio.

[14] A browser cache stores recently used web resources on the visitor's device according to server and browser policy. Reuse can make a later visit feel much quicker.

[15] For a landscape photograph, the long side is usually the width. For a portrait photograph, it is usually the height. The shorter side is calculated from the original aspect ratio.

can use WebP while JPEG remains a dependable alternative.[16]

The visitor does not choose a format manually. The page tells the browser which versions exist, and the browser chooses a format it understands. The original photograph keeps its own source format and remains separate from these browsing copies.

17.4 When thumbnails are created

Thumbnails can be prepared when photographs are imported or uploaded, rebuilt through Admin maintenance, or generated through a guarded repair process when the public page discovers that an expected copy is missing. Preparing them near upload time gives later visitors the smoothest experience because the required files already exist before the first visit.[17]

A gallery owner should review thumbnail maintenance after:

- importing a large existing folder tree;
- changing enabled thumbnail sizes or quality;
- restoring a backup that contains originals and database data but an incomplete thumbnail cache;
- moving an installation between servers with different image-format support;
- seeing repeated original-file fallbacks or missing previews in System Health.

Generated thumbnails are reproducible cache data and can be rebuilt from the source photographs. The sources themselves require permanent backup protection.

17.5 DNG and RAW display masters

DNG photographs can remain the archival source while PHP Gallery creates a browser-display master for thumbnail generation and public viewing. The display master is generated derivative data, not a replacement for the original DNG. If the required local conversion tool is unavailable or conversion fails, the Admin upload and thumbnail reports identify the failed DNG derivative. Thumbnail maintenance can rebuild the master later; deleting generated derivatives does not delete the source DNG.

Runtime diagnostics exposes the active DNG conversion policy together with the server's actual GD, Imagick/ImageMagick, and format capabilities. The source policy can use automatic fallback with full RAW decoding first and the embedded camera preview second, explicitly prefer full RAW decode, or prefer the embedded preview. Color handling can force browser-safe sRGB, preserve the decoded appearance as closely as possible, or use camera white balance when available. These choices affect generated browser derivatives only. They never rewrite the archival DNG source, and their availability still depends on the image delegates and memory limits provided by the host.

[16] Compression efficiency varies with the photograph and quality settings. Fine texture, noise, gradients, and sharp graphic details can produce different results.

[17] Large imports can require meaningful processing time and storage. Running preparation as an intentional Admin task gives the owner a clearer opportunity to monitor completion.

17.6 What visitors receive

The gallery sends the browser a list of thumbnail copies that really exist for a card. The browser considers the space available on the page, the sharpness needs of the screen, the supported format, and its own loading decisions. It then requests an appropriate file.

For example, suppose a photograph has 300, 600, 800, and 960-pixel copies:

- a small phone card may receive the 300-pixel copy;
- a medium card may receive the 600-pixel copy;
- a large or high-density card may receive the 800 or 960-pixel copy;
- the large viewer may request a still larger preview prepared for that purpose.

The exact choice can differ between browsers and screens. This flexibility is useful because the server does not need to guess one universal size.

17.7 Responsive browser selection

Responsive browser selection is the supported legacy mode under **Admin > Theme > Layout**. The page exposes the available thumbnail candidates immediately and the browser chooses the one that best matches the rendered card and display density. JavaScript is not required for this selection.[18]

Responsive selection usually transfers one chosen thumbnail for each loaded card and is the normal choice when predictable browser-native behavior matters most.

Situation	Likely result
Narrow mobile card	A smaller thumbnail candidate is usually sufficient.
Wide desktop card	The browser may choose a medium or larger candidate.
High-density display	A higher-resolution candidate may be selected for the same CSS size.
JavaScript unavailable	Responsive candidate selection still works.

The 300-pixel copy acts as a fallback when available. Its presence does not force a modern browser to download it first because the larger choices are presented at the same time.

17.8 Progressive thumbnail sharpening

Progressive thumbnail sharpening is the default and safe fallback under **Admin > Theme > Layout**. The gallery first presents a small thumbnail, usually 300 pixels, and later replaces relevant cards with larger copies. On slower connections this can make the collection recognizable earlier, at the cost of potentially transferring both the small and replacement thumbnail.

17.8.1 What happens on the screen

1. The page reserves the final card geometry.

[18] The underlying web standard calls the candidate list a *srcset*. Gallery owners can rely on the visual behavior without learning that markup term.

2. Small thumbnails begin filling visible cards.
3. Relevant cards are queued for a larger candidate.
4. The larger copy loads while the small one remains visible.
5. The card is replaced after the larger copy is ready.

Larger upgrades are queued rather than started for every card at once, and visible cards receive priority over distant content. The current progressive option applies to photo cards inside an open gallery. Gallery covers, collage cells, home-page gallery cards, and Admin images continue using responsive browser selection. The Admin interface labels progressive sharpening as Default and responsive selection as Legacy.

17.8.2 The transfer tradeoff

Progressive mode can transfer two files for one photograph: the small first copy and the sharper replacement. Responsive mode often transfers one browser-selected copy. Progressive mode therefore favors earlier visual population, while responsive mode usually reduces total transfer for the same card.

17.9 Which mode should an owner choose?

Choose **Responsive browser selection** when efficient browser-native selection and typically one thumbnail transfer per loaded card suit the collection.

Keep **Progressive thumbnail sharpening** for a small first image followed by bounded upgrades on relevant cards. Test both supported renderers with representative galleries and devices when choosing the best presentation for your visitors.

Useful questions include:

- Do visitors see the first photographs sooner?
- Does the small version look acceptable during sharpening?
- How much extra data is transferred on a typical visit?
- Do phones and high-density screens sharpen at a comfortable pace?
- Does the chosen mode suit the audience's likely connection quality?

17.10 Loading priorities and stable layout

The first visible photographs receive earlier loading attention because they shape the visitor's first impression. Photographs farther down the page use lazy loading, which means the browser can wait until they approach the visible area. This avoids spending connection capacity on the bottom of a long gallery while the visitor is still looking at the top.[19]

The gallery knows the width and height of indexed photographs. It uses those proportions to reserve stable card shapes before image loading finishes. Text, buttons, and neighboring cards therefore keep their positions as thumbnails appear.

[19] Lazy loading timing belongs partly to the browser. Browsers consider scrolling, viewport distance, connection state, and their own implementation policy.

Visitors who prefer reduced motion receive a calm presentation without a required continuous loading animation. Photograph links remain usable throughout loading and sharpening.

17.11 Thumbnail quality and storage

Higher quality preserves more fine detail and can create larger files. Lower quality saves space and transfer while introducing more visible compression. The best setting depends on subject matter, screen use, hosting capacity, and audience. Detailed foliage, hair, text, and fine architecture can reveal compression more readily than broad soft backgrounds.

Every additional enabled size and format uses storage because it creates another copy for each photograph. The benefit is more precise delivery to different screens. System Health and storage statistics help an owner understand the resulting cache. A large source archive may produce a substantial thumbnail collection, yet those files remain reproducible from the originals.

17.12 Thumbnail compatibility and maintenance modes

The maintenance panel has two compatibility policies. **Modern** generates WebP thumbnails wherever the server can write WebP. **Legacy** generates JPEG fallback thumbnails as well as WebP. After switching an installation permanently to Modern, **Remove legacy JPG thumbnails** deletes only generated JPEG thumbnail files; originals, WebP derivatives, and DNG display masters are preserved.

Gallery consistency. Open **Maintenance Center > Gallery consistency**. Its tabs group Installation, System tools and Gallery consistency. The former Overview maintenance tools, including media renaming and thumbnail controls, now live in System tools. Saved maintenance links redirect here; the main vertical menu gains no item. **Analyze gallery** first creates a read-only report of physical folders, primary names, parent relationships, sidecar titles, original files, expected thumbnails and their metadata. It also finds unindexed photographs, unowned derivatives and pending image moves. Missing originals and unavailable evidence remain explicit findings.

The Overview shortcut retains System Health warnings beside the Maintenance Center title.

The initial scope is **Names and paths (DB only)**: it compares gallery metadata without opening photographs or physical folders. **Files and thumbnails** checks real files, headers, times and metadata in batches. **Deep content verification (slow)** additionally reads checksums and decodes pixels; large collections require substantially more work. Each report names its actual scope. Expanded rows show **Now** and **After repair** values for proposed title, path, parent or sidecar changes, before any repair is chosen.

Findings are grouped by reason with exact counts and a short action or manual status. Photo/file examples combine a known original's variants under its catalog filename; technical cache names and path hashes stay in nested details. A question mark explains the current state, why it matters and the repair result. Hover or focus opens help, activation pins it, and Escape or outside click closes it. Native disclosures remain available. Unassigned previews remain manual; unused size/format records may be historical, not damaged images. Only bounded displayed photo IDs are looked up in the catalog, without reading photo files; names-only reports perform no photo lookup. The export stays complete.

Only affected galleries appear in compact expandable rows. Select individual galleries or use **Repair all available**; manual findings remain separate. Catalog progress counts records completed in batches, disk discovery shows its actual entry count without inventing a total, and repairs confirm one task per batch. The JSON export includes all findings beyond the paged list and detail

samples. The browser needs JavaScript and keeps its checkpoint in the Admin session.

Repairs revalidate catalog and source ownership, preserve originals and valid thumbnails, and stop visibly on failure. Lost responses require a status check, never automatic write replay. Confirmed findings disappear after each successful batch; empty gallery rows vanish. Select another subset immediately without a new analysis. Manual and unselected findings remain; exported receipts retain history. Older completed jobs adopt this list on opening. Catalog proposals refresh from DB without inspecting photos again. Delayed earlier batch requests are refused; external catalog changes still require a new report. Small RGB comparisons flag suspicious image content for manual review; similarity does not prove image identity. Unsupported decoding, ambiguous files and missing originals are not automatically repaired. Hidden/internal storage and non-image attachments retain their separate maintenance tools.

Disk names also govern import. A sidecar title survives only when its sanitized form agrees with the physical leaf. Otherwise the disk leaf supplies the title. New imported folders are canonicalized deepest first through the guarded subtree mover. Conflicting destinations refuse import; unrelated existing galleries are not silently renamed. Explicit consistency repair handles historical discrepancies.

A browser failure remains visible and never starts server generation silently. This choice applies to both Create all and Create missing; the global warning opens these controls first. Consistency repair always uses browser workers. Keep the tab open. Only requested variants are generated and published; valid unrelated files retain their bytes and timestamps. The server validates source identity and stored output before advancing the task count. Lost replies first reconcile stored output. Unsupported RAW/HEIC sources and browser errors remain visible without PHP generation fallback. Results retain a labeled list of remaining findings and show confirmed variant counts. Browser processing progress sits with the main progress above the finding list.

A full **Check missing thumbnails** pass inspects imported photographs and records galleries with missing, stale, or wrong-ratio generated variants without creating anything. The stored result enables **Create missing thumbnails**, which targets only the reported images. **Create all thumbnails** in Maintenance covers all galleries; the same action in a gallery's Images editor covers that gallery and, when **Include subgalleries** is selected, its descendants. These Admin operations remain the primary deliberate maintenance workflow. Public browsing does not start an unbounded repair sweep, but the current renderer has a small guarded background warmup for specific cards that had to fall back to authorized original media because an expected derivative was missing.

Refresh thumbnail database reconciles database metadata with existing generated files and removes wrong-ratio generated files instead of publishing them. **Delete all thumbnails** is a separately confirmed destructive cache operation; source photographs remain untouched and the derivatives can be rebuilt later.

Browser-side thumbnail rebuild is selected by default for these regeneration actions when browser processing is available. Clear its checkbox to use server generation. The server sends original files to the authenticated browser in bounded source ZIP chunks, the browser generates thumbnail variants, and prepared thumbnail ZIP batches are uploaded back for server validation and installation. The source-chunk and batch limits are configured with the browser-upload settings so the workflow does not depend on one oversized PHP request.

17.12.1 Guarded public thumbnail warmup

When the server has rendered a publicly authorized photograph through an original-media fallback, it can mark that exact photograph and the missing supported sizes as a background warmup

candidate. The browser waits for idle time, stops when the page is hidden, sends at most two candidates per request, makes at most four requests from one page load, and spaces requests by about 1.8 seconds. The server independently caps the normalized submitted list at 24 image IDs, but the actual generation budget is smaller: an anonymous request processes at most two source images within roughly four seconds, while an authenticated administrator request processes at most four within roughly eight seconds.

A candidate carries an HMAC token tied to the image ID, gallery ID, filename, relative path, and gallery folder that the server originally rendered. Before generating anything, the endpoint reloads the image/gallery, verifies that token, repeats current visitor access checks, normalizes the requested sizes to the configured thumbnail set, consults thumbnail metadata when available, and skips variants that are already renderable. A per-image 60-second cooldown prevents repeated expensive retries. A single global non-waiting worker lock means concurrent public traffic cannot stack several image-decoding jobs; a busy request simply reports that another warmup worker is active. The warmup is enabled by default through the internal `thumbnail_background_warmup_enabled` application setting and writes bounded diagnostic events to the Admin log when logging storage is available.

Security note. Background warmup does not turn a private or otherwise inaccessible photograph into a public repair target. The signed candidate proves that the server rendered that image as a fallback candidate, and the endpoint still repeats the current access policy before opening the source or writing derivatives.

17.13 If a thumbnail is missing

A missing thumbnail may appear as a slower original-file fallback, an empty preview, or a maintenance warning, depending on the image and available files. Open the thumbnail maintenance tools, inspect the affected gallery or sizes, and rebuild the missing variants. Review write permissions and image-format support when rebuilding repeatedly fails.

After repair, open the gallery with an empty browser cache and confirm that covers, cards, and the large viewer receive the expected previews.

18 Lightbox, maps, EXIF, and voting

18.1 Compact browsing and keyboard focus

The photo viewer fits narrow screens, with the counter separated from its controls. Opening a photo or map moves keyboard focus into that layer; **Tab** and **Shift+Tab** stay inside it. **Escape** closes the map before the underlying photo, and closing returns focus to the opening control. At 100% a single finger swipes between photos; above that scale one finger pans and two fingers pinch. Closing a direct photo restores the album context for language switching and login return links. Map actions without available location data are hidden. Interactive markers identify route starts, destinations, waypoints or photo locations.

Tag results use readable single-column cards on narrow screens. Search suggestions show plain description text while retaining useful link labels. Authored photo captions/titles take precedence over a localized numbered fallback. Public controls and media/map panels follow the selected Theme's foreground and panel colors. The picture game retains an album breadcrumb link and compact photo comparisons. Floating navigation uses small icons; widgets retain their saved placement and return to the page flow when previews or controls leave no safe floating space.

18.2 Asynchronous lightbox metadata

The lightbox is the large photograph viewer that opens above the gallery page. It lets visitors concentrate on one photograph while retaining forward, backward, close, full-screen, slideshow, map, and other available controls.

The gallery prepares the viewer when it becomes useful and obtains information for nearby photographs in manageable groups. A visitor can therefore move through a large gallery without requiring the first page to carry every title, preview, map point, and vote at once.[20]

The Left and Right arrow keys move one photograph backward or forward. `Shift+Left` and `Shift+Right` move ten positions for faster travel through a large result set. The jump follows the same stable ordered sequence as ordinary navigation, wraps at the ends, and also works when the active result set is a Smart Gallery. The translated keyboard-help text lists these shortcuts; no separate graphical jump buttons are added.

The viewer usually starts with an appropriately sized preview. A larger or original source can be used where the action and access policy call for it. Nearby previews may be prepared quietly so the next photograph appears smoothly.

While a new photograph is being prepared, the viewer uses the same progress area that reports full-quality loading. Rapid forward or backward movement always gives the newest requested photograph ownership of that indicator, so an older preview or metadata request cannot leave the viewer permanently busy or replace the current selection. If a photograph cannot be presented, the progress area shows a recoverable failure message; the visitor can continue with the forward, backward, close, or reopen controls. These loading states do not change gallery access, protected-media, Smart Gallery, map, slideshow, or no-JavaScript rules.

[20] The current viewer normally requests information in nearby groups of photographs. Access checks are repeated for each request so private gallery rules continue to apply.

18.3 Adjusting slideshow speed

Start the slideshow with `S` or its existing control. Move the pointer to reveal two compact outlined rows: **Photo change speed** and **Transition speed**. Each range offers **Fast**, **Medium**, and **Slow**; the current choice appears beside it. Both ranges run from Slow on the left to Fast on the right. Arrow keys adjust a focused range without navigating photographs. Actual input restarts the inactivity timer. All controls, including optional developer diagnostics, disappear after inactivity even when a range retains focus or the pointer rests over it. Dragging keeps controls visible until release; movement reveals them again.

On mobile, both speed ranges and the soundtrack panel hide with the idle controls. Tap the photograph to reveal them again; photos and music keep playing.

Photo display intervals are 1, 2, and 4 seconds; transition durations are 0.5, 1, and 2 seconds. Medium preserves the existing defaults. Changes apply immediately and independently: the interval replaces the pending wait, while the next blend uses the new transition duration. An animation already running finishes normally, and the next photograph still waits for its full-quality image to be ready. Choices remain during stop/resume and close/reopen within the same viewer instance; reloading restores Medium. No Save action is needed.

18.4 Zooming and panning a photograph

The lightbox zoom range is 100–400%. Use the minus and plus controls, or select the percentage control to return to 100%. Keyboard shortcuts are `+` or `=` for zoom in, `-` or `_` for zoom out, and `0` for reset. Focus indicators and the current percentage remain exposed to assistive technology.

On desktop, normal mouse-wheel movement and two-finger trackpad scrolling pan an enlarged photograph in both axes; at 100% they do not change the fitted image. A trackpad pinch or `Ctrl` plus wheel zooms the photograph around the pointer, while `Command` plus wheel remains a browser shortcut. On touchscreens, a two-finger pinch keeps its midpoint fixed and one finger pans an enlarged image. At 100% a one-finger horizontal swipe navigates photographs; switching from swipe to pinch and from pinch to pan does not advance the gallery.

At 100%, the photograph is fitted and centered in the viewer. Full-screen mode uses the same zoom and pan behavior, and its Close and other controls remain clickable while the image is enlarged.

Fit by content automatically hides confidently detected black borders stored inside a screenshot, fitting the remaining content proportionally. The frame icon beside the zoom percentage offers **Show original frame**; select it again to restore content fit. This session choice follows navigation and resets zoom to 100% when changed. Space required by the content's own aspect ratio remains visible around it. Detection reads only a small sample of the active decoded image, at most 256 pixels per edge, without machine learning or extra image requests. Matched opposite borders can support darker content edges. One edge with sustained variation can also confirm a sharp brightness step beside a flat or sparsely lit opposite edge. An additional symmetry check measures both black bands independently and verifies sharp steps at both boundaries with sustained detail on at least one side. This also recognizes near-black scenes. Small compression differences at the border can use a bounded shared cut; every additional line must still pass the black-band checks, and both content transitions are verified again. A uniform dark fade or a gradual boundary remains unchanged. Uncertain edges and text in borders stay visible while other proven borders can fit; unavailable pixel reads keep the whole frame. Files, thumbnails and downloads remain unchanged.

Manual navigation and slideshows prepare the crop before the next photograph appears. The previous photograph retains its geometry while loading; during the transition it fades out as the next

one fades in. Differing image shapes therefore do not leave old content visible until an abrupt removal. The slideshow blend excludes the viewer background so removing the incoming cover keeps the same colors and brightness, including changes between cropped and original frames.

During slideshow navigation, the covered gallery retains its modal isolation without being reactivated between photographs. Closing the viewer restores the original background state, including nested editors and maps. Native fullscreen retains an opaque backing, and touch swipe styling preserves the photograph fade. In fullscreen, the covered page header and cards stop painting while their layout and scroll position remain intact. Leaving fullscreen restores them; nested maps and native dialog hosts retain their own visibility. A Smart Gallery reuses an unchanged source-gallery link and updates its address, breadcrumb and accessible name when the source changes.

Zooming above 100% immediately requests the protected full-quality source for the active photograph. The existing preview remains visible until the larger image is ready, and the current zoom and pan position are preserved. In normal, full-screen, and mobile full-screen modes, this transfer shows the same progress bar with a percentage and a byte count. Changing photographs, closing the viewer, or starting a newer full-quality request cancels the previous transfer. A failed or cancelled full-quality request leaves the protected preview in place.

Navigation, slideshow start, close, and reopen return the viewer to a centered 100%. Entering or leaving full screen keeps the current scale when the photograph itself has not changed and adjusts pan to the new viewport. Reduced-motion preferences remove the short discrete zoom transition.

Zoom affects presentation only. It does not alter the stored photograph or bypass media authorization. Password, share-link, NSFW, Smart Gallery, map, voting, pagination, and lightbox metadata rules continue to apply.

18.5 EXIF and map policy

Gallery maps and the lightbox map share compact translucent Theme chrome, a small title and transparent Close action. Reset, zoom and the native **Keep current centered** switch wrap within one map toolbar. Fullscreen and slideshow split maps retain their pane geometry and use the same controls. Narrow/touch actions have larger targets; route rendering, viewport memory, provider attribution, Escape and focus return remain available.

EXIF is information many cameras and phones save inside a photograph. It can include capture date, camera, lens, exposure, dimensions, orientation, and GPS coordinates. PHP Gallery can use this information for date suggestions, maps, technical details, ordering assistance, and duplicate review.

Maps load when a visitor asks to see them, which saves work for visitors interested only in the photographs. A gallery can show individual photo locations and an overview of available points. Parent and child settings determine which galleries expose this information. The global EXIF/GPS public-display setting is the fallback for galleries whose override is set to inherit. Admin also shows how many galleries currently have an explicit override and can reset all of those overrides back to inherit in one confirmed settings save. The reset is schema-guarded: if the override state cannot be verified, the application refuses the change and directs the administrator to System Health instead of guessing.

Selecting **Open photo** in a gallery-map marker opens that photograph in the current lightbox when it belongs to the active gallery. This also works when the photograph is outside the currently displayed pagination page: the viewer requests only a bounded metadata window around the selected photograph. In full-screen split-map mode the map stays open while the photograph

pane changes. From a normal map overlay, the map closes and reveals the selected photograph in the viewer. If the enhanced viewer cannot resolve the selection, the same control follows the canonical photograph page; it does not link directly to the raw image file.

> **Security note.** GPS data can reveal sensitive locations. Administrators should review inherited map settings, image metadata, and public access before publishing personal photographs.

A gallery with a saved flight route can open its map before the first photograph is uploaded, including pages with no lightbox markup. The map fits all stored route points and retains ordinary reset controls without treating route points as photo markers. Without stored coordinates it shows the normal empty-map message. Gallery access and effective map settings still apply; loading remains on demand.

Leaving photo fullscreen with `F`, its control, or the browser's `Escape` closes the split map and returns to the ordinary lightbox with the same photograph and zoom. Pending map loading cannot reopen the pane after exit. In CSS fullscreen, one `Escape` performs the same transition. `M` toggles the map independently while fullscreen; a later `Escape` closes the ordinary lightbox.

18.6 Flight maps and navigation data

Admin > Maintenance > Maps and navdata uses the compact translucent group treatment from Features and Content and display. The navdata name and trailing arrow open the same manager. Counts and the last import time remain visible; question-mark help and Import details hold longer explanations. Update navdata remains an explicit action with in-place progress and the normal form fallback.

Flight-oriented collections can use the optional SimBrief integration to retrieve the latest operational flight plan for a Pilot ID or pilot name. The compact editor accepts one visible identifier: digits alone select the Pilot ID path; other text selects the pilot-name path. Existing integrations that submit separate fields remain compatible, with Pilot ID preferred when both are provided. The editor creates an editable Markdown description and route draft; the OFP and available route coordinates attach after the gallery is saved. A failed SimBrief request leaves the ordinary description field and manual route editor available. The pre-create draft is private and temporary as described in the gallery creation section.

The SimBrief preview fills the existing route editor with the complete filed route, adding missing departure and arrival airports once while preserving route instructions and airport/runway tokens. Description drafts in all maintained languages keep the complete route, including its destination on long routes. The status explains that the OFP and available map points will attach when you save the gallery; importing alone does not save them. If you change the route, identifier, description, or source language while the request runs, the response preserves your form values and asks you to import again. After saving, the map retains the original OFP coordinates.

A gallery with a saved local SimBrief OFP PDF offers **View flight plan (OFP)** and **Download OFP PDF** in its description, even with no photographs. The PDF opens in its own locally rendered document lightbox; its page controls never navigate through gallery photographs. **Fit whole page** is the initial mode, sizing each page to both the usable stage width and height and centering it clear of the header and toolbar. Choose **Fit width** for width-first display with vertical scrolling, or **Actual size (100%)** for the PDF screen-pixel scale. The active mode is indicated on the toolbar. Zoom in/out, 100% zoom reset to the last chosen fit, touch pinch/pan, keyboard page navigation, and fullscreen remain available.

Historical OFP attachments and pre-dispatch review. For an existing gallery without a valid OFP PDF, open the administrator's **API** editor tab and use **Create historical OFP / Open prefilled SimBrief**. A modal first shows the selected gallery and editable departure/destination ICAOs, date, UTC time, aircraft type, flight details, route, flight level, alternate and passengers. Verified saved OFP values are labelled as such; route-map fields and a single gallery date are marked uncertain, while unknown values stay blank. The overlay does not infer an original flight from gallery titles or EXIF timestamps. **Cancel / Back** causes no navigation, provider request or gallery change.

The review dialog also works above the open right-hand Admin panel without closing that editor. Its fields remain editable, `Tab` moves between them, and `Escape` or **Cancel / Back** closes only the review dialog and returns focus to the editor.

Even with a valid PDF already attached, **Prepare revised OFP in SimBrief** opens this same editable review without changing the stored document. After generating and downloading the new PDF on SimBrief, use the separate confirmed **Replace existing OFP PDF** upload. The **Origin of the uploaded PDF** selector only records provenance; choosing a retrospective origin never creates or downloads a file. The departure-date picker has its own full-width row so the native control remains visible in the modal on narrow screens.

Enter the cruise flight level as hundreds of feet, for example `350`; the redirect sends the documented `fl=FL350` representation. An unsigned browser reaches the SimBrief login screen before the dispatch fields. Reaching that screen does not verify acceptance of a historical date; check the date after signing in and before generating.

Only **Open SimBrief Dispatch** sends the reviewed, URL-encoded options to SimBrief's official HTTPS dispatch page in a separate protected browser tab. The administrator may need to sign in and must choose **Generate** on that site. No developer key, OAuth or background request is needed. Historical dates use the documented DDMMMYY format, but SimBrief may reject or adjust old dates. A newly generated retrospective plan is not the original flight-day OFP: weather, NOTAMs, AIRAC, routes and fuel can differ.

After downloading the new PDF, return to the same gallery's **API** tab and use **Upload OFP PDF** to attach one validated document, at most 25 MiB. Select whether it is a retrospective generation or a manually supplied document. A valid existing PDF instead offers **View OFP**, **Download PDF**, and a separate collapsed **Replace existing OFP PDF** workflow requiring explicit confirmation. The canonical OFP manifest records provenance; gallery dates, descriptions, media, routes, visibility, and original OFP JSON stay unchanged. This is a single-gallery action, not a bulk rebuild.

The saved PDF is served through the gallery-scoped `gallery_ofp_pdf` route on the same browser origin as the gallery. Its `index.php` query links also work in subdirectory installations without URL rewriting. A fresh anonymous visitor can view and download an OFP of a public or unpublished gallery via its direct URL without administrator login. Unpublished means unlisted, not private: the original PDF has the same visibility as the gallery page, including password, share-token, and NSFW access gates. Truly private source galleries without a valid visitor grant and generated private OFP-page child galleries remain protected. A verified administrator may access a private source PDF except in anonymous preview mode. Forbidden, missing, and invalid PDFs return the same uncached plain-text `404`, without rendering an HTML gallery page or revealing filesystem paths. Older `media` URLs with `ofp=1` remain compatible.

The compact **View flight plan** and **Download OFP PDF** buttons share one row. An administrator can optionally create a private OFP-page subgallery from a separate action below them. A localized question-mark help button explains the conversion on hover, keyboard focus, or tap without occupying space in the gallery description.

Automatic whole-page and width fits recompute for each PDF page's actual dimensions and orientation, window resizing, device rotation, and fullscreen transitions. Manual zoom retains its chosen CSS scale and pan during ordinary resizes; selecting a fit mode or resetting zoom restores automatic fitting. The automatic calculation excludes the previous manual pan margins, so reset and width/page fitting keep the PDF readable and proportional even after resizing a zoomed document. When document rendering fails, the viewer offers the original protected PDF. The existing gallery visibility, password, and NSFW authorization rules still guard viewing and downloading. In CI, the required Chromium job retains the detailed `browser-map.log` as an artifact whenever the document-viewer browser contract needs diagnosis.

Left and Right arrows move by one PDF page; holding either arrow continues at a controlled rate after the previous page finishes rendering. Navigation stops at the first or last page without wrapping. Releasing the key, leaving the viewer or changing fullscreen ends the held-key sequence. Page navigation preserves the chosen fit or manual zoom. Editable controls retain their arrow keys; `Tab` and `Shift+Tab` move through the viewer controls with visible focus, without outlining the whole PDF stage.

The `F` shortcut and the fullscreen button control only the OFP document viewer; `Escape` first leaves fullscreen and then, in normal view, closes the document. Typing in editable fields, modifier shortcuts, and repeated keys do not activate fullscreen. If the browser refuses a native fullscreen request, the same viewer uses a CSS-only fullscreen layout. In fullscreen, the header and toolbar float above the PDF, fade after a short period of pointer inactivity, and reappear on pointer or keyboard activity. Hovered or keyboard-focused controls remain visible; showing the controls does not resize or rescale the PDF.

Normal mouse-wheel movement and two-finger trackpad scrolling pan only the PDF page, including horizontal scrolling and **Fit width**; they do not zoom the document, viewer controls, or browser. `Ctrl` plus wheel or a trackpad pinch zooms the PDF at the pointer, while the browser retains `Command` plus wheel. On phones and tablets, one finger pans the document and two fingers pinch to zoom; the remaining finger can continue panning after a pinch. The PDF HUD keeps a fixed dark, high-contrast palette independent of the gallery theme and stays at its original scale in both normal and fullscreen modes. **Fit whole page**, **Fit width**, and **Actual size (100%)** restore automatic display.

Manual route text is resolved offline-first. Imported OurAirports database rows are checked first, then the small bundled CSV fallback. A valid cached provider result may be used next; when an administrator has linked a configured Navigraph account and remote lookup is allowed, an unknown identifier can be resolved through Navigraph and cached under the known AIRAC cycle. The Navigation Data manager shows local database and bundled counts, provides a lookup tester for identifiers such as airports/navaids/fixes, and can refresh the local OurAirports airport/navaid dataset. Navigraph OAuth/account storage is optional and can be disconnected without removing local data.

With JavaScript enabled, entering or focusing a nonempty route, requesting a SimBrief draft, or saving route-bearing input also checks local navigation-data freshness in the background. Both **Flight Maps** and **Navigation Data** must be enabled. OurAirports data is refreshed when the last successful import is at least a week old, with an hour between failed attempts. Gallery saving does not wait for this request; the editor draft, open side panel, and browser address stay in place. A refresh failure still allows the gallery to be saved.

After the check, unresolved saved manual routes can gain missing coordinates from current navigation data. Existing coordinates and SimBrief OFP geometry are preserved, and a newer route edit or deletion prevents the background result from overwriting that state. Only saved routes are completed; unsaved browser text is not sent with the refresh. Changed public map contexts are refreshed through the shared panel workflow. Import and route writes require verified

database storage; missing or uncertain storage prevents the optional write while ordinary gallery editing remains available.

Public map rendering never calls SimBrief, Navigraph, or another live planning service. It renders only coordinates already stored with the gallery. Saved SimBrief OFP coordinates therefore remain usable even if the remote provider is later unavailable. Manual route text can also bypass identifier lookup completely with explicit `NAME@latitude,longitude` points.

18.7 Voting

Public cards and lightbox controls share image-vote state. The current public vote is a reversible positive vote: pressing the active up-vote again removes it. The displayed score is the sum of stored image votes; there is no separate whole-gallery voting endpoint in the current implementation. Logged-in votes are associated with the account, while anonymous votes use the application's privacy-preserving visitor hash. New positive votes are rate-limited; revoking an existing vote is allowed immediately. Server-rendered forms provide direct behavior, while JavaScript submits normal interactions asynchronously and synchronizes the score across matching views. The server validates image identity, gallery policy, access, schema readiness, CSRF, and request integrity.

18.8 Picture Game

Picture Game is an optional public comparison mode. An administrator enables it for a gallery branch; eligible public photographs from that gallery and its descendants can then participate subject to the ordinary access and visibility rules. The public game shows two photographs side by side, records which pair was presented, and lets the visitor choose the preferred image before moving to the next comparison. The gallery can also present leading results accumulated by the game.

Because presenting a new pair creates persistent game state even before the visitor chooses a winner, Picture Game requires its database schema to be known and available. If the required storage cannot be verified, the write is refused rather than silently falling back to an untracked comparison. The global feature switch can hide Picture Game without deleting existing game data or per-gallery enable settings.

19 Duplicate Photo Detector

The Admin Duplicate Photo Detector analyzes a selected gallery branch by default and can expand to global scope when the administrator enables the corresponding option. It reports exact and possible duplicate pairs, provides gallery and photo context links, records review decisions, and delegates confirmed deletion to the normal gallery image-deletion service.[21]

19.1 Evidence categories

19.1.1 Exact and possible findings

An exact duplicate contains the same file content. The application recognizes this through a checksum, which acts like a very detailed digital fingerprint calculated from the file. Matching fingerprints provide strong evidence that two files contain the same bytes.

A possible duplicate has supporting similarities without an exact fingerprint match. The detector can compare file size and available camera information such as dimensions, capture time, camera, lens, exposure, aperture, focal length, ISO, and orientation. Two separately exported copies of one photograph may differ as files while retaining enough shared information to deserve review.

Missing camera information simply gives the detector fewer clues. The results remain suggestions for a person to inspect. Open both photograph links, compare their gallery context and quality, and delete only the copy whose removal fits the collection.

19.2 Persistent review ledger

The ledger belongs to the authenticated administrator. Pair entries store canonical image identifiers, with the lower identifier in the low column and the higher identifier in the high column. Gallery entries apply to the exact stored gallery. Parent and child gallery decisions remain independent.

Available review actions include:

- ignore the selected pair;
- ignore all findings from the left image's exact gallery;
- ignore all findings from the right image's exact gallery;
- clear the current administrator's ledger;
- delete a confirmed duplicate through validated normal image deletion.

AJAX actions refresh the detector fragment in the existing Admin right-side panel. Direct POST requests use redirect behavior as a fallback. Server-owned scan jobs preserve scope between continuation requests, and deletion prunes the affected image from current job results.

[21] The detector remains a review tool. Evidence supports administrator judgment because matching file size and photographic metadata can describe distinct photographs under some workflows.

20 Access control and security

20.1 Layered access evaluation

Privacy is evaluated before protected output is assembled. Gallery visibility and inherited access rules are combined with per-photo visibility, password/share-link state, and age-restriction rules where configured. The same authorization applies to gallery pages, thumbnails, previews, original files, lightbox metadata, maps, search results, and downloads.

Admin operations require an authenticated account. Forms include a private request token, and the server validates the target gallery, photograph, and requested operation before changing persistent data.[22]

20.2 Passwords and share links

Protected galleries store password hashes and grant session-scoped access after successful verification. Share links use server-generated token state and expiry policy. Password reset uses one-time hashed tokens, an optional recovery email, configured lifetime, and the selected mail transport. Use HTTPS, strong administrator credentials, restricted database accounts, and protected mail/API secrets.

A listed password-protected gallery can appear as a locked card with its title and link; the locked card does not show its description or cover image.

Administrator cookie protection, generated URL schemes and viewer HTTPS checks share one transport policy. Direct TLS is recognized without proxy settings. When a reverse proxy terminates TLS and forwards HTTP to PHP, configure its exact IP address or CIDR in `security.trusted_proxies` and enable the actual protocol header in `security.trusted_proxy_protocol_headers` before upgrading. Supported headers are `x-forwarded-proto` and `x-forwarded-ssl`; client-IP header trust is configured separately. The proxy must overwrite client-supplied values in enabled headers. Untrusted, malformed, multiple or conflicting protocol assertions cannot prove HTTPS. Both configured proxy hosting and direct HTTPS remain supported; forwarded headers alone no longer upgrade public URLs or set the administrator cookie Secure flag.

20.3 Viewer accounts and invite-only registration

Viewer accounts are an optional identity layer for personalized gallery features. They are separate from the administrator account and, by themselves, do not unlock a private or password-protected gallery. Existing gallery passwords, share links, Smart Gallery access, and media authorization continue to decide what a visitor may see.

The complete viewer-account subsystem is wrapped by a master switch in **Admin > Features**. This master feature is disabled by default. While it is off, the public gallery shows no viewer Login/Account controls, viewer routes are unavailable, and the **Viewer accounts** entry is hidden from the Admin Account menu. Existing viewer identities, favourites, and private collections remain stored; the switch controls reachability rather than deleting data.

After enabling the master feature, the administrator can open **Account > Viewer accounts**. That page retains the separate **Viewer accounts (invite-only)** mode switch. The mode switch

[22] The technical name for the private form token is a CSRF token. Gallery owners normally encounter it only through ordinary Admin forms.

controls viewer login/invitations inside the enabled subsystem, while the master Features switch controls whether the subsystem is exposed at all.

An administrator manages viewer identities from **Account > Viewer accounts**. The existing invite-only workflow remains available: the administrator may optionally bind an invitation to an intended email address and copy its show-once secret link. Opening the link does not create an account. The recipient submits an email address, receives a verification message through the configured mail transport, confirms the verification in the browser, and then chooses a password of at least 15 characters. There is no general **Register** or **Sign up** page.

The same Admin page can also create a viewer account directly. Enter the viewer email and either provide a temporary password that meets the viewer password policy or leave the password blank so the gallery generates a strong one. The temporary password is shown once after creation. If the optional account-created notification is sent, the message contains the trusted login address and instructions but never the temporary password; provide that password separately through a trusted channel. With the master feature enabled, direct account creation remains available while the subordinate viewer frontend mode is disabled so accounts can be prepared in advance, but those viewers cannot sign in until that viewer mode is enabled.

A directly created viewer cannot use the temporary password as a durable login. After the first successful credential check, the browser is sent to the private first-login page and must choose a different compliant password. Until that replacement succeeds, the gallery does not establish the normal viewer identity, issue **Remember me**, or allow viewer favourites/collections authority. The ordinary forgotten-password reset can also replace the temporary credential through its independently verified recovery flow. A simultaneous administrator login in the same browser remains independent.

Existing viewer accounts are listed on the Admin page with their account state and security controls. An active viewer can be suspended with **Suspend** or signed out everywhere; a suspended viewer can be reactivated with **Restore**. Suspension blocks viewer authentication and invalidates the viewer's existing session, remember-me, reset, pending verification/email-change, and prepared collection-share authority through the central viewer lifecycle service. Restoration makes the account active again but does not revive any credential or session that existed before suspension, so the viewer must authenticate again normally. A forced first-login password replacement remains required across suspend/restore. **Sign out everywhere** leaves an active account active while revoking its viewer login authority on all devices. These security controls remain available to the administrator while the master feature is enabled but the subordinate viewer frontend mode is disabled. Turning the master feature off hides the viewer-management surface as well. None of these switches or security controls removes favourites, private collections, collection items, photographs, galleries, Smart Galleries, gallery share links, or administrator authentication.

Admin deletion remains a separate destructive action with explicit confirmation. It removes the viewer identity and viewer-owned authentication, lifecycle, favourite, and collection state through the prepared database relationships. It does not delete photographs, galleries, Smart Galleries, gallery share links, or administrator accounts/sessions.

The public header shows **Login** only while viewer accounts are enabled. After sign-in it shows **Account**. The account page shows the verified email, links to the private **Favourites** and **Collections** pages, **Change password**, **Change email**, **Delete account**, and a viewer-only **Log out** action. Logging out a viewer or changing viewer account authority does not log out an administrator who happens to be authenticated in the same browser session.

A signed-in viewer can mark an authorized photograph as a favourite from the small heart control on a gallery card or in the lightbox. The same photograph is kept synchronized between the card and lightbox, and the private **Favourites** page lists saved photographs that the viewer is still

allowed to open. Removing a favourite deletes only the viewer's saved reference; it does not delete or modify the photograph.

A favourite is never an access grant. If the source gallery later becomes private, its password/share permission expires, or the photo is otherwise no longer authorized for that request, the saved favourite does not bypass those rules and the source metadata is not shown on the Favourites page. A browser that is simultaneously signed in as administrator and viewer still needs the viewer's independent source authorization before the viewer can save that image as a favourite.

A signed-in viewer can also create private named **Collections**. A collection is an ordered list of references to existing photographs; it is not a gallery, does not copy photographs, and does not change the administrator-owned gallery hierarchy. From an authorized gallery card, Smart Gallery card, or the private **Favourites** page, the viewer can use **Add to collection** and choose one of their own collections. The private Collections area supports creating and renaming a collection, removing photographs, moving visible photographs up or down, and deleting the collection itself.

Collection ownership and photograph access are separate decisions. Saving a photograph stores only its internal image reference and order. The collection does not store a gallery password, gallery share token, filesystem path, thumbnail path, or a remembered access decision. Each time the collection is opened, the gallery re-evaluates the current source-gallery and photo authorization. If a previously saved photograph is no longer authorized, it is omitted without exposing its title, filename, gallery title, thumbnail, EXIF data, or reason for denial. The saved reference can remain so the photo becomes visible again if normal source access later returns.

This rule also applies when the same browser is signed in as both administrator and viewer. Administrator visibility does not become viewer collection authority, and a collection can never be used as a second route around gallery passwords, share grants, age restrictions, or media-serving checks. Favourites and collections are independent: a photo may be in either, both, or neither, and deleting a collection never deletes a favourite or the underlying photograph.

A viewer may create one unlisted, read-only share link from an owned collection. The link expires after 30 days and can be replaced or revoked at any time. The complete secret link is shown only immediately after creation or replacement; later the collection page shows only that a share is active plus its creation and expiry times. Replacing a share makes the previous link unusable. Revoking a share removes only the sharing authority and does not delete the collection, its items, favourites, or photographs.

A recipient does not need a viewer account. Opening the secret link validates it, establishes only a narrow session permission for that one collection, and redirects to a clean address without the secret. The link is reusable until expiry/revocation, so automatic mail or chat link scanners do not consume it. The shared page is unlisted, non-cacheable, and marked not for search indexing.

The share grants access only to the collection container. It never unlocks a source gallery or photo. On every shared view, each stored photo reference is checked again against the recipient's current source-gallery authorization. A password-protected source remains hidden until that recipient independently unlocks it through the normal gallery mechanism; an independent existing gallery-share permission may likewise authorize it normally. Even when the same browser is also signed in as administrator, administrator bypass is not used to populate the shared collection. Direct thumbnail/media routes continue to enforce their own existing source rules. Inaccessible items are omitted without exposing their hidden title, filename, path, gallery, or reason.

Suspending the collection owner revokes active collection shares, and restoring the account does not revive them. **Sign out everywhere** only revokes viewer login authority and does not revoke an otherwise valid share. Deleting the owner or collection invalidates its share without deleting source photographs. The global Viewer Accounts feature switch also gates collection sharing and remains disabled by default.

Viewer **Remember me** uses a dedicated rotating viewer credential rather than the administrator persistent-login token. Restoring that credential signs in only the viewer and does not count as recent password reauthentication. Forgotten-password requests return a generic response and use a two-step reset confirmation so automated link scanners cannot reset a password by opening a URL.

Sensitive account changes require recent viewer password proof. A viewer restored only through **Remember me** is therefore asked to confirm the current viewer password before changing the password, starting an email-address change, or deleting the account. The return path is limited to those account actions and cannot redirect to an arbitrary page.

Changing the viewer password uses the same viewer password policy as activation and reset. A successful change invalidates the old viewer password authority, viewer sessions, and viewer remember credentials according to the security lifecycle and then asks the viewer to sign in again. Saved favourites remain attached to the same viewer account. An administrator signed in through the same browser session remains signed in.

Changing the viewer email is staged. Submitting a new address does not replace the current verified address. The gallery first sends a verification link to the candidate address within the viewer mail-abuse limits. Opening that link only validates it and prepares a short-lived confirmation state; it does not change the account. The final change occurs only after the viewer submits the separate protected confirmation form. A successful switch signs the viewer out so the new verified email can be used for the next login.

Delete account is a permanent viewer-only action. After recent password confirmation, the viewer must explicitly confirm deletion. The operation removes the viewer account and viewer-owned data such as favourites and private collections according to the prepared database lifecycle. It does not delete gallery photographs, galleries, Smart Galleries, or their existing share links, and it does not destroy a simultaneous administrator login.

Viewer security and private account/favourite/collection/lifecycle pages are served as private, non-cacheable responses. Invitation, verification, reset, and email-change links use the configured trusted application origin. Viewer verification, reset, and email-change mail remains subject to the viewer address, visitor, and global abuse budgets before the configured PHP mail or SMTP transport is used.

Unlisted read-only collection sharing is available only inside the Viewer Accounts feature. Public collection discovery, public viewer profiles, recipient editing/collaboration, viewer uploads/comments, open registration, viewer social login, TOTP, passkeys, and magic-link login remain unavailable.

20.4 Administrator account and mail settings

The account area supports username and password changes, an optional recovery address, password-reset lifetime, and a test message. Recovery mail can use PHP `mail()` or an authenticated SMTP transport with host, port, encryption, username, and password settings. Treat SMTP credentials like database credentials and never include them in exports or diagnostic screenshots.

Google login is optional. The administrator configures the OAuth client and authorized callback/redirect URI, then links a Google identity only while already authenticated with the normal administrator password session. An unlinked Google identity is rejected at the login page and is not allowed to create or claim an administrator profile. After linking, only that stored Google identity can sign in through the Google button for the profile. Disconnecting the link requires the current administrator password. External login does not replace password login; if OAuth configuration or identity-link storage is missing or cannot be verified, ordinary password

authentication remains independent.

The login form accepts the administrator username or the configured recovery email. Authentication attempts are throttled by both visitor and normalized submitted identifier when the rate-limit schema is available. The throttle table stores hashed subjects rather than raw IP addresses or raw submitted identifiers. Current policy limits repeated failed logins within a 15-minute window and password-reset requests within longer bounded windows; successful login clears the relevant login throttle state.

Password recovery deliberately returns a generic request result so the public form does not confirm whether a submitted username/email exists. Reset links use one-time hashed token storage and the configured 15-to-1440-minute lifetime. Completing a reset changes the password, marks the token used, and revokes persistent-login tokens for that account. The Account page can send a test recovery email through the configured PHP `mail()` or SMTP transport.

The **Keep me signed in** option uses a hashed browser token so an administrator can survive ordinary shared-host PHP session cleanup without storing the raw persistent credential in the database. Persistent login remains optional. It is revoked by the appropriate account/logout security workflows and can be unavailable while its schema state cannot be verified, without disabling the current ordinary PHP session.

20.5 Database readiness contract

System health groups checks by runtime and recovery, access and sign-in, galleries and files, maps and routes, public features and reports, and tools. A checkmark means required storage was found; a dash means the integration is disabled in configuration. Neither proves a complete workflow or external API connection. Question marks explain the purpose of each check. Problems appear first within their group with visible impact and guidance; **Details** retains safe database identifiers and request references. Opening the page does not repair data or run migrations. Viewer diagnostics saves its preference in place, with a native Save fallback; an uncertain save locks until a passive reload.

Features that depend on database schema use one readiness contract. The important distinction is between a schema that was successfully inspected and found to be older, and a schema that cannot currently be inspected.

State	Meaning	Application behavior	Owner action
Available	Required storage was found and checked.	Normal reads and writes for the capability.	None.
Missing	Inspection succeeded, but a required object is absent.	A documented older-schema path may remain available; otherwise the feature asks for a migration.	Back up and apply the pending migration.
Unknown	The schema could not be inspected reliably.	Protected reads may fail closed or optional read-only output may be omitted. Dependent writes are blocked.	Repair database connectivity, selected schema, or metadata permissions.

State	Meaning	Application behavior	Owner action
Disabled	An optional feature is turned off.	The feature is not used.	None unless the feature should be enabled.

System Health and **Runtime Diagnostics** use these states consistently. A **Missing** result normally points to migration work; **Unknown** is an operational problem and should not be treated as proof that a table or column never existed. Diagnostics may show the capability name, status, validated database-object names, suggested checks, and a request reference. They exclude SQL text, raw database exceptions, credentials, tokens, cookies, and private filesystem paths.

20.6 Security-sensitive exceptions

A few capabilities have stricter or more specific behavior:

NSFW Guard. If inspection confirms the older schema without NSFW fields, the site retains its pre-feature behavior until migration. If the fields are **Unknown**, public requests that could expose age-restricted media fail closed and NSFW editing is paused.

Gallery access. A database is treated as the older pre-protection format only when inspection succeeds and all core access fields are absent. A mixed layout is an incomplete migration, so protected routes remain unavailable until it is repaired. The older publication value `draft` is written only when the field definition is known to require it; current schemas use `unpublished`.

Share-link revocation. Revocation may use the smaller known field set needed to clear the validating hash because that operation reduces access. It still stops when its own required fields are **Unknown**.

Persistent Admin login. A missing remember-login table disables persistent browser tokens but does not prevent an ordinary password login from creating the current PHP session.

Password reset. Recovery requires both the recovery-email field and reset-token storage. Missing storage produces migration guidance; **Unknown** storage produces a temporary failure instead of an incorrect account/token result.

Google identity links. External login requires both valid OAuth configuration and known identity-link storage. If that storage is missing or unknown, password login remains available independently.

20.7 Writes, maintenance, and credentials

Operations that change files, credentials, or persistent records perform their readiness check before the first target-changing step. This includes gallery/photo mutations, uploads, WebDAV writes, gallery migration, thumbnail maintenance, database repair/cleanup, and application update activation. An **Unknown** result stops the operation before the target is changed. A confirmed **Missing** result may use an older-schema compatibility path or migration bootstrap only where that behavior is documented.

Credential revocation follows the same restrictive principle as share-link revocation: it may use a smaller known field set when those fields are sufficient to disable access. Creating or trusting a credential still requires the complete storage used by that authentication path.

When a write is blocked as **Unknown**, repair database inspection before retrying. When the state becomes **Missing**, back up and apply the required migration. When it becomes **Available**, retry the operation once and check both filesystem and database results. Resume an existing recoverable migration/update job rather than starting a duplicate job.

20.8 Optional features

Optional presentation features can degrade without disabling the main gallery when doing so cannot reveal protected content or grant access. A map may be omitted if its optional storage cannot be read, for example, but saving a GPS/lightbox override still requires the relevant field to be known.

Voting and Picture Game both write persistent state. Picture Game records a displayed pair before a winner is chosen, so loading a new pair is also a write boundary and requires known storage. AI queues, telemetry maintenance, navigation tokens, saved SimBrief route data, and similar optional writes follow the same rule. A feature may return a read-only result without optional persistence only where the interface makes that limitation clear.

20.9 Operational security

Recommended practices include:

- Serve the application through HTTPS.
- Restrict database credentials to the application schema and required operations.
- Keep `config.php`, backups, caches, and generated archives outside public discovery wherever the hosting layout permits.
- Apply migrations and updates through authenticated maintenance workflows.
- Review audit logs and security diagnostics.
- Test protected galleries through anonymous visitor sessions.
- Maintain coordinated filesystem and database backups.

20.10 Media storage boundaries and recoverable migration

Public gallery media, covers, and gallery branding are authorized before streaming. Restricted or age-gated gallery images use private cache semantics even when an administrator is viewing them; an authenticated response is not automatically safe for shared caches. Stored cover, branding, and image paths are confined to their owning gallery directory, including checks of actual symbolic-link targets. Global Theme backgrounds and branding resolve only within their owned cache directories, and image responses accept supported raster types rather than active SVG content. The retained original Theme background is available only to an authenticated administrator; the normal public derivative remains viewable.

For a new gallery target, containment resolves the existing parent and gallery root, allowing configured filesystem aliases such as macOS `/var` to `/private/var`. Unresolved existing entries

and parents outside the root are refused. These observations do not prevent concurrent filesystem changes after the check.

During gallery-to-gallery migration, asset manifest enumeration and authenticated API-key file delivery independently verify the final file's containment in the source gallery. An incoming asset is copied to a unique temporary file in its destination directory; its copied byte count and any manifest SHA-256 are checked before installation by rename. A failed copy removes the staging file without creating a partial final asset. Matching installed files are reused on retry, while conflicting files and symbolic-link destinations are refused. Browser-prepared store-only ZIP uploads also reject duplicate normalized entry names, invalid CRC-32, and incomplete or inconsistent central-directory records.

21 Theme, presentation, and localization

The Theme area controls site identity, colors, dimensions, page width, backgrounds, branding, favicon assets, language, gallery-card presentation, lightbox defaults, GPS presentation, favorite navigation, and public thumbnail rendering. Settings use normalized scalar values or focused service models.

Under **Theme > Layout > Cards & badges**, choose the global default breadcrumb style. The gallery editor's **Display > Navigation and maps** controls and Smart Gallery presentation controls can inherit or override it. Each selector shows a real rendered breadcrumb example inside a native radio card, so administrators can choose by appearance. The inherit card in gallery selectors previews the currently effective Theme style. Native radio controls support keyboard selection and work without JavaScript. The stable IDs are `minimal`, `chevron`, `pills`, `surface`, `ribbon`, `nodes`, `tabs`, `tiles`, and `gradient`. Ribbon forms arrow segments, Connected nodes link decorative dots, Tabs are compact, Tiles are raised rectangular cards, and Gradient adds a Theme-colored surface with an accent stripe. All ancestors remain reachable as the path wraps, including without JavaScript.

The Theme editor does not use a separate light/dark-mode switch. Instead, the saved color controls and an optional Custom CSS skin define the public visual palette directly. It also sets gallery-card and subgallery layouts, the default breadcrumb style, and up to three header shortcuts. Each shortcut can target the main page, one gallery, or remain empty. Anonymous visitors see only configured gallery shortcuts that are still public and listed; duplicate or deleted gallery selections are discarded when the settings are saved. Leaving all three slots empty hides those favorite-gallery buttons. Gallery cards may also show a configurable contained-picture count badge. When that badge is effective for a gallery, opening the gallery keeps the same branch count visible in the hero panel, counting images in the current gallery and its accessible subgalleries under the same public/access rules as the card. Public cards stack nonzero PDF and music counts compactly below the photo icon with tighter gaps between badges. When the photo-count badge is enabled, the opened gallery's action bar shows those media counts inline immediately to its left. Each count covers unique files in the gallery and its subgalleries, counting a recorded SHA-256 once and following public visibility, hidden-state, access and capability rules. Missing, unknown or dirty derived snapshots omit that badge; a ready zero count remains hidden. Per-gallery presentation can inherit the site defaults or override the choices offered by the gallery editor. Breadcrumbs wrap onto additional lines on narrow screens; every ancestor remains an accessible link, including without JavaScript.

The **Animations** subsection sets the opening and closing duration for the public information panel on gallery cards and the Admin side panel. Each value ranges from 0 to 800 ms; 0 disables that motion. The defaults are 320 ms and 260 ms respectively. The Admin value also times gallery-tree expansion; reduced motion disables that reveal. The public panel contains the gallery title, description, date, and complete tag list. Its native disclosure works without JavaScript; with JavaScript, the panel stays within the viewport and can be dismissed by clicking elsewhere or pressing Escape. The interaction honors the browser's reduced-motion preference.

If the durable initial-pass receipt is missing, the first successful public HTML request for the home, a gallery, or a Smart Gallery queues the media-count pass after the response. Page rendering does not scan original files.

21.1 Theme control reference

The current Theme page is divided into **Appearance**, **Branding & media**, **Layout**, **Language**, and **Custom CSS**. The following reference records the owner-visible controls and the normalization

enforced by the current build.

Area	Control	Current behavior
Appearance	Site identity and colors	Site name plus accent, dark-accent, page-background, panel, open-gallery panel, header-title, and gallery-title colors. The “dark accent” is a hover/outline color, not a dark-mode selector.
Appearance	Corners and font	Corner radius is 0–32 pixels. Font mode is **Classic serif** or **Clean sans-serif**. The live preview updates while these controls are edited.
Appearance	Public page width	**Default**, **Wider**, **Full width**, or **Custom**. Custom width is clamped to 1024–2048 pixels and defaults to 1440 when no valid value exists.
Appearance	GPS pin	When the GPS-maps feature is enabled, photo-card pins and their background underlay can be shown or hidden separately. Pin size is 14–48 pixels with 26 as the default. The underlay slider is displayed as 0–48 pixels, but the current server normalizer treats 0 or another non-positive value as unset and restores 22 pixels; use the separate underlay checkbox to hide the background.
Appearance	Public tag pages	Dedicated tag-page columns and rows use the same safe maxima as other public grids: 1–12 columns and 1–50 rows. Tag-result cards can use the vertical or horizontal gallery-card design independently from ordinary gallery listings.
Appearance	Gallery hero tags	Sort by usage or alphabetically. Either show all immediately or show 1–200 tags before the in-place expansion control; the default threshold is 20. Optional scrolling uses 1–12 visible rows, default 5, and activates only when the rendered tags actually wrap beyond the configured rows.
Appearance	Interface animations	Set the gallery-card public-information-panel duration to 0–800 ms (default 320 ms) and the Admin side-panel duration to 0–800 ms (default 260 ms). A duration of 0 disables that motion.

Area	Control	Current behavior
Layout	Header shortcuts	Three ordered slots. A slot can be empty, point to the main page, or point to a gallery found by the gallery picker. Public visitors see only public/listed gallery targets.
Layout	Default breadcrumb style	Set the global default under **Theme > Layout > Cards & badges**. Gallery overrides can inherit it; invalid values safely fall back to the built-in `chevron`.
Appearance	Gallery-card design	**Vertical system** places the photograph above the description; **Horizontal system** places it beside the description. Fresh installations start with vertical cards; migration preserves older installations' appearance. A physical gallery can inherit or override the Theme choice where the presentation schema is available.
Layout	Picture-count badge	Global contained-picture badge is enabled by default. Individual galleries can inherit it or save an override. When enabled for the opened gallery, the hero panel also shows the current gallery's branch image count, including accessible subgalleries.
Layout	Public thumbnail renderer	**Progressive thumbnail sharpening** is the default and safe fallback. **Responsive browser selection** remains the supported legacy choice with complete server-rendered candidates and no-JavaScript behavior. Covers and collages retain their responsive pipeline.
Layout	Pagination	Global pagination is off by default. Columns default to 3 and are clamped to 1–12; rows default to 3 and are clamped to 1–50. Effective items per page are columns multiplied by rows.
Layout	Main-page grid	The front page has its own 1–12-column and 1–50-row grid. This does not change grids inside galleries.

Area	Control	Current behavior
Layout	Per-gallery grid reset	**Reset all custom gallery grids** clears database overrides, restores default subgallery-inheritance state, and removes matching grid keys from `gallery.json` sidecars so a later discovery pass cannot re-import stale grid settings.
Layout	Lightbox default	**Single image**, **Picture strip**, or **3D carousel** while the lightbox-modes feature is enabled. The classic single-image mode is the fallback and the only mode exposed when that feature is disabled.
Branding	Public header banner	Optional Theme-level JPG/PNG/GIF/WebP banner, maximum 8 MB. It replaces the visible site title when no gallery-specific banner is configured.
Branding	Header separator	Optional JPG/PNG/GIF/WebP separator, maximum 8 MB. Width 0 follows the responsive page width; a positive width is clamped to 160–3840 pixels. Height defaults to 72 and is clamped to 8–512 pixels. Normal mode preserves aspect ratio and treats height as a maximum; stretch mode uses the configured render box exactly and can distort the image intentionally.
Branding	Favicon	Upload JPG/PNG/GIF/WebP, drag a square crop, and zoom the crop from 1–3 times. Saving produces 32-, 48-, and 180-pixel PNG favicon variants.
Branding	Theme background	Preserve the original upload and optionally generate a WebP derivative when GD/WebP support is available. The optimized longest side is 1024–3840 pixels, default 1920, with 128-pixel slider steps. It can be regenerated or removed without deleting the original.

Area	Control	Current behavior
Branding	Background visibility/fallback	Background visibility is 0–100 percent, default 65. The gallery fallback source can be none, a new Theme upload, an existing gallery image, or a collage generated from public galleries. Separate actions reset all per-gallery background selections, remove the Theme background, or remove the favicon.
Custom CSS	Preset skin or upload	Presets are the CSS files currently present in `custom_css/`; selecting one copies it to `public/assets/custom.css`. A manually uploaded `.css` file replaces that active file. The active custom stylesheet loads after built-in CSS and before generated Theme rules.
Custom CSS	Reset actions	**Reset custom CSS** deletes the active custom stylesheet and clears its preset marker. **Reset to CSS** clears saved palette/font/radius/background-source/GPS visual overrides so the underlying CSS becomes authoritative again; structural page-width settings are intentionally preserved.
Language	Admin/public language	Admin language is stored for the administrator's session/browser; the public language is the site-wide anonymous default. The viewer selector is an independent visitor override, described below.
Language	Pack maintenance	Supported packs show string counts and coverage. The JSON editor validates an object containing string values, imports/exports JSON, and can clear authenticated missing-key diagnostics. English remains the source/fallback catalog.

21.2 Advanced appearance and manual CSS overrides

Open **Theme > Appearance > Advanced appearance** for gallery-grid gaps (0–64 pixels), gallery-card internal padding (0–48 pixels), card elevation (current, none, soft, raised), and public content typography (80–140 percent). The defaults retain the installed appearance: 16-pixel spacing/padding, current elevation and 100-percent typography leave existing component rules intact. Sliders have numeric alternatives and visible units; reset restores only the selected control. The existing preview follows unsaved changes; save Theme to apply them.

Transparent site header defaults off. Enabling it removes the public top navigation container's background, image, blur, border highlight, shadow and pseudo-element layers. The logo, title, navigation, language selector and administrator links retain their dimensions and click targets. The hero panel and Admin interface keep their backgrounds. Choose a readable Header text color for the underlying page background; resetting the switch restores the existing header backdrop independently.

Open **Theme > Custom CSS > CSS overrides / Editor** to edit the final public CSS layer. **Save overrides**, **Discard / reload** and confirmed **Clear overrides** operate independently from ordinary Theme saves and the preset/upload asset. The editor marks unsaved text and warns before navigation. **Preview draft** links the draft stylesheet inside an isolated, sandboxed public sample; it does not save or style the Admin page. Ordinary selectors, custom properties, media queries and `!important` are supported. The visible selector example is illustrative.

The cascade order is built-in styles, installed `public/assets/custom.css`, generated Theme rules, and finally `public/assets/custom-overrides.css` on public pages. Specificity and importance still apply. Both custom assets remain installation-owned; deployment/integrity inventories exclude them. Manual overrides survive updates, presets, uploads and unrelated resets. Saving or clearing changes the SHA-256 stylesheet revision or removes its link, so new public loads use the current CSS.

Only authenticated administrators with CSRF authority may read the editor or invoke its actions; saved CSS itself is a public asset. Input must be UTF-8 and at most 256 KiB, without binary control characters. A fixed-path writer lock, staged byte verification and atomic replacement preserve the prior file on failure. Stale revisions are refused: copy the draft, reload the saved version and reconcile before retrying. CSS syntax mistakes use browser recovery and can be corrected by reopening the editor; they do not corrupt Theme settings. Missing/unwritable storage reports a bounded error.

Visual editor availability. The launch button remains visible and disabled until JavaScript loads the workspace and validates the protected preview address. If startup fails, its localized explanation identifies an invalid address, a different origin, a missing preview marker, unavailable modules, or failed initialization. Continue in Manual CSS; ordinary Theme controls and explicit saving remain available. With JavaScript disabled or blocked, the initial message says that the editor is not ready. The preview uses the current installation path and browser origin, including subdirectories and nonstandard ports, independently of the configured public domain. A redirect to another origin is still refused. If the entire editor section is absent, check deployed files; a disabled launcher and a preview refusal are different states.

Live Visual CSS Editor. The visual editor opens the real public home or gallery page in a script-disabled preview, with the Admin controls outside the public document. Switch between signed-in and anonymous visibility and desktop, tablet or mobile widths. Select an element to inspect its stable selector, match count, effective style and any existing visual-editor value. Inspection alone does not change CSS. A deliberate edit uses a limited set of validated properties; advanced CSS variables, functions and rules remain available in Manual CSS. Choose the site, tablet (maximum width 768 pixels) or mobile (maximum width 480 pixels) scope explicitly. The preview width does not create responsive rules automatically. If existing CSS contains `@import`, the visual preview is refused while the CSS draft remains unchanged and available in Manual CSS. The shared-layout controller prepares the marked stylesheet URLs and the layout view only renders them. Bootstrap carries preview context to dependent resources only on same-origin, in-mount GET/HEAD requests with an exact-origin referrer; cross-origin resources receive no preview referrer. The server refuses an import before its target can load. The preview also

refuses an installed `public/assets/custom.css` file larger than 8 MiB or a stylesheet it cannot inspect safely; this bound applies only to preview inspection. Manual CSS remains available, and reducing an oversized installed file to 8 MiB or less permits visual inspection again.

Selecting and styling elements. Ordinary clicks in the preview select elements but do not activate links or actions. To navigate, select a link and choose **Follow link**; **Style element** opens the selected element's controls. Use Tab to reach focusable preview controls; Enter or Space selects the focused element without activating it, and the inspector's ancestor buttons move selection to a containing element. Resize handles have accessible directional labels but use pointer or touch dragging; Escape cancels an active drag. For keyboard edits, use the properties offered by **Style element**.

Visual session controls. **Reset visual session** restores the CSS and pending image captured when the workspace opened, clears visual undo/redo history and keeps the workspace, preview route, audience and viewport open; it sends no request. **Clear draft** empties the CSS draft and removes the pending image, while **Undo clear draft** restores that one-step snapshot. **Restore saved CSS** replaces the draft with the saved snapshot and clears the pending image; when unsaved work exists, confirm the replacement. It does not save or fetch current server state: the textarea and CSS revision are set to the last confirmed saved snapshot rendered on this page or returned by the last successful Save. Later saves from another tab are not reconciled; a subsequent explicit Save can therefore be refused as stale. Escape first closes an open inspector or popover; otherwise choose **Apply to CSS editor & exit**, **Discard session** or **Continue editing**. Discard restores the entry CSS and file and closes the workspace. These session actions do not send a request. The separate confirmed **Clear overrides** action remains the way to delete the persisted stylesheet. Use **Collapse controls** and **Expand controls** to change the workspace panel layout.

Color controls preserve alpha. A hero color change accounts for the theme's real pseudo-element overlay; resetting it removes only the editor-owned companion rule and preserves an independent override or hand-written CSS. Supported resize handles appear only for the approved hero, public layout, card and media profiles. A drag previews temporary styles, commits one undoable edit when released, and can be cancelled with Escape. Media retains its aspect ratio; inline text and ambiguous layout axes are not resized.

Styling profiles. The **Text**, **Header**, **Hero**, **Card**, **Media**, **Layout** and **Links and buttons** profiles expose only their supported controls. Links and buttons support foreground and background color with alpha, typography, borders, radius, padding and shadow; headings and paragraphs offer text decoration. Header and hero expose separate standard and WebKit backdrop filters. Reset removes only the managed property; Undo and Redo preserve each edit as one CSS-draft step.

The Page width control offers **Default**, **Wide**, **Custom** and **Full width**; custom values range from 1024 to 2048 pixels. Custom width has a synchronized slider and editable numeric pixel field. A separate readout shows the effective rendered main-content width in the preview, including viewport clamping. Preview changes the header, main area and footer together. Reset removes the managed width declaration and reveals the underlying Theme or Manual CSS draft value. Background opacity affects only the image layer, not page content. Preview and draft edits do not save Theme settings.

Page width precedence. An unsaved visual width is a CSS-draft override that takes precedence over the saved Theme Appearance width in the preview. After the existing explicit CSS save,

the rule also applies on public pages; neither the draft nor saved CSS changes the Theme setting.

The visual draft stays in memory until **Apply to CSS editor & exit** merges its managed block into the existing CSS textarea. **Cancel & exit** restores the entry draft. Neither action publishes CSS: only the existing explicit **Save overrides** request saves the stylesheet. A selected background image also remains a browser-only pending file until reviewed and submitted with that save. The save validates CSS and image revisions together and stages the new background and stylesheet as one coordinated operation; CSS-only saves remain compatible. A stale revision keeps the local draft and file for review, while an invalid image receives a validation error. If an operational failure leaves database rollback uncertain, retain immutable background assets that may still be referenced and review Theme settings before retrying. The editor does not write a second stylesheet, setting or server draft. The background review targets only the global Theme image. Its source label reports the actual authorized image visible on the current page: the global image, a Theme gallery fallback, a gallery-specific background, or no image. If a gallery-specific background or Theme fallback is visible, the editor disables **Replace or choose image** and **Remove background** and guides you to Home; it does not silently change that gallery's cover or background source. Choose **Keep current**, **Replace or choose image** or **Remove background** as an in-memory operation. Remove deletes only the global Theme image and its optimized derivative; it preserves the Theme gallery fallback mode, per-gallery background choices and gallery cover assets. Replace, Remove and a pending file take effect only through the existing explicit Save, which sends the global target, selected operation and background revision. Keep remains available to discard a pending global operation while viewing an independent gallery background. If the global image or its revision cannot be read safely, only image operations become unavailable; the existing localized message explains this, while Manual CSS and CSS-only Keep Save remain usable. A readable state with no saved global image is normal and allows the first upload. If a safe preview URL cannot be prepared, the visual launch remains unavailable; background draft controls may still initialize, but no preview frame is shown. When the current preview target is the global Theme image, **Background fit** offers **Cover** or **Contain**; **Background position** shows the effective horizontal and vertical percentages and lets you edit each from 0 to 100%, with its numeric field synchronized to the slider. Each reset removes only the editor-managed declaration and reveals the underlying Theme or handwritten CSS draft value; each edit can be undone. These controls write only the site-scope managed rules for `body.public-page.theme-background-image`; they do not change Theme settings or gallery-specific and Theme-fallback hero images. If stronger CSS prevents an override from taking effect, check Manual CSS for `!important` or a more specific selector. Unsupported current effective values are reported for review in Manual CSS rather than silently converted. To replace an unsupported position, enter valid percentages for both axes; a partial pair leaves the draft unchanged, and an empty field does not mean 0%.

21.3 Lightbox browsing modes

The Theme editor defines the default public lightbox presentation. **Single image** keeps the classic viewer. **Picture strip** adds nearby thumbnail previews below the active photograph. **3D carousel** places a small set of neighboring photographs behind the active one to provide more visual context while browsing.

A physical gallery can inherit the Theme default or save its own lightbox-mode override. Smart Gallery presentation can also resolve an effective lightbox mode from its own presentation settings and the site defaults. The modes share the same authorized lightbox metadata and navigation pipeline, so fullscreen, slideshow, zoom, maps, voting, and progressive quality behavior continue to follow their ordinary rules. Disabling the global lightbox-modes feature switch hides these browsing controls without changing stored photographs.

21.4 Choosing the interface language

PHP Gallery treats the language of the program interface separately from the text that you write yourself. Changing the interface language changes buttons, menu items, help text, status messages, dialogs, and other application wording. Gallery and photo titles/descriptions can have optional maintained-language versions, but they are edited and saved separately; tags and other owner content remain unchanged.

Open **Theme > Language**. Two independent selectors are available:

- **Admin interface language** changes the language seen by the administrator in the current browser.
- **Public visitor language** sets the default interface language for public pages.

The optional **Viewer language selector** lets individual visitors override that public default in their own browser. The administrator chooses which maintained languages appear in the public selector; disabling the feature returns visitors to the site-wide public language. Admin and public languages do not have to match.

Public pages use a compact language control with native-language labels and locally served flag icons. Choosing a language keeps the visitor on the same public page and remembers the choice for later visits in that browser. **Use site default** removes the personal override. The equivalent `?lang=` parameter remains available for direct links and no-JavaScript use.

21.4.1 Customizing the viewer selector

The selector has five presets: **Classic**, **Solid pills**, **Outline**, **Soft cards**, and **Minimal**. **Settings > General** provides the basic preset/flag choices, while **Theme > Language** contains the detailed design controls and live preview. These appearance settings do not change the offered languages, site default, Admin language, or a visitor's personal choice.

21.4.2 Multilingual gallery and photo text

Gallery and photo editors keep the existing title and description as source content. Use **Language of the current title and description** to classify that text as English, Czech, German, Swedish, or leave it **Not specified**. Existing installations are not automatically classified as English.

Open **Other languages** with the globe control to enter optional translated titles and descriptions. Gallery titles and descriptions fall back independently. A saved photo-language row is instead treated as one caption variant: blank fields remain hidden rather than mixing source-language text underneath translated photo text. Leaving both translated fields blank removes that language row. Saving occurs through the normal gallery/photo form and remains in the existing Admin side panel.

Public pages use the viewer's selected maintained language. Matching text is shown on cards, gallery descriptions, photo overlays, lightbox metadata, search results, alt text, and SEO metadata. Missing gallery fields fall back to source content; a present photo caption variant displays only its saved translated fields. Translations do not change URLs, slugs, folders, filenames, ordering, visibility, passwords, NSFW rules, or media authorization.

If OpenAI text assistance is configured, **Suggest translation with OpenAI** sends the source field and inserts a reviewable draft in the target field. The draft is never saved or published

automatically. Manual translation remains fully available without a provider.

21.4.3 Available languages and their current state

English is the canonical source language, the new-installation default, and the fallback language. Czech, German, and Swedish are also maintained as complete catalogs. These four are the currently selectable interface languages.

Language	Code	Current role
English	`en`	Complete canonical source, new-installation default, and fallback language.
Čeština	`cs`	Complete maintained selectable Czech translation.
Deutsch	`de`	Complete maintained selectable German translation.
Svenska	`sv`	Complete maintained selectable Swedish translation.

Important. Only English, Czech, German, and Swedish are currently selectable. English remains the fallback if a maintained translation is missing.

21.4.4 How translation packs work

A translation pack is simply a dictionary of named interface messages. The program asks for a stable message key such as “save gallery” or “delete selected photos”, and the active language pack supplies the wording that should be shown. The internal key stays the same in every language; only the displayed text changes.

The **Detected language packs** table shows coverage for the supported languages, and the **Diagnostics** subsection records missing keys observed during an authenticated Admin session. English supplies fallback wording when a maintained translation lacks a key.

The **Pack editor** is intended for careful maintenance or for importing a prepared translation. Language data is stored as JSON. In that editor, the text on the left side of each JSON pair is the stable key and should not be translated. The text on the right side is the value that may be translated. Placeholders such as `\{count\}`, `\{gallery\}`, or `\{time\}` represent live values inserted by PHP Gallery and must keep the same names in every language.

A language pack can also be exported as JSON, translated outside PHP Gallery, and imported again. Imports are accepted only when the file is a JSON object containing text values, so a translator can work on the pack without receiving access to the gallery server.

For developers and maintainers, the maintained catalogs live under `app/lang/`. JSON is the canonical editable format. English, Czech, German, and Swedish PHP dictionaries are retained only as compatibility fallbacks for installations that do not have the corresponding JSON file. Server-rendered pages and browser JavaScript use the same active-language plus English-fallback model, so a translation does not need to be duplicated in separate front-end and back-end catalogs.

Custom CSS supports installation-specific visual refinement. Theme-generated CSS exposes validated variables and computed selectors through a dedicated route. Gallery branding can override selected site-level assets while preserving effective fallback behavior.

21.5 Gallery hero tags

The **Appearance > Gallery tags** subsection controls the compact tag collection below the title of an open gallery. The tag area uses the complete width of the hero panel, so tags can continue wrapping toward the right edge instead of being limited by the normal readable paragraph width. Direct gallery tags and tags collected from contained content remain separate semantic groups, but each group follows the same global display policy.

The same subsection also controls the public tag landing page. **Galleries per row** and **Rows per page** define the tag-result gallery grid and its page capacity, while **Gallery-card design** selects the vertical or horizontal card presentation for those results. These values affect public pages opened through a tag; they do not change ordinary gallery pages. If the dedicated values have not been saved, the global Theme grid and gallery-card design remain the fallback. The global pagination switch still determines whether a long tag result is split into pages. From **Edit tags**, **Configure tag display** opens this subsection directly, and **Manage tag metadata** provides the reverse contextual link.

By default, the initial server-rendered page shows 20 tags. The limit can be changed from 1 to 200 with either a range slider or an exact number field. When more tags exist, the ellipsis control opens an **All tags** panel containing every direct and contained tag in its original group order. The panel uses the same animation and dismissal behavior as gallery cards, keeps the header dimensions stable, and wraps long names. With JavaScript, it stays inside the viewport and can be closed through the same control, an outside click, or Escape; Escape returns focus to the control. All tags are already in the HTML, so opening the panel needs no server request. Without JavaScript, native disclosure opens the panel below the header and keeps long lists reachable by scrolling. Enable **Display every tag immediately** to show every tag from the first response; an exact-limit collection also needs no disclosure control.

Tag ordering is configurable as **Most used first** or **Alphabetical**. **Most used first** is the default. Usage is the total number of direct tag assignments to galleries plus direct tag assignments to photographs across the installation. Equal usage counts fall back to a natural case-insensitive alphabetical order and then a stable identifier tie-break. Direct gallery tags are sorted within their group and contained tags are sorted within their group; the groups are not mixed together.

The tag preview can also use an internal scrollbar. This option is enabled by default with a five-row height cap; the row setting can be changed from 1 to 12 with a slider or exact number field. The server supplies the height cap in the initial markup, so the configured preview does not wait for JavaScript or change geometry when the module starts. Wrapping follows the available width, and scrolling is needed only when the preview exceeds the cap. The open overlay is not clipped by this cap. Disabling the scrollbar allows the preview to grow naturally.

22 Downloads, sharing, and transfer

Gallery downloads prepare authorized content as ZIP archives and stream the finished archive to the browser. A normal gallery download traverses the permitted gallery subtree; Smart Gallery downloads re-evaluate the saved query and current access rules; Picture Manager selection downloads package only the currently selected owned photographs; the administrator can also request an all-galleries archive. Archive entry names are normalized to prevent traversal and keep deterministic paths.

On a modern browser, a public gallery or Smart Gallery download first requests a private, bounded manifest. The browser then requests each authorized original through an independent media endpoint and assembles the ZIP locally, showing progress and allowing cancellation or retry without holding one long PHP archive request open. Every source request repeats the current gallery, Smart Gallery membership, visitor visibility, media-version, and path-containment checks; an image identifier alone never grants access. File-count, cumulative source-byte, duplicate-name, memory, and ZIP64 limits keep the workflow bounded. Direct links and no-JavaScript clients retain the existing server-side ZIP fallback.

Generated gallery/all/selection ZIPs use a content signature derived from the relevant gallery/image metadata so a still-current cached archive can be reused and a changed collection produces a new cache identity. This *content signature* is cache invalidation metadata, not a public HMAC download authorization token. Authorization is checked by the download controller before an archive is served.

22.1 Optional original PDF and music files in ZIP downloads

New and upgraded installations include photographs only. In Admin Settings, the Media category provides two independent global switches for PDF and music originals. The physical gallery editor provides corresponding gallery switches and individual original-file preferences in the PDF and music managers. All three levels default to OFF; inclusion is permitted when the global, source-gallery, or item switch is ON. A higher-level switch overrides lower-level ZIP preferences without changing the saved lower-level choices or standalone music/PDF download permissions.

Archive inclusion never bypasses inherited gallery access, visibility or hidden state, feature master switches, file-integrity checks or Smart Gallery rules. Smart ZIPs include originals only from physical galleries contributing at least one authorized matching photograph; they do not sweep unrelated source trees. Eligible files use the `PDF/` and `Music/` ZIP folders, preserve original extensions, and are deduplicated by recorded content identity. Browser manifests, individual media delivery and bounded server ZIP fallback recheck current authorization; progress, cancellation, size limits and ZIP64 continue to apply. Media-count badges remain inventory counts, not the number of items selected for an archive.

22.2 Gallery-to-gallery migration

For a direct import, open the source gallery's **API** tab, generate an API key and copy the **Gallery import URL** and one-time key. In the target gallery, open **Edit > Media > Import gallery**, paste both values and start the transfer. This creates a new child gallery with originals and existing thumbnails. **Include subgalleries** also transfers the descendant tree. **Connection options** keeps the reconnect interval collapsed until needed. The editor stays open with progress and error feedback. JavaScript is required; help opens beside its trigger and closes with Escape or an outside click.

The gallery editor's API area supports both transfer directions between compatible PHP Gallery installations. **Export this gallery to another instance** treats the current gallery as the source and uses a gallery-scoped API key created for the receiving parent gallery. **Import into this gallery from another instance** treats the current gallery as the receiving parent and uses an API key created for the remote source gallery. The import creates a new child gallery below that parent; it never overwrites or moves the selected receiving gallery.

Include subgalleries is enabled by default. When selected, the versioned manifest describes the source gallery and its complete descendant hierarchy in parent-first order. The target recreates that hierarchy below the new imported root and preserves supported gallery/image metadata, translations, flight-map state, originals, existing thumbnails, and gallery branding assets. Turning the option off transfers only the selected source gallery. The current compatibility policy requires source and target to run exactly the same application version.

The receiving server creates a deterministic package plan bounded by its upload limits. Each original remains grouped with its already-generated thumbnails, while gallery-level assets can be packaged independently. ZIP entries use stable asset keys, store existing bytes without recompressing photographs, and are checked against the manifest's sizes and SHA-256 checksums before installation. Exact API-key scope, gallery-tree structure, target paths, database readiness, and asset completeness are validated at their mutation boundaries.

The administrator chooses a reconnect interval from 5 to 300 seconds. After a timeout, reload, or interrupted connection, the active side asks the durable target job which asset keys are already present and skips completed packages instead of blindly retransmitting them. Folder mappings and received assets are saved incrementally so the same transfer can resume safely. Completion remains a separate step and is refused until every declared asset is present. Cancelling the browser workflow stops further requests; credentials should still be revoked when a temporary migration key is no longer needed.

22.3 Mobile WebDAV transfer

Settings > Uploads creates a WebDAV-compatible connection for one selected gallery. Record a recognizable label, choose the gallery, create the connection, and copy the generated server URL, username, and password into the mobile application. The password is shown only once and stored as a hash. Existing rows show their last-use time and can be deleted immediately from Admin.

Use the WebDAV target type in clients such as PhotoSync. Current server ingestion accepts JPG, PNG, GIF, and WebP, so enable HEIC-to-JPEG conversion in the client where available. WebDAV uploads do not bypass gallery ingestion rules: the credential is gallery-scoped, authentication is required on every write, the gallery/image schema must be known, and accepted media is then scanned, optionally auto-renamed, and processed through the ordinary thumbnail pipeline.

22.4 Public discovery and selected-photo tools

The public `/robots.txt` response describes crawler boundaries, while `/sitemap.xml` lists eligible public gallery and image URLs with image metadata and last-modified information. Unpublished, private, password-protected, age-restricted, or otherwise unauthorized content is not made public through these outputs. Canonical URLs, social metadata, and structured image data use the same access-aware public path model.

The Picture Manager is a logged-in administrator overlay on the public gallery view, not a visitor feature. An administrator can select photographs and physical subgalleries, delete a mixed selection, or copy it into a new physical gallery below a chosen parent. Photo-only selections can also

be dragged onto a visible child gallery, moved or copied through the searchable destination picker, shared through the native device share sheet where supported, or downloaded. Native sharing falls back to the selected-photo ZIP when the browser cannot share the prepared files. Smart Galleries are excluded from physical subtree operations. The selection itself is only browser state and does not grant permission. Every mutation/ZIP request rechecks the normal logged-in account, CSRF token, source gallery, selected ownership, and destination where applicable. Ordinary visitors do not receive these controls. Picture Manager is intentionally independent from the Inline Administration capability: disabling Inline Administration removes direct public-page edit, add, delete, and reorder affordances, while an independently enabled Picture Manager can still provide its selection, deletion, move, copy, physical-gallery creation, and selection-export workflows.

The compact top toolbar shows the selection count when visible. Open it to select all items, clear the selection, share or delete, and choose a destination for moving or copying photos. Expand **New gallery from selection** only when creating a gallery; its parent defaults to the current gallery. The plain **?** control opens selection, drag, copy and move guidance; click outside or press Escape to dismiss it. Picker instructions and the gallery directory link appear while the picker has focus. Progress, success and errors remain visible; the idle ready message is omitted.

With no selection, the manager is hidden above the page. Move the pointer to the top centre edge to reveal and open it. Selected photos or physical subgalleries keep it visible. Keyboard focus also reveals it; on touch devices, select a card first. Open help remains available until dismissed.

23 Maintenance, updates, and recovery

23.1 Migrations

Migration callbacks use explicit database connections and independent migration models. Replacing files does not reload a runtime model already included by the updater; `require_once` retains its old definitions. Initial content evidence therefore uses a separate bootstrap with fixed migration limits. FULL tests execute the real callback after an old-model/file-swap scenario with populated physical galleries, Smart Galleries, PDF and music. They also cover the previous seed symbol, replay, optional readiness deferral and SQL rollback. Database owners run this vector on disposable MySQL/MariaDB; source owners use an isolated SQLite adapter. Fresh-install tests remain separate coverage.

Schema changes live in timestamped PHP files under `database/migrations/`. The migration runner prevalidates pending definitions, applies them in filename order, and records successful versions. New schema work receives a new migration file so existing installations retain a deterministic history.[23]

Database-readiness checks are remembered only for the current PHP request. This normally reduces repeated metadata queries, but a migration can change the answer while the same Admin, installer, updater, or command-line process is still running. The migration runner therefore clears those remembered answers after schema-changing statements and after successful repair callbacks. A post-migration check then reads the new database catalog instead of reusing an answer from before the migration. Ordinary data-only migration statements do not clear the readiness cache because they cannot create or remove a table, field, or index.

Version 0.106 added the `maintenance_jobs` table through an ordinary migration. Run it from the Gallery administration page as usual if it has not already been applied. It requires the installation's normal table-creation authority, but does not require database triggers, stored routines, `SUPER`, a global server-variable change, or separate database-administrator intervention. Until the migration is available and verified, the Maintenance Center refuses to start a job instead of attempting maintenance without durable checkpoints. The cooperative and thumbnail identity migrations use the same normal upgrade workflow.

23.2 How the gallery checks database readiness

The shared readiness states and their owner actions are defined in Section 20.5. Maintenance uses the same contract when checking tables, fields, and indexes needed by password reset, NSFW Guard, upload automation, thumbnail metadata, and other schema-dependent features.

During an upgrade, **Missing** can be expected when new application files arrive before their migration has run. **Unknown** instead means PHP Gallery could not inspect the selected database reliably. For **Missing**, back up and apply the pending migration. For **Unknown**, check database connectivity, the selected schema, and metadata-inspection permissions before changing schema.

After repair, reload **System Health** and retest the affected feature. Keep any request reference with the time and application version when asking a hosting provider or maintainer for help.

[23] Some migrations include repair callbacks in addition to SQL. Compatibility tests protect partial-update scenarios where an older runner encounters newer migration definitions.

23.3 Gallery trash and recovery

Trash uses compact translucent groups with visible preference switches and contextual question-mark help. With JavaScript, changes save immediately together with retention and batch values. During the write, Trash controls lock; confirmed state refreshes in place and returns focus to the edited field. A refused or unverified result requires a passive state reload without replaying the write. Without JavaScript, use the normal Save form. Narrow screens retain labelled metadata and explicit actions without horizontal scrolling. Restore, permanent deletion and Empty Trash retain their separate confirmations.

Open **Maintenance > Trash** to review recoverable gallery deletions. Each active row shows the original path, deletion time, gallery/photo counts, stored size, lifecycle status, and automatic-purge deadline when that policy is active. Problem entries remain visible for inspection; PHP Gallery does not guess that an ambiguous filesystem/database state is safe to destroy.

The main Trash switch controls future gallery deletes and is enabled by default. Turning it off does not hide or delete existing entries: restore and manual permanent deletion remain available. Automatic purge is a separate opt-in and is off by default. Its retention defaults to 30 days, its maintenance batch is bounded, and automatic deletion pauses whenever Trash is disabled. Enabling automatic purge after a pause gives current recoverable entries a fresh full retention window before scheduled deletion can begin.

Migrations `202609070001_gallery_trash_bin.php` and `202609070002_gallery_trash_state_machine.php` add the durable operation records. When Trash is enabled, deletion checks this schema before the first filesystem move or database-row deletion. Confirmed missing or unknown storage stops recoverable deletion and directs the administrator to apply/check migrations; it does not silently fall through to permanent deletion. Scheduled Site Maintenance reconciles stale in-progress operations conservatively and runs expired auto-purge work only when both Trash and automatic purge are enabled.

Trash payloads live in protected persistent `data/gallery-trash/` storage rather than the cache or live gallery tree. Deployment packages carry the directory's access-policy file but exclude stored trash contents. Include that runtime directory and the database in backup and restore planning; neither half alone is a complete recoverable entry.

23.4 Maintenance Center

Open **Maintenance > Maintenance Center** for a guided **Analyze**, **Review**, **Execute**, and **Verify** workflow across routine maintenance owners. Analyze is read-only for gallery, telemetry, log, Trash, cache, thumbnail, and database content; it writes only the durable checkpoint needed to resume the inspection.

The compact Review has three tabs: Cleanup, Media, and Database. Optional tasks use shared switches with short labels; open the adjacent question-mark details for an explanation. Safety preflight and final verification run automatically. They are not user choices and do not appear as disabled switches. Deep thumbnail verification remains available as an explicit opt-in, even when analysis finds no thumbnail work.

Expand technical activity when diagnosing a run. Errors retain their useful status and identify the safe source basename and line when available, without exposing a private path or stack trace. Undefined PHP function, class, or constant errors can also show the safe symbol name; raw exception text remains hidden. Deep media verification is routed to the thumbnail administration owner.

The server loads the reviewed subsystem dependencies again for each separate analysis or execution request before running its task. Checking for pending migrations only reads the migration ledger; a missing ledger or failed inspection is treated as pending work. Apply migrations through the explicit migration workflow; Analyze does not create or repair migration storage.

The reviewed plan is bound to the current application version, applied migration revision, Maintenance Center registry revision, and plan content. If one of those inputs changes, execution refuses the stale plan and asks for a new analysis. The server rechecks task dependencies, capability state, and database-table eligibility at execution time; browser checkboxes are presentation input rather than authorization.

Execution advances through bounded checkpoints, so you can reload or close the page and resume the same job later. Only one running or paused central job owns the durable mutation claim, and two browser tabs cannot advance one job concurrently. Automatic Site Maintenance yields before its first mutation while that claim is held. Public read-only gallery requests remain available.

Pause and Cancel are cooperative: they take effect between bounded operations and do not undo work already committed. A database `ANALYZETABLE` or `OPTIMIZETABLE` already running must return before cancellation can continue. The center processes at most one currently eligible table per browser step, skips protected, unknown, oversized, or newly ineligible tables, and records an uncertain atomic outcome instead of blindly replaying a table after a lost response.

The final report separates removed rows and files, analyzed or optimized tables, skips, warnings, and errors. It reports physical before/after storage only when inventory evidence supports that conclusion. The generic workflow never applies migrations, performs schema repair, installs updates, deletes Admin-log archives, empties all Trash entries, or regenerates media; use the specialist pages for those deliberate operations.

Download/cache cleanup advances through the manifest cache, legacy download-artifact cache, and generated-ZIP cache as separate bounded steps. These caches are independent optimizations: if one is unavailable on the current host, the center preserves cleanup already completed, records a safe warning, and continues with the other owners and later tasks. The warning can identify the operation and a stable error category, but does not expose a raw exception message, SQL, credential, private path, or stack trace. After an update changes this execution contract, an older ready plan is intentionally stale; run Analyze again before continuing.

Cache lifecycle work admits at most 32 short browser steps per central run. Completed regions are not forced to restart while another region waits for retry. Deferred or unavailable work keeps its checkpoint; later tasks continue. The Admin log records the reason, step count, coverage and pending bytes under `maintenance_center.cache_lifecycle_deferred`. Pause and Cancel also work while a request is pending: the control follows the current step, and cancellation releases the central claim through a final guarded step. A service restart preserves the job; after updating, reload the Maintenance Center to continue or cancel that existing run.

23.5 Thumbnail maintenance

Maintenance tools inspect generated variants, identify missing or stale derivatives, and rebuild selected sizes. Browser-assisted rebuilding can prepare source chunks where configured. Background warm-up handles small public-rendering gaps with limited requests, while explicit Admin maintenance remains the primary large-scale repair path.

23.6 SEO request guard and public query hygiene

The maintenance area includes an optional public SEO request guard, enabled by default. For anonymous public GET requests it keeps an explicit allowlist of query parameters for each route. Known analytics parameters such as UTM fields and common ad-click identifiers are ignored because they do not change rendered content. Unexpected parameters are rejected before the normal public renderer with a 404, `noindex,nofollow`, and no-cache response so crawler-generated parameter spam does not multiply indexable gallery URLs.

Admin routes and machine endpoints such as setup, WebDAV, upload automation, gallery-migration receivers, and the maintenance cron endpoint are outside this public query guard. Signed-in administrators are also exempt so diagnostic/admin workflows are not blocked by public crawler rules. Rejected-query logging can be enabled independently and is sample-limited per UTC day to prevent a crawler flood from filling the Admin log. The same service supplies fallback canonical URLs when a public page has not already emitted its own canonical tag.

23.7 Database maintenance

Database inspection inventories schema objects, references, ownership categories, retention policy, and cleanup confidence. Logical cleanup uses allow-listed rules with deterministic selection, limited batches, and audit records. Schema repair provides a dry-run and idempotent migration path. `ANALYZETABLE` can refresh optimizer statistics for selected tables. Physical `OPTIMIZETABLE` is available only as a preview followed by separately confirmed selected-table operation; it is never an automatic cleanup step.[24]

23.8 Storage statistics and Complete report

The Files view also shows counts and bytes for PDF and music originals, plus an overall media total. Update all reads PDF/music/OFP totals from the database after photo batches. The first update imports missing accounting from at most two physical galleries per step, including empty galleries. Hidden, private and disabled files and tracks with downloads denied still consume storage. Each central music asset counts once, including unassigned assets; unmigrated local originals count separately. Smart playlists add no copy. Generated PDF pages already count as photographs. Indexes, locks, conversion state, unindexed files, Trash and other caches are outside these media totals. Unreadable indexes or unsafe/missing files produce partial-accounting warnings without repair. Passive pages read cached results; attachment mutations mark them stale when the advisory revision write succeeds. After upgrading from photo-only snapshots, apply the accounting migration and run Update all once. Attachment uploads, replacements and deletions maintain category totals from committed metadata; repeat updates do not reopen attachment files or indexes. External filesystem changes require explicit accounting invalidation before Update all; see `docs/GALLERY_PDF.md`. With durable thumbnail metadata, ordinary photos are accounted for during the initial database pass without additional photo-batch requests. Only DNG display masters retain bounded file checks. Installations without thumbnail metadata retain the full bounded filesystem fallback; a failed required thumbnail aggregate preserves the last successful snapshot rather than reporting zero. For performance diagnostics, export the latest Admin log event `storage.refresh_all.completed`. Its run ID links progress checkpoints; the summary includes wall/phase times, request counts, media totals, observed file calls and read bytes, inspection source counts and bounded per-table ANALYZE times. File-call counts are application observations, not physical disk I/O. Private paths and workflow capabilities are excluded; a logging failure

[24] Storage-engine allocation values such as data-free estimates describe engine state and do not guarantee a precise number of filesystem bytes returned after optimization.

does not fail the update. The final inspection records separate times for schema and migration reads, source discovery and reads, SQL-literal extraction, identifier indexing and matching, cleanup counts, legacy findings, report assembly, thumbnail distribution and saving. Read byte counts and table/column comparison counts expose the workload; nested times overlap. Source references use a fresh whole-word index per file and SQL literal, preserving production/test separation and same-literal evidence. Unusual identifiers retain regex matching. No persistent analysis cache is used. Required photo-query errors and prematurely empty derivative batches fail the file stage and preserve the last successful snapshot. Later stages continue with an overall partial result; a query failure never means completed scanning.

Detailed **Storage statistics** are calculated only when an administrator requests them, so the normal dashboard does not perform a full derivative-filesystem scan on every visit. The Files view separates indexed source photographs from generated JPEG/WebP thumbnails and DNG browser-display masters, shows source file-type and size distributions, largest galleries and sources, unknown sizes, and scan warnings. A cached snapshot is marked stale when gallery data changes. Refreshing the snapshot scans expected generated files in small browser-driven batches. The same area links to database storage details and database maintenance.

The Database tab reads information-schema storage statistics separately from the filesystem scan. It distinguishes Gallery-owned tables from the whole selected database where possible and shows the largest tables rather than forcing a full table inspection into the ordinary dashboard request. The **Update all** action refreshes the file scan, updates database-table estimates through bounded `ANALYZETABLE` requests, and then runs the read-only database inspection. Progress is kept in a server-owned workflow, and the selected view refreshes in place when the steps finish; repeating the action during the same administrator's active workflow resumes its saved stage. Partial failures are reported by stage and written to the Admin log. This action does not remove data, repair the schema, optimize tables, or execute the Maintenance Center plan.

Maintenance > Complete report builds one self-contained HTML report covering storage, database structure and size, gallery structure, image and EXIF statistics, GPS overview, telemetry, operational logs, feature/settings state, and runtime diagnostics. Image and generated-media work is processed in bounded browser-driven batches so the report remains practical on shared hosting. Transient timeout, rate-limit, and server failures are retried with short increasing delays, while high-cardinality EXIF and GPS summaries remain bounded to control session-storage and memory use on large libraries. The selected telemetry period affects only the telemetry section; image statistics always cover the complete image library. GPS place summaries use local approximate clustering and offline reference data; probable simulator or game screenshots are excluded from real-world place frequency where the available evidence supports that classification.

The completed report is returned to the browser as a download and is not saved as a report file on the server. To keep a PDF copy, open the generated HTML and use the browser's Print or Save as PDF function; print styling is included in the report. Treat any exported report as administrative data because it can contain detailed operational and collection statistics.

23.9 Telemetry and privacy controls

The compact workspace separates **Overview**, **Collection and privacy**, **History and maintenance**, and **Reports**. Its query URL preserves the selected tab across reload and browser history. Each preference saves independently after a change; native forms retain explicit Save without JavaScript. Question marks explain collection, privacy, limits and defaults. An unverified save locks its own form and offers a passive reload without retrying the write. Segmented HTML reports open in a new tab; Settings and Logs are separate destinations. Rollup/purge remains an explicit action, with technical results in a disclosure. Report links display the standalone

HTML document directly in the new tab, which remains available for reading.

In Overview, click a ranked photo name to open its public detail, or a gallery name to open that gallery. Existing access checks still apply. Six small daily bar charts accompany the totals. Each chart has its own scale; hovering shows the date and exact value, and **Daily detail** provides the same values in a keyboard-accessible table. Missing or expired days remain gaps. Retention may leave totals without daily detail. Image bytes include photographs and thumbnails, excluding downloads. Unavailable public-path metadata leaves ranking names readable without links. The small photo icon uses the same preview as the gallery Images editor: hover or keyboard focus reveals it, clicking pins it, and Escape closes it. The bounded thumbnail loads only when the preview is opened.

The optional **Telemetry** area records privacy-controlled operational measurements and presents them as Admin reports. Available controls cover the master telemetry switch, public usage measurements, performance, cache and database events, honoring Do Not Track, excluding administrators, the maximum counted photo-view duration, and retention for raw, hourly, and daily data. Reports include measures such as anonymous sessions, page views, photograph opens and capped view time, client errors, measured media bytes, browser mix, and cache/database activity where those categories are enabled.

The telemetry subsystem is designed not to store raw IP addresses, raw browser user-agent strings, raw referrer URLs, names, email addresses, account identifiers, request bodies, or exact visitor locations. Exports contain aggregated anonymous telemetry and session hashes rather than raw visitor identifiers. Maintenance rolls up and purges telemetry according to the configured retention policy, and the Admin screen can export a standalone HTML report for offline review. Review the privacy controls before enabling collection on a public site.

SQL diagnostics have their own retention: seven days by default, configurable from one to 90 days. This does not shorten hourly or daily usage history. Maintenance compacts historical daily summaries one day at a time while preserving counts, durations, photo identity and bot segments. Browser, OS, viewport, country and referrer details are neutralized for known daily metric families; browser mixes continue to come from sessions. Unknown metrics retain their dimensions. The database migration preserves measurements and rebuilds aggregate indexes around short physical keys; plan temporary disk space and a DDL maintenance window. Deleted data pages become reusable; allocated database size may require manual physical optimization. Actual savings depend on the stored data.

Scheduled telemetry is eligible every five minutes, with a ten-second normal slice and up to eight batches of 5000 expired rows per table (40000 rows). Tables alternate by batch and the first table rotates between slices. The shared clock may reduce actual progress. Maintenance Center keeps its four-second UI slice. Successful public visits to home, gallery and smart-gallery pages dispatch a private telemetry worker after rendering. Shared throttling admits work about every five minutes when traffic exists, independently of thumbnail windows. The visitor never waits for retention SQL and receives no maintenance token. External cron is optional; without incoming requests, work waits for a later visit. Preview, media and failed responses cannot trigger work. Telemetry OFF and central maintenance ownership remain respected; retention ages are unchanged.

23.10 Runtime diagnostics, integrity, and gallery benchmark

Runtime diagnostics provides a copyable environment and support view for PHP, database-readiness classes, hosting limits, GD, Imagick/ImageMagick, and relevant media-format support. It also exposes the DNG conversion policy described earlier. The readiness panels separate presentation/reporting dependencies, authentication/security dependencies, and mutation-sensitive dependencies so an administrator can distinguish an optional missing capability from a condition

that must block a write.

The same bounded model reports gallery edit-revision readiness and unresolved image moves. A missing or unknown required schema is an action state and stops the affected mutation before target files or rows change. Pending-move summaries contain only capped operation/gallery identities and safe state categories; use the documented command-line recovery workflow for an explicit operation rather than expecting the diagnostics page to move files.

The separate **Development diagnostics** switch is administrator-only. When enabled for a signed-in administrator it adds viewer instrumentation for preload, cache, memory, network, and frame-timing work in the public/lightbox/fullscreen experience. It does not turn those diagnostics on for ordinary anonymous visitors. Keep it off during normal ownership unless you are investigating rendering or performance behavior; the gallery benchmark also depends on this development-diagnostics context.

The lightbox DEV dashboard opens as a compact overview. Use Freeze or Resume to hold or continue observations, Collapse to keep the overview, and Expand to open separate pages for media, preloading, graphs, viewport, and lifecycle details. Pages fit the available viewport without scrollbars, preserve your selected page during updates, and remain within the viewer in native fullscreen. The media page distinguishes the displayed photograph from a pending navigation target and shows loaded dimensions, displayed quality, and requested quality. Source labels omit credentials, query parameters, and fragments.

The three graphs have separate labeled scales: retained decoded-cache pixel estimates, cache entries and active detached loads, and animation-frame intervals with a 16.7 ms reference. Up to 90 samples at 350 ms intervals cover about 31 seconds. The pixel estimate excludes the live image, GPU allocations, and browser heap; frame intervals include scheduling and background throttling rather than measuring render cost alone. A past successful decode does not prove that the viewer still retains that image. Unsupported browser measurements show an unavailable marker. Diagnostics observe existing work without requesting extra media or metadata; closing stops monitoring and teardown removes the panel. When DEV is off, the dashboard module is not loaded.

The Admin **Integrity** workflow checks integrity-managed core files against the installed manifest and helps identify incomplete or unexpected deployment changes. It is a runtime verification tool; coordinated backups are still required because an integrity manifest does not contain user photographs or replace recovery media.

When development diagnostics are available, a signed-in administrator can start a **Public gallery load benchmark** for the current gallery. The browser loads the gallery several times in a hidden same-origin frame using an anonymous preview, combines PHP render counters with browser navigation timing, and then offers the benchmark log as JSON. Use the same gallery, presentation mode, and network/cache conditions when comparing two changes; the benchmark is most useful as a before/after measurement rather than as a universal score.

23.11 Scheduled site maintenance

The Media and cache tab uses the same compact translucent groups as Features and Content and display. Complete groups flow down one to three columns, with independent heights. Question marks open contextual help; scan status and the reason for an unavailable action remain visible. Schedule preferences save immediately on change, together with their time and numeric values. Pending writes lock only their form. An unverified result retains the intent and offers a reload without replaying the write. Without JavaScript, use the normal Save form. Thumbnail generation, cleanup, reset and token rotation remain explicit tasks with their existing safeguards.

In Admin > Maintenance > Media and cache, use *Clean updater cache now* to run one bounded step in place. The step removes only already staged updater trash in `cache/updates/prune-trash`; it preserves photos, thumbnails, downloads, rollback backups and unknown cache data. Force bypasses the timer, but respects active updates and maintenance locks. If debt remains, use *Continue cleanup*. The result reports actual files and bytes removed. For unattended progress, run `scripts/site_maintenance.php` with `-cache-only` every minute from hosting cron. This cleanup is independent of thumbnail scheduling and does not establish an overall cache quota. Compatibility settings and schedule diagnostics remain available in the native disclosures; Maintenance Center retains ownership of download-cache cleanup.

Measured cache retention. Automatic work rotates persistent database regions. Last service and the last successfully completed check are separate. Thirty-second opportunities serve the oldest due work for at most five seconds; partial areas resume their saved cursor and yield to other due areas. Healthy completed areas are due after one hour; errors wait fifteen minutes. Root discovery finds new directories without creating one database row per file. Drained absent identities are retained seven days and pruned in small batches.

The main cache card shows recorded size, confirmed total removal and the verified queue. The question-mark help contains scope, coverage and UTC times; technical details remain expandable. A small waiting bar counts down to the scheduled check, never scan completion. The local timer redraws four times per second; a passive AJAX read refreshes visible evidence every fifteen seconds without starting work. Historical measurements, current manual-cycle counters and lifetime receipts remain separate.

Search, cleanup and refresh use labeled outline icons. Manual scan activity stays visible above expandable details; its spinner uses CSS and respects reduced motion. Help has short labeled sections, and operational dates use structured UTC fields. Each thin owner bar shows measured eligible bytes as a share of recorded size, never scan completion or the live queue. Maintenance subtab URLs survive reload and browser history; the anchor is aligned after deferred content loads.

Historical flat download ZIPs expire after seven days from payload modification; changing permissions or reading them does not renew expiry. Native abandoned temporary ZIP outputs retain six-hour inactivity. Only recognized paths with positive ZIP proof and free producer/serving leases are reclaimed. Historical update ZIPs require a verified Gallery source snapshot: the installed application stays in place, and two previous recovery copies are retained, counting the latest job rollback and selected backup first. Active or unproven recovery remains protected. Unfinished regions may reuse spare admission within the existing five-second burst; dense catalogs stream names with bounded memory. After ninety seconds without progress, a foreground-contended autonomous worker gets only one forty-millisecond slice. Explicit OFF and all ownership guards remain effective.

The regional sample shows the area, state, last successful completed check and next check or retry in compact rows. Technical details retain the relative path, observation round, last service, both planned dates, failures and saved cursor. Dates explicitly use UTC; missing completion remains unrecorded. The cursor is a position, never a percentage. The sample does not cover every region. AJAX updates preserve the same localized presentation, open details, panel and URL; reading starts no work.

The compact overview labels last-known measurements and their age, completed coverage, current work, pending eligible debt and the last confirmed deletion separately. A new partial check keeps the completed measurement visible; a smaller completed check can replace it. Historical bytes are not current size and deletion is not subtracted from them. Reading status never scans storage. DB checkpoints own cursors; private disk journals retain names and detached payloads. Unfinished

legacy work is drained safely before conversion. Invalid checkpoints remain unverified. OFF preserves the registry and progress; the existing authenticated server continuation needs no open browser. A killed chain still needs a later restart opportunity.

Registered disposable browser profiles and versioned release working copies have a fixed 24-hour lifetime. Use does not renew expiry. New browser profiles record creation; older workspaces use a filesystem-derived age anchor saved once. Pins and unsafe or busy ownership can defer physical removal. Persistent release preparation/qualification storage remains protected.

The same Media and cache page adds *Scan cache* and an explicit *Clean eligible cache* approval for the exact preview. Inventory is non-destructive; continuation stays on the page. The compact table shows observed and eligible bytes by owner, candidate examples, partial scans and actual reclamation. Completed test audits and verified downloaded CI diagnostics expire after 24 hours; generated managed ZIPs remain available for seven days. New audit ownership markers also permit expired interrupted reports once their producer lease is free. Updater retention has no time limit: current installed files and the latest successful rollback retain current/previous versions; active jobs, selected beta recovery and rollback sources remain protected. Older verified completed/cancelled work is reclaimable; failed jobs and unversioned legacy backups require recovery review. Unknown content, originals, resume state and `.keep` pins are preserved. GitHub response metadata expires after one day, and SimBrief drafts after their existing 30-minute expiry plus a one-minute grace. Hosting cron with `-cache-only` also advances this independent lifecycle; it never auto-approves a manual preview. Dense, slow or unreadable directories are explicitly partial and preserved. Expiry establishes eligibility, not instant deletion. The existing managed-download capacity limit remains; there is no global cache quota. Inventory controls are excluded from measured payload totals.

Shared hosts such as WEDOS may disable `disk_free_space()`. Automatic cleanup continues with explicitly unknown free capacity, successful scheduler and candidate checkpoints, and the same bounded journals. Known capacity below 16 MiB still blocks admission; actual checkpoint write failures stop work. Unknown capacity is neither a measured reserve nor an account quota. No manual Scan action is required.

Automatic cache cleanup is enabled on installation and upgrade through the canonical Admin Features preference; explicit OFF survives updates. One ordinary GET visit can start an authenticated server chain that continues without an open browser. New inventory is due every 30 seconds and independent workers admit up to 5 seconds of checkpointed inventory and immediate reclamation of verified eligible data. Unfinished cursors, originals, pins and active/recovery data remain protected. Waiting occupies a PHP worker; configured-origin HTTP access and sufficient runtime are required. A terminated process or failed handoff can interrupt the chain; a later suitable visit or optional independent scheduler can recover it. Actual removals are coalesced in the Admin log at most once per five minutes, without individual filenames or empty checks.

A measured eligible backlog of at most 128 MiB receives an attempt of up to 600 ms and eight short slices where response finishing is supported. Larger backlogs receive 120 ms and two slices. Without that support, admission is 40 ms and two slices; the client may wait for this short shutdown work. These are admission deadlines, not OS-call latency guarantees. PHP retains its worker even after response finishing. A shared 60-second throttle, nonblocking worker locks, foreground leases and arrival signals make cleanup yield to other visits. Each inner slice admits at most 60 ms, inventory at most 5000 observations and a 4096-name catalog, deletion 180 operations, with eight MiB incremental memory admission and eight MiB PHP memory headroom. Many tiny files therefore continue across visits even below 128 MiB. Known committed eligible candidates can be reclaimed before the whole scan finishes; scan and deletion alternate fairly.

An unapproved manual preview idle for fifteen minutes receives a new scheduled identity; its old ID is refused, never automatically approved. Approved deletion cursors survive interruption. Attempts, actual progress, stale observations, next due, current pending bytes and successfully

removed bytes remain separate from historical measurements; HTTP 200 alone proves no progress. Optional CLI or protected HTTPS cron can supply opportunities on an idle installation, with at most ten seconds and 32 slices per invocation. No hosting setup is needed for visitor-driven operation. See `docs/CACHE_MAINTENANCE.md`. Fixed workspace lifetime remains 24 hours, ZIP retention seven days and updater recovery contextual without a TTL.

Manual runs immediately show a diagnostic ID and a trace linking to filtered Admin Logs. Events record locks, checkpoints, observed entries, removed bytes and safe failure codes. Busy results identify the owner; force respects it. Failed log writes are visible in diagnostics, with best-effort fallback to PHP's error log. Hard process termination can leave an incomplete trace.

Use *Copy log* inside the disclosure to copy its current JSON trace. A thin progressbar uses known updater phase counters; unknown totals remain indeterminate. *Stop update* uses safe pre-activation cancellation and never forces an activation-stage stop. Busy refusal remains visible. Read-only progress refreshes while the panel is visible and does not advance the update. Failed phases show the safe updater error/reference, also included in the copied trace. A phase counter at 100% does not mean successful validation. Copied checkpoints also include age, worker-lock observation, counter movement and available retry history. A stored running status alone does not prove live work; an unlocked worker is only an instantaneous observation.

Routine maintenance can run in bounded slices instead of one long request. The maintenance service chains safe web requests when traffic is available and can also be called from hosting cron with the CLI command shown in Admin. It can check thumbnail state, refresh metadata, process cleanup tasks, roll up telemetry, and archive eligible Admin-log days according to their configured policies. The hidden web-cron token is rotatable; rotating it invalidates previously published web-cron URLs. Maintenance is non-destructive by default and does not replace explicit confirmation for database optimization or other consequential actions.

23.12 Cron Overview and visitor scheduling

Open **Maintenance > Cron Overview** for a dense, sortable table of automatic updates, log archival, site/cache maintenance, updater-trash cleanup, telemetry retention and one-shot attachment-count reconciliation. Each owner supplies its effective cadence, next eligible threshold and actual last-attempt evidence. Unknown evidence remains unknown; admission is not completion. Ordinary visitor and crawler PHP page requests provide scheduling opportunities. The shared scheduler admits at most one bounded background owner per tick, retaining owner locks, enablement, UTC windows and the Maintenance Center claim. Scheduled gallery publication still happens before the triggering page renders.

Sort the column headings; the visible page refreshes existing cells about once per second without executing jobs. Observation pauses while hidden and backs off on errors. Play submits one protected task action in place, overriding only its time gate; native POST remains available without JavaScript. A task can be busy, continued or deferred rather than complete. Next eligible is a threshold, not an exact execution promise: without visitors, work waits for the next request. Play is directly before the task name. The pause icon beside the heading freezes only this live preview; resume reads immediately and background tasks continue. Future thresholds retain their absolute time with the remaining duration in parentheses. Task names link to their existing settings or operational controls. No next deadline is labelled Disabled, Completed or Nothing pending cleanup where appropriate; Unknown is reserved for unavailable evidence.

The collapsed **Optional hosting cron** section creates or regenerates a copyable protected URL and can revoke it. The URL respects installation subdirectories and query routing without rewriting. Its example uses `curl -fsS` every one to five minutes, optionally. It calls the same due-only scheduler and never disables visitor scheduling. Treat the URL as a secret. No hosting cron,

browser job timer, resident process or external queue is required. The 256-bit token and shared locks are supplemented by one accepted hosting call per minute. Repeated calls cannot force work past its owner gates. Network/ICMP flood protection remains the hosting provider's responsibility.

23.13 Admin log history and archives

The compact log table keeps event names, messages, severity, timestamps and instance counts visible. Open the event name for IDs, request/route metadata, the first and latest occurrence, and individual group members; the code icon beside a message reveals its structured context. Copy and download icons have descriptive tooltips and accessible names. Row actions export one event or the entire group indicated by that row; inside group details you can also copy, save or select individual members. Every export includes the full diagnostic text and context.

Search includes event names, messages, JSON context and request/route metadata. A row marked **Match in details** matches inside its collapsed context. Live filtering commits only the newest request, keeping rows, counts and the URL tied to the same result even during rapid typing. For automatic cache failures, `cache.automatic_failed` records the entry point, failure stage, exception class, source filename/line and validated SQLSTATE/driver codes when available. It omits raw messages, SQL, private paths and credentials. Repeated failures are coalesced for five minutes; changed diagnostics retain a thirty-second minimum. Logging failure preserves the worker checkpoint and safe refusal. Complete pending migrations when schema diagnostics require them; observe automatic progress through ordinary visits and passive status without starting a manual scan.

Use the checkboxes and the header checkbox to select entries on the current page. In Admin bulk lists with item checkboxes, click one row and Shift-click another in the same list to apply the second checkbox's resulting state to enabled visible rows between them on the current results page. The range anchor resets when live filtering, pagination, or another row replacement changes the results. Severity filters continue to control which log rows are visible; **Selected**, **This page** and **All filtered** retain their existing log export scopes. The toolbar offers those scopes for both copying and saving a UTF-8 TXT file. Selecting a group includes all its instances, including those not yet loaded; overlapping individual selections appear only once. **All filtered** exports matching raw events across all pages in the chosen time order. Selection clears when filters or pagination replace the rows. PHP reads large exports in batches; copying still needs the complete chosen text in browser memory, so prefer saving for very large histories. If clipboard access is unavailable, a dialog selects the complete text for manual copying. TXT downloads also work without JavaScript; the existing full CSV/JSON ZIP export remains available.

Everyday administrative work creates a useful history: uploads, gallery changes, security decisions, maintenance results, warnings, and other operational events can leave a record in the Admin log. That history is valuable when you want to understand what happened, when it happened, and which account or request was involved. It is also normal for the table to become large on an active installation. Version 1.1.2 therefore treats log care as its own explicit maintenance task rather than allowing generic database cleanup to remove audit history unexpectedly.

The Admin log screen can show individual entries or collapse repeated events into a summary. A group may represent repeated thumbnail warnings, upload results, or maintenance messages; opening it shows the individual records. Filters and workflow status controls help narrow operational review, while bounded exports preserve selected history without loading the entire table in one request. Large lists, raw-instance expansion, and exports are processed in limited portions, so requesting a complete history does not require the browser or PHP to hold the entire table in memory at once.

The owner can choose how long recent logs should remain in the live database. Choosing *Forever*

disables automatic archival. Choosing a finite live period does not immediately throw older records away. The maintenance workflow first selects complete calendar days that are outside the live retention window, writes a protected archive containing both machine-readable JSON and a readable static HTML report, checks that the archive contains the expected number of records, and only then removes those same rows from the live table. The archive is stored under `data/admin-log-archives/`, which is application data rather than a public gallery area and carries an additional web-server protection rule.

Archive maintenance is resumable. It writes temporary files before promoting a completed archive, records a small state file, and uses a lock so two maintenance requests do not process the same history simultaneously. If hosting limits, a connection interruption, or another problem stops the work, the Admin screen can show the incomplete state and the next maintenance attempt can verify, continue, or recover it. An archive is never considered a reason to delete live records merely because a file with a similar name exists; the row-count and archive metadata checks are part of the safety boundary.

Administrators should still include the database and the `data/admin-log-archives/` directory in coordinated backups. The live database contains recent operational history, while the archive directory contains older history selected by the explicit retention policy. Keeping both gives the owner a continuous record and makes a restored installation easier to investigate after an update, migration, or unexpected event.

23.14 Complete current feature reference

This reference is the manual's completeness checklist for Version 1.1.2. It lists the current product-level capabilities exposed by the public routes, Admin navigation, gallery/photo editors, maintenance tools, and optional integrations. Internal helper functions are not listed as separate features when they exist only to implement one row below.

Feature	Implemented behavior
Filesystem-first galleries	Nested physical folders mirrored by database records, unlimited practical hierarchy, discovery/import, manual creation, physical moves, clean public paths, sidecar metadata, and stable sibling/photo ordering.
Gallery metadata	Title, description, maintained translations, manual date/range, EXIF date suggestions, slug, folder name, parent, order, tags, cover/title picture, uploaded gallery thumbnail, branding, background, and presentation overrides.
Gallery access	Public, unpublished, private, password locking, expiring/non-expiring share links, inherited access evaluation, administrator access, and NSFW / 18+ confirmation boundaries.
Bulk gallery administration	Branch-aware deletion, visibility updates, EXIF/GPS map override changes, voting changes, filename-display changes, Picture Game changes, path regeneration, thumbnail/scanning operations, and sibling reorder/date-sort workflows.

Feature	Implemented behavior
Smart Galleries	Saved nested Boolean queries over physical galleries, tags, dates, EXIF/GPS, text, AI metadata, duplicate state, file properties, and private editorial rating; public/unlisted/root/attached placement; per-parent above/below ordering; cycle protection; independent presentation overrides; complete paged lightbox traversal; bounded ZIP download.
Photograph editor	Title/description/translations, visibility, NSFW, sort order, private editorial rating, tags, per-photo thumbnail bounds, EXIF/GPS review, internal AI metadata review, and OpenAI draft tools.
Bulk photograph administration	Move to existing/new gallery, deletion, cover assignment, visibility, NSFW, thumbnail creation, and drag/filename ordering.
Tags	Shared gallery/photo tags, normalized reusable tag names/slugs, public descriptions, usage counts, weighted editor suggestions, Admin rename/edit/delete, public tag landing pages, tag-specific grid/card settings, hero tag ordering/disclosure/scrolling.
Public home and gallery pages	Hierarchical cards, breadcrumbs, covers/collages, pagination, direct photograph grids, gallery counts, dates, descriptions, branding/backgrounds, favorite header shortcuts, contextual logged-in Admin edit/remove/reorder controls, floating back-to-top assistance for long listings, responsive public paths, and query-string fallback when rewriting is disabled.
Public search	Optional site/branch search across gallery/photo titles, descriptions, filenames, tag metadata, maintained-language text, and available internal AI searchable text; minimum query length and bounded result count; access/listing policy applied before results are returned.
Robots, sitemap, and SEO	Access-aware `robots.txt`, image-aware `sitemap.xml`, canonical/social/structured metadata, plus the route-specific query-string guard described above.
Public media authorization	Dedicated original/thumbnail/cover/branding/background/favicon routes re-evaluate gallery/image visibility and protected access instead of exposing arbitrary filesystem paths.
Thumbnail generation	Responsive size variants, JPEG/WebP compatibility policy, responsive and progressive selected-gallery renderers, lazy/near-viewport behavior, per-gallery/photo size bounds, DNG display masters, metadata reconciliation, Admin/browser rebuild workflows, and signed access-checked low-budget public fallback warmup.
Lightbox	Authorized asynchronous metadata, keyboard/touch navigation, slideshow/fullscreen, single-image/picture-strip/3D-carousel browsing modes, EXIF/map overlay, voting state, 100–400% zoom with pan/pinch/cursor anchoring, immediate original-quality promotion when zooming, and Shift+Arrow jumps of 10 photographs.

Feature	Implemented behavior
EXIF and GPS	Camera/lens/exposure/date/GPS extraction, global default with per-gallery inherit/force-on/force-off, photo pins, gallery map data, date suggestions, duplicate evidence, and privacy-sensitive public suppression.
Voting	Per-image reversible positive vote with score synchronization, anonymous visitor hash or logged-in ownership, rate limiting for new likes, and per-gallery enable/disable.
Picture Game	Optional branch-level pair comparison using eligible public images, persistent pair/winner state, leaderboard/result aggregation, and schema-safe refusal when storage is unknown.
Picture Manager	Logged-in public-view administrative overlay with mixed photograph/physical-subgallery selection, recoverable deletion, chosen-parent physical-gallery creation with subtree copying and rollback, photo-only move/copy/drag/share actions, native multi-file Web Share where supported, and selected-photo ZIP fallback/download.
Downloads	Authorized gallery ZIPs, Smart Gallery ZIPs, selected-photo ZIPs, administrator all-gallery ZIP, normalized archive paths, cache content signatures, bounded Smart Gallery source-count/byte limits, and safe cache cleanup.
Browser uploads	Existing/new-gallery upload flows, explicit browser or server processing choice, complete prepared-thumbnail validation, bounded workers/batches with singleton oversize packages below the hard PHP limit, format-policy hinting, local ZIP import, and post-scan automatic renaming.
Filesystem discovery	Session-backed bounded folder scan; skips symlink/hidden/generated directories; imports image-bearing hierarchy in place; can move unmanaged photos into an existing gallery or delete safe unmanaged candidate trees; reports metadata-only sidecars separately.
Filesystem metadata sidecars	Optional per-gallery `gallery.json` written from editable gallery state and read during discovery/parent repair for supported metadata; useful for filesystem portability but not a replacement for database backup.
Upload API	Gallery-scoped one-time-visible API keys, central API manager, key revocation, shared ingestion pipeline, and authenticated automation endpoint.
Windows companion	Watch-folder uploads, manual bulk upload, optional local responsive-thumbnail generation, parallel/pipelined work, retry/state files, system tray, optional SimConnect camera coordinates, confirmed-success source deletion, and optional local AI metadata worker.
Mobile WebDAV	Gallery-scoped one-time password credentials, Basic authentication, WebDAV discovery/write methods required by mobile transfer clients, JPG/PNG/GIF/WebP ingestion, last-use tracking, and revocation.

Feature	Implemented behavior
Media Renamer	Dry-run physical filename plan, default `\{gallery_path\}_\{seq4\}` and documented wildcard placeholders, collision/safety handling, gallery-order context, coordinated database/public-path/derivative/cache updates, bounded site-wide application, and post-upload auto-rename integration.
Metadata Organizer	Preview-only EXIF date grouping with min/max date filters, title-derived child folders, reuse of matching direct date children, confirmed physical moves with derivatives, and bounded apply batches; location grouping remains planned rather than active.
Duplicate Photo Detector	Exact SHA-256 and metadata-supported possible pairs, context links, deletion workflow, per-admin reviewed-pair/exact-gallery ledger, and persistent review state.
OpenAI text assistance	Per-account encrypted API key/model settings, optional explicit thumbnail-consent switch, gallery/photo description drafting, text cleanup, context summary, visible-content description from small generated thumbnails, translation suggestions, and manual-review-only insertion.
Internal AI metadata	Separate searchable machine metadata and queue tables, authenticated Windows worker claims/results, read-only photo-editor inspection, Smart Gallery/search evidence, model-generation tracking, and forced branch reprocessing.
Flight maps and SimBrief	Manual route text, explicit coordinate syntax, SimBrief latest-OFP fetch, editable generated Markdown, saved OFP data and coordinates, and public rendering from stored points only.
Navigation data	OurAirports local database import, bundled offline CSV, cached provider points, optional Navigraph OAuth/package/AIRAC integration, offline-first resolver, and Admin lookup tester.
Gallery migration	Same-version source-push/target-pull API transfer, versioned manifests, metadata/asset checksums, one-asset bounded requests, durable target status, reconnect/resume, and separate completion.
Theme and layout	Direct palette colors, font mode, rounded corners, page width, gallery/card grids, pagination, card layout, count badge, public thumbnail renderer, lightbox default, GPS-pin presentation, three favorite/header shortcuts, and live preview. There is no separate light/dark-mode switch in the current Theme UI.
Branding and browser identity	Site header banner, separator dimensions/stretching, optimized background and opacity, per-gallery overrides, public cover assets, square-cropped favicon with 32/48/180-pixel generated variants, and custom CSS presets/editor.
Localization	English canonical fallback plus maintained selectable Czech, German, and Swedish UI packs, per-viewer browser preference, flag/native-name selector designs, shared browser/server catalogs, pack diagnostics/editor/import/export, and per-gallery/photo maintained-language content.

Feature	Implemented behavior
Feature switches	Global route-aware switches for public search, lightbox modes, Picture Manager, image voting, Picture Game, downloads, viewer accounts and private collections, gallery maps, flight maps, navigation data, SimBrief, OpenAI, local AI metadata, upload API, mobile WebDAV, gallery migration, Media Renamer, and telemetry.
Central Settings	Guided staged Setup Wizard with final approval, live preview and safe owner adapters; searchable categorized registry, keyboard-driven Spotlight navigation, canonical save delegation for approved global values, specialist deep links, inheritance-aware ownership, default/source labels, and secret redaction.
Admin gallery workspace	Visibility filter with hidden-row deselection, persistent per-branch collapse state, Collapse/Expand all, hierarchy reordering, visible-page public reorder tools, side-panel editors, and contextual public-page edit/remove/visibility controls for signed-in administrators.
Administrator authentication	Username-or-recovery-email login, CSRF/session protection, hashed rate-limit subjects, optional durable login token, password reset by PHP mail/SMTP, account/profile changes protected by current-password checks, and optional Google sign-in that must first be linked from an authenticated password session and requires the current password to disconnect.
Viewer accounts, lifecycle, favourites, private collections, and unlisted sharing	Administrator direct viewer creation/deletion plus invite-only registration, Admin suspend/restore/sign-out-everywhere controls with central viewer-authority revocation, forced replacement of Admin-provisioned temporary passwords before normal viewer authority, scanner-safe email verification and activation, separate viewer login/logout, dedicated rotating remember-me credential, generic scanner-safe password reset, recent password reauthentication for sensitive account actions, password/email change, viewer self-deletion, private no-store account/favourite/collection/lifecycle responses, favourite controls on authorized cards/lightbox images, viewer-owned ordered collections with live source authorization, one revocable 30-day unlisted read-only collection link with show-once secret and clean redirect, recipient-context source ACL intersection without Admin bypass, and strict separation from gallery authorization; no open signup, public collection discovery/profiles, recipient collaboration, comments, or uploads.
Database readiness/System Health	Tri-state schema inspection separating Available, Missing, Unknown, and Disabled where applicable; distinct security/authentication, mutation-sensitive, and optional presentation/report capability panels; writes fail closed on unknown required schema.

Feature	Implemented behavior
Database maintenance	Explicit schema/migration/code-reference inventory, storage information, dry-run cleanup candidates, bounded logical cleanup, dedicated legacy repair readiness, selected ANALYZE, selected-table optimization preview, and separately confirmed OPTIMIZE TABLE.
Storage statistics	On-demand source/generated-media scan, file-type/size distribution, largest galleries/files, DNG master and thumbnail accounting, stale snapshot handling, separate database/table usage view, combined **Update all** refresh for files, database estimates, and read-only inspection, and bounded progress.
Complete report	Self-contained HTML export covering storage, database, gallery/image/EXIF/GPS statistics, feature/settings state, telemetry, logs, and runtime diagnostics, with bounded image/generated-media processing, transient-request retries, and bounded high-cardinality summaries.
Telemetry	Optional privacy-controlled anonymous usage/performance/cache/database measurements, DNT respect, administrator exclusion, bounded view time, raw/hourly/daily retention, rollups/purge, Admin dashboard, and HTML export.
Admin logs	Structured events, filters/status, repeated-event grouping with member expansion, bounded exports/ZIP export, configurable live retention, protected daily JSON+HTML archives, resume/recovery state, and archive download/view tools.
Maintenance Center	Durable Analyze–Review–Execute–Verify workflow, grouped task review with no-work optional items in a collapsed disclosure, discoverable opt-in deep verification and full optimization, server-authorized task plan, bounded checkpoints, pause/resume/cancel, central mutation claim, one-table physical database steps, and verified before/after report.
Scheduled site maintenance	Daily UTC window, optional request-triggered continuation, bounded image batch/time budget, CLI/web-cron support, run-one-slice action, interrupted-state reset, rotatable hidden web-cron token, thumbnail/metadata/cleanup/telemetry/log-archive tasks, and yielding while the Maintenance Center owns the central mutation claim.
Runtime diagnostics	PHP/hosting/database readiness, GD/Imagick/ImageMagick/format support, DNG policy, copyable support summary, administrator-only development diagnostics for viewer preload/cache/memory/network/frame timing, and development-only benchmark entry points.
Integrity	Installed core-manifest hash verification for managed application files plus release-time manifest generation/validation; separate from user-data backup.
Gallery benchmark	Development-only authenticated browser benchmark that uses anonymous preview loads, same-origin hidden frames, server render counters and navigation timing, then exports JSON for before/after comparisons.

Feature	Implemented behavior
Updater	Stable/beta discovery, cached patch notes, resumable durable jobs, Range download resume, archive/manifest verification, staging, rollback snapshot, activation, migrations, cleanup, locks/stale-lock recovery, retry/cancel before activation, file rollback after activation, stable restore/clean reinstall/beta install, CLI continuation, and emergency `reset.php`.
Production packaging	One portable PHP folder/ZIP builder, positive inventory, complete source-byte verification, release metadata and core-manifest generation.

23.15 Application updates

Automatic stable checks use the saved 5/15/30/60-minute interval, defaulting to five minutes. The first admitted gallery visit after the interval, including an anonymous visitor or allowed robot, can start the update. After a successful response, independent authenticated server requests continue three-second worker slices until completion. Login, JavaScript, an open Updates panel and hosting cron are unnecessary. Blocked SEO requests do not start this work. The configured installation URL must be reachable from the server with valid TLS. Lost handoffs expire after 120 seconds; later visits recover them. GitHub backoff and worker failures may delay progress. Five minutes controls discovery eligibility, not total installation time; a future release still needs an eligible visit. See `docs/AUTOUPDATE.md`.

The compact release summary keeps installed and available versions, channel, status, and the main actions together; repository and GitHub API details remain in a collapsed diagnostics section. When a job completes, the browser refreshes the summary from local metadata in place, keeps the panel open and the URL unchanged, and hides an obsolete stable-update action. This refresh does not perform another GitHub check. If it fails, use the status-refresh button to retry the local read without repeating the installation.

On Windows, an updater checkpoint commit allows up to ten atomic replacement attempts, with 50 ms pauses and a 450 ms total pause limit, to handle brief file-sharing locks. If the commit still fails, the previous checkpoint is preserved. A persistent permission problem requires fixing storage access before retrying the job. Windows uses the same bounded retry policy when migrating orientation metadata in `gallery.json`; a concurrent edit or persistent refusal preserves the existing document and stops the migration.

The updater checks configured channels, validates packages, stages content, verifies required runtime files and sizes, preserves overwritten files in recovery storage, and activates the update. Rate-limit handling and retry state prevent repeated remote requests during provider wait periods. A resumable job card appears in both **Status** and **Advanced tools**, so a beta install, restore, or reinstall continues to show its progress where it was started. The page may be closed and reopened without discarding completed checkpoints.

The Admin update area also shows cached release information and versioned patch notes for the available channel, so an owner can review a release before starting its job. Stable and beta channels remain separate choices; the Beta label belongs to the release choice in the Admin interface and does not change the updater's durable job or manifest terminology.

Patch notes are stored as raw Markdown in the installed `PATCH_NOTES.md` and the versioned cache. The Admin viewer renders the selected version when it is displayed, including entries loaded from older caches; previously cached HTML is discarded rather than trusted. Reading the viewer

uses locally bundled or cached notes without making another GitHub request. A separate, bounded release-discovery operation may refresh remote notes; when that fails, installed notes remain available.

Markdown links such as `[issue](https://example.org/issue/123)`, angle-bracket autolinks, and plain HTTP(S) addresses become clickable. They open in a new tab with `noopener` and `noreferrer` protection. Other URL schemes remain escaped, inert text. The renderer preserves headings, lists, emphasis, fenced blocks, and literal inline code; it does not turn URLs inside code spans into links. Link punctuation, underscores, and query parameters are retained without accepting arbitrary HTML.

Temporary download or hosting failures may be retried. A package whose files do not match its own integrity manifest is different: downloading the same published build again cannot change those bytes. PHP Gallery therefore asks the administrator to cancel that prepared job and choose a newer release or beta code instead of offering an endless retry loop.

Stable discovery and package work are separate bounded operations. A discovery request may create a durable background job, while a later safe request or the CLI runner advances a short slice through download, archive validation, extraction, manifest verification, staging, rollback backup, activation, migrations, and cleanup. Range requests can resume interrupted downloads, worker locks prevent concurrent advancement, and a stale lock can be recovered after its owner expires. On idle shared hosting, schedule `phpscripts/application_update.php` from cron, for example every five minutes. Each invocation performs bounded discovery or advances existing background work; correctness does not depend on `set_time_limit()` or `ignore_user_abort()`.

If the source server ignores Range, three incomplete full-response restarts switch the same job to the published `production.zip`. Separate bounded metadata requests require the release tag to resolve to the original full commit and require the asset's canonical URL, byte count and SHA-256. The archive digest is checked before extraction even when optional per-file publisher checks are disabled. Flat production archives and wrapped source archives use the same later installation stages. Missing or unmatched assets retain finite failure handling; request limits are not extended. An older installed updater cannot use this repair before receiving it. Verify that first upgrade on a staging copy or deploy the complete compatible release manually and use **Finish FTP update**.

Current releases also reconcile the small set of application-owned server-policy files during update preparation and activation. This protects installations whose older updater skipped a policy file: the validated release copy is compared and published atomically, rollback metadata is extended before replacement, and a migration-backed bootstrap path can repair an already activated older tree. The reconciliation is limited to known policy paths and does not alter gallery media or user configuration.

Before activation, an administrator can cancel or retry a recoverable job. After activation, the Admin recovery action can restore managed application files from the saved rollback snapshot, but file rollback does not reverse database migrations. Stable restore, clean reinstall, and beta installation remain resumable jobs with their own package validation. The standalone `reset.php` emergency path is a separate recovery tool for returning to the stable branch when the normal Admin route cannot be used; it still requires the same careful backup and integrity review.

During the short activation critical section, a dependency-free early runtime gate prevents new ordinary requests from loading a partially replaced application. Such requests receive a private, non-cacheable `503ServiceUnavailable` response until activation completion is durably recorded. The authenticated update recovery/status path remains available, so an interrupted activation can continue. Corrupt or incomplete gate state fails closed instead of presenting the installation as healthy.

The same early runtime boundary converts uncaught and catchable fatal PHP failures into bounded `500` responses when PHP can still control the response. Production hosting must keep `display_errors` disabled; otherwise the PHP runtime itself may print warning or fatal details before application-level handling can format a safe response.

> **Important.** Run a coordinated backup before an application update. Keep the backup available until runtime checks, migrations, public galleries, protected access, uploads, and Admin operations have been verified.

Before activation, updater archive validation also requires the complete new request runtime and its reviewed module plan. A partial archive is refused while prepared files remain outside the active installation. Keep all shipped runtime files together during manual deployment; replacing only the bootstrap would leave an incomplete application.

Updater activation uses the reviewed updater subset of the production inventory. Incoming inventories list WinApp sources separately so older installed updaters can validate the release before installing its reader. The CMS updater does not install those sources. Older source archives without that inventory retain a bounded path based on their owned manifest; a malformed present inventory is refused. Obsolete cleanup requires prior ownership evidence and preserves unknown local files. Rollback uses the server-written snapshot index and refuses unindexed files or paths redirected by symbolic links or Windows junctions. These file safeguards do not reverse database migrations.

Failed jobs from automatic, Admin/API and Advanced commit installs share three transient retries after 60, 120 and 240 seconds. Undefined functions and deterministic integrity failures do not repeat the same defective package. New jobs pin an immutable trusted GitHub commit. The publisher integrity sweep is retained but disabled until explicitly enabled; the short stage is labelled **Preparing package**. Staging and activation check actual copy bytes. Non-core assets and rollback snapshots additionally preserve exact bytes through raw SHA-256 identities. At least 600 seconds after the first failure, an untouched pre-activation job releases its active slot. A safe post-activation failure creates one separate rollback job restoring the actual previous managed files and channel settings, including deleted files and removal of newly created paths. Both identities, archives, backups and diagnostics remain preserved. A failed rollback cannot create another rollback.

The sole public maintenance switch is `maintenance.lock` in the installation root. An empty manually created file also enables maintenance; removing it ends manual maintenance. Active jobs and rollback records alone do not close the website. Inspect updater-owned markers before removal after an interrupted write. Legacy activation markers are imported before admission, with their exact bytes retained privately. The marker stays outside deployment and rollback payloads. Runtime commit ends the isolated automatic rollback window. Application files continue to be updated in place.

File restoration requires verified backup bytes, matching actual schema/ledger and proof that no migration effects occurred. Shared runtime leases drain before activation; supported targets keep the gate through migrations and required finalization. They then commit the usable runtime and reopen before physical cleanup, which may pause or retry while the website remains available. Ordinary use during the observation window invalidates rollback safety even with unchanged schema. Entered/unverified migrations, unknown DB state, missing proof or unsupported older packages stay failed with repair instructions. Database migrations and application data are never reversed. Automatic discovery OFF and beta still prevent new automatic installation; capability OFF stops installer work. After safe restoration, discovery can skip known failed versions and install a newer stable release. Gated failures need hosting CLI cron or an authenticated Admin request; denied visitors cannot run the scheduler. No idle worker waits through backoff.

Applied migration rows remain history when their files were intentionally retired from the installed manifest. Database evidence format 2 retains the complete ledger with timestamps and actual schema, recording that absence explicitly. Missing declared files and changed bytes still refuse activation; old saved witnesses retain their original strict comparisons. An old `activate` checkpoint alone does not prove partial activation: pointer release requires unchanged originals, verified backups and absent created paths under start, worker and exclusive runtime locks. Admin requests release their own bootstrap read lease only after authentication and CSRF. Integrity progress shows the verified-file cursor; entering each new stage resets the previous counters. The compact progress surfaces remain synchronized on desktop and narrow screens. The normal action remains Update followed by automatic continuation. Details call the lifetime counter Processing runs: it includes short worker requests and waits, and Retry keeps the same job and count. Waiting for other requests is explicit and uses slower polling. Unmanaged local PHP files remain preserved. Private worker diagnostics include stage/reference, exception owner and line, safety receipts and the validated migration identity when relevant; unavailable Admin logging falls back to the PHP error log without blocking work.

An older stuck updater does not receive this mechanism merely because a new release is published. In maintenance mode, stop workers, preserve coordinated backups and all failed-job evidence, establish actual database compatibility, deploy the complete verified compatible fixed release and run forward migrations. Only after verified repair may the administrator archive the matching old active pointer. Never replace only the bootstrap, edit a job's target/package, discard a live activation gate or restore application files alone. See `docs/AUTOUPDATE.md` for the complete repair procedure and limits. That procedure includes the reported `c2e2bc77` installation and its pinned older job: after separate approval, deploy the complete compatible fixed release in maintenance mode, preserve the old job and reconcile its owner only after verified forward repair. Retry never changes a job's target. For overwrite-only FTP deployment, upload the complete release without deleting old local files, jobs or backups. Choose **Reset updater / finish FTP update** on the job card or in Advanced tools. It stops previous jobs at a safe boundary, preserves their records and gate receipts, then starts fresh forward completion. **Install stable** is always available and downloads the latest stable branch head, overwriting every packaged file even at the same version. It also replaces stuck or paused jobs without requiring FTP. Both actions preserve configuration, media, unlisted local files and backups. File-content differences are warnings with the relative file and expected/actual digests in the log when the optional verifier is enabled. Its sweeps are currently disabled for all installation jobs. Safe archive paths, required files and runtime ownership remain enforced. With the optional verifier enabled, a missing, malformed or stale manifest also becomes a warning and cannot stop either recovery action. Reset closes visitor admission before existing requests drain, so ongoing traffic cannot prevent its start. Actual activation and migrations wait for those requests to finish. Ordinary automatic updates also close admission before waiting for existing readers, so new visitors cannot indefinitely delay activation. While the gate is closed, private workers retain a paced successor when waiting for old readers; another visitor is not required. A one-second handoff delay runs after locks are released. The configured Admin session name survives post-response session close, keeping authenticated recovery reachable. The start log names the target version; a separate completion event records the previous and installed versions only after durable completion. During activation, visitors see a compact English maintenance page instead of a plain server message. It refreshes the current page every three seconds, even without JavaScript, and offers **Try again**. The self-contained light/dark layout needs no database or application assets. HTTP 503 remains in effect; POST and machine replies never refresh automatically. Busy takeover is shown explicitly and continues automatically. FTP retry rechecks installed files instead of downloading. Recovery runs forward migrations; database changes are never reversed. **Pause** works at the next safe checkpoint in every stage; **Continue update** resumes the same job. A partial activation stays gated. Advanced commit installation also accepts older supported release commits; database migrations are never

reversed and a compatible DB backup may be needed.

23.16 Integrity manifest

`app/core-manifest.json` records the application version and hashes for integrity-managed core files. The local generator under `scripts/generate_manifest.php` calculates final hashes after runtime changes. Documentation and user-data inclusion follow the generator's established policy.

During deterministic release preparation, the manifest generator accepts the optional `SOURCE_DATE_EPOCH` build timestamp for its `generated_at` field; without it, normal generation uses the current clock. The hosted release workflow derives the epoch from the release metadata and uses it for both the manual PDFs and the final manifest, so identical prepared inputs retain reproducible timestamps. The manifest still has to be regenerated after changing an integrity-managed application file.

24 Optional background: how PHP Gallery works

The following pages describe the request path used by the application. They are mainly relevant to maintenance and development; ordinary gallery administration does not require them.

The generated module plan declares shared dependencies and one canonical file order, preserving the complete route closure and include order. Keep dependency inputs and the generated plan together when changing routes. Request/session adapters own transport access; language, navigation tokens, gallery grants, NSFW acknowledgement, and flash behavior retain their established owners. The lightbox separately owns navigation generations and decoded resources, while the viewer keeps presentation and authorized quality selection. This separation provides regression boundaries; it does not establish faster browser operation. See `docs/BROWSER_LIFECYCLE.md` and `docs/COMPATIBILITY_LIFECYCLE.md`.

24.1 Bootstrap, routing, and dispatch

Ordinary requests enter through `public/index.php`; the repository-root `index.php` is a supported compatibility entry point for other hosting layouts. The public entry loads `app/early\_runtime.php` before `app/bootstrap.php`, then both layouts reach the same normal request pipeline.

The small request-local kernel under `app/runtime/` coordinates the existing request lifecycle through Request, Router, RouteRegistry, and ModuleLoader before invoking the procedural controller. The canonical route and security table remains in `app/bootstrap/dispatch.php`. A reviewed generated plan loads logical modules for the selected route; production performs no source scan or dependency compilation and has no load-all fallback. Existing PHP 8.1 and shared-hosting requirements remain sufficient, without Composer or a Node build. Legacy CLI/test consumers may use `app/bootstrap\_full.php` for the complete procedural API.

Shared Admin help and factory-value markup belong to the existing `app/views/admin\_ui.php` owner; Maintenance sections remain deferred. Theme reuses the complete gallery-access policy closure already needed by its picker and preview, while scoped saves use canonical mutation responses. These owners add no logical module or runtime file. Hosted preparation and FULL verify the unchanged duplicate-ownership limits.

The loader validates the complete module plan in each request. It checks canonical layer/path order in one pass, checks each selected physical path once within a load preflight, and counts completed files without repeatedly scanning module lists. Later loads check their paths afresh. No validation cache is shared between requests, so updates and concurrent requests need no additional cache invalidation. Optional OPcache remains a server setting; verify its state in the web PHP runtime and keep timestamp validation or deployment resets effective. The existing Test Run trace separates plan loading, full validation, dependency planning and includes. Developer measurements use disposable public/Admin requests and distinguish opcode-cold and warm samples; they do not establish production latency. The compiler explicitly reviews the private tracing callback; only local loader operations supply it, with no additional module dependency.

Shared updater discovery reads safe errors and opaque references from the lightweight job reader. Public galleries do not load the updater worker for these values. Explicit Admin/background installer owners retain the complete worker; optional subsystem diagnostics observe already-loaded owners.

Thumbnail delivery reuses the existing complete public-media module. Compilation accepts this dependency only while its full file closure already belongs to the thumbnail owner. Route file limits and the duplicate-ownership ratchet remain enforced.

Bootstrap reads local configuration and establishes the request context: database access where required, language, session state, security prerequisites, and shared runtime services. Routing then translates the incoming URL or query-style fallback into an internal route plus parameters such as a gallery path, page number, media size, or share token. Dispatch calls the controller registered for that route.

The ordering is deliberate. A request must be classified before code decides how to serve it, and access checks must run before protected output is assembled. A direct-looking image or thumbnail URL is therefore still an application request; the media controller authorizes it before returning bytes.

```
browser request
  -> index.php or public/index.php
  -> app/bootstrap.php
  -> cms_run() -> Gallery\Core\Kernel
  -> Request -> Router -> RouteRegistry
  -> ModuleLoader -> app/bootstrap/dispatch.php
  -> controller function
  -> service functions
  -> HTML, JSON, media, archive, or redirect response
```

24.2 Controllers

Controllers own HTTP-level coordination. They receive a resolved route and request context, validate the operation, call the services needed for that operation, and choose the response type. A public gallery route normally returns HTML; an Admin side-panel action may return JSON or an HTML fragment; a media route returns an authorized file; a download route can stream an archive.

Public gallery rendering begins in `app/controllers/public_gallery.php`, with related responsibilities split into focused controller files. Uploads, maintenance, updates, downloads, media access, and Admin features have their own entry points, while shared domain rules remain in services.

Controllers are not an authorization shortcut. Thumbnail, map, lightbox, search, download, and direct-media routes repeat the relevant access decision instead of assuming that a visitor previously opened the gallery page successfully.

24.3 Services

Reusable application rules live in `app/services/`. Services resolve galleries and images, evaluate access, normalize settings, manage uploads and mutations, prepare thumbnail/media manifests, perform searches, maintain metadata, and coordinate other domain work. A controller should be able to ask for one of these operations without reimplementing its rules.

The separation matters because the same photograph can be reached from several routes. Access policy, thumbnail selection, or deletion semantics should not depend on whether the request came from a gallery card, search result, lightbox, Admin tool, or direct link. Source photographs remain in the gallery tree; services coordinate the database records and generated derivatives that describe or support them.

24.4 Views and browser modules

Views render prepared data into HTML. They are responsible for structure and presentation, not for deciding whether a visitor is authorized or how a thumbnail is generated. Server output includes semantic links, forms, titles, alternative text, and initial media candidates before JavaScript enhancement begins.

Browser modules add lightbox behavior, search suggestions, voting, upload progress, drag-and-drop actions, thumbnail upgrades, diagnostics, and the Admin side panel. The code remains framework-free. Public pages therefore retain a useful server-rendered baseline, while authenticated workflows can add richer in-place interactions.

The Admin side panel is dynamic: its fragment can be replaced without navigating away from the page. Modules that own panel controls consequently use delegated or lifecycle-aware event binding so a freshly rendered copy continues to work and an old fragment no longer owns listeners.

24.5 Public selected-gallery render pipeline

A selected-gallery request first resolves the gallery and its effective access policy, including inherited protection, password/share state, publication state, and age restrictions. Only permitted content proceeds to rendering.

The controller then loads the direct child galleries and the current page of photographs, together with the metadata needed for titles, descriptions, tags, dates, votes, ordering, and navigation. Thumbnail metadata is prepared in batches, and request-local media manifests describe the authorized candidates available for each visible image.

The view receives that prepared model and renders the page title, branding, navigation, breadcrumbs, cards, pagination, lightbox configuration, and footer assets. Browser modules enhance the result after delivery. Access remains a server responsibility; JavaScript can make an interaction faster or more convenient, but it does not turn an unauthorized candidate into an authorized one.

This server-first split also preserves useful behavior when scripts are blocked or late. Direct links, semantic content, and the initial image choices are already present in the HTML, while JavaScript handles the features that genuinely need an interactive client.

25 Optional background: what the database remembers

The database stores application state that does not belong in image filenames: descriptions, privacy, ordering, tags, votes, dates, public addresses, accounts, and maintenance history. The overview below is enough for backup planning and hosting discussions; column-level definitions remain in the database reference [3].

25.1 Principal domains

Domain	Persistent responsibility
Users and authentication	Administrator identity, password hashes, recovery email, external identity links, reset tokens, and scoped credentials.
Galleries	Folder path, hierarchy, public path, metadata, visibility, access, branding, presentation overrides, and operational ordering.
Images	Gallery ownership, relative path, filename, metadata, dimensions, visibility, checksum, EXIF data, ordering, and derived state.
Tags	Reusable tag identity plus gallery and image relationships.
Votes	Gallery and image scoring records with viewer association.
Thumbnails	Valid generated variant inventory, dimensions, format, status, and derivative version.
Settings	Scalar application, theme, feature, update, maintenance, and rendering values.
Audit and telemetry	Administrator events, operational diagnostics, privacy-controlled measurements, and rollups.
Duplicate ledger	Per-administrator canonical pair decisions and exact-gallery suppression rules.
Jobs and tokens	Bounded browser, detector, migration, WebDAV, reset, sharing, and maintenance state where persistence is required.

25.2 Relations and cascades

Foreign keys express ownership where schema evolution and compatibility permit it. Cascades remove dependent workflow records when their parent identity disappears. Content deletion continues through application services so filesystem and database effects remain coordinated. Unique constraints encode identities such as canonical duplicate pairs, exact administrator-gallery rules, slugs, tokens, and metadata variants.

25.3 Application settings

The `app_settings` table stores key-value configuration. `app_setting()` reads values, `set_app_setting()` persists normalized values, and focused services interpret domain-specific settings. Public language configuration uses `public_language` for the site default, `public_language_selector_enabled` for the default-on viewer override feature, and `public_language_selector_languages` for its ordered non-empty language list. The public thumbnail renderer uses `public_thumbnail_rendering_mode` with `responsive` and `progressive` values.

26 Gallery-scoped API-key reference

Complete reference for the original gallery-key protocol. Authentication does not expose automatic browsing. All examples contain synthetic identities; use only galleries and keys that you are authorized to operate.

26.1 Scope and support boundary

This reference describes the existing gallery-scoped `pgu_` HTTP API. It is intended for independently written clients, not only the Windows uploader. The reviewed baseline is PHP Gallery `1.0.4`, commit `150160d3771f97f5b96dab86e98ac6fe5365ccb4`. This identifies the source checked for this edition; it is not runtime version negotiation or a promise that every older installation has every optional action.

Only explicit operations authorized by a gallery key belong here. Automatic browsing, server-wide gallery discovery/listing/search, filesystem discovery, Watcher traversal, browser-approved linking and the resource API from issue #345 are not part of this public contract. WebDAV is a separate protocol with separate credentials. An inventory request compares caller-supplied file candidates; it is not an endpoint for browsing all galleries. Internal post-upload registration is server implementation, not an exposed scan command.

The key fixes the upload target. It is not necessarily upload-only: when gallery migration is enabled, that same key can export the authorized gallery and its descendants, and a destination key can import a new child tree. Do not hand it to an untrusted party as a write-only credential. Choosing `include_subgalleries=0` limits one export request, not the authority of the key itself. No new permissions, endpoints or server behavior are introduced by this documentation.

26.2 Operation index

The first nine actions below share index.php?page=`upload_automation_upload`. Migration uses the eight individually named routes. Every call requires the appropriate gallery key; specialized actions add their own gates. The document covers 17 operations. Identifiers and code remain identical in all languages.

Action or route	HTTP method and request encoding
`upload`	POST; multipart/form-data
`inventory`	POST; application/json
`revoke`	POST; application/json | form
`ai_next_job`	POST; application/json
`ai_heartbeat`	POST; application/json
`ai_complete`	POST; application/json
`ai_asset`	POST; form
`flight_report_status`	POST; application/json
`flight_report`	POST; multipart/form-data
`gallery_migration_manifest`	GET / POST; query / form
`gallery_migration_asset`	GET / POST; query / form

Action or route	HTTP method and request encoding
`gallery_migration_package`	POST; form
`gallery_migration_receive_manifest`	POST; form
`gallery_migration_receive_asset`	POST; multipart/form-data
`gallery_migration_receive_package`	POST; multipart/form-data
`gallery_migration_receive_status`	GET / POST; query / form
`gallery_migration_receive_complete`	POST; form

26.3 Connection, keys and request conventions

In Admin, open the intended gallery, then its API tab. Generate a separately labelled key for each client and copy both the exact upload endpoint and the key. The raw key is displayed once; only its SHA-256 hash is stored. Ordinary issued keys use `pgu_` followed by 64 hexadecimal characters. This API has no refresh-token exchange or general expiry field. A key must remain active and not revoked; it can also cease to work when its gallery or required storage is unavailable. Rotate by creating and verifying a replacement for the same gallery before revoking the old key. Do not publish credentials in source, screenshots, URLs, logs or shared shell history.

Use the copied canonical front-controller URL, preserving any installation subdirectory. The portable upload URL is `<base>/index.php?page=upload_automation_upload`. `/api/upload` is a rewrite-dependent compatibility alias, not the canonical address. Migration uses the same `<base>/index.php?page=...` form with the exact route below. API keys are not Admin sessions; key creation and global key listing require authenticated Admin UI/CSRF and are not documented key-authenticated operations.

Field	Type, requirement and meaning
`X-Gallery-API-Key`	Recommended header. First nonempty dedicated-header value wins after trimming.
`Authorization:Bearer`	Fallback when the dedicated header is empty. The bearer is still the original gallery key, not a linked access token.
`api_key`	Legacy scalar form field, used only if neither header supplied a usable credential. Not read from the query or a JSON object.
`Accept:application/json`	Request JSON refusals even for multipart calls. Binary success responses retain their actual media type.
`action`	For upload/automation, nonempty form value takes precedence over JSON; omitted/empty selects upload. Use only explicitly documented actions.

Send exactly one credential form. An invalid higher-priority key is not rescued by a valid lower-priority one. Keep TLS verification enabled and use HTTPS for remote deployments; the legacy

handler is not a substitute for HTTPS enforcement at the hosting boundary. Refuse redirects while holding credentials, especially changes of host or scheme. Do not reuse browser cookies or expose the key in public JavaScript. Cross-origin browser CORS/preflight support is not promised by this contract.

All code below uses synthetic URLs and shell variables. Set `API_URL` to the copied endpoint and supply `PG_API_KEY` securely in the process environment; no real key belongs in the examples. A curl transport exit code is not sufficient: inspect the appended HTTP status and the JSON `ok` field. Do not use redirect following or disable certificate verification.

```
API_URL='https://gallery.example/photos/index.php?page=upload_automation_upload'
: "${PG_API_KEY:?Set PG_API_KEY securely before running requests}"
# Never add -L or --insecure to credential-bearing calls.
```

26.4 Upload photographs

Operation: `POSTindex.php?page=upload_automation_upload`, `multipart/form-data`, `action=upload` (also the default). Authentication and `upload_api` must be available. Do not manually set a multipart boundary when using curl. No Admin cookie or CSRF field is required for these key-authenticated calls.

Field	Type, requirement and meaning
`image` / `images[]`	Binary; one file or repeated files, required. Prefer one spelling per request; a nonempty images[] input wins. The server validates filenames and actual image bytes.
`gallery_id`	Optional integer assertion. A positive value different from the key gallery returns 403. Omitting it does not broaden authority.
`create_thumbnails`	Optional scalar flag; default true. True spellings: 1/true/yes/on, case-insensitive. Use 0 to suppress server derivative completion. Existing valid client derivatives may still be installed.
`image_client_ids[]`	Optional per-original correlation strings; [A-Za-z0-9_-], 1..80 characters. Aligned with original-file positions, not persistent operation IDs and not idempotency keys.
`client_thumbnails[]`	Optional binary derivative files. Each requires the corresponding three aligned thumbnail metadata arrays below.
`thumbnail_client_ids[]`	String; required for each thumbnail. Must match a valid `image_client_ids` value in this request.
`thumbnail_sizes[]`	Integer target long side in pixels; one of 300, 600, 800, 960, 1280, 1600. Actual decoded long side must not exceed the declaration by more than 4 pixels.
`thumbnail_formats[]`	String jpg or webp (normalized to lowercase); detected MIME must match. Valid extra JPG variants may be counted as skipped on a WebP-only installation, rather than rejecting the original.
`sim_location_source`	Optional string; only `simconnect_camera` enables the following shared camera coordinates.

Field	Type, requirement and meaning
`sim_camera_latitude` / `sim_camera_longitude`	Optional finite numbers, both required for accepted camera metadata. Latitude -90..90; longitude -180..180; rounded to 7 decimals. Invalid metadata is ignored, not a new GPS write endpoint.
`sim_camera_altitude`	Optional finite altitude in feet (legacy SimConnect unit; no conversion by this handler), rounded to 2 decimals; absent/invalid becomes null. One camera tuple applies to the accepted images of the request.

Baseline image types are JPG/JPEG, PNG, GIF and WebP. HEIC/HEIF and DNG depend on the actual server converters; do not infer support from a filename alone. No PDF, music, arbitrary ZIP or JSON/base64 image upload belongs to this operation. PHP `upload_max_filesize`, `post_max_size`, `max_file_uploads` and the hosting proxy can impose tighter bounds. There is no advertised universal legacy batch size or per-request rate budget. Use modest batches; do not copy limits from the separate linked API.

The server natural-sorts uploaded filenames and may choose unique names or apply configured renaming. Do not assume response positions preserve arbitrary input order. When using position-based thumbnail correlation, submit originals already in the same natural filename order; a single original with its derivatives avoids ordering ambiguity. Re-uploading an original is not checksum-based deduplication and can create another file. Read the returned filenames and image IDs, not a guessed URL. Modern derivative policy is WebP-first; disabling derivative creation does not guarantee previews already exist.

```
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --form 'action=upload' --form 'image=@photo.jpg' \
  --form 'create_thumbnails=1' \
  --write-out '\nHTTP %{http_code}\n' "$API_URL"
```

```
{
  "ok": true, "gallery_id": 42, "gallery_title": "Example",
  "gallery_url": "https://gallery.example/photos/gallery/example",
  "edit_url": "/photos/index.php?page=admin_edit_gallery&id=42",
  "uploaded": 1, "scanned": 1, "image_ids": [501],
  "filenames": ["photo.jpg"], "renamed": 0,
  "rename_warnings": [], "rename_failures": [],
  "thumbnails": 6, "thumbnail_failed": 0, "thumbnail_errors": [],
  "post_upload_errors": [],
  "sim_camera_metadata": {"attached": 0, "skipped": 0, "error": ""},
  "client_thumbnails": {
    "installed": 0, "skipped": 0, "failed": 0, "errors": []
  }
}
```

The example is one possible response, not guaranteed counts or URL spelling. `uploaded`, `scanned`, `renamed`, `thumbnails` and `thumbnail_failed` are integer counters; `image_ids` is an integer list; `filenames`, `rename_warnings`, `rename_failures`, `thumbnail_errors` and `post_upload_errors` are string lists. Nested camera counts/error and client-thumbnail counts/errors must also be inspected. HTTP 200 with `ok=true` can coexist with failed derivative, GPS or bookkeeping

work. It is not an atomic all-or-nothing transaction. The upload catch maps failures to 422; shared authentication failures are described below.

The following optional example attaches one prepared WebP derivative to one original. The thumbnail must already have the declared dimensions and actual WebP content; neither its filename nor its MIME declaration can bypass server validation. Multiple originals can instead use repeated images[] fields, with aligned `image_client_ids`[] in the server filename order.

```
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --form 'action=upload' --form 'images[]=@photo.jpg' \
  --form 'image_client_ids[]=photo-001' \
  --form 'client_thumbnails[]=@photo-300.webp;type=image/webp' \
  --form 'thumbnail_client_ids[]=photo-001' \
  --form 'thumbnail_sizes[]=300' --form 'thumbnail_formats[]=webp' \
  --form 'create_thumbnails=1' \
  --write-out '\nHTTP %{http_code}\n' "$API_URL"
```

26.5 Inventory: compare supplied file candidates

Operation: `POSTindex.php?page=upload_automation_upload`, `application/json`, `action=inventory`. It returns matches for supplied candidates in the one authorized gallery, not a complete photo catalogue or a server-wide search. An empty files list can check gallery identity and returned capabilities without uploading anything. JSON calls must authenticate in a header.

Field	Type, requirement and meaning
`action`	Required string inventory.
`files`	Array of candidate objects; omitted or non-array normalizes to empty. Invalid rows are discarded. Up to 5000 distinct SHA-256 values; duplicate hashes replace the earlier candidate.
`files[].sha256`	Required 64 hexadecimal characters; trimmed and lowercased. Hash the exact original bytes sent to the server.
`files[].filename`	Optional string; path components are removed. Empty becomes sha256. This is a display/correlation name, not a destination path.
`files[].size`	Optional integer bytes; normalized to at least zero. Positive size provides a secondary match check and is required for disk fallback.
`files[].client_id`	Optional trimmed string echoed for mapping; default sha256. It is not the `image_client_ids` upload identifier or a durable receipt.
`deep_check`	Optional JSON boolean, default false; only literal true activates it. Disk fallback considers at most the first 8 normalized candidates. This bounds candidate count, not the directory traversal time or number of directory entries inspected.

```
python - <<'PYJSON' > inventory.json
import hashlib
```

```
import json
from pathlib import Path
path = Path("photo.jpg")
digest = hashlib.sha256(path.read_bytes()).hexdigest()
print(json.dumps({"action": "inventory", "deep_check": False,
    "files": [{"client_id": "photo-1", "filename": path.name,
               "size": path.stat().st_size, "sha256": digest}]}))
PYJSON
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --header 'Content-Type: application/json' \
  --data-binary @inventory.json \
  --write-out '\nHTTP %{http_code}\n' "$API_URL"
```

```
{
  "ok": true, "action": "inventory", "gallery_id": 42,
  "gallery_title": "Example", "gallery_folder": "example",
  "checked": 1, "matched": 1,
  "existing": [{
    "client_id": "photo-1", "filename": "photo.jpg", "size": 1024,
    "sha256": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
    "remote_filename": "photo.jpg", "remote_relative_path": "photo.jpg",
    "remote_image_id": 501, "matched_by": "checksum_sha256"
  }],
  "deep_check_applied": false,
  "fingerprint": "opaque-server-fingerprint",
  "server_time": "2026-11-09 10:00:00",
  "capabilities": {"flight_reports_v1": true}
}
```

HTTP 200 returns `ok`, `action`, integer `gallery_id`, string `gallery_title` and `gallery_folder`, integer `checked` and `matched`, an `existing` list, boolean `deep_check_applied`, opaque `fingerprint`, SQL-style `server_time`, and optional `capabilities`. Each match echoes `client_id`/filename/size/sha256 and supplies `remote_filename`/`remote_relative_path`/`remote_image_id`/`matched_by`. Paths are metadata, not public media URLs. `checksum_sha256` denotes an indexed match; `disk_sha256` can have `remote_image_id=0`. No row means no observed match, not proof that a previous request never committed. Keep fingerprints opaque; SQL-style timestamps carry no explicit timezone offset.

26.6 Partial results, timeouts and recovery

After a confirmed success, retain the receipt and inspect warnings. After a timeout, disconnect, malformed response or truncated JSON, mark the upload outcome unknown and retain the original file. Do not resend automatically, delete local files, change the target, or assume that 422 implies rollback. Probe inventory with the original bytes and, only when necessary, a small explicit `deep_check` request. A positive match observes content, not request-specific completion; a disk match does not prove database registration. Reconcile the gallery and warnings before an operator deliberately decides whether another upload is appropriate. This protocol exposes no general photo-upload operation-status receipt, cancellation endpoint, universal Idempotency-Key or exactly-once guarantee.

26.7 Revoke the current key

Operation: POST to the upload endpoint, `action=revoke`, as JSON or URL-encoded form. No `token_id` selector is accepted for this action: it revokes the credential authenticating this request, for its gallery. The key cannot create a replacement or revoke arbitrary other keys. Stop and reconcile pending work first. This is an explicit destructive credential action; never run it as a routine connection test. It does not delete photos. Subsequent use of the revoked key receives 401 on the enabled upload route; replay is not a second successful revocation receipt.

```
# OPTIONAL: this invalidates the currently supplied key.
curl --silent --show-error --proto '=https' --max-time 30 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --data-urlencode 'action=revoke' \
  --write-out '\nHTTP %{http_code}\n' "$API_URL"
```

```
{"ok": true, "action": "revoke", "gallery_id": 42,
 "token_id": 7, "message": "API key revoked."}
```

Success is HTTP 200. A failed revocation returns 422 with ok=false/error. Losing the response requires checking current key usability or Admin key state; it does not authorize a new key automatically.

26.8 Specialized AI worker contract

The following four actions reuse the upload endpoint and gallery key, but are specialized worker operations, not generic photo browsing. The installation must enable `upload_api` and `ai_image_metadata` and have the AI queue/metadata schema. The worker must hold the current `job_id` plus the secret `claim_token` as well as the gallery key. Claims are scoped to that gallery; `worker_id` is diagnostic, not authentication. Do not log claim tokens. No automatic browser, filesystem crawler or perpetual polling loop is part of these examples.

Common AI responses: 403 if AI metadata is disabled; 409 if its schema is missing; 503 if storage cannot be verified; 400 for an unknown ai_ action; 422 for other caught worker failures. The shared upload authentication/feature refusals still precede these checks. A lost claim response is not permission to complete another job: only a received claim token proves possession. Completion retry semantics described below do not make photo uploads idempotent.

26.9 Claim one AI job: `ai_next_job`

POST, application/json, action=`ai_next_job`. This is a state-changing allocation: missing jobs may be enqueued and one job claimed for direct images in the key gallery. It is not a read-only connection probe.

Field	Type, requirement and meaning
`worker_id`	Optional string; default unknown-worker; unsupported characters replaced with underscore, at most 190 characters.
`model_name` / `model_version`	Optional strings; defaults local-image-metadata / 1; whitespace collapsed, at most 120 characters each. Keep stable to identify the intended model/version queue.

Field	Type, requirement and meaning
`lease_seconds`	Optional integer seconds, default 600. Nonpositive values use default; otherwise clamped to 60..3600. Same normalization applies to heartbeat.

```
{"action":"ai_next_job","worker_id":"example-worker",
 "model_name":"local-image-metadata","model_version":"1",
 "lease_seconds":600}
```

```
{
  "ok": true, "asset_action": "ai_asset",
  "job": {
    "job_id": 91, "claim_token": "REPLACE_WITH_RECEIVED_CLAIM_TOKEN",
    "lease_seconds": 600, "lease_expires_at": "2026-11-09 10:10:00",
    "model_name": "local-image-metadata", "model_version": "1",
    "gallery_id": 42, "gallery_title": "Example",
    "image": {
      "id": 501, "filename": "photo.jpg", "title": "Example",
      "relative_path": "photo.jpg", "mime_type": "image/jpeg",
      "file_size": 1024, "modified_at": "2026-11-09 09:00:00",
      "checksum_sha256": "
    aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
      "width": 1600, "height": 900
    }
  }
}
```

HTTP 200 returns a job object with the complete fields illustrated above, or job=null with `poll_after_seconds`=60 and a message. Integer fields are `job_id`, `lease_seconds`, `gallery_id`, `image.id`, `image.file_size`, `image.width`, `image.height`; other shown job/image fields are strings. Respect the suggested idle delay and the lease deadline. The placeholder is deliberately not a valid claim token.

26.10 Renew an AI claim: `ai_heartbeat`

POST, application/json, action=`ai_heartbeat`. Extends the current live claim; it cannot revive an expired or foreign claim.

Field	Type, requirement and meaning
`job_id` / `claim_token`	Required positive integer and exact secret string from the claim response. The gallery key is still required in the header.
`lease_seconds`	Optional; same 600 default and 60..3600 second bounds as claim.
`progress_percent`	Optional integer; default 0, clamped to 0..99. Completion is a separate action.
`message`	Optional string; default Worker heartbeat.; stored text limited to 500 characters.

```
{"action":"ai_heartbeat","job_id":91,
 "claim_token":"REPLACE_WITH_RECEIVED_CLAIM_TOKEN",
 "lease_seconds":600,"progress_percent":40,"message":"Analyzing image"}
```

```
{"ok":true,"job_id":91,"message":"Heartbeat accepted."}
```

HTTP 200 confirms renewal; 409 returns ok=false/error when the claim is invalid or expired. Common AI errors also apply.

26.11 Complete or fail an AI job: `ai_complete`

POST, application/json, action=`ai_complete`. Success stores model metadata and searchable text; it is not an arbitrary photo-title/tag/settings mutation API.

Field	Type, requirement and meaning
`job_id` / `claim_token`	Required claim identity, as for heartbeat.
`status`	Optional string; default succeeded. Send succeeded or failed explicitly. Current handler treats every other value as failure; do not rely on misspellings.
`metadata`	Success: JSON object, default empty. Worker-owned structured metadata, no fixed required tag vocabulary. Non-object/non-array values fail with 422. Do not store secrets or diagnostics as searchable content.
`searchable_text`	Success: optional string, default empty. Combined with scalar metadata text to depth 5, diagnostic keys excluded, whitespace normalized; final text limited to 60000 characters.
`error`	Failure: optional string, limited to 2000 characters. A server fallback message is used when omitted. Do not include credentials.

```
{"action":"ai_complete","job_id":91,
 "claim_token":"REPLACE_WITH_RECEIVED_CLAIM_TOKEN",
 "status":"succeeded","metadata":{"objects":["aircraft"]},
 "searchable_text":"Aircraft on the apron"}
```

```
{"ok":true,"job_id":91,"status":"succeeded","message":"AI metadata stored."}
```

Failure acknowledgement is also HTTP 200 with status=failed and message="AI job failure recorded."; it does not mean analysis succeeded. An invalid/expired claim gives 409. Repeating a successful completion with the same already-succeeded job/claim can acknowledge the stored result without replacing it; this does not verify that a repeated payload is identical. A failure may be requeued by server policy (maximum 5 attempts, exponential backoff capped at 3600 seconds). The client must not manufacture a new claim or blindly resubmit after 409.

26.12 Fetch the claimed AI input: `ai_asset`

POST to the upload endpoint using application/x-www-form-urlencoded (multipart also populates the required form fields). Required fields are action=`ai_asset`, positive integer `job_id` and the exact `claim_token`. Unlike the other AI actions, this handler reads `job_id` and `claim_token` from form data, not JSON. Success is HTTP 200 with image bytes, detected Content-Type, Content-Length, Content-Disposition attachment, X-Content-Type-Options nosniff and Cache-Control no-store, private. The server can return an existing display representation rather than original RAW bytes; do not assume its hash equals the original checksum in the job. It does not generate a missing derivative just to serve this action.

```
# CLAIM_TOKEN and JOB_ID come from a successful ai_next_job response.
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --data-urlencode 'action=ai_asset' \
  --data-urlencode "job_id=$JOB_ID" \
  --data-urlencode "claim_token=$CLAIM_TOKEN" \
  --dump-header ai-asset.headers --output ai-asset.response \
  --write-out 'HTTP %{http_code}\n' "$API_URL"
```

An invalid claim or unavailable asset returns JSON 409 with ok=false/error; other common AI failures remain JSON. Check HTTP status and Content-Type before treating the saved response as an image.

26.13 Look up a PDF report receipt: `flight_report_status`

POST to the upload endpoint, application/json, action=`flight_report_status`. This is the specialized report attachment protocol, not unrestricted PDF management. The inventory response can advertise capabilities.`flight_reports_v1`=true; older responses may omit capabilities. That advertisement is not a readiness guarantee. On the audited revision it is unconditional after successful inventory and does not independently verify PDF storage or the PDF presentation switch. Do not infer other permissions or UI availability from it.

Field	Type, requirement and meaning
`report_id`	Required string, 1..80 ASCII characters [A-Za-z0-9_-]. Stable caller identity reused for lookup and upload.
`language`	Required string en/cs/de/sv, normalized to lowercase. The Czech wire code is cs, not the manual filename suffix CZ.
`sha256`	Required 64-hex digest of the expected PDF; trimmed and lowercased. Identity is `report_id` + language within the key gallery.

```
{"action":"flight_report_status","report_id":"flight-demo-001",
 "language":"cs",
 "sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"}
```

```
{"ok":true,"action":"flight_report_status","found":false,
 "conflict":false,"report":null}
```

HTTP 200 has found boolean, conflict=false, and report=null when no verified attachment is observed, otherwise report={id:string, sha256:string, size:integer bytes, filename:string}. A matching receipt has found=true. The lookup is based on verified indexed files; found=false is not proof that no lost write ever happened. Conflicting content or duplicate identities return HTTP 409 with ok=false, action, conflict=true and error. Invalid input or unverifiable status returns 422; a missing/non-JSON status payload returns 400. No overwrite flag is offered.

26.14 Attach a PDF report: `flight_report`

POST to the upload endpoint, multipart/form-data, action=`flight_report`. Required scalar fields `report_id`, language and sha256 use the same rules as status. Also send `session_id` (1..80 characters [A-Za-z0-9_-]) and one binary file in report. The `session_id` is validated but is not part of the stored report identity. The actual PDF must match the submitted SHA-256; valid files are 16 bytes through 25 MiB (26214400 bytes), with PDF signature, EOF marker and accepted detected PDF MIME. Hosting limits may be lower.

```
PDF_SHA=$(python -c \
 'import hashlib; print(hashlib.sha256(open("report.pdf","rb").read()).hexdigest())
    ')
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_API_KEY" \
  --header 'Accept: application/json' \
  --form 'action=flight_report' --form 'report_id=flight-demo-001' \
  --form 'session_id=session-demo-001' --form 'language=cs' \
  --form "sha256=$PDF_SHA" --form 'report=@report.pdf;type=application/pdf' \
  --write-out '\nHTTP %{http_code}\n' "$API_URL"
```

```
{"ok":true,"action":"flight_report","found":true,"conflict":false,
 "report":{"id":"0123456789abcdef0123456789abcdef",
 "sha256":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
 "size":2048,"filename":"flight-report-0123456789abcdef-cs.pdf"}}
```

HTTP 200 returns a stored or already-existing matching receipt. Resubmitting the same `report_id`/language and identical PDF can return that receipt; another checksum for the same identity returns 409 and is not overwritten. Missing/incomplete PDF, validation/storage failure or checksum mismatch returns 422. After a lost response, query `flight_report_status` with the original identity/checksum before any deliberate retry. This specific receipt behavior must not be generalized to ordinary photo upload, inventory or arbitrary PDF editing. The API does not expose report deletion, replacement or media browsing.

26.15 Migration: authority and data contract

Migration is an explicit transfer of the gallery authorized by a source key into a new child below the gallery authorized by a destination key. It is not server discovery or a general browsing API. Each installation must enable `gallery_migration` and have the required schema. The audited `protocol_version` is 2; source and destination `app_version` strings must match exactly. Preserve the server-produced manifest, its identities, order and optional fields instead of synthesizing a manifest from a local directory.

The source export includes descendants by default. Set `include_subgalleries`=0 to request only the root. This parameter narrows that export request, not the permanent authority of the key: a source asset request may authorize a descendant in the same source tree. Treat a key as access

to export data, not merely upload permission. A target key authorizes creation of an imported child/tree, not arbitrary root creation or mutation of unrelated galleries. Imported metadata can retain source visibility; use a protected destination and review the result before exposing it.

All migration URLs use index.php?page=<route>. GET fields belong in the query; POST scalar fields use application/x-www-form-urlencoded or multipart/form-data. JSON serialized inside `manifest_json` or `assets_json` is a form value, not an application/json request body. On GET/POST routes the POST field wins when both locations supply the same field. Always use a header credential for GET. Successful JSON uses application/json; charset=utf-8. Error payloads have ok=false and error:string, with action-specific status codes below. Wrong methods return 405. Do not depend on localized error-message wording.

Field	Type, requirement and meaning
`scope`	Asset selector, string image (default) or gallery.
`source_gallery_id`	Source gallery integer. Send the exact manifest value; source-side zero/omission falls back to the key root. Destination references must identify the manifest entry.
`kind`	Required string: image original or thumbnail; gallery `cover_image_path`, `banner_image_path`, `logo_image_path` or `separator_image_path`.
`source_image_id`	Integer for an image asset; historical alias `image_id` is accepted when `source_image_id` is absent. No image ID is needed for gallery artwork.
`size` / `format`	Thumbnail identity only: size integer pixels, format jpg or webp. Copy the descriptor, not a guessed size. size here is not byte length; byte length is `file_size`.
`file_size` / `checksum_sha256` / `mime_type`	Descriptor metadata: byte count, SHA-256 digest and MIME string. Relay unchanged and verify downloaded bytes where a digest is supplied.
`filename` / `relative_path` / `asset_key` / `label`	Descriptor values, not arbitrary path access. `asset_key` is an opaque 24-hex identity added to transfer-plan entries; label is display text. Never construct filesystem URLs from paths.

Migration transfers the assets enumerated by this protocol, not a complete installation backup. General PDF-document sidecars and music repositories/playlists are not automatically covered. Do not invent downloadable URLs or promise transfer of unlisted hidden sidecars. Keep a separate filesystem/database backup when preserving the entire installation.

26.16 Export the manifest

```
GET|POST index.php?page=gallery_migration_manifest
```

Authenticate with the source gallery key. Optional `gallery_id` is a positive integer assertion; a different root returns 403. `include_subgalleries` defaults to true; the values 0, false, off and no (case-insensitive after trimming) select root only. Success is HTTP 200 with ok, `app_version`, `protocol_version` and manifest. Exceptions in authentication, schema lookup or manifest generation are all mapped to 401 by this handler; 401 is therefore not exclusively

a bad-key diagnosis.

Field	Type, requirement and meaning
`protocol_version` / `app_version`	Integer 2 and exact source application-version string. The destination requires the same application version.
`migration_id` / `source_instance_id`	Opaque strings generated by the source. Preserve both; the target uses source/manifest identity to derive a resumable job.
`source_gallery_id` / `source_gallery_folder`	Integer authorized root ID and its gallery-relative folder string, not an authorization token or direct media URL.
`include_subgalleries` / `generated_at` / `counts`	Boolean export selection, server SQL timestamp string and object {galleries:integer, images:integer, assets:integer}.
`galleries[]`	Parent-first entries: `source_id`, `parent_source_id` (root 0), `source_folder`, `folder_name`, `relative_folder` (root empty), gallery, `gallery_assets`[], images[] and `flight_map` (nullable).
`gallery` / `gallery_assets` / `images` / `flight_map`	Root mirrors retained for compatibility alongside galleries[]. Preserve them; do not merge different source snapshots.

The nested gallery metadata carries only fields present in storage, plus tags (string), `description_layout_semantics_version`, `cover_source_id` and optional `cover_selection_source_ids` (integer array, including empty Automatic selection). Optional `content_language` and translations depend on localization schema. Image metadata includes `source_id`, `source_gallery_id`, tags and assets, plus present persisted attributes listed below. Numeric database values can be strings, and optional values can be null: do not coerce or translate these version-bound transfer records. These are migration records, not general PATCH operations. Pass the entire server-issued object unchanged.

```
gallery:
title, slug, description, sort_order, visibility, voting_enabled,
show_filenames, newest_uploads_first, gallery_date, gallery_date_end,
description_layout, count_badge_visibility, picture_game_enabled,
gps_map_enabled, grid_columns, grid_rows, grid_use_for_subgalleries,
thumbnail_min_size, thumbnail_max_size, background_source,
banner_image_path, logo_image_path, separator_image_path,
cover_image_path, nsfw_enabled

images[]:
relative_path, filename, title, description, width, height, mime_type,
file_size, modified_at, checksum_sha256, sort_order, visibility,
exif_taken_at, exif_camera_make, exif_camera_model, exif_lens_model,
exif_focal_length, exif_aperture, exif_exposure_time, exif_iso,
gps_lat, gps_lng, gps_altitude, gps_extracted_at, nsfw_enabled,
thumbnail_min_size, thumbnail_max_size
```

```
SOURCE_BASE='https://source.example/photos'
: "${PG_SOURCE_KEY:?Set PG_SOURCE_KEY securely}"
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_SOURCE_KEY" \
  --header 'Accept: application/json' \
  --get --data-urlencode 'include_subgalleries=0' \
  "$SOURCE_BASE/index.php?page=gallery_migration_manifest" \
  --output manifest-response.json --write-out 'HTTP %{http_code}\n'
```

```
# Validate HTTP/ok before using the file. Extract only the manifest member.
python -c 'import json; r=json.load(open("manifest-response.json")); assert r["ok"]
    is True; json.dump(r["manifest"],open("manifest.json","w"))'
```

When present, `flight_map` is an object with `map_source_type`:string, `route_text`:string, `resolved_points_json`:string, `unresolved_points_json`:string, `point_count`:integer and `resolved_at`:string. The two *_json fields contain serialized JSON strings, not nested arrays. Keep them unchanged with the version-bound manifest; no separate route creation API is implied.

26.17 Download one source asset

```
GET|POST index.php?page=gallery_migration_asset
```

Use the source key and the asset selectors defined above. The server resolves the selected original, existing thumbnail or gallery artwork within the authorized root/tree. A source path supplied by the caller cannot choose an arbitrary file. No manifest `job_id` is required on this source route. Descendant authorization is checked for each request, independently of whether the preceding manifest excluded descendants. Success is HTTP 200 binary with detected Content-Type, Content-Length and attachment Content-Disposition. Exceptions, including authentication or missing files, map to 404 JSON; wrong method is 405. Do not save a JSON error as a photograph.

```
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_SOURCE_KEY" \
  --header 'Accept: application/json' \
  --get --data-urlencode 'scope=image' \
  --data-urlencode 'source_gallery_id=12' \
  --data-urlencode 'source_image_id=34' --data-urlencode 'kind=original' \
  "$SOURCE_BASE/index.php?page=gallery_migration_asset" \
  --output source-asset.bin --write-out 'HTTP %{http_code}\n'
# IDs must come from the actual authorized manifest, not this example.
```

26.18 Download a source ZIP package

```
POST index.php?page=gallery_migration_package
```

Source key, form fields: `assets_json` (required JSON-encoded nonempty descriptor array, at most 2048 assets) and `include_subgalleries` (optional, true by default). Obtain the exact asset group from the destination package plan. The source resolves each descriptor, verifies available size/checksum evidence and emits a ZIP using stored entries (ZIP compression method 0). ZipArchive support is required. Success: HTTP 200 application/zip, Content-Length, attachment filename php-gallery-migration-package.zip. Exceptions including invalid credentials, descriptor/size/hash problems or ZIP failure return 422 JSON. Wrong method returns 405.

```
# package-assets.json is one packages[i].assets array from the target plan.
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_SOURCE_KEY" \
  --header 'Accept: application/json' \
  --data-urlencode 'assets_json@package-assets.json' \
  --data-urlencode 'include_subgalleries=0' \
  "$SOURCE_BASE/index.php?page=gallery_migration_package" \
  --output transfer.zip --write-out 'HTTP %{http_code}\n'
```

26.19 Prepare the destination job

```
POST index.php?page=gallery_migration_receive_manifest
```

Destination gallery key, required form field `manifest_json` containing only the full manifest object from the source response. This is a mutation: it prepares a server job and the imported child gallery/tree below the key gallery. Reuse the exact manifest and destination to recover a compatible existing job; do not obtain a changed snapshot or choose a new target after an ambiguous response. The server-generated `job_id` is opaque; persist it with source/destination identity and manifest, but without secrets. Application versions must match exactly and `protocol_version` must be supported. Exceptions (authentication included) return 422 with ok=false/error; wrong method returns 405.

Field	Type, requirement and meaning
`ok` / `app_version` / `job_id`	HTTP 200: true, destination version string and job identity string.
`compatibility`	Object {ok:boolean, `source_version`:string, `target_version`:string, policy:"exact", message:string}. Incompatible requests fail before successful preparation.
`target_gallery_id` / `imported_root_gallery_id` / `gallery_ids`	Authorized parent integer; created/imported root integer; array of imported gallery integers. Parent and imported root are distinct concepts.
`assets[]` / `packages[]`	Ordered asset descriptors with `asset_key`/label; package objects {`package_id`:string, `asset_keys`:string[], assets:object[], `asset_count`:integer, `source_bytes`:integer}. Use the returned plan, not independently regrouped ZIPs.
`counts` / `status`	Counts object {galleries:integer, images:integer, assets:integer, packages:integer, received:integer}. status is the job-status object described below (nullable in the controller envelope).

Package planning respects destination upload limits and a soft batch target. One original plus its derivatives can exceed the soft target while staying within the hard limit; a group above the hard limit is rejected. Do not advertise a universal 24 MiB upload maximum merely because that is a fallback planning target. The plan may contain no assets for an empty gallery, which still needs completion.

```
TARGET_BASE='https://target.example/gallery'
: "${PG_TARGET_KEY:?Set PG_TARGET_KEY securely}"
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_TARGET_KEY" \
  --header 'Accept: application/json' \
  --data-urlencode 'manifest_json@manifest.json' \
  "$TARGET_BASE/index.php?page=gallery_migration_receive_manifest" \
  --output job-response.json --write-out 'HTTP %{http_code}\n'
# Preserve the validated job response before transferring any bytes.
```

26.20 Upload one asset to the destination

```
POST index.php?page=gallery_migration_receive_asset
```

Destination key and multipart/form-data. Required: `job_id`, the asset selector fields and a binary file in asset. Selectors must identify a planned manifest asset; the server checks job ownership, descriptor, content size/hash and destination safety. Transfer the returned asset order so originals precede their thumbnails. Gallery artwork requires scope=gallery and its exact kind. Success is HTTP 200 with ok=true, `job_id`, `asset_key`, `target_gallery_id` (the imported gallery receiving this asset), result, received and `total_assets`. This `target_gallery_id` can differ from the key parent. Exceptions including invalid credentials, unknown job/asset, missing upload or checksum mismatch return 422; wrong method 405.

Field	Type, requirement and meaning
`result:imageoriginal`	{scope:"image", kind:"original", `source_image_id`:integer, `image_id`:integer}; source and destination IDs differ.
`result:imagethumbnail`	{scope:"image", kind:"thumbnail", `source_image_id`:integer, `image_id`:integer, size:integer, format:string}.
`result:gallery`	{scope:"gallery", kind:string, `relative_path`:string}; kind is the artwork column from the descriptor.

```
{"ok":true,"job_id":"0123456789abcdef0123456789abcdef",
 "asset_key":"0123456789abcdef01234567","target_gallery_id":41,
 "result":{"scope":"image","kind":"original",
 "source_image_id":34,"image_id":100},"received":1,"total_assets":3}
```

26.21 Upload a planned ZIP package

```
POST index.php?page=gallery_migration_receive_package
```

Destination key, multipart/form-data: `job_id` and `package_id` are required strings from preparation; package is the required binary ZIP. No `assets_json` is accepted as a replacement destination plan. Send the unmodified ZIP generated by the source for that exact package. The receiver checks expected entry names/count, rejects duplicate/unexpected entries and compressed entries (only stored method 0), verifies asset evidence and uses its authorized job mapping. Some assets can be installed before a later failure; this is not an atomic rollback protocol. After an ambiguous upload, call `receive_status` for the same job before deciding to resend.

```
{"ok":true,"job_id":"0123456789abcdef0123456789abcdef",
 "package_id":"abcdef0123456789abcdef01",
 "asset_keys":["0123456789abcdef01234567"],
 "received":1,"total_assets":1}
```

HTTP 200 fields above contain strings/string arrays and integer counts. Authentication, job/plan mismatch, invalid ZIP, missing package, wrong content or incomplete transfer exceptions return 422 JSON. Wrong method returns 405. A package already recorded as received is not permission to switch its content or move it to a different job. Use current status and original package identity for recovery.

26.22 Reconcile destination progress

```
GET|POST index.php?page=gallery_migration_receive_status
```

Destination key and required `job_id`. Optionally include the six asset-selector fields to ask about one manifest asset. The same target-parent ownership is checked on every call. This handler reconciles stored receipt evidence against files; even GET can update recovery bookkeeping, so do not describe it as a side-effect-free snapshot. Missing/unmatched optional asset selection produces `asset_key`="" and `asset_received`=false rather than a global file-absence guarantee. Invalid key/job/target, corrupt job or reconciliation exceptions return 422; wrong method 405.


```
{"ok":true,"action":"status",
 "job_id":"0123456789abcdef0123456789abcdef","mode":"source_push",
 "target_gallery_id":40,"imported_root_gallery_id":41,"gallery_ids":[41],
 "received":1,"total_assets":3,"received_packages":0,"total_packages":1,
 "received_asset_keys":["0123456789abcdef01234567"],
 "asset_key":"","asset_received":false,
 "updated_at":"2026-11-09 12:00:00","server_time":"2026-11-09 12:00:03"}
```


The complete HTTP 200 shape is shown above. IDs/counts are integers, `gallery_ids` is an integer array, `received_asset_keys` is a string array, times are server SQL strings, and `asset_received` is boolean. `received_packages` counts packages for which every constituent asset is received. mode is `source_push` for jobs started through this public preparation route. There is no completed flag in this response. received==`total_assets` means bytes are accounted for, not that final metadata application succeeded; call `receive_complete` separately.

26.23 Complete the destination job

```
POST index.php?page=gallery_migration_receive_complete
```

Destination key and required form field `job_id`, no file. The server requires the authorized target job, all expected assets and usable schema, then applies metadata, source-to-target image/cover mappings and related finalization. This can change the imported gallery presentation/visibility. A lost success response is not proof of rollback. Preserve the original job and inspect its result; do not start another import merely because completion was ambiguous. No public cancel/delete-job or rollback endpoint is defined by this contract.


```
{"ok":true,"job_id":"0123456789abcdef0123456789abcdef",
 "target_gallery_id":40,"target_parent_gallery_id":40,
 "imported_root_gallery_id":41,"gallery_ids":[41],
 "assets_received":3,"total_assets":3,
 "gallery_url":"https://target.example/gallery/gallery/imported/",
 "edit_url":"https://target.example/gallery/index.php?page=admin_edit_gallery&id=41&
    tab=admin-edit-api#admin-edit-api"}
```


HTTP 200 contains the fields above; IDs/counts are integers, `gallery_ids` is an integer array and URLs are strings (use the returned values, not this illustrative spelling). Authentication, incomplete assets, target/schema/metadata failures return 422 with ok=false/error, wrong method 405. The Admin `edit_url` is a browser link requiring its normal Admin authentication; it does not grant API-key access to Admin functions.

26.24 HTTP errors, limits and compatibility

The following matrix describes handler behavior on the reviewed baseline, after the request reaches PHP. It is not a uniform REST status policy. Disabled-feature routing, invalid installation/schema state, PHP body limits, a reverse proxy or an upstream error can produce another status or HTML before the handler. Send Accept: application/json; validate status, Content-Type and JSON shape independently. Do not infer write rollback from any HTTP error or failed JSON parse. Do not branch on translated free-form error messages.

Operation group	Failure interpretation
All upload actions	405 wrong method; 503 token schema unavailable/unverifiable; 401 missing/invalid/revoked key; 404 key gallery missing. These occur before action dispatch.
`upload` / `inventory` / `revoke`	Upload: 403 positive gallery assertion mismatch, 422 caught upload failure. Revoke: 422 failed revocation. Inventory has no dedicated local exception-to-status mapping; infrastructure exceptions need not produce the usual JSON envelope.
`ai_*`	403 disabled AI feature, 409 missing queue schema, 503 unverifiable schema, 400 unknown AI action, 422 caught AI failure; heartbeat/complete/asset 409 for an invalid claim or unavailable claimed input.
`flight_report_status` / `flight_report`	Status: 400 missing JSON body; both: 409 conflicting PDF identity, 422 invalid input or status/storage failure.
`gallery_migration_manifest`	403 root assertion mismatch; 401 any caught exception including invalid key, unavailable schema or export failure; 405 wrong method.
`gallery_migration_asset`	404 caught authentication/resolution/read preparation failure; 405 wrong method. A failure after streaming begins need not be recoverable as clean JSON.
`gallery_migration_package` / `gallery_migration_receive_*`	422 any caught exception including key/target/job/version/payload/ZIP/integrity problems; 405 wrong method. Authentication errors are not universally 401 on these routes.

```
{"ok":false,"error":"Invalid or revoked API key."}
```

```
{"ok":false,"action":"flight_report_status","conflict":true,
 "error":"A different PDF is already stored for this report identity."}
```

These are separate error examples, not a universal shape requiring action or conflict. For a key-only caller without Admin cookies, a disabled `upload_api` or `gallery_migration` route normally returns 404 with {"ok":false,"error":"`not_found`"} when JSON is requested; an authenticated Admin context can instead get 403 `feature_disabled`. Keep API calls independent of browser sessions. The two JSON helpers are not identical: upload uses application/json, migration explicitly adds charset=utf-8. Do not assume all success/error replies set identical cache/security headers. Treat all authenticated metadata as private and never cache credentials.

Operational limits are the intersection of application validation, installed feature/schema support, PHP `upload_max_filesize/post_max_size/max_file_uploads`, web-server body/time limits

and client memory/time limits. A host can return 413 or 429 without those being uniform application-level guarantees. No universal Retry-After, pagination contract, resumable multipart upload, chunk API, rate quota or exactly-once guarantee is defined here. Retain all original files and uncertain-operation evidence. Inventory and report receipts have different meanings. Respect optional/missing response fields on older servers; no general API-version endpoint is promised. Migration alone requires exact application-version equality.

26.25 Complete Python upload, inventory and migration example

Save the following complete script as `gallery_api_example.py`. It needs Python 3.10+ and no third-party package. `PG_BASE`, `PG_SOURCE_BASE` and `PG_TARGET_BASE` are explicit installation-base URLs including any subdirectory, without index.php, query or fragment. Supply keys in environment variables only. The example deliberately bounds each file/binary response to 64 MiB and JSON to 8 MiB, with a 120-second socket timeout; these are client-example limits, not server limits. It buffers bounded files and uses ASCII multipart filenames; larger or differently named uploads require an appropriately streaming/encoding-aware client. urllib may use the configured operator proxy. TLS verification stays enabled and every redirect is refused.

The migrate command performs a complete new manifest -> destination preparation -> ordered per-asset transfer -> completion sequence for the explicitly selected source/target keys. It uses individual-asset routes so a ZIP tool is unnecessary; the package routes above provide a separate batching option. It refuses an existing work directory rather than accidentally starting a second import. Private manifest/job files survive an error; they may contain private gallery metadata, but never credentials. After an error, inspect the saved job and query its status manually. Do not simply rerun into a different directory. Missing job-response.json after preparation failure still requires reconciling the original saved manifest and target before another attempt. The script never removes source photographs or automatically retries writes. Printed upload JSON can contain warnings even when ok=true.

```
"""Example client for the documented gallery-key API. Author: Rudolf Klusal.

Requires Python 3.10+ and the standard library. It never follows redirects,
retries writes, deletes original files, or enumerates unrelated galleries.
The bounds below belong to this example, not to PHP Gallery's API contract.
"""

import argparse
import hashlib
import json
import os
from pathlib import Path
import secrets
import ssl
import sys
import urllib.error
import urllib.parse
import urllib.request

# Example-only memory/time bounds; stop explicitly when an input is larger.
FILE_LIMIT = 64 * 1024 * 1024
JSON_LIMIT = 8 * 1024 * 1024
TIMEOUT_SECONDS = 120
UPLOAD_ROUTE = "upload_automation_upload"
ASSET_FIELDS = (
    "scope", "source_gallery_id", "kind", "source_image_id", "size", "format"
```

```
)


class NoRedirect(urllib.request.HTTPRedirectHandler):
    """Refuse all redirect responses so credentials never follow a new URL."""

    def redirect_request(self, req, fp, code, msg, headers, newurl):
        """Return no replacement request; urllib reports an HTTP error."""
        return None


class Api:
    """Call fixed gallery routes with one installation-specific key."""

    def __init__(self, base: str, key: str) -> None:
        """Validate the explicit HTTPS installation base and retain its key."""
        parsed = urllib.parse.urlsplit(base)
        if (parsed.scheme != "https" or not parsed.hostname
                or parsed.username or parsed.password
                or parsed.query or parsed.fragment or not key):
            raise ValueError("Use an HTTPS installation base and a nonempty key")
        self.base = base.rstrip("/")
        self.key = key
        self.opener = urllib.request.build_opener(
            NoRedirect(), urllib.request.HTTPSHandler(
                context=ssl.create_default_context()
            )
        )

    def call(self, route: str, fields: dict | None = None,
             payload: dict | None = None,
             upload: tuple[str, str, bytes] | None = None,
             binary: bool = False) -> dict | bytes:
        """POST once; return verified JSON success or binary bytes, else raise.

        :param route: Fixed documented front-controller route.
        :param fields: Scalar form fields, not JSON request-body fields.
        :param payload: JSON body; mutually exclusive with fields/upload.
        :param upload: Multipart (field name, ASCII filename, file bytes).
        :param binary: Read a bounded binary attachment instead of JSON.
        :return: Successful JSON object or received binary bytes.
        :raises RuntimeError: HTTP, transport, body-limit or JSON failure.
        """
        if payload is not None and (fields is not None or upload is not None):
            raise ValueError("JSON and form/multipart cannot be combined")
        headers = {"X-Gallery-API-Key": self.key, "Accept": "application/json"}
        if upload is not None:
            field, filename, content = upload
            if len(content) > FILE_LIMIT:
                raise ValueError("File exceeds the example's memory bound")
            filename.encode("ascii")
            if any(c in field + filename for c in '\r\n"\\'):
                raise ValueError("Unsafe multipart field/filename")
            boundary = "pg-example-" + secrets.token_hex(16)
            chunks = []
            for name, value in (fields or {}).items():
                chunks.append((f"--{boundary}\r\nContent-Disposition: "
                               f'form-data; name="{name}"\r\n\r\n'
                               f"{value}\r\n").encode("utf-8"))
```

```
            chunks.extend([
                (f"--{boundary}\r\nContent-Disposition: form-data; "
                 f'name="{field}"; filename="{filename}"\r\n'
                 "Content-Type: application/octet-stream\r\n\r\n").encode(),
                content, f"\r\n--{boundary}--\r\n".encode(),
            ])
            data = b"".join(chunks)
            headers["Content-Type"] = f"multipart/form-data; boundary={boundary}"
        elif payload is not None:
            data = json.dumps(payload).encode("utf-8")
            headers["Content-Type"] = "application/json"
        else:
            data = urllib.parse.urlencode(fields or {}).encode("ascii")
            headers["Content-Type"] = "application/x-www-form-urlencoded"
        url = self.base + "/index.php?" + urllib.parse.urlencode({"page": route})
        request = urllib.request.Request(url, data=data, headers=headers,
                                         method="POST")
        try:
            with self.opener.open(request, timeout=TIMEOUT_SECONDS) as response:
                limit = FILE_LIMIT if binary else JSON_LIMIT
                body = response.read(limit + 1)
                mime = response.headers.get_content_type()
                if response.status != 200 or len(body) > limit:
                    raise RuntimeError("Unexpected status or oversized response")
        except urllib.error.HTTPError as exc:
            status = exc.code
            exc.close()
            raise RuntimeError(f"HTTP {status}; no automatic retry") from None
        except (urllib.error.URLError, TimeoutError, OSError) as exc:
            raise RuntimeError("Transport failed; write outcome may be unknown")
    from exc
        if binary:
            if mime in ("application/json", "text/html"):
                raise RuntimeError("Expected attachment, received JSON/HTML")
            return body
        if mime != "application/json":
            raise RuntimeError("Expected JSON, received another media type")
        result = json.loads(body)
        if not isinstance(result, dict) or result.get("ok") is not True:
            raise RuntimeError("Server did not confirm ok=true; inspect safely")
        return result


def read_file(path: Path) -> bytes:
    """Read one bounded original without modifying it; raise if too large."""
    with path.open("rb") as stream:
        content = stream.read(FILE_LIMIT + 1)
    if len(content) > FILE_LIMIT:
        raise ValueError("File exceeds the example's 64 MiB limit")
    return content


def save_json(path: Path, value: dict) -> None:
    """Exclusively save non-secret transfer metadata in the private workdir."""
    with path.open("x", encoding="utf-8") as stream:
        json.dump(value, stream, ensure_ascii=False, indent=2)
        stream.write("\n")
```

```
def migrate(directory: Path, include_children: bool) -> dict:
    """Run a new explicit migration; preserve evidence and stop on any failure.

    :param directory: New private directory; an existing directory is refused.
    :param include_children: Explicitly include the authorized source tree.
    :return: Successful completion receipt. Never retries ambiguous writes.
    """
    source = Api(os.environ["PG_SOURCE_BASE"], os.environ["PG_SOURCE_KEY"])
    target = Api(os.environ["PG_TARGET_BASE"], os.environ["PG_TARGET_KEY"])
    directory.mkdir(mode=0o700, parents=False, exist_ok=False)
    envelope = source.call("gallery_migration_manifest", {
        "include_subgalleries": "1" if include_children else "0"
    })
    save_json(directory / "source-response.json", envelope)
    manifest = envelope["manifest"]
    # This source snapshot is retained BEFORE any destination mutation.
    save_json(directory / "manifest.json", manifest)
    prepared = target.call("gallery_migration_receive_manifest", {
        "manifest_json": json.dumps(manifest)
    })
    save_json(directory / "job-response.json", prepared)
    job_id = prepared["job_id"]
    status = target.call("gallery_migration_receive_status", {"job_id": job_id})
    received = set(status["received_asset_keys"])
    for asset in prepared["assets"]:
        if asset["asset_key"] in received:
            continue
        selectors = {k: asset[k] for k in ASSET_FIELDS if k in asset}
        content = source.call("gallery_migration_asset", selectors, binary=True)
        expected_size = int(asset.get("file_size") or 0)
        expected_hash = asset.get("checksum_sha256") or ""
        if expected_size > 0 and len(content) != expected_size:
            raise RuntimeError("Source size changed; stop and retain the job")
        if expected_hash and hashlib.sha256(content).hexdigest() != expected_hash:
            raise RuntimeError("Source checksum changed; stop and retain the job")
        target.call("gallery_migration_receive_asset",
                    {"job_id": job_id, **selectors},
                    upload=("asset", "asset.bin", content))
    result = target.call("gallery_migration_receive_complete", {"job_id": job_id})
    save_json(directory / "completion.json", result)
    return result


def main() -> None:
    """Dispatch one explicit upload/inventory or a new authorized migration."""
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("operation", choices=("upload", "inventory", "migrate"))
    parser.add_argument("path", type=Path)
    parser.add_argument("--include-children", action="store_true")
    args = parser.parse_args()
    if args.operation == "migrate":
        result = migrate(args.path, args.include_children)
    else:
        client = Api(os.environ["PG_BASE"], os.environ["PG_API_KEY"])
        content = read_file(args.path)
        filename = args.path.name
        if args.operation == "upload":
            result = client.call(UPLOAD_ROUTE, {"action": "upload"},
                                 upload=("image", filename, content))
```

```
        else:
            result = client.call(UPLOAD_ROUTE, payload={
                "action": "inventory", "files": [{
                    "client_id": "example-1", "filename": filename,
                    "size": len(content),
                    "sha256": hashlib.sha256(content).hexdigest(),
                }]
            })
    print(json.dumps(result, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    try:
        main()
    except (KeyError, ValueError, OSError, RuntimeError) as error:
        detail = str(error) if isinstance(error, RuntimeError) else type(error).
    __name__
        print(f"Stopped: {detail}. Preserve inputs and inspect "
              "the original gallery/job; do not blindly repeat writes.",
              file=sys.stderr)
        sys.exit(1)
```

```
export PG_BASE='https://gallery.example/photos'
: "${PG_API_KEY:?Set securely}"
export PG_API_KEY
python gallery_api_example.py inventory photo.jpg
# The next command stores a photograph. Do not blindly repeat it.
python gallery_api_example.py upload photo.jpg

export PG_SOURCE_BASE='https://source.example/photos'
export PG_TARGET_BASE='https://target.example/gallery'
: "${PG_SOURCE_KEY:?Set securely}" "${PG_TARGET_KEY:?Set securely}"
# Keys must be exported into the child process environment.
export PG_API_KEY PG_SOURCE_KEY PG_TARGET_KEY
# Creates a child in the target; the workdir must not exist.
python gallery_api_example.py migrate ./migration-evidence
# Add --include-children only for an explicitly authorized subtree transfer.
```

For a retained job, the following status request does not create a new import. `JOB_ID` is copied from the validated job-response.json. Decide how to recover from the returned evidence; absence of a completion flag is intentional. A finalization request is a separate deliberate mutation, not a harmless connectivity test.

```
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_TARGET_KEY" \
  --header 'Accept: application/json' --data-urlencode "job_id=$JOB_ID" \
  "$TARGET_BASE/index.php?page=gallery_migration_receive_status"
# Only after reviewing the original job and its complete asset receipts:
curl --silent --show-error --proto '=https' --max-time 120 \
  --header "X-Gallery-API-Key: $PG_TARGET_KEY" \
  --header 'Accept: application/json' --data-urlencode "job_id=$JOB_ID" \
  "$TARGET_BASE/index.php?page=gallery_migration_receive_complete"
```

26.26 Implementation map and maintenance

The public contract above is intentionally separate from internal function documentation. The following source owners support verification of types, gates and error behavior. Preserve comments/docstrings and review Markdown plus all four manual editions when changing these wire contracts. A branch-only implementation is not automatically a supported released API. Do not add automatic browsing, server-wide listing or browser-approved APIs merely because their routes exist. docs/API.MD is the English reference; the corresponding manual chapters contain the same technical content in the edition language.

```
app/bootstrap.php
app/bootstrap/routing.php
app/bootstrap/dispatch.php
app/controllers/upload_automation.php
app/services/upload_automation.php
app/models/upload_automation.php
app/views/upload_automation.php
app/services/ai_image_analysis.php
app/services/upload_flight_reports.php
app/services/gallery_pdf_documents.php
app/controllers/gallery_migration.php
app/services/gallery_migration/versions.php
app/services/gallery_migration/manifest.php
app/services/gallery_migration/metadata.php
app/services/gallery_migration/assets.php
app/services/gallery_migration/target_setup.php
app/services/gallery_migration/install.php
app/services/gallery_migration/packages.php
app/services/gallery_migration/jobs.php
app/services/gallery_migration/recovery.php
winapp/uploader/simconnect_location.py
winapp/uploader/flight_delivery.py
docs/HTTP_ENTRYPOINTS.md
docs/COMPATIBILITY_LIFECYCLE.md
docs/AGENTS.md
docs/LATEX_BUILD.md
```

27 Optional reference for developers

This reference is for people changing PHP Gallery itself. Owners who only need publication and maintenance checks can continue with Section 28.

27.1 Repository organization

Location	Responsibility
`app/controllers/`	HTTP controllers and response coordination.
`app/services/`	Reusable domain and infrastructure services.
`app/views/`	Layout and reusable server-rendered view helpers.
`app/lang/`	Server and browser translation catalogs.
`database/migrations/`	Timestamped schema evolution.
`public/assets/`	Framework-free JavaScript, CSS, and public assets.
`scripts/`	Migration, integrity, deployment, and maintenance utilities.
`tests/`	Standalone PHP and focused JavaScript tests.
`winapp/`	Windows-specific tools and integrations.

For new or materially changed Admin interfaces, follow the permanent visual guidance in `docs/ADMIN_DESIGN.md`. It defines the compact media-player and WinApp direction and feature-scoped CSS review.

27.2 Coding conventions

New PHP files use strict types and four-space indentation. Controllers coordinate request behavior, while services own reusable logic. New JavaScript and CSS remain framework-free and belong under public assets. Migrations use timestamp-prefixed filenames. Existing explanatory comments, docstrings, and PHPDoc receive preservation and correction as behavior evolves.

Declaration docstrings are required for named functions, methods, and class-like declarations, including interfaces, traits, and enums. Place PHPDoc/JSDoc immediately above the declaration and Python docstrings first inside its body. Named functions and methods need a factual purpose summary, a typed and described parameter entry for each explicit parameter, and a typed return entry, including void. Classes need a factual purpose summary. Anonymous functions, closures, arrow functions, callbacks, variables, constants, and class properties do not require declaration docstrings; named declarations nested inside callbacks remain checked. File attribution, native signature typing, operational-policy comments, and parser coverage remain separate requirements.

Atomic modules should own one cohesive responsibility. Shared contracts deserve normalized inputs, explicit outputs, and focused tests. Dynamic Admin and public fragments require lifecycle-safe setup and teardown.

27.3 Authoring code before source checks

Readability guidance covers code, TeX and Markdown source. Prefer handwritten lines near 80 display columns; consider natural breaks at 81–100 and seek readable splitting above 100. This

is advice, not a maximum or CI gate. Put lengthy arguments and collection entries on separate indented lines, group conditions and separate logical steps/declarations with idiomatic blank lines. Wrap documentation prose at existing spaces while preserving paragraphs, TeX commands, non-breaking bindings and Markdown structure. Preserve semantics, types, comments and docstrings; allow necessary longer literals and URLs.

For an optional whole-tree report, open **Code readability report** in GitHub Actions, choose **Run workflow** and select only the branch. Its workflow, checker, rules and source use that branch's exact event SHA; the summary records the branch/SHA. Download the artifact for complete HTML, Markdown, JSON and per-file diagnostics covering PHP, JS, MJS, Python, CSS, TeX and Markdown. Findings never fail CI or rewrite source; tooling failures remain visible. The workflow appears in the sidebar after reviewed default-branch installation. Rules, exclusions, Unicode/tab measurement and focused examples are in `docs/CODE_READABILITY.md`; the central optional audit profile is `readability`. Required qualification remains unchanged.

The canonical `AGENTS.md` requires each named declaration and its complete documentation in the same edit. Start with a factual purpose before tags, document every explicit parameter (including defaults and added parameters), actual return variants and concrete collection/object shapes. PHP/Python native types remain independent. Anonymous callbacks need no declaration docstring; named declarations inside them still do. Runtime policy sites independently need purpose, type, units, scope, consumers and rationale immediately before the site. Checked examples in `docs/AGENT_AUTHORING.md` are parsed by the existing scanners in a registered regression, with negative fixtures preserving failures for incomplete contracts.

Python sources consistently use typed Google-style `Args:` and `Returns:` sections. Each parameter entry names the argument, its type and its actual meaning; procedures document `None`. Native annotations remain independent. Legacy tag inputs stay supported by the scanner for compatibility, while authored Python docstrings use one consistent format.

GitHub Actions runs the source preflight before hosted preparation. Coding agents follow CI-first: identify the current authorized working branch and HEAD, verify task-scoped branch/push access, finish source/tests/documentation, commit and push that exact branch. Never directly mutate `develop` or `main`. Missing authorization or CI access is BLOCKED. Local audits, generators and test loops are exceptional diagnostics, not routine handoff gates. Hosted CI retains the immutable merge-base with `origin/main`; releases retain the previous stable tag. Red or slow CI does not justify local-only qualification.

A requested implementation or fix authorizes automatic creation/checkout of a suitable codeword-feature/* branch from verified codewordorigin/develop, commits, exact branch push and hosted BASIC. Routine branch permission needs no repeat question. Integration, squash, release, tags and publication still require separate explicit approval under

ilepathAGENTS.md.

For disposable MySQL diagnostics, `scripts/gallery_workflow_mysql.php` offers `--audit-php-regression` through the central PHP regression suite. Isolate the checkout separately to isolate its cache as well.

Hosted qualification covers PHP/Node/WinApp tests, source contracts, required Chromium, real MySQL/MariaDB workflows, PHP compatibility and required Linux/Windows package checks; macOS runs independently as an advisory check. Every mandatory job and the exact-SHA gate must pass. The release workflow compiles all four PDF manuals once after the final source edits, not during routine development.

The hosted portable WinApp job installs the existing Pillow requirement into the Python interpreter selected for the audit, so tray-badge pixel coverage runs instead of skipping. Local runs without

Pillow may still skip this optional visual fixture; the headless hosted job does not install pystray.

The title-picture regression extends the already registered Admin side-panel delegation and browser lifecycle suites, plus the existing gallery edit concurrency HTTP/database suite. These fixtures remain under `scripts/audit.php`; they add no YAML test owner or alternate runner, and the protected evaluator registry is unchanged.

Reports record source HEAD, resolved comparison SHA and dirty state. A modified checkout is feedback, not clean-commit qualification. The linked `source-failures.md` lists all failed changed-source findings, blockers and findings in over-budget categories; complete value-free JSON retains the evidence and historical debt stays identified. Candidate workflows upload these together. Repair the whole batch before rerunning; unchanged input is not a repair. Finish all four manual sources and push the authorized working branch. Hosted Candidate preparation regenerates artifacts after the final edit and runs the mandatory full CI matrix for the exact prepared SHA. Record SHA, base, profile/result and material incomplete coverage. Measure first-attempt results and repair cycles from actual reports; expected savings remain unknown until observed.

27.4 Adding a feature

A typical feature change follows this sequence:

1. Trace current implementation and identify the owning controller, services, views, browser modules, persistence, and tests.
2. Define access, CSRF, scope, fallback, migration, and no-JavaScript contracts.
3. Add focused services and route integration.
4. Add append-only migration work where persistent schema changes are required.
5. Add translated interface strings.
6. Add focused automated tests and documented manual verification.
7. Update permanent Markdown references and all four TeX manual sources whenever current user, administrator, architecture, or maintainer guidance changes.
8. Hosted Candidate preparation runs `scripts/prepare_candidate.php` and its read-only check: runtime modules, indexed production inventory and complete-byte integrity hashes are generated in dependency order. It commits only changed artifacts back to the same authorized branch, refuses concurrent writers, and qualifies the exact resulting SHA. Agents push focused source commits and inspect CI; they do not duplicate generation locally.
9. Report the GitHub Actions URL, checked working branch, exact prepared candidate SHA, immutable comparison base, required job results and manual or missing coverage. A stale, missing, skipped, cancelled, failed or blocked mandatory job is not green qualification. Fix source and push the same branch for fresh CI.

27.5 Preparing source candidates for CI

Atomic source commits are allowed during development, but qualification must use a candidate whose generated artifacts match the exact source revision. Permanent technical documentation is part of the source edit. Changes to administrator behavior, architecture, runtime ownership,

security, packaging, or CI also require reviewing all four TeX manual editions, not only user-facing pages. Routine development leaves `PATCH_NOTES.md` and the tracked manual PDFs unchanged.

The issue-362 source workflow also provides an unprivileged recovery artifact for prepared outputs. Both PHP quick jobs run preparation and its read-only check; after successful preparation, only the PHP 8.5 job uploads the complete bytes of `app/runtime/modules.php`, `app/production-files.json`, and `app/core-manifest.json` before feature tests. The artifact name starts with `issue-362-prepared-` and includes the exact source SHA. It lets the owner review and commit those prepared bytes. It is not FULL qualification or publisher authority; verify the SHA and inspect all three files before use.

Preparation in a temporary CI checkout does not update the local checkout. Persist all three canonical outputs on the feature branch and require final-SHA hosted re-preparation to reproduce them before claiming a runnable checkout. Startup checks every registered route, including optional routes with a disabled capability. When localhost smoke is explicitly requested, reload the exact mount and a content page after persistence; correlate any error reference with the PHP emergency log and repeat the browser check after repair.

1. Hosted Candidate preparation runs `scripts/prepare_candidate.php` and its read-only check: runtime modules, indexed production inventory and complete-byte integrity hashes are generated in dependency order. It commits only changed artifacts back to the same authorized branch, refuses concurrent writers, and qualifies the exact resulting SHA. Agents push focused source commits and inspect CI; they do not duplicate generation locally.
2. Report the GitHub Actions URL, checked working branch, exact prepared candidate SHA, immutable comparison base, required job results and manual or missing coverage. A stale, missing, skipped, cancelled, failed or blocked mandatory job is not green qualification. Fix source and push the same branch for fresh CI.

The current qualification and release assembly sequence is described in the maintainer verification section and `RELEASE.md`.

28 Checking your gallery before publication

Testing for a gallery owner means walking through the site as the intended audience. A technically healthy page can still contain a private location, an unclear title, an incorrect date, an accidental download permission, or an awkward mobile layout. A short human review catches these presentation and privacy concerns.

28.1 A visitor-centered publication check

Open a private browser window and follow the same link you plan to share. Check:

1. The title and first paragraph explain the collection.
2. The chosen cover remains clear at a small size.
3. Child galleries appear in a sensible order.
4. Photograph titles and descriptions match the visible subjects.
5. Private photographs and galleries stay hidden.
6. Password or share access works from a fresh session.
7. GPS maps reveal only intended locations.
8. The first page loads comfortably on a phone.
9. The large viewer, forward and backward controls, and close action feel natural.
10. Search, tags, voting, and downloads behave according to the gallery's purpose.

Ask another person to try the gallery without instructions. Their first click and first question often reveal navigation or wording that felt obvious only to the owner.

28.2 Automated checks for software maintainers

The central audit covers clipboard MIME/name handling and synthetic browser paste events, ordinary multipart upload and panel persistence. These fixtures do not read the operating-system clipboard. Record actual Windows Chrome/Edge/Firefox, Linux Chromium/Firefox and macOS Safari/Chrome/Firefox results using the manual matrix in `TESTING.md`, for both browser and server preparation on direct-page and side-panel forms. Unexecuted checks remain pending.

The registered miniature tests cover bounded raster headers, cancellation and resource disposal. The Chromium fixtures also check 100-item FileList parity, a 320-pixel drawer, reset/navigation cleanup, focus and classic/prepared retry without duplicate ingestion. Observed URL/decode counts are not a whole-process memory measurement; real OS clipboard delivery and Firefox/Safari checks remain manual.

Based on hosted timeout evidence, the Windows feature job uses `phpscripts/audit.php--suite=winapp-partitioned`. It places all classes from `test_native_gallery_viewer.py` into one child, the small and large thumbnail-cache benchmark classes `ThumbnailCacheSmallBenchmarkTests` and `ThumbnailCacheLargeBenchmarkTests` into separate children, the `GalleryPreloadWorkerTests` class into a fourth, and the exact remaining inventory into a fifth. All five serial processes retain the existing 90-second child limit; the coordinator has a 480-second outer bound for these child caps and inventory/report handling. The central audit verifies all five schema-1 reports

and the complete disjoint test-ID union before merging actual outcomes. Missing, timed-out, malformed, duplicate or unsuccessful evidence fails. The ordinary full-suite owner keeps its existing single-process behavior and limit. Earlier full-process and two-, three- and four-partition timeouts remain failed runs. Selected-child logs flush the full failure traceback immediately after each FAIL/ERROR terminal record, preserving its reason if a later timeout prevents the final JSON report.

The **Issue 381 Watcher recovery** workflow also offers a manual browser diagnostics mode. It runs only the central `phpscripts/audit.php--suite=browser-map` on the selected feature SHA, retaining reports and screenshots. Separate concurrency prevents that diagnosis from cancelling normal source/WinApp checks. It preserves central limits and assertions and does not replace the complete integration FULL. Interactive fixtures activate their own browser tab before trusted input; emulated DOM focus alone does not prove native fullscreen entry.

For coding agents, GitHub Actions is the default and authoritative verification environment (CI-first). The central audit commands below describe the hosted runner; agents do not execute them locally by default. Focused local reproduction is reserved for an observed environment-specific CI failure, genuinely inaccessible CI or an explicit request. Local evidence never replaces required hosted coverage.

```
# Hosted source feedback (exceptional local diagnosis only):
php scripts/audit.php --profile=release-preflight
php scripts/audit.php --profile=quick
# Hosted full source qualification:
php scripts/audit.php --profile=full
# Hosted authoritative release job:
php scripts/audit.php --profile=release
```

The current qualification and release assembly sequence is described in the maintainer verification section and `RELEASE.md`.

The registered breadcrumb Theme test compares the Theme and physical-gallery style lists in `docs/ADMIN_SETTINGS_INVENTORY.md` with `breadcrumb_style_registry()`. It also protects the `chevron` fallback and the gallery-only `inherit` sentinel. When adding a style, update both documented lists; `inherit` is not a Theme visual style.

28.3 Manual browser verification

28.3.1 Network and interaction checks

Browser-dependent software checks use representative data, anonymous and authenticated sessions, an empty cache, mobile and desktop viewports, JavaScript-disabled operation where relevant, reduced-motion preferences, and network throttling.

For public thumbnail rendering, inspect the Elements and Network panels. Responsive mode should expose its complete active candidate set. Progressive mode should expose a small active candidate and inert larger candidates before activation, then perform at most the documented concurrent upgrades for relevant cards. Check layout stability, cache reuse, error fallback, lightbox interaction, maps, voting, protected media, and direct links.

28.4 Release hygiene

Follow `RELEASE.md` for the complete release sequence. Compare the exact previous stable tag, complete the release notes and permanent documentation, and align all four maintained manual editions. Rebuild each PDF with the indexed multi-pass sequence in `docs/LATEX_BUILD.md` and check compiler success, references, index entries, and layout warnings. Visual PDF review is reserved for an explicit request or diagnosis of a concrete layout problem; routine successful builds need no additional visual-approval gate.

The commands in this section describe hosted release ownership and the retained local maintainer recovery path. Normal preparation and qualification run on GitHub; agents do not execute these commands locally by default. Local recovery requires an explicit request or genuinely unavailable CI and remains BLOCKED / not CI-qualified until hosted evidence exists. The later review workflow performs protected merge and tagging; GitHub Release publication stays manual.

The current qualification and release assembly sequence is described in the maintainer verification section and `RELEASE.md`.

Create a production folder or ZIP only when explicitly requested; hosted release assets use the same PHP builder automatically. Pass the candidate epoch with `--timestamp` for deterministic metadata. The builder refuses stale manifests and existing outputs; `scripts/release_files.php` inspects membership only. Required Linux/Windows packaging owners cover the real builder independently of the database matrix; macOS repeats the same suite in a separate advisory run. The main-only locked TinyTeX cache warmer requires reviewed installation before release branches can reuse its exact key. A CMS-only release does not require a new WinApp installer. Commit, tag, publish, upload, and deploy require explicit authorization. CI tests PHP 8.1 as the compatibility floor and PHP 8.5 as the newest representative. All three database variants run on PHP 8.5; historical evidence retains its actual versions.

During ordinary development, update all four `.tex` sources together and review their actual explanations; do not rebuild the tracked PDFs after every change. The PDFs may lag behind the sources until final release preparation. Automated version/date replacement is not semantic documentation generation: maintainers must add new feature, security, compatibility, and operational content in English, Czech, German, and Swedish themselves.

The current qualification and release assembly sequence is described in the maintainer verification section and `RELEASE.md`.

29 Making the gallery feel fast

Perceived speed is not one number. Visitors notice when the page first appears, when the first photographs become recognizable, whether scrolling stays responsive, how quickly controls react, and how long a larger image takes after they ask for it. Connection quality, server distance, page size, thumbnail policy, branding assets, and the selected renderer all contribute.

29.1 Practical ways to improve speed

- Keep the number of cards per page reasonable for the target audience and hosting plan.
- Generate the configured thumbnail sizes in advance for large public collections.
- Avoid unnecessarily large hero, background, logo, and other theme assets.
- Prefer the responsive thumbnail renderer when predictability and no-JavaScript behavior matter most; use the progressive renderer when its staged sharpening suits the collection and browser mix.
- Place the gallery on hosting geographically close to its main audience where practical.
- Test with an empty cache. A warm developer browser can hide expensive first-visit transfers.
- Check PHP/database latency separately from image-transfer time before assuming that every slow gallery is a thumbnail problem.

29.2 Slow-connection test

Open the gallery in a private browser window, select a representative mobile viewport, enable network throttling in developer tools, and reload with an empty cache. Watch the order in which the page, covers, visible cards, and later image upgrades arrive. Repeat with the renderer or page-size setting you intend to compare; otherwise cached files make the comparison unreliable.

29.3 Technical measurements for maintainers

Public render profiling and browser diagnostics are useful when a subjective report needs numbers. Record at least:

- time to first byte and server-side render duration;
- HTML transfer size;
- request count and bytes transferred for initial thumbnails;
- time until the largest visible content has settled;
- layout shifts while cards and media load;
- main-thread work attributable to browser modules;
- number and size of progressive candidate upgrades;
- database-query totals from the development profiler where available;
- image decode/memory pressure when testing very large originals or deep lightbox zoom.

The development profiler and captured benchmark records are intended for before/after comparisons across renderer modes and gallery layouts. Use representative high-density screens and constrained connections rather than relying only on a fast desktop with a warm cache.

30 Troubleshooting

30.1 Application does not start

Confirm the PHP version, required extensions, configuration location, database connectivity, writable paths, and web-root routing. Review PHP and web-server logs. Run syntax checks on recently changed PHP files and inspect pending migrations.

30.2 Gallery or images are missing

Confirm filesystem paths, gallery discovery state, database records, visibility, parent access rules, password unlock state, NSFW policy, supported file type, and image relative path. Test as both administrator and anonymous visitor.

30.3 Thumbnails are missing or soft

Review thumbnail metadata and maintenance reports. Confirm generated sizes, format support, configured bounds, source geometry, derivative status, and public endpoint authorization. In responsive mode, inspect the browser-selected candidate and `sizes`. In progressive mode, inspect the small candidate, queue state, selected replacement, and decode result.

30.4 Admin actions leave the side panel

Confirm the matching JavaScript module loaded, delegated handlers recognized the form, AJAX headers and response contracts are correct, the returned fragment contains expected lifecycle markers, and generic Admin form handlers exclude feature-owned forms. Direct POST behavior can still provide the documented fallback route.

30.5 Migration fails

Keep a verified database backup and record the migration filename, Gallery version, database-server version, bounded on-screen message, request reference, and existing schema/ledger state. Retry through the Gallery administration page so the normal idempotent compatibility path can finish an interrupted migration. Version 1.1.2 does not need triggers, stored routines, `SUPER`, `log_bin_trust_function_creators`, or a global database change. If the bounded error remains, inspect Runtime Diagnostics and give the request reference to the maintainer or hosting provider; do not paste database credentials or raw production data into a support report.

30.6 PDF manual compilation fails

Run the commands in `docs/LATEX_BUILD.md`. Confirm `pdflatex` and `makeindex` are installed. A first MiKTeX run may request common packages. Delete ignored intermediate build files after a failed package transition and repeat the documented sequence.

31 Backup and recovery checklist

1. Export the complete application database.
2. Copy `galleries/` with source files and gallery-owned assets.
3. Copy local configuration and relevant installation secrets securely.
4. Preserve custom theme assets and custom CSS.
5. Record the application version and pending migration state.
6. Store backups outside the active web tree.
7. Test restoration periodically in an isolated environment.
8. Verify public routes, protected access, Admin login, thumbnails, uploads, and migrations after restoration.

Updater-created backups preserve overwritten application files for update recovery. A complete operational backup also includes the database, gallery tree, local configuration, and installation-owned assets.

31.1 Validate a recovery set

Keep the database export, original photographs, local configuration, relevant `data/` storage, Trash contents, custom assets, and exact application version together as one coordinated recovery set. A provider snapshot or a bounded maintenance window can prevent writes from splitting database and file state. Store an off-host copy and agree how much recent work may be lost and how quickly service must return.

The CLI entry point `scripts/recovery.php` can inventory that set and validate restored file samples and manifest hashes in a separate location. Follow `docs/RECOVERY_ASSURANCE.md` for the evidence format and commands. The tool does not execute a database dump, restore production, or verify a backup provider. Its synthetic corruption drill proves the checker detects fixture damage, not that your actual backup can restore the site.

Before opening a restored installation, isolate email, scheduled jobs, integrations, and write access from production destinations. Verify the restored database, relationships, Admin login, private-media denial, and Trash restoration. Record backup age, elapsed recovery time, and the operator's observations without copying secrets into reports. Use `docs/RECOVERY_OFF_HOST.md` for the full procedure. Updater file rollback still does not reverse database migrations.

32 Maintainer verification and release evidence

32.1 Source qualification and selected feature workflows

Feature pushes and the final squash receive BASIC syntax and declaration checks. In GitHub Actions, choose **Integrate into develop**, select main or the intended feature in **Use workflow from**, and select the source branch. The owner's explicit dispatch runs that revision's actual workflow, audit and tests. Changes to tests or registries need no preliminary installation on main.

The source is one commit above observed develop. Hosted preparation generates runtime modules, inventory and manifest, folds changed generated data into the final candidate and obtains fresh BASIC. A separate authority checkout inspects the clean candidate. FULL retains all 27 required native owners: shared source contracts, portable Node/WinApp, Linux/Windows packaging, four shards for each of three databases, four shards for each PHP 8.1/8.5 source environment, required Chromium and the aggregate. Missing, failed, cancelled or skipped mandatory owners block integration.

After preparation, the selected feature workflow dispatches macOS packaging as an independent advisory run on the same exact candidate and base. It executes the central package suite and retains its actual result and diagnostics. Runner queues, timeout, cancellation, failure or dispatch refusal never block integration. Separate concurrency and check names protect required FULL. Historical owner contracts 1–3 keep their original mandatory macOS proof; authenticated immutable workflow bytes determine the applicable contract.

Successful execution is sealed before the protected non-force fast-forward. `qualify_only` retains qualification without writing develop. Supply its run ID as `qualified_run` to publish the identical candidate, complete input tree and base without another FULL. Changed inputs require new execution. The publisher still checks actual owner/run identities, source leases and branch protections.

After successful integration and record upload, the workflow deletes the selected remote feature branch with an exact candidate-SHA lease. Newer commits prevent deletion. `qualify_only` and `recover_existing` retain the branch. Cleanup failure leaves the completed integration and retained evidence intact.

For an explicit CI-only installation ahead of a release, base the feature on main and dispatch **Basic source checks** with `install_ci`. Complete FULL and the authoritative release audit produce a bot PR; one real human approval installs it through normal protected auto-merge.

The publisher binds the same verified FULL to both Checks and a standard Actions commit status, because dispatch-suite Checks can be absent from GitHub's branch-rule summary. It rechecks the status identity, state and run before each protected fast-forward.

After qualification and protected installation, a sole HTTP 422 rejection for the new required matrix check permits up to six publication attempts with two-second waits. Each retry revalidates the active run, BASIC/FULL evidence, original check IDs and branch leases. Other failures stop; FULL is not replayed.

32.2 Release assembly from qualified develop

Registered Markdown version markers accept soft line breaks within the same paragraph. Preparation and consistency checks preserve the existing wrapping and historical version prose while updating the registered value. WinApp README, agent instructions and tests alone do not trigger an installer rebuild; runtime and build-input changes still do. The draft attachment reader

uses the same exclusions when reusing verified published installer bytes. Recovery after tagging preserves Q, M and the tag.

In GitHub Actions, run **Begin Release** from the branch containing the intended workflow revision, normally `develop`, and enter the version including `v_`, for example `v_0.136.2`. The workflow separately reads current protected develop, freezes its SHA, creates `release/v_0.136.2` and explicitly dispatches assembly on that release branch. Its new workflow and scripts execute before they reach main. No BASIC/FULL is repeated. Initialization authenticates the original maintainer and initiating run, attempt and version; the bot identity alone grants no authority. Retry can reuse an unmoved ref and avoids duplicate active assembly. If the release ref has advanced, continue its existing assembly directly. PR approval and publication remain separate maintainer actions.

Before PR review, assembly retains the frozen main history and prepared tree. After approval and tagging, unchanged develop fast-forwards to the actual tagged main commit. Both branch refs then match; no content-copy commit is created. Its status represents verified assembly and qualified source ancestry; functional FULL is not repeated. Parallel develop work is preserved. The permanent receipt records the actual main result, tree and base. Develop's history policy must admit reviewed main merges while retaining non-force updates, required Actions checks and no bypass. The legacy linear-history requirement explicitly blocks readiness and PR creation; only the maintainer changes installed policy. A later release can join existing divergent histories without rewriting them. Historical content-copy receipts remain readable; immediate ref realignment requires separate authorization. The next release trusts protected develop without historical reconciliation archives. Only the post-assembly observer handles approval, including approval submitted early; successful origin Git push needs no immediate REST readback.

A commit admitted to protected develop is qualified. Create the separately authorized `release/v_X.Y[.Z]` branch from that source. Release does not rerun BASIC/FULL or download older qualification archives. It freezes source A, updates metadata and patch notes, aligns all four manual editions, compiles their PDFs with the locked TinyTeX toolchain and reproducible timestamp, and refreshes inventory and the complete-file manifest last. Changed WinApp inputs require the installer; unchanged inputs retain its independent version and verified existing assets.

New patch notes link every evidenced issue or PR directly addressed by a described change in that release. Complete commit metadata includes changed paths for each commit, supporting comparison with the source diff and evidence digests. Previous notes supply style only; historical examples and routine release bookkeeping establish no current issue work. Release origin messages and PR wrappers add no fixed references. Actual new work on older issues remains eligible. Copilot may return `NOTES_REVIEW_REQUIRED` when extraction, consolidation or grounded synthesis cannot finish reliably, including an unresolved material issue relationship. Validation then uses the same external JSON handoff described below. Format checks do not prove semantic relevance.

Editorial notes preserve completed maintainer prose or use the existing bounded Copilot generation from complete Git evidence. Automatic work admits at most 100000 diff bytes, 20 source files and 180000 prompt bytes before provider setup. Larger changes, missing credentials, explicit external mode or Copilot/quota/invalid-output failures create `NOTES_PENDING`. Download `release-notes-handoff` from the Actions run and give `prompt.txt` to your chosen AI. The 30-day artifact includes the full template, commit evidence, source diff and hash manifest. Review its single-line JSON response, then select the same release branch in **Release qualification > Run workflow**, paste it into `external_notes` and run. The bundle README provides CLI continuation when the button is not yet registered on main. JSON escapes retain Markdown newlines. Repository, version, branch, source SHA and prompt hash must match freshly rebuilt evidence; changed evidence needs a new bundle. Existing notes remain authoritative. Pending handoff uses no waiting runner and creates no prepared commit, qualification check, manuals build or release PR. Continuation adds no BASIC/FULL. Invalid output is never applied. Build failures,

missing attachments and stale generated hashes stop assembly; they do not introduce another source qualification stage. Lifecycle schema 4 records the qualified develop source and assembly outcome.

The existing stable tag remains immutable after CI-only main maintenance. Initialization follows actual first-parent main history instead of requiring a new tag or a version-specific exception. Git leases still protect concurrent writers. Original release identities and published assets stay unchanged.

32.3 Review, version and complete attachments

The workflow opens the exact bot-authored PR to main. Approve Q once. Automation performs the protected standard merge, verifies its parents/tree, creates the immutable version tag, attempts safe develop synchronization and fills an unpublished release draft with the original ZIP, metadata, checksums, all four PDFs, notes and permanent assembly/completion evidence. It adds missing verified assets and preserves identical existing ones.

Wait for `DRAFT_READY`, then manually choose **Publish release**. `TAGGED/DRAFT_PENDING` is recoverable for the same Q; do not rebuild the candidate or move a correct tag. Parallel develop work blocks unsafe synchronization, not a correct completed tag or draft. After `DRAFT_READY` and retention of completion evidence, the final job automatically removes the exact release branch with a Q SHA lease. It rechecks uploaded asset hashes, reviewed merge/tag, main/develop ancestry and active PR/workflow dependencies. Incomplete synchronization keeps the branch; failed proof blocks cleanup while preserving the draft. Retry only the failed cleanup job; immutable Q and retained evidence support an already deleted branch. This starts with the first release containing the change in qualified develop. Manual publication and browser/hosting smoke tests remain separate. Historical branches retain the separately approved retirement workflow. See `RELEASE.md`, `docs/RELEASE_LIFECYCLE.md`, `docs/CI_COVERAGE_REUSE_CONTRACT.md` and `docs/LATEX_BUILD.md`.

33 Glossary

AJAX Browser request used to update a page region while preserving the surrounding interface.

Branch scope A gallery and its descendants, when the selected feature defines recursive scope.

Canonical pair Two image identifiers stored in a stable low-to-high order.

CSRF token Session-bound value used to validate intentional state-changing form submissions.

Derivative Generated media representation such as a JPEG or WebP thumbnail.

EXIF Camera and capture metadata embedded in supported image files.

Integrity manifest Versioned list of core paths and expected hashes.

Lightbox Fullscreen or overlay photo viewer with navigation and related controls.

Media manifest Request-local rendering data prepared from thumbnail metadata for visible images.

NSFW guard Age and gallery/image policy controlling access to restricted media.

Progressive sharpening Small-first thumbnail pipeline that activates a decoded larger candidate later.

Responsive selection Browser-native candidate selection from an immediately active source set.

Selected gallery Gallery represented by the current public route.

Side panel Admin right-side contextual interface loaded and refreshed through fragment requests.

Thumbnail bounds Gallery-aware minimum and maximum generated sizes eligible for selection.

34 References

References

[1] PHP Gallery project, *README.md*, local product overview, feature guide, requirements, and installation documentation, version 1.1.2.

[2] PHP Gallery project, *ARCHITECTURE.md*, local application architecture and subsystem lifecycle reference, version 1.1.2.

[3] PHP Gallery project, *DATABASE.md*, local migration and persistent schema reference, version 1.1.2.

[4] PHP Gallery project, *CODEMAP.md*, local source ownership and maintenance map, version 1.1.2.

[5] PHP Gallery project, *docs/ADMIN_SETTINGS_INVENTORY.md*, canonical global Settings ownership, fallback, sensitivity, and migration inventory, version 1.1.2.

[6] PHP Gallery project, *TESTING.md*, local automated and manual verification guide, version 1.1.2.

[7] PHP Gallery project, *AGENTS.md*, local repository contribution and security guidelines.

[8] The PHP Documentation Group, *PHP Manual*, https://www.php.net/manual/en/.

[9] Oracle Corporation, *MySQL Reference Manual*, https://dev.mysql.com/doc/.

[10] MariaDB Foundation, *MariaDB Server Documentation*, https://mariadb.com/docs/server/.

[11] WHATWG, *HTML Living Standard: Images*, https://html.spec.whatwg.org/multipage/images.html.

[12] OWASP Foundation, *Web Application Security Guidance*, https://owasp.org/.

Index

3D carousel, 106
503 response, 95

Admin
    temporary support access, 7
Admin dashboard, 7
Admin login, 7
admin logs, 123
administration, 58
    side-panel mutations, 62
administrator account, 94
AI metadata, 74
alternative text, 47
anonymous preview, 8
API
    pgu_, 139
API migration, 110
architecture, 135
audiences, 49
authentication, 91

backup, 113
bootstrap, 135
browser modules, 137
browser-side thumbnail rebuild, 80

canonical URL, 116
captions, 47
clean URLs, 6
client gallery, 49
coding style, 163
collection design, 9
collection sharing, 93
Complete report, 117
configuration
    base URL, 65
controller, 136
controller dispatch, 135
controllers, 135
cooperative galleries, 28
core manifest, 134
cron
    site maintenance, 119
CSRF, 91
Custom CSS, 99

daily administration, 53
data ownership, 51
database, 3
    benefits, 51
    for gallery owners, 51
    mutation safety, 96
    readiness, 95, 113
database schema, 138
date range, 19
development, 163
device pixel ratio, 76
discovery, 71
DNG, 77
documentation, 1
downloads, 110
duplicate ledger, 90
Duplicate Photo Detector, 90

event gallery, 49
EXIF, 83
    date organizer, 67
    user review, 48

family archive, 49
favicon, 99
favourites, 92
feature inventory, 124
feature reference, 124
feature switches, 66
Features
    Admin, 66
filenames
    renaming, 68
filesystem
    for gallery owners, 51
filesystem-first design, 3
first hour, 7
flight maps, 86

gallery
    automatic organization, 67
    creating the first gallery, 7
    creation, 11
    deletion, 24
    display overrides, 20
    editing, 11
    moving, 23
    quick feature controls, 58
    reordering, 23
    restoration, 114
    trash, 114
    visibility, 20

gallery benchmark, 118
gallery branding, 23
gallery cover, 23
gallery creation
  remembered defaults, 12
gallery dates
  EXIF review, 19
gallery description, 9
gallery editor, 11
  controls, 20
  side panel, 12
gallery hierarchy, 55
  depth, 9
  planning, 9
gallery metadata, 19
gallery migration, 110
  atomic installation, 97
gallery owner, 7
gallery report, 117
gallery title, 9
gallery visibility, 10
gallery.json, 71
getting started, 7
glossary, 177
Google login, 94
  database readiness, 96
GPS, 83

historical archive, 50

image comparison, 89
image metadata, 46
image previews, 76
image sitemap, 111
images, 69
installation, 5
institutional archive, 50
integrity checking, 118

JPEG, 76
  thumbnail use, 76

language, 107
  Czech, 108
  English, 108
  German, 108
  Swedish, 108
launch checklist, 53
layout stability, 79
lazy loading, 79
lightbox, 83
  browsing modes, 106
  zoom, 84
localization, 99
  gallery content, 107
  language selection, 107
  photo titles, 107
location privacy, 48
log archive, 123

maintenance, 113
  central workflow, 114
Maintenance Center, 114
manual
  how to use, 1
  scope, 1
maps
  database readiness, 97
  travel use, 49
MariaDB, 4
media
  filesystem containment, 97
media maintenance, 68
Media Renamer, 68
metadata organizer, 67
migrations, 138
  missing objects, 113
monthly maintenance, 53
MySQL, 4

navigation data, 86
NSFW, 91
NSFW Guard, 96

OpenAI, 74
  database readiness, 97
ownership checklist, 53

page speed
  thumbnails, 76
pagination, 55
  Theme settings, 99
password protection, 91
password reset
  database readiness, 96
PDF
  gallery documents, 15
performance, 170
permissions
  filesystem, 4
photo ordering, 48
photo organization, 46
photo title, 47
photo workflow, 46
PHP, 4
Picture Game, 89

database readiness, 97
picture manager, 111
picture strip, 106
pinch gesture, 84
planning galleries, 9
portfolio, 49
privacy
telemetry, 117
privacy planning, 10
product overview, 3
progressive renderer, 78
progressive sharpening, 78
proofing workflow, 49
public content widgets, 55
public gallery, 54
publication checklist, 167
publishing workflow, 49

quality assurance, 167
query-string routing, 6
query-string spam, 116

RAW photographs, 77
recovery, 173
assurance, 173
render pipeline, 137
request
life cycle, 135
requirements, 4
responsive renderer, 78
robots.txt, 111
routing, 54, 135
Runtime diagnostics, 118

scheduled maintenance, 119
schema inspection, 113
search, 56
security, 91
SEO request guard, 116
services, 135, 136
Settings
central hub, 64
Setup Wizard, 64
website address, 65
setup, 5
Setup Wizard, 64
SHA-256, 90
share links, 91
database readiness, 96
shared hosting, 3
side panel
mutation completion, 62
sidecar metadata, 71
SimBrief, 86
database readiness, 97
pre-create draft, 12
retrospective OFP, 87
sitemap.xml, 111
Smart Galleries, 25
cycle safety, 26
downloads, 27
lightbox, 27
presentation, 26
SMTP, 94
speed, 170
srcset, 78
storage statistics, 117
storytelling, 48
System Health
database inspection, 113
database readiness, 95

tagging strategy, 47
tags, 56
administration, 65
gallery hero, 109
metadata, 65
practical use, 47
telemetry, 117
database readiness, 97
testing, 167
theme, 99
controls, 99
gallery tags, 109
thumbnail
browser selection, 78
choosing a renderer, 79
definition, 76
formats, 76
generation, 77
lazy loading, 79
maintenance, 77
missing, 82
progressive mode, 78
quality, 80
rebuild, 82
responsive mode, 78
self-healing, 81
sizes, 76
storage, 80
thumbnail compatibility, 80
thumbnail maintenance, 80
thumbnail warmup, 81
thumbnails, 76
translation packs, 107

diagnostics, 108
fallback, 108
JSON, 108
trash
automatic purge, 114
travel archive, 49
troubleshooting, 172
database connection, 113

updates, 113
database readiness, 96
upload automation, 74
upload methods, 46
uploads, 69
database readiness, 96
URL rewriting, 6
use cases, 49

viewer account management, 91
viewer accounts, 91
viewer collections, 93
viewer invitations, 91
viewer login, 91
views, 135, 137
virtual galleries, 25
visitor experience, 54
visitor preview, 8
voting, 83
database readiness, 97

WebDAV, 74, 110
mobile upload, 111
WebP, 76
thumbnail use, 76

ZIP, 110
optional media, 110
zoom controls, 84